

算法板子

竞赛代码模板速查手册

打印用 • 比赛翻阅 • 开局粘贴

2026 年 7 月 23 日

目录

第一章 总览、复杂度与学习路线	15
1.1 先判断题目属于哪一类	15
1.2 复杂度速查（单次操作约 1 秒时的经验值）	15
1.3 分层路线	15
1.3.1 L0: 实现基本功	15
1.3.2 L1: 线性与排序	16
1.3.3 L2: 基础图和数学	16
1.3.4 L3: 基础 DP 和经典数据结构	16
1.3.5 L4: 字符串、树和提升题	16
1.3.6 L5: 银牌常见进阶	16
1.4 赛场决策树	16
1.5 正确性证明的四种常用模板	16
1.6 提交前检查	17
第二章 竞赛模板	18
2.1 C++ 竞赛实现规范	19
第三章 STL 常用函数速查	20
3.1 容器	20
3.1.1 vector	20
3.1.2 array	20
3.1.3 string	21
3.1.4 set / multiset	21
3.1.5 map	21
3.1.6 unordered_map / unordered_set	21

3.1.7	priority_queue	22
3.1.8	deque	23
3.1.9	stack	23
3.1.10	list	23
3.2	常用算法	24
3.3	pair / tuple	24
3.4	bitset	25
3.4.1	bitset 优化背包 / 子集和可行性	27
3.4.2	bitset 优化传递闭包 (Floyd)	27
3.4.3	bitset 优化字符串匹配 (Shift-And)	27
3.5	__int128	28
3.6	__builtin 系列	28
3.7	C++ 常用函数速查	29
3.7.1	数值函数	29
3.7.2	区间操作函数	30
3.7.3	排列与组合	31
3.7.4	全排列函数详解 (next_permutation / prev_permutation)	32
3.7.5	排列序 (康托展开与逆康托展开)	36
3.7.6	字符串处理函数	37
3.7.7	其他实用工具	38
第四章	位运算	40
4.1	基本性质	40
4.1.1	分配律与吸收律	40
4.1.2	异或的重要性质	40
4.1.3	$O(1)$ 求连续自然数的异或和	41
4.1.4	与/或的单调性	42
4.1.5	常用恒等式速查	42
4.2	竞赛常用性质科普	42
4.2.1	按位独立性 (Bitwise Independence)	42
4.2.2	异或与线性代数	43

4.2.3	位运算与集合论的对应	43
4.2.4	popcount 的性质与应用	43
4.2.5	高位/低位操作	44
4.2.6	位运算与不等式	44
4.2.7	SOS DP (子集和 / 超集和)	44
4.3	常用技巧	45
4.4	位运算与异或	45
4.5	线性基 (XOR Basis)	46
4.6	线性基: 参数、调用与改造	46
4.6.1	0.1 它解决什么问题	46
4.6.2	0.2 可直接使用的模板	47
4.6.3	0.3 例题: P3812 线性基	47
4.6.4	0.4 常见错误	49

第五章 基础算法 50

5.1	前缀和	50
5.1.1	一维前缀和	50
5.1.2	二维前缀和	50
5.2	差分	50
5.2.1	一维差分	50
5.2.2	二维差分	51
5.3	二分查找	51
5.3.1	整数二分	51
5.3.2	浮点二分	51
5.4	三分法	52
5.4.1	浮点三分法	52
5.4.2	整数三分法	53
5.5	双指针 / 滑动窗口	54
5.6	自定义排序	54
5.7	枚举与剪枝	55
5.7.1	子集枚举	55

5.7.2	回溯剪枝	55
5.7.3	折半搜索 (Meet-in-the-middle)	55
5.8	前缀和、差分与扫描	56
5.8.1	一维前缀和	56
5.8.2	一维差分	56
5.8.3	扫描线思想	57
5.9	排序、二分、双指针	57
5.9.1	二分查找	57
5.9.2	二分答案	57
5.9.3	双指针/滑动窗口	58
5.9.4	三分与凸性	58
5.10	贪心	59
5.10.1	典型证明模式	59
5.10.2	反悔贪心	59
5.10.3	差分约束式贪心	59
5.11	单调栈与单调队列	60
5.11.1	单调栈	60
5.11.2	单调队列	60
5.12	离散化与坐标压缩	60
5.13	常见构造套路	61
5.14	随机化与哈希技巧	61
5.15	位集优化	61
5.16	快速小技巧	62

第六章 数学 63

6.1	GCD / LCM	63
6.2	快速幂 (取模)	63
6.3	逆元 (费马小定理)	64
6.4	质数筛	64
6.4.1	埃氏筛	64
6.4.2	线性筛 (欧拉筛)	64

6.5	组合数（预处理阶乘逆元）	65
6.6	高斯消元	65
6.7	模运算与快速幂	66
6.8	最大公约数、扩展欧几里得与同余	66
6.9	质数与筛法	67
6.9.1	试除与质因数分解	67
6.9.2	埃氏筛与线性筛	67
6.9.3	约数枚举与整除分块	67
6.10	欧拉函数、莫比乌斯与积性函数	68
6.11	组合数与阶乘（Binomial Coefficients）	68
6.11.1	5.1 质数模且 n 不大	68
6.11.2	卢卡斯定理（Lucas Theorem）	69
6.11.3	非质数模	69
6.12	矩阵快速幂与线性递推（Matrix Exponentiation and Linear Recurrence）	70
6.13	容斥与组合计数	70
6.14	概率与期望	71
6.15	常见数列与技巧	71
6.16	生成函数基础	72
6.17	Burnside 引理与 Pólya 计数定理（Burnside's Lemma and Pólya Enumeration Theorem）	72
6.18	Prüfer 序列与矩阵树定理（Prüfer Sequence and Matrix-Tree Theorem）	73
6.19	二次剩余与 Tonelli - Shanks 算法（Quadratic Residues and Tonelli - Shanks Algorithm）	74
6.20	置换与循环	75
6.21	快速傅里叶变换与数论变换（了解）	75
6.22	高斯消元	76
6.23	数学与组合计数：参数、调用与改造	77
6.23.1	3.1 高级容斥与倍数容斥	77
6.23.2	3.2 卢卡斯定理（Lucas Theorem）与组合数模质数	78
6.23.3	3.3 中国剩余定理与扩展中国剩余定理	79
6.23.4	3.4 高斯消元、秩、解空间与行列式	81

6.23.5	3.5 莫比乌斯反演	82
6.23.6	3.6 原根、Baby-step Giant-step 离散对数与扩展算法	84
6.23.7	3.7 生成函数、数论变换、Burnside 引理与 Pólya 计数定理	85
6.23.8	3.8 二次剩余	87
6.23.9	3.9 Prüfer 序列与矩阵树	89
6.23.10	7.1 矩阵快速幂与线性递推	89
6.23.11	7.2 高斯消元求实数方程组	90
第七章	数据结构	93
7.1	单调栈	93
7.2	单调队列	93
7.3	并查集	94
7.4	字典树 (Trie)	94
7.4.1	异或字典树 (01-Trie)	95
7.5	哈夫曼树 (Huffman Tree)	96
7.6	树状数组 (BIT)	98
7.7	线段树	99
第八章	数据结构	101
8.1	STL 必会组件	101
8.2	并查集 (Disjoint Set Union, DSU)	101
8.3	树状数组 (Fenwick Tree)	102
8.4	线段树	102
8.4.1	常见维护	102
8.4.2	线段树上的二分	103
8.5	稀疏表 (Sparse Table, ST)	104
8.6	堆与可并堆思想	104
8.7	单调结构与差分约束	104
8.8	字典树 (Trie, Prefix Tree)	105
8.9	可持久化数据结构	106
8.10	分块与莫队算法 (Mo's Algorithm)	106
8.10.1	块状数组	106

8.10.2 莫队	107
8.11 平衡树思想（了解 Treap）	107
8.12 撤销与可回滚结构	108
8.13 李超线段树（Li Chao Segment Tree）	108
8.14 区间最值势能线段树（Segment Tree Beats）	109
8.15 高级数据结构：参数、调用与改造	111
8.15.1 2.1 高阶前缀和与多阶差分	111
8.15.2 2.2 多懒标记与仿射标记线段树	112
8.15.3 2.3 线段树上二分	114
8.15.4 2.4 莫队、带修改莫队与回滚莫队	114
8.15.5 2.5 三维偏序的 CDQ 分治与动态规划优化	116
8.15.6 2.6 可持久化字典树（Persistent Trie）	118
8.15.7 2.10 李超线段树：整条直线与线段插入	120
8.15.8 2.11 区间最值势能线段树（Segment Tree Beats）	122
8.15.9 7.3 可持久化并查集	125
第九章 字符串	127
9.1 KMP	127
第十章 字符串算法	128
10.1 选择算法	128
10.2 Knuth–Morris–Pratt 字符串匹配算法（KMP）	128
10.3 Z 函数	129
10.4 字符串哈希	129
10.5 Manacher	129
10.6 字典树与 Aho–Corasick 多模式匹配自动机（AC 自动机）	130
10.7 后缀数组与最长公共前缀（Suffix Array and Longest Common Prefix, LCP）	131
10.8 后缀自动机（Suffix Automaton, SAM）	132
10.9 回文自动机（Palindromic Automaton, PAM, 了解）	133
10.10 字符串常见组合	134
10.11 字符串高级算法：参数、调用与改造	134
10.11.1 4.1 Booth 最小表示法	134

10.11.2 4.2 回文自动机 (Palindromic Automaton)	136
10.11.3 4.3 后缀数组、最长公共前缀、后缀自动机与 Aho–Corasick 失配树 . .	137
10.11.4 7.4 后缀数组与 LCP	139
10.11.5 7.5 AC 自动机与 fail 树	141

第十一章 图论 143

11.1 最短路	143
11.1.1 Dijkstra (堆优化)	143
11.1.2 SPFA (Bellman-Ford 队列优化)	144
11.2 拓扑排序	145
11.3 多源 BFS	145
11.4 BFS 求最小环 (无权图)	146
11.5 最小生成树 (Kruskal)	147
11.6 欧拉路 (Hierholzer)	148
11.7 最大流 (Dinic)	148

第十二章 图论 151

12.1 图的表示与基础遍历	151
12.2 有向无环图 (Directed Acyclic Graph, DAG) 与拓扑排序	151
12.3 最短路	152
12.4 最小生成树	153
12.4.1 Kruskal	153
12.4.2 Prim	153
12.5 强连通分量 (Strongly Connected Components, SCC) 与缩点	153
12.6 割点、桥、双连通	154
12.7 二分图	154
12.7.1 染色判定	154
12.7.2 最大匹配	155
12.7.3 带权匹配	155
12.8 网络流	155
12.8.1 建模	155
12.8.2 Dinic	156

12.8.3 费用流	156
12.9 树上算法	157
12.9.1 DFS 序与子树	158
12.9.2 最近公共祖先 (Lowest Common Ancestor, LCA) 倍增	158
12.9.3 树上差分	158
12.9.4 直径与重心	159
12.9.5 树形 DP	159
12.9.6 重链剖分 (Heavy-Light Decomposition, HLD)	159
12.9.7 点分治 (入门)	160
12.9.8 树上启发式合并 (Disjoint Set Union on Tree, DSU on Tree)	161
12.9.9 长链剖分	162
12.9.10 Link-Cut Tree 动态树 (LCT)	162
12.10 欧拉路、欧拉回路 & 哈密顿回路	164
12.11 图论组合套路	165

第十三章 树 166

13.1 树上 BFS / DFS	166
13.2 LCA (倍增)	167
13.3 树上算法	168
13.3.1 DFS 序与子树	168
13.3.2 最近公共祖先 (Lowest Common Ancestor, LCA) 倍增	168
13.3.3 树上差分	169
13.3.4 直径与重心	169
13.3.5 树形动态规划 (Tree Dynamic Programming)	169
13.3.6 重链剖分 (Heavy-Light Decomposition, HLD)	170
13.3.7 点分治 (入门)	170
13.3.8 树上启发式合并 (Disjoint Set Union on Tree, DSU on Tree)	171
13.3.9 长链剖分	172
13.3.10 Link-Cut Tree 动态树 (LCT)	172
13.4 点分治	173
13.5 虚树	174

13.6 二进制提升的更多用法	174
13.7 树上高级算法：参数、调用与改造	175
13.7.1 2.7 树上启发式合并 (Disjoint Set Union on Tree)	175
13.7.2 2.8 长链剖分	177
13.7.3 2.9 点分治与点分树	178
13.7.4 2.12 伸展树、文艺平衡树与 Link-Cut Tree 动态树	179

第十四章 动态规划 182

14.1 01 背包	182
14.2 完全背包	182
14.3 多重背包 (二进制拆分)	183
14.4 分组背包	183
14.5 最长上升子序列 (LIS)	184
14.6 最长公共子序列 (LCS)	184
14.6.1 还原一组 LCS	184
14.7 区间 DP	185
14.8 树上 DP (树形 DP)	186
14.8.1 树上最大权独立集 (没有上司的舞会)	186
14.8.2 树形背包 (有依赖的背包 / 选课)	187
14.8.3 换根 DP (二次扫描)	187
14.8.4 树的直径 (树形 DP 求法)	188
14.9 DP 四问法	188
14.10 线性 DP	189
14.11 背包	189
14.12 区间 DP	190
14.13 树形 DP 与换根 DP	190
14.14 有向无环图动态规划与路径计数 (Directed Acyclic Graph Dynamic Programming)	191
14.15 状态压缩动态规划	191
14.16 数位动态规划 (Digit Dynamic Programming)	191
14.17 概率动态规划与期望动态规划 (Probability and Expected-Value Dynamic Programming)	192

14.18 记忆化搜索	192
14.19 动态规划优化 (Dynamic Programming Optimization)	192
14.19.1 滚动数组	192
14.19.2 单调队列优化	193
14.19.3 斜率优化 (Convex Hull Trick, CHT)	193
14.19.4 分治优化	193
14.20 组合游戏动态规划 (Combinatorial Game Dynamic Programming)	194
14.21 常见 DP 误区	194

第十五章 博弈专题 195

15.1 Nim、Bash 与反常 Nim 取石子游戏	195
15.2 通用 Sprague-Grundy 函数、最小未出现值与有向无环图博弈	196
15.3 Nim 变体：取法集合与 Sprague-Grundy 周期	197
15.4 极大极小搜索 (Minimax) 与 Alpha-Beta 剪枝	197
15.5 博弈题检查清单	198
15.6 动态规划进阶：参数、调用与改造	198
15.6.1 6.1 复杂数位动态规划与自动机数位动态规划	198
15.6.2 6.2 所有子集和动态规划、轮廓线动态规划与状态压缩	199
15.6.3 6.3 单调队列、斜率优化与李超树优化动态规划	200
15.6.4 6.4 分治优化、四边形不等式、树形背包与 Steiner 树状压动态规划	202
15.6.5 2.13 动态动态规划与矩阵线段树	203
15.6.6 7.8 概率动态规划的自环方程	204
15.7 博弈论：参数、调用与改造	205
15.7.1 1.1 必胜态/必败态动态规划 (Win/Lose Dynamic Programming)	205
15.7.2 1.2 有向无环图上的 Sprague-Grundy 函数与最小未出现值	206
15.7.3 1.3 多个独立子游戏为什么异或	208
15.7.4 1.4 普通 Nim：不只判断，还要恢复必胜操作	208
15.7.5 1.5 Bash、反常 Nim 与阶梯 Nim	209
15.7.6 1.6 Sprague-Grundy 函数打表、周期与大范围取石子	211
15.7.7 1.7 图游戏建模、状压博弈和记忆化	213
15.7.8 1.8 极大极小搜索 (Minimax) 与 Alpha-Beta 剪枝	214

15.7.9 1.9 状态去重、哈希与有环博弈	215
-----------------------------------	-----

第十六章 计算几何	217
------------------	------------

16.1 基础	217
16.2 三分法求单峰极值（几何应用）	217

第十七章 计算几何与计算技巧	218
-----------------------	------------

17.1 浮点规范	218
17.2 向量与叉积	218
17.3 直线、线段与交点	218
17.3.1 圆几何	219
17.4 多边形	220
17.5 凸包（Andrew）	220
17.6 旋转卡壳	221
17.7 扫描线	221
17.8 最近点对	221
17.9 半平面交（Half-Plane Intersection, HPI, 了解）	222
17.10 随机化与哈希技巧	223
17.11 位集优化	224
17.12 快速小技巧	224
17.13 计算几何高级算法：参数、调用与改造	224
17.13.1 5.1 极角排序与凸包	224
17.13.2 5.2 旋转卡壳、凸多边形内点与最近点对	225
17.13.3 5.3 半平面交（Half-Plane Intersection）	226
17.13.4 5.4 圆几何与扫描线	227
17.13.5 7.6 圆与直线、最小圆覆盖	228
17.13.6 7.7 扫描线求矩形并面积	230
17.14 超过十行模板的针对性自测	232
17.14.1 基础、枚举、差分与二分	232
17.14.2 数学、筛法、生成函数与树计数	233
17.14.3 数据结构：下标、懒标记、版本与回滚	235
17.14.4 图论：路径、连通性、流和树上边界	237

17.14.5 动态规划和字符串	240
17.14.6 进阶专题：整体二分、时间分治、可满足性与匹配	241

第十八章 对拍 243

18.1 对拍脚本	243
18.1.1 Linux / macOS (Bash)	243
18.1.2 Windows (BAT)	244
18.2 随机数据生成器 (gen.cpp)	245
18.3 暴力解法 (brute.cpp)	247
18.4 交互题对拍	248

第十九章 竞赛环境速查 250

19.1 Windows 命令行 (CMD / PowerShell)	250
19.1.1 编译与运行	250
19.1.2 文件操作	251
19.1.3 进程与调试	251
19.1.4 常用技巧	251
19.2 VS Code 快捷键速查	252
19.2.1 文件与编辑	252
19.2.2 查找与替换	253
19.2.3 多光标与选择	253
19.2.4 终端	253
19.2.5 调试	253
19.2.6 其他实用操作	254
19.3 VS Code 竞赛配置	254
19.3.1 一键编译运行 (tasks.json)	254
19.3.2 调试配置 (launch.json)	255
19.3.3 推荐插件	255
19.3.4 推荐用户设置 (settings.json 片段)	255

第二十章 11. 题型/关键词 → 算法索引 257

20.1 一、博弈论	257
----------------------	-----

20.2 二、高级数据结构 258

20.3 三、数学与组合计数 260

20.4 四、字符串算法 262

20.5 五、计算几何 264

20.6 六、动态规划进阶 265

20.7 七、按题面快速查找 267

20.8 八、常见算法组合 268

1 总览、复杂度与学习路线

1.1 先判断题目属于哪一类

1. 静态数组/区间：前缀和、差分、排序、双指针、二分、树状数组、线段树。
2. 图/网格可达性：BFS、DFS、连通分量、拓扑、最短路、并查集。
3. 选择最优方案：贪心（交换论证/单调性）或 DP（状态有重叠）。
4. 计数/取模：组合数、容斥、DP、生成式递推、矩阵快速幂。
5. 字符串模式：KMP/Z/哈希；多模式匹配用 AC，回文用 Manacher。
6. 树上问题：先想 DFS 序、子树、倍增、LCA、树形 DP，再考虑重链/点分治。
7. 双方对抗/取石子：先判无偏性与终止规则，再选 Win/Lose、SG 或 Minimax。

1.2 复杂度速查（单次操作约 1 秒时的经验值）

规模	通常可接受	典型算法
$n \leq 20$	$O(2^n n)$	状压、子集枚举、折半搜索
$n \leq 40$	$O(2^{(n/2)})$	Meet-in-the-middle
$n \leq 200$	$O(n^3)$	Floyd、区间 DP、矩阵转移
$n \leq 2000$	$O(n^2)$	二维 DP、朴素 LIS、部分图算法
$n \leq 2e5$	$O(n \log n)$	排序、堆、树状数组、线段树、Dijkstra
$n \leq 1e6$	$O(n)$ 或 $O(n \log \log n)$	线性扫描、线性筛、哈希
$n \leq 1e18$	$O(\log n)$ / $O(\log^2 n)$	快速幂、矩阵快速幂、数论分块

若题目有 T 组数据，先把所有规模相加；不要只看单组上限。

1.3 分层路线

1.3.1 L0：实现基本功

C++ 输入输出、引用/指针、`vector/string/pair/tuple`、排序比较器、`lambda`、`lower_bound`、位运算、递归栈、`long long`、模运算规范。

1.3.2 L1: 线性与排序

前缀和/差分 → 双指针/滑动窗口 → 二分查找/二分答案 → 离散化 → 排序贪心 → 单调栈/单调队列。

1.3.3 L2: 基础图和数学

BFS/DFS/连通分量 → 拓扑 → 并查集 → Dijkstra/MST → 模运算、筛法、组合数、逆元。

1.3.4 L3: 基础 DP 和经典数据结构

线性 DP、背包、区间 DP、树形 DP → 树状数组、线段树、ST 表、堆、Trie。

1.3.5 L4: 字符串、树和提升题

KMP/Z/哈希/Manacher → LCA/树上差分/直径 → SCC/割点桥/二分图匹配 → 0-1 BFS、差分约束、网络流。

1.3.6 L5: 银牌常见进阶

CDQ/整体二分、点分治、虚树、LCT/DSU on Tree、凸包与旋转卡壳、后缀数组/SAM/PAM、矩阵快速幂、状压与数位 DP 优化、基础博弈。

1.4 赛场决策树

先读约束

- ├ n 很小 ($\leq 20/40$) ? → 子集枚举 / 折半搜索 / 状压 DP
- ├ 需要连续区间? → 前缀和 / 差分 / 双指针 / 单调队列
- ├ “最小的最大值/最大的最小值”? → 二分答案 + 可行性检查
- ├ 图上可达/层数? → BFS; 有权最短路? → 0-1 BFS / Dijkstra
- ├ 边权全为 1 且有状态? → 分层 BFS / 乘积图
- ├ 选取若干对象最优? → 先尝试贪心交换论证, 再写 DP
- ├ 多次区间修改/查询? → 差分 / 树状数组 / 线段树
- ├ 树上路径/子树? → DFS 序、LCA、树上差分、重链或点分治
- ├ 字符串匹配/回文/多模式? → KMP/Z、Manacher、AC 自动机

1.5 正确性证明的四种常用模板

- **不变量**: 循环/数据结构每一步都维持同一个性质, 如 Dijkstra 已确定点的最短路不再改变。

- **交换论证**：把任意最优解改造成贪心解而不变差，如按结束时间排序的区间调度。
- **归纳/状态转移**：证明 DP 状态覆盖所有合法前缀，转移恰好枚举最后一步。
- **势能/单调性**：证明每次操作让某个量单调变化，从而保证双指针、并查集、单调栈的线性复杂度。

1.6 提交前检查

- 是否把 `int` 乘法提升成 `1LL * a * b`？
- 模数不是质数时是否误用了逆元？
- `vector` 是否提前 `reserve`，递归深度是否可能爆栈？
- 图是否 0/1 下标一致，是否忘记反向边或重边？
- 二分边界是否满足“左闭右闭/左闭右开”约定？
- 多测是否清空邻接表、全局答案、标记数组、缓存？
- 几何是否统一使用 `long double` 与 `eps`，叉积是否溢出？

2 竞赛模板

- `#define int long long` 之后 `main` 必须写 `signed main`, 否则编译错误。
- 位运算中常数 1 默认是 32 位, 当 $i \geq 31$ 时 `1 << i` 溢出变负数, 必须写 `1LL << i`。
- `inf = 2e18` 做加法时可能溢出 (如 `inf + x` 爆 long long), 松弛前先判 `if(dist[u] != inf)`。

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'

const int inf = 2e18;
const int N = 1e6 + 10;

void solve() {
    int n;
    cin >> n;
}

signed main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int t = 1;
    cin >> t;
    while (t--) {
        solve();
    }
    return 0;
}
```

- 不需要多测时把 `cin >> t` 注释掉即可。
- 多测时注意：solve() 里不要中途 `return`（会导致后续输入串流）；全局数组/容器必须清空干净。
- 局部变量必须初始化！`int sum`；在 OJ 上可能是乱码，必须写 `int sum = 0`；。

2.1 C++ 竞赛实现规范

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;
using u64 = unsigned long long;
using i128 = __int128_t;
constexpr int INF = 0x3f3f3f3f;
constexpr i64 LINF = (1LL << 62);

void print_i128(i128 x) { // 需要输出超出 long long 时使用
    if (x < 0) { cout << '-'; x = -x; }
    string s;
    do { s.push_back('0' + x % 10); x /= 10; } while (x);
    reverse(s.begin(), s.end()); cout << s;
}
```

- `ios::sync_with_stdio(false)`；`cin.tie(nullptr)`；放在 main 开头。
- 需要修改容器元素时使用引用：`for (auto &x : a)`。
- 大对象传参用 `const vector<int>&`；避免递归中反复复制。
- `sort` 比较器必须是严格弱序，不能写 `<=`。
- 计数、乘积、距离默认 `long long`；乘法可能达到 `1e18` 时显式 `1LL` 或 `__int128`。

3 STL 常用函数速查

3.1 容器

3.1.1 vector

```
vector<int> a(n, 0);           // n 个元素初始化为 0
a.push_back(x);               // 尾部插入
a.pop_back();                 // 尾部删除
a.size();                     // 元素个数（返回 size_t，比较时注意）
a.empty();                    // 是否为空
a.clear();                    // 清空
a.resize(m, val);              // 重设大小
a.erase(a.begin() + i);       // 删除第 i 个元素 O(n)
// 去重：
sort(a.begin(), a.end());
a.erase(unique(a.begin(), a.end()), a.end());
```

3.1.2 array

```
array<int, N> a;                // 固定大小数组，N 必须是编译期常量
array<int, 5> a = {1, 2, 3, 4, 5}; // 初始化
array<int, 5> a = {};           // 全部初始化为 0
a.size();                       // 元素个数（编译期确定，始终为 N）
a.fill(val);                    // 全部填充为 val
a.front(); a.back();            // 首尾元素
a.data();                       // 底层数组指针
a[i];                           // 随机访问 O(1)
// 支持比较运算符（按字典序），可以直接当 map 的 key
// 与 C 数组的区别：可以赋值、比较、作为函数返回值
```

- array 不能用变量做大小，array<int, n> 编译错误（n 必须是 constexpr）。
- 局部 array 不会自动初始化为 0！必须写 array<int, 5> a = {}; 或 a.fill(0);。

- 竞赛中常用场景：需要把固定长度的数组当 map / set 的 key 时，用 array 比手写 struct 方便。

3.1.3 string

```
string s;
getline(cin, s);           // 读一整行（注意前面 cin>>n 后要 cin.ignore()）
s.substr(pos, len);        // 从 pos 开始取 len 个字符
s.find("abc");             // 查找子串，找不到返回 string::npos
s += "xyz";               // 拼接
to_string(123);           // int -> string
stoi("123");              // string -> int
stoll("123456789012");    // string -> long long
```

3.1.4 set / multiset

```
set<int> s;
s.insert(x);              // 插入
s.erase(x);              // 删除所有值为 x 的元素
s.count(x);               // 0 或 1
s.lower_bound(x);         // >= x 的第一个迭代器
s.upper_bound(x);         // > x 的第一个迭代器
// multiset 删除单个元素：
auto it = ms.find(x);
if (it != ms.end()) ms.erase(it); // 只删一个！
// 如果写 ms.erase(x) 会删掉所有等于 x 的元素！
```

3.1.5 map

```
map<int, int> mp;
mp[key] = val;            // 插入/修改
mp.count(key);            // 是否存在
mp.erase(key);           // 删除
// 遍历：
for (auto& [k, v] : mp) { ... }
// 注意：mp[key] 如果 key 不存在会自动插入默认值 0！
// 只查询用 mp.count(key) 或 mp.find(key)
```

3.1.6 unordered_map / unordered_set

```

unordered_map<int, int> mp; // 哈希表, 平均  $O(1)$  查找/插入/删除
mp[key] = val;           // 插入/修改 (同 map)
mp.count(key);           // 是否存在
mp.erase(key);           // 删除
for (auto& [k, v] : mp) { ... } // 遍历 (无序!)

unordered_set<int> us;
us.insert(x);
us.count(x);              // 0 或 1
us.erase(x);

// 与 map/set 的区别:
// 1. 无序, 不支持 lower_bound / upper_bound
// 2. 平均  $O(1)$ , 但最坏  $O(n)$  (哈希冲突)
// 3. 不能直接用 pair/vector 做 key (需要自定义哈希)

```

- 被卡哈希: Codeforces 等平台可以构造数据卡 unordered_map 的默认哈希, 导致退化为 $O(n^2)$ 。
- 防卡方法: 自定义哈希函数加随机种子:

```

struct custom_hash {
    size_t operator()(int x) const {
        static const size_t FIXED_RANDOM =
            chrono::steady_clock::now().time_since_epoch().count();
        x ^= FIXED_RANDOM;
        return x ^ (x >> 16);
    }
};

unordered_map<int, int, custom_hash> mp;

```

- 用 mp.reserve(n) 预分配空间、mp.max_load_factor(0.25) 降低负载因子, 可以显著提速。
- 需要有序遍历或 lower_bound 时, 必须用 map / set。
- unordered_map 的 [] 操作同样会自动插入默认值, 注意与 map 一样的坑。

3.1.7 priority_queue

```

// 大根堆 (默认)
priority_queue<int> pq;

```

```
// 小根堆
priority_queue<int, vector<int>, greater<int>> pq;
pq.push(x);
pq.top();    // 堆顶（不弹出）
pq.pop();    // 弹出堆顶
```

3.1.8 deque

```
deque<int> dq;
dq.push_front(x); dq.push_back(x);
dq.pop_front();   dq.pop_back();
dq.front();       dq.back();
dq[i];            // 支持随机访问
```

3.1.9 stack

```
stack<int> stk;
stk.push(x);      // 压栈
stk.pop();        // 弹栈（无返回值！）
stk.top();        // 取栈顶元素
stk.empty();      // 是否为空
stk.size();       // 元素个数
// 注意：stack 没有 clear()，清空方式：
// while (!stk.empty()) stk.pop();
// 或直接 stk = stack<int>();
```

3.1.10 list

```
list<int> lst;
lst.push_back(x); lst.push_front(x); // 头尾插入 O(1)
lst.pop_back();   lst.pop_front();   // 头尾删除 O(1)
lst.front();      lst.back();        // 访问头尾
lst.insert(it, x); // 在迭代器 it 前插入 O(1)
lst.erase(it);    // 删除迭代器指向的元素 O(1)
lst.size();       lst.empty();
lst.remove(val);   // 删除所有值为 val 的元素
lst.sort();        // 链表专用排序（归并）O(n log n)
lst.unique();      // 去除相邻重复元素（需先排序）
lst.merge(lst2);   // 合并两个有序链表
lst.reverse();     // 反转链表
lst.splice(it, lst2); // 将 lst2 全部移接到 it 前面 O(1)
// 注意：list 不支持随机访问（没有 []），遍历只能用迭代器
```



```
// 适用场景：频繁在中间插入/删除，且不需要随机访问
```

3.2 常用算法

```
// 排序
sort(a.begin(), a.end());           // 升序
sort(a.begin(), a.end(), greater<int>()); // 降序

// 二分查找（要求有序）
lower_bound(a.begin(), a.end(), x); // >= x 的第一个位置
upper_bound(a.begin(), a.end(), x); // > x 的第一个位置

// 去重
unique(a.begin(), a.end()); // 返回去重后的尾迭代器

// 全排列
next_permutation(a.begin(), a.end()); // 下一个排列，到最后返回 false

// 最值
*max_element(a.begin(), a.end());
*min_element(a.begin(), a.end());

// 累加
accumulate(a.begin(), a.end(), 0LL); // 注意初始值类型！
```

- 自定义 sort 的比较函数必须用严格小于 <，不能用 <=，否则破坏严格弱序，导致随机 RE/WA。
- accumulate 初始值写 0 会按 int 累加溢出，必须写 0LL。
- vector.size() 返回无符号数，a.size() - 1 当 size 为 0 时会下溢变成极大值！

3.3 pair / tuple

```
// pair
pair<int, int> p = {1, 2};
auto [x, y] = p; // C++17 结构化绑定
p.first; p.second; // 访问元素
// pair 支持比较运算符（先比 first，再比 second）
// 可以直接作为 map 的 key、放入 set、sort
```

```
// make_pair
auto p2 = make_pair(3, 4); // 自动推导类型

// tuple (三元及以上)
tuple<int, int, int> t = {1, 2, 3};
auto [a, b, c] = t; // C++17 结构化绑定
get<0>(t); get<1>(t); get<2>(t); // 访问元素
auto t2 = make_tuple(1, 2, 3);
// tuple 也支持比较运算符 (按字典序)
// 竞赛中三元排序直接用 tuple 最方便
```

- 多关键字排序用 `vector<tuple<int,int,int>>` 比写结构体更快。
- `tie(a, b, c) = make_tuple(x, y, z)` 可以一行赋值多个变量。
- C++17 的结构化绑定 `auto [a, b, c] = t` 最简洁，推荐使用。

3.4 bitset

`bitset<N>` 是固定大小的位集合，`N` 必须是编译期常量。空间 $N/8$ 字节，位运算操作是 $O(N/64)$ (64 位压缩)。

```
bitset<1000> bs; // 全部初始化为 0
bitset<8> bs2("10101010"); // 从字符串初始化 (注意: 字符串最右对应第 0 位)
bitset<8> bs3(0xAA); // 从整数初始化

// 单位操作
bs.set(i); // 第 i 位置 1
bs.set(i, 0); // 第 i 位置 0
bs.reset(i); // 第 i 位置 0
bs.flip(i); // 第 i 位取反
bs.test(i); // 返回第 i 位的值 (bool)
bs[i]; // 同 test(i), 但不做越界检查

// 全局操作
bs.set(); // 全部置 1
bs.reset(); // 全部置 0
bs.flip(); // 全部取反
bs.count(); // 1 的个数 (popcount)
bs.size(); // 总位数 N
bs.any(); // 是否存在至少一个 1
bs.none(); // 是否全为 0
```

```

bs.all();           // 是否全为 1 (C++11)

// 位运算 (O(N/64), 比手写快很多)
bitset<1000> a, b;
a & b; a | b; a ^ b; ~a; // 返回新 bitset
a &= b; a |= b; a ^= b; // 原地修改
a <<= k; a >>= k;       // 左移/右移 k 位
(a & b).count();        // 两个集合的交集大小

// 转换
bs.to_ulong(); // 转为 unsigned long (N <= 32 时)
bs.to_ullong(); // 转为 unsigned long long (N <= 64 时)
bs.to_string(); // 转为 "010101..." 字符串

// 查找 (非标准但 GCC 支持)
bs._Find_first(); // 第一个 1 的位置 (没有则返回 N)
bs._Find_next(i); // i 之后第一个 1 的位置 (没有则返回 N)
// 枚举所有 1 的位置:
for (int i = bs._Find_first(); i < (int)bs.size(); i = bs._Find_next(i)) {
    // 处理位置 i
}

```

- bitset 的大小 N 必须是编译期常量，不能用变量！
- bs[i] 的 i 从 0 开始，第 0 位是最低位。
- 从字符串构造时，字符串的最后一个字符对应第 0 位（最低位），容易搞反。
- _Find_first 和 _Find_next 是 GCC 扩展，非标准，但竞赛中可用。

- **bitset 优化 DP:** 将布尔 DP 数组改用 bitset，转移用位移 + 或运算，复杂度除以 64。
经典例题：背包可行性判断 $bs = (bs \ll w[i])|$ 。
- **bitset 求交集/汉明距离:** $(a \oplus b).count()$ 即汉明距离， $(a \& b).count()$ 即共同元素个数。
- **bitset 优化图算法:** 传递闭包 $O(n^3/64)$ ，二部图匹配加速等。
- **bitset 加速暴力:** $n \leq 40000$ 的 $O(n^2)$ 布尔运算用 bitset 可以压到 $O(n^2/64)$ ，约等于 $n \leq 5000$ 的常规暴力速度。

3.4.1 bitset 优化背包 / 子集和可行性

判断若干物品能凑出哪些重量（布尔 DP），把 `dp[j]` 改为 bitset 的第 j 位，转移用左移 + 或，复杂度 $O(nV/64)$ 。

```
// bs[j] = 1 表示重量 j 可以被凑出
bitset<MAXV> bs;
bs[0] = 1; // 重量 0 总能凑出（什么都不选）
for (int i = 0; i < n; i++)
    bs |= bs << w[i]; // 01 背包：每个物品做一次整体左移再或
// bs[j] 为 1 即容量 j 可达；bs.count() 为可达重量种类数

// 完全背包可行性：物品可重复选，按位逐位递推
// for (int i = 0; i < n; i++)
//     for (int j = w[i]; j < MAXV; j++) if (bs[j-w[i]]) bs[j] = 1;
```

- MAXV 必须 $\geq$ 最大可能重量 +1，且是编译期常量。
- `bs = bs << w[i]` 一次只把每个物品计入一次，是 01 背包语义；完全背包需另写逐位递推。

3.4.2 bitset 优化传递闭包（Floyd）

求有向图可达性矩阵， $O(n^3)$ 的 Floyd 用 bitset 整行或运算压到 $O(n^3/64)$ 。

```
bitset<MAXN> reach[MAXN]; // reach[i][j]=1 表示 i 能到达 j
// 初始化：reach[i][i]=1, 有边 i->j 则 reach[i][j]=1
for (int k = 0; k < n; k++)
    for (int i = 0; i < n; i++)
        if (reach[i][k])
            reach[i] |= reach[k]; // i 能到 k, 则 k 能到的 i 都能到
```

3.4.3 bitset 优化字符串匹配（Shift-And）

在文本 t 中查找模式串 p （长 m ），用 bitset 维护“当前已匹配前缀”集合， $O(nm/64)$ 。

```
// B[c] 第 j 位为 1 表示 p[j]==c
bitset<MAXM> B[256], D;
for (int j = 0; j < m; j++) B[(unsigned char)p[j]][j] = 1;
for (int i = 0; i < (int)t.size(); i++) {
    D = ((D << 1) | bitset<MAXM>(1)) & B[(unsigned char)t[i]];
    if (D[m - 1]) {
```

```

        // 在文本位置 i - m + 1 处匹配成功
    }
}

```

Shift-And 的状态 $D[j]=1$ 表示 $p[0..j]$ 是 t 当前结尾的后缀。把 $|$ 换成“或非”、判 $D[m-1]==0$ 即为 Shift-Or 变体，原理相同。

3.5 `__int128`

GCC 扩展的 128 位整数，范围约 $[-1.7 \times 10^{38}, 1.7 \times 10^{38}]$ 。用于中间计算结果超过 `long long` 但最终结果不超过的情况。

```

__int128 a = 1;
// 不能直接 cin/cout, 需要手写 IO
void print128(__int128 x) {
    if (x < 0) { putchar('-'); x = -x; }
    if (x > 9) print128(x / 10);
    putchar(x % 10 + '0');
}

__int128 read128() {
    __int128 x = 0; int f = 1; char c = getchar();
    while (c < '0' || c > '9') { if (c == '-') f = -1; c = getchar(); }
    while (c >= '0' && c <= '9') { x = x * 10 + c - '0'; c = getchar(); }
    return x * f;
}

// 典型应用：大数乘法取模，避免 long long 乘法溢出
int mul_mod(int a, int b, int mod) {
    return (__int128)a * b % mod;
}

```

- `__int128` 不能直接用 `cin/cout/printf`，必须手写 IO。
- `__int128` 是 GCC 扩展，MSVC 不支持。竞赛 OJ 一般都支持。
- 不能用于 `bitset` 的下标或 `vector` 的大小。

3.6 `__builtin` 系列

```
// 二进制中 1 的个数
__builtin_popcount(x);    // unsigned int 版
__builtin_popcountll(x);  // unsigned long long 版

// 末尾 0 的个数（即最低位 1 的位置）
__builtin_ctz(x);         // x != 0 时才有意义
__builtin_ctzll(x);

// 前导 0 的个数
__builtin_clz(x);         // x != 0 时才有意义
__builtin_clzll(x);

// 奇偶性（1 的个数是否为奇数）
__builtin_parity(x);      // 奇数个 1 返回 1
__builtin_parityll(x);

// 常用推导
// 最高位 1 的位置 = 63 - __builtin_clzll(x)    (x > 0)
// 最低位 1 的位置 = __builtin_ctzll(x)        (x > 0)
// 向上取 2 的幂 = 1LL << (64 - __builtin_clzll(x - 1)) (x > 1)
// 向下取 2 的幂 = 1LL << (63 - __builtin_clzll(x))    (x > 0)
// log2(x) 下取整 = 63 - __builtin_clzll(x)    (x > 0)
```

- `__builtin_clz(0)` 和 `__builtin_ctz(0)` 是未定义行为！使用前必须判断 $x \neq 0$ 。
- `int` 类型用不带 `ll` 的版本，`long long` 类型必须用 `ll` 版本，否则结果错误。

3.7 C++ 常用函数速查

3.7.1 数值函数

```
// <algorithm> 头文件
min(a, b); max(a, b);    // 最小值/最大值
min({a, b, c, d});       // 多个值取最小 (C++11, 初始化列表)
max({a, b, c, d});

swap(a, b);              // 交换两个值
clamp(x, lo, hi);        // 限制 x 在 [lo, hi] 范围内 (C++17)

// <numeric> 头文件
accumulate(a.begin(), a.end(), 0LL);    // 累加（注意初始值类型！）
accumulate(a.begin(), a.end(), 1LL, multiplies<>()); // 累乘
iota(a.begin(), a.end(), 0);             // 填充 0, 1, 2, ...
```

```

partial_sum(a.begin(), a.end(), pre.begin()); // 前缀和
adjacent_difference(a.begin(), a.end(), d.begin()); // 差分
inner_product(a.begin(), a.end(), b.begin(), 0LL); // 内积
gcd(a, b); lcm(a, b); // 最大公约数/最小公倍数 (C++17)
midpoint(a, b); // (a+b)/2 不溢出 (C++20)
reduce(a.begin(), a.end()); // 并行累加 (C++17, 无序)

// <cmath> 头文件
abs(x); // 绝对值 (整数用 abs, 浮点也可以)
ceil(x); floor(x); round(x); // 上取整/下取整/四舍五入
sqrt(x); cbrt(x); // 平方根/立方根
log(x); log2(x); log10(x); // 自然对数/以2为底/以10为底
pow(x, y); // x 的 y 次方 (浮点, 整数慎用!)
fmod(x, y); // 浮点取模
hypot(x, y); // sqrt(x*x + y*y), 不溢出

```

- `pow(x, y)` 返回浮点数, `pow(5, 2)` 可能返回 24.999..., 赋给 `int` 变成 24! 整数幂次用快速幂。
- 整数除法上取整: $\lceil a/b \rceil = (a + b - 1) / b$ ($a, b > 0$ 时)。不要用 `ceil((double)a / b)`, 浮点不精确。
- `abs(INT_MIN)` 溢出! `INT_MIN` 的绝对值超过 `INT_MAX`。

3.7.2 区间操作函数

```

// 排序
sort(a.begin(), a.end()); // 升序
sort(a.begin(), a.end(), greater<int>()); // 降序
stable_sort(a.begin(), a.end()); // 稳定排序
partial_sort(a.begin(), a.begin()+k, a.end()); // 只排前 k 小
nth_element(a.begin(), a.begin()+k, a.end()); // 第 k 小放到位置 k, O(n)

// 二分查找 (要求有序)
lower_bound(a.begin(), a.end(), x); // >= x 的第一个位置
upper_bound(a.begin(), a.end(), x); // > x 的第一个位置
binary_search(a.begin(), a.end(), x); // 是否存在 x (bool)
equal_range(a.begin(), a.end(), x); // 返回 {lower_bound, upper_bound} 的 pair

// 查找
find(a.begin(), a.end(), x); // 找第一个 x, 返回迭代器
count(a.begin(), a.end(), x); // 统计 x 的个数
count_if(a.begin(), a.end(), [](int v){ return v > 0; }); // 条件统计

```

```

any_of(a.begin(), a.end(), pred);    // 是否存在满足条件的
all_of(a.begin(), a.end(), pred);    // 是否全部满足条件
none_of(a.begin(), a.end(), pred);   // 是否全不满足条件

// 最值
*max_element(a.begin(), a.end());    // 最大值
*min_element(a.begin(), a.end());    // 最小值
auto [mi, ma] = minmax_element(a.begin(), a.end()); // 同时求最小和最大的迭代器

// 去重（要求有序）
auto it = unique(a.begin(), a.end()); // 返回去重后的尾迭代器
a.erase(it, a.end());                // 真正删除多余元素

// 反转、旋转、填充
reverse(a.begin(), a.end());          // 反转
rotate(a.begin(), a.begin()+k, a.end()); // 左旋 k 位
fill(a.begin(), a.end(), val);        // 填充
fill_n(a.begin(), k, val);            // 填充前 k 个
generate(a.begin(), a.end(), [&]() { return rng(); }); // 用函数生成

// 拷贝、变换
copy(a.begin(), a.end(), b.begin());  // 拷贝到 b
transform(a.begin(), a.end(), b.begin(), [](int x) { return x*2; }); // 变换

```

- `nth_element` 是 $O(n)$ 的，比 `sort` 后取第 k 个快很多。常用于求中位数。
- `partial_sort` 只排前 k 小， $O(n \log k)$ ，当 $k \ll n$ 时比完整排序快。
- `rotate` 实现数组左旋非常方便， $O(n)$ 。

3.7.3 排列与组合

```

// 全排列
// next_permutation: 生成下一个字典序排列，到最后返回 false
// prev_permutation: 生成上一个字典序排列，到最前返回 false
vector<int> a = {1, 2, 3};
do {
    // 处理当前排列 a
} while (next_permutation(a.begin(), a.end()));
// 注意：要枚举所有排列，a 必须先排序为最小排列！

// 枚举子集（用 bitmask）
for (int mask = 0; mask < (1 << n); mask++) {

```



```

// mask 的第 i 位为 1 表示选第 i 个元素
for (int i = 0; i < n; i++)
    if (mask >> i & 1) { /* 选了第 i 个 */ }
}

// 枚举大小为 k 的子集 (用 bitmask)
// 见位运算章节

// is_permutation: 判断两个序列是否互为排列
is_permutation(a.begin(), a.end(), b.begin()); // bool

```

- `next_permutation` 会修改原数组！如果要枚举所有排列，初始必须是最小排列（升序）。
- 全排列总数是 $n!$ ， $n \geq 12$ 时 $n! > 10^8$ ，注意复杂度。
- `prev_permutation` 生成上一个排列，初始必须是最大排列（降序）才能枚举所有排列。

3.7.4 全排列函数详解（`next_permutation` / `prev_permutation`）

C++ STL 提供的 `next_permutation` 和 `prev_permutation` 是生成排列的核心工具。它们原地修改序列，将其变换为字典序的下一个/上一个排列，返回 `bool` 表示是否成功（到达最后/最前时返回 `false` 并回绕为最小/最大排列）。时间复杂度单次 $O(n)$ ，枚举所有排列总计 $O(n! \cdot n)$ 。

```

// 函数签名
bool next_permutation(BidirIt first, BidirIt last);
bool next_permutation(BidirIt first, BidirIt last, Compare comp);
bool prev_permutation(BidirIt first, BidirIt last);
bool prev_permutation(BidirIt first, BidirIt last, Compare comp);
// 返回值：成功生成下一个/上一个排列返回 true
//          已经是最大/最小排列时回绕并返回 false

// ===== 用法一：枚举所有排列 =====
// 必须先排序为最小排列（升序），然后用 do-while 循环
vector<int> a = {1, 2, 3};
sort(a.begin(), a.end()); // 必须先升序排列！
do {
    // 处理当前排列，例如输出
    for (int x : a) cout << x << " ";
    cout << endl;
} while (next_permutation(a.begin(), a.end()));
// 输出：1 2 3 / 1 3 2 / 2 1 3 / 2 3 1 / 3 1 2 / 3 2 1

```

```
// ===== 用法二：有重复元素的排列（自动去重） =====
// next_permutation 天然支持重复元素，不会生成重复排列
vector<int> a = {1, 1, 2};
sort(a.begin(), a.end());
do {
    for (int x : a) cout << x << " ";
    cout << endl;
} while (next_permutation(a.begin(), a.end()));
// 输出: 1 1 2 / 1 2 1 / 2 1 1 (只有 3 种, 不是 3!=6 种)

// ===== 用法三：只排列部分元素（前 k 个的全排列） =====
// 枚举 n 个元素中选 k 个的全排列 (P(n,k))
int n = 4, k = 2;
vector<int> a = {1, 2, 3, 4};
sort(a.begin(), a.end());
do {
    // 只看前 k 个元素，它们是一种选法的排列
    for (int i = 0; i < k; i++) cout << a[i] << " ";
    cout << endl;
    // 将后面的部分反转，使 next_permutation 跳到下一组前 k 个
    reverse(a.begin() + k, a.end());
} while (next_permutation(a.begin(), a.end()));

// ===== 用法四：求当前排列之后的第 k 个排列 =====
vector<int> a = {1, 2, 3, 4};
int k = 5;
while (k-- && next_permutation(a.begin(), a.end()));
// a 变成第 5 个后继排列

// ===== 用法五：对字符串使用 =====
string s = "abc";
sort(s.begin(), s.end());
do {
    cout << s << endl;
} while (next_permutation(s.begin(), s.end()));
// 输出: abc / acb / bac / bca / cab / cba

// ===== 用法六：自定义比较函数（降序排列枚举） =====
vector<int> a = {3, 2, 1}; // 按 greater 的最小排列
do {
    for (int x : a) cout << x << " ";
    cout << endl;
} while (next_permutation(a.begin(), a.end(), greater<int>()));
// 枚举所有"降序字典序"的排列
```

- **初始状态很重要：**用 `next_permutation` 枚举所有排列，初始必须是升序（字典序最小）；用 `prev_permutation` 则初始必须是降序（字典序最大）。否则只能枚举到从当前位置到末尾的部分排列。
- **回绕行为：**当 `next_permutation` 在最大排列上调用时，会将数组变回最小排列并返回 `false`。这就是 `do-while` 的终止条件——回绕后 `false` 退出循环。
- **复杂度：** $n!$ 增长极快， $n = 8$ 时 $n! = 40320$ ， $n = 10$ 时 $n! = 3628800$ ， $n = 12$ 时 $n! \approx 4.8 \times 10^8$ 。竞赛中 $n \leq 8$ 比较安全， $n \leq 10$ 勉强可行。
- **不要用 while：**写 `while (next_permutation(...))` 会跳过初始排列！必须用 `do { ... } while (next_permutation(...))`。

- **判断是否为排列：**用 `is_permutation(a.begin(), a.end(), b.begin())` 判断 a 和 b 是否互为排列。
- **`next_permutation` 的原理：**从右往左找第一个 $a[i] < a[i + 1]$ 的位置，再从右往左找第一个 $a[j] > a[i]$ 的位置，交换 $a[i]$ 和 $a[j]$ ，最后反转 $a[i + 1 \dots]$ 。
- **求排列数/排名：**需要知道某排列是第几个，用康托展开（见下节）。
- **竞赛经典场景：**暴力枚举所有排列检查是否满足条件（如 N 皇后的暴力解、TSP 暴力等）。

DFS 递归生成全排列

当需要在生成过程中剪枝（如 N 皇后），或需要更灵活的控制（如只要部分元素的排列），用 DFS 递归比 `next_permutation` 更方便。

```
// 生成 {1, 2, ..., n} 的全排列（不重复元素）
vector<int> path;
vector<bool> used;

void dfs(int n) {
    if ((int)path.size() == n) {
        // 得到一个排列 path，处理它
        for (int x : path) cout << x << " ";
        cout << endl;
        return;
    }
    for (int i = 1; i <= n; i++) {
        if (used[i]) continue;
        path.push_back(i);
```

```

        used[i] = true;
        dfs(n);
        path.pop_back();
        used[i] = false;
    }
}

// 调用: used.assign(n + 1, false); dfs(n);

// 有重复元素的 DFS 全排列 (去重版)
// nums 必须先排序!
vector<int> nums; // 已排序
vector<int> path;
vector<bool> used;

void dfs2(int n) {
    if ((int)path.size() == n) {
        for (int x : path) cout << x << " ";
        cout << endl;
        return;
    }
    for (int i = 0; i < n; i++) {
        if (used[i]) continue;
        // 去重: 同一层中, 相同值只选第一个未用的
        if (i > 0 && nums[i] == nums[i-1] && !used[i-1]) continue;
        path.push_back(nums[i]);
        used[i] = true;
        dfs2(n);
        path.pop_back();
        used[i] = false;
    }
}

// 调用: sort(nums.begin(), nums.end()); used.assign(n, false); dfs2(n);

```

- DFS 版本可以方便地在递归中**剪枝**, 例如 N 皇后问题在放置每个皇后时检查冲突, 大幅减少搜索量。
- DFS 生成**部分排列** (选 k 个): 只需把终止条件改为 `path.size() == k`。
- 去重的关键条件 `!used[i-1]` 表示: 如果前一个相同元素没有被选 (说明是同层回溯回来的), 则跳过当前元素, 避免重复。

3.7.5 排列序（康托展开与逆康托展开）

康托展开：将一个排列映射为它在所有排列中的字典序排名（0-indexed）。逆康托展开：由排名恢复排列。时间复杂度均为 $O(n^2)$ （用树状数组可优化到 $O(n \log n)$ ）。

```
// 康托展开：求排列 p 在所有 n! 个排列中的排名（0-indexed）
// p 是 {0, 1, ..., n-1} 的排列
int cantor(vector<int>& p) {
    int n = p.size();
    int rank = 0;
    vector<int> fact(n, 1);
    for (int i = 1; i < n; i++) fact[i] = fact[i-1] * i;
    for (int i = 0; i < n; i++) {
        int cnt = 0; // 统计 p[i] 后面比 p[i] 小的个数
        for (int j = i + 1; j < n; j++)
            if (p[j] < p[i]) cnt++;
        rank += cnt * fact[n - 1 - i];
    }
    return rank;
}

// 逆康托展开：由排名 rank 恢复排列（0-indexed）
// 返回 {0, 1, ..., n-1} 的第 rank 个排列
vector<int> inv_cantor(int rank, int n) {
    vector<int> fact(n, 1);
    for (int i = 1; i < n; i++) fact[i] = fact[i-1] * i;
    vector<int> avail; // 可用元素
    for (int i = 0; i < n; i++) avail.push_back(i);
    vector<int> res;
    for (int i = 0; i < n; i++) {
        int idx = rank / fact[n - 1 - i];
        rank %= fact[n - 1 - i];
        res.push_back(avail[idx]);
        avail.erase(avail.begin() + idx);
    }
    return res;
}
```

- 如果排列是 $\{1, 2, \dots, n\}$ （1-indexed），只需将输入减 1 再调用，或将 avail 改为 $1 \sim n$ 。
- 用树状数组替代内层循环可以优化到 $O(n \log n)$ （统计前缀中比当前小的个数）。
- 康托展开在排列哈希、状态压缩（如 8-puzzle）中非常有用。

- $n \leq 20$ 时 $n!$ 超过 `long long` 范围，需要用高精度或只处理小规模。 $n \leq 12$ 时 `int` 足够， $n \leq 20$ 用 `long long`。

3.7.6 字符串处理函数

```
// <string> 头文件
s.substr(pos, len);           // 子串
s.find("abc");                // 查找子串位置，找不到返回 string::npos
s.rfind("abc");               // 从右往左找
s.find_first_of("aeiou");     // 找第一个属于给定字符集的字符
s.find_last_of("aeiou");      // 找最后一个属于给定字符集的字符
s.find_first_not_of("01");    // 找第一个不在给定字符集中的字符
s.replace(pos, len, "new");    // 替换子串
s.insert(pos, "abc");          // 在 pos 处插入
s.erase(pos, len);            // 删除从 pos 开始的 len 个字符
s.starts_with("pre");          // 是否以 "pre" 开头 (C++20)
s.ends_with("suf");           // 是否以 "suf" 结尾 (C++20)
s.contains("sub");             // 是否包含子串 (C++23)

// 类型转换
to_string(123);                // int -> string
to_string(3.14);               // double -> string
stoi("123");                   // string -> int
stoll("123456789012");         // string -> long long
stod("3.14");                  // string -> double

// <cctype> 字符分类 (传入 char)
isalpha(c); // 是否是字母
isdigit(c);  // 是否是数字
isalnum(c);  // 是否是字母或数字
isupper(c);  // 是否是大写字母
islower(c);  // 是否是小写字母
isspace(c);  // 是否是空白字符
toupper(c);  // 转大写
tolower(c);  // 转小写

// 整行读入
getline(cin, s); // 读一整行 (注意前面 cin>>n 后要 cin.ignore())

// 字符串流 (<sstream>)
stringstream ss("1 2 3");
int a, b, c;
ss >> a >> b >> c; // 按空格分割解析
```

- `s.find()` 找不到时返回 `string::npos`（一个极大的无符号数），不是 `-1`！判断时用 `if (s.find(x) != string::npos)`。
- `stoi` 对超出 `int` 范围的字符串会抛异常，大数用 `stoll`。
- `getline` 前如果用了 `cin >>`，必须先 `cin.ignore()` 消耗掉残留的换行符，否则读到空行。

3.7.7 其他实用工具

```
// 二分答案的 lambda 写法
auto check = [&](int mid) -> bool {
    // 判断 mid 是否可行
    return true;
};

// 自定义哈希 (pair 做 key)
struct pair_hash {
    size_t operator()(const pair<int,int>& p) const {
        return hash<long long>()(((long long)p.first << 32) | p.second);
    }
};

unordered_map<pair<int,int>, int, pair_hash> mp;
unordered_set<pair<int,int>, pair_hash> st;

// 离散化
vector<int> vals(a.begin(), a.end()); // 复制原数组
sort(vals.begin(), vals.end());
vals.erase(unique(vals.begin(), vals.end()), vals.end());
// 映射: 原值 -> 离散化后的编号
for (int i = 0; i < n; i++)
    a[i] = lower_bound(vals.begin(), vals.end(), a[i]) - vals.begin();
// 反映射: vals[a[i]] 即为原值

// iota 快速填充连续序列
vector<int> idx(n);
iota(idx.begin(), idx.end(), 0); // idx = {0, 1, 2, ..., n-1}
// 按其他数组的值排序下标 (间接排序)
sort(idx.begin(), idx.end(), [&](int i, int j) {
    return a[i] < a[j];
});

// merge 合并两个有序序列
```

```
merge(a.begin(), a.end(), b.begin(), b.end(), c.begin());

// set_intersection / set_union / set_difference (要求有序)
vector<int> res;
set_intersection(a.begin(), a.end(), b.begin(), b.end(), back_inserter(res));
// 类似地: set_union, set_difference, set_symmetric_difference
```

- 离散化是竞赛高频操作，用于将大值域压缩到 $[0, n)$ ，配合树状数组/线段树使用。
- 间接排序（按下标排序）可以保留原数组不变，同时知道排序后每个元素的原始位置。
- `merge` 和集合运算函数要求输入有序，输出也有序。

4 位运算

4.1 基本性质

- 与 $\&$: 同 1 为 1。交换律、结合律。 $a \& 0 = 0$, $a \& a = a$, $a \& (\sim a) = 0$ 。
- 或 $|$: 有 1 为 1。交换律、结合律。 $a | 0 = a$, $a | a = a$, $a | (\sim a) = -1$ (全 1)。
- 异或 $\oplus$: 不同为 1。交换律、结合律。 $a \oplus 0 = a$, $a \oplus a = 0$, $a \oplus (\sim a) = -1$ 。
- 取反 $\sim$: $\sim a = -a - 1$ (补码性质)。

4.1.1 分配律与吸收律

- 与对或的分配律: $a \& (b | c) = (a \& b) | (a \& c)$
- 或对与的分配律: $a | (b \& c) = (a | b) \& (a | c)$
- 异或对与的分配律: $a \& (b \oplus c) = (a \& b) \oplus (a \& c)$
- 吸收律: $a \& (a | b) = a$, $a | (a \& b) = a$
- 德摩根律: $\sim (a \& b) = (\sim a) | (\sim b)$, $\sim (a | b) = (\sim a) \& (\sim b)$

4.1.2 异或的重要性质

- 自逆性: $a \oplus b \oplus b = a$ (异或同一个数两次等于没异或, 用于加密/交换)。
- 无进位加法: 异或等价于不考虑进位的二进制加法。
- 前缀异或: $a_l \oplus a_{l+1} \oplus \dots \oplus a_r = \text{pre}[r] \oplus \text{pre}[l-1]$, 类比前缀和。
- 交换两数 (不用临时变量): $a \hat{=} b$; $b \hat{=} a$; $a \hat{=} b$; (但实际竞赛中直接用 swap 更安全)。
- 线性基: n 个数的所有异或组合可以用至多 $\log V$ 个基底表示。

4.1.3 $O(1)$ 求连续自然数的异或和

求 $1 \oplus 2 \oplus \dots \oplus m$ 是一个经典结论。令 $S(m) = \bigoplus_{i=1}^m i$ ，则结果只取决于 $m \bmod 4$ ：

- $m \bmod 4 = 0$ 时， $S(m) = m$
- $m \bmod 4 = 1$ 时， $S(m) = 1$
- $m \bmod 4 = 2$ 时， $S(m) = m + 1$
- $m \bmod 4 = 3$ 时， $S(m) = 0$

为什么有这个结论：核心引理是每 4 个对齐到 4 的倍数的连续整数，异或起来等于 0。即对任意 $k \equiv 0 \pmod{4}$ ：

$$k \oplus (k+1) \oplus (k+2) \oplus (k+3) = 0.$$

原因：这 4 个数的低 2 位恰好取遍 00, 01, 10, 11，异或为 0；而高位（第 2 位及以上）四个数完全相同，相同的数异或偶数次也为 0。所以整块异或为 0。

把 $S(m)$ 写成 $0 \oplus 1 \oplus \dots \oplus m$ （多异或一个 0 不影响结果），再从 0 开始每 4 个分一组：(0 1 2 3)(4 5 6 7)⋯，每个完整块都异或为 0，于是只剩下结尾不满一块的零头：

- $m \equiv 3$ ：恰好若干完整块，全抵消， $S(m) = 0$ 。
- $m \equiv 0$ ：到 $m-1$ ($\equiv 3$) 为完整块抵消，再异或上多出来的 m ， $S(m) = m$ 。
- $m \equiv 1$ ： $S(m) = S(m-1) \oplus m = (m-1) \oplus m$ 。此时 m 为奇数， $m-1$ 与 m 只差最低位，故 $(m-1) \oplus m = 1$ 。
- $m \equiv 2$ ： $S(m) = S(m-1) \oplus m = 1 \oplus m$ 。此时 m 为偶数（最低位为 0），故 $1 \oplus m = m+1$ 。

推论（区间异或和）：利用前缀异或，任意区间 $\bigoplus_{i=l}^r i = S(r) \oplus S(l-1)$ ，同样 $O(1)$ 。

```
// 1 ^ 2 ^ ... ^ m, O(1)
int xor_n(int m) {
    switch (m & 3) {          // m & 3 等价于 m % 4
        case 0: return m;
        case 1: return 1;
        case 2: return m + 1;
        default: return 0;    // case 3
    }
}

// 区间异或和 1 ^ (l+1) ^ ... ^ r, O(1)
int xor_range(int l, int r) {
    return xor_n(r) ^ xor_n(l - 1);
}
```

4.1.4 与/或的单调性

- 与运算越与越小： $a \& b \leq \min(a, b)$ 。连续与一串数，结果单调不增。
- 或运算越或越大： $a | b \geq \max(a, b)$ 。连续或一串数，结果单调不减。
- 推论：子数组的 AND 值最多有 $O(\log V)$ 种（每次变化至少少一个 1）；OR 同理。
- 应用：枚举子数组的 AND/OR 值时，可以利用单调性 + 双指针/二分优化。

4.1.5 常用恒等式速查

- $a \& (a - 1)$ ：去掉 a 最低位的 1。应用：统计 1 的个数、判断 2 的幂。
- $a \& (-a)$ ：取出 a 最低位的 1 (lowbit)。应用：树状数组。
- $a | (a + 1)$ ：将 a 最低位的 0 置为 1。
- $a \& (a + 1)$ ：将 a 末尾连续的 1 全部清零。
- $a | (a - 1)$ ：将 a 末尾连续的 0 全部置 1。
- $(a \oplus b) | (a \& b) = a | b$
- $(a \oplus b) \& (a \& b) = 0$ （异或和与互斥）
- $a + b = (a \oplus b) + 2 \cdot (a \& b)$ （加法 = 无进位和 + 进位）
- $a + b = (a | b) + (a \& b)$ （另一个加法分解）

4.2 竞赛常用性质科普

4.2.1 按位独立性 (Bitwise Independence)

位运算中，AND / OR / XOR 对每一位的运算是独立的——第 i 位的结果只取决于所有操作数的第 i 位。

竞赛意义：遇到“对一组数做位运算后求最优值”的题目，可以逐位贪心或逐位 DP，把一个 $\log V$ 位的问题拆成 $\log V$ 个独立的 0/1 问题。

典型应用：

- 求“选若干数使 OR 最大”——从高位到低位贪心，每一位独立决策。
- 求“所有子数组 AND 之和”——对每一位分别统计有多少子数组该位为 1。
- 位运算卷积 (AND/OR/XOR 卷积) 的正确性也依赖于按位独立性。

4.2.2 异或与线性代数

异或运算构成 $\text{GF}(2)$ （二元域）上的加法，AND 构成乘法。因此：

- n 个数的所有异或组合构成一个线性空间，维度 $\leq \log V$ 。
- 线性基（Gaussian Elimination on $\text{GF}(2)$ ）可以在 $O(n \log V)$ 时间内求出基底。
- 线性基支持：求最大/最小异或值、判断某个值能否被异或出、求第 k 小异或值。
- 异或方程组可以用高斯消元求解（开关灯问题、翻转问题）。

4.2.3 位运算与集合论的对应

将整数看作集合（第 i 位为 1 表示元素 i 在集合中），位运算对应集合运算：

- $a \& b = A \cap B$ （交集）
- $a | b = A \cup B$ （并集）
- $a \oplus b = A \Delta B$ （对称差）
- $\sim a = \bar{A}$ （补集）
- $a \& (\sim b) = A \setminus B$ （差集）
- $(a \& b) == a$ 等价于 $A \subseteq B$ （子集判断）

竞赛意义：状压 DP 中，集合操作全部用位运算实现，理解这层对应关系可以快速写出状态转移。

4.2.4 popcount 的性质与应用

$\text{popcount}(x)$ 表示 x 二进制中 1 的个数。常用性质：

- $\text{popcount}(a \oplus b) = a$ 和 b 不同位的个数（汉明距离）。
- $\text{popcount}(a \& b) = a$ 和 b 同时为 1 的位数。
- $\text{popcount}(a | b) = \text{popcount}(a) + \text{popcount}(b) - \text{popcount}(a \& b)$ （容斥）。
- $a + b = 2 \cdot (a \& b) + (a \oplus b)$ ，因此 $\text{popcount}(a) + \text{popcount}(b) = \text{popcount}(a \oplus b) + 2 \cdot \text{popcount}(a \& b)$ 。

竞赛应用：

- 状压 DP 中按 popcount 分层枚举（先枚举选 1 个的状态，再选 2 个……）。
- 判断两个状态的“距离”用汉明距离。
- 用 `__builtin_popcountll(x)` 获取， $O(1)$ 。

4.2.5 高位/低位操作

- 最高位位置: $\lfloor \log_2 x \rfloor = 63 - \text{__builtin_clzll}(x)$ ($x > 0$)。
- 最低位位置: $\text{__builtin_ctzll}(x)$ ($x > 0$)。
- 只保留最高位的 1: $1LL \ll (63 - \text{__builtin_clzll}(x))$ 。
- 只保留最低位的 1: $x \& (-x)$ (lowbit)。
- 将最高位以下全部填 1: 先求最高位位置 k , 然后 $(1LL \ll (k+1)) - 1$ 。

竞赛意义:

- 贪心从高位到低位确定答案时, 需要快速定位最高位。
- 树状数组的核心操作就是 lowbit。
- 求“不超过 x 的最大 2 的幂”等问题直接用最高位操作。

4.2.6 位运算与不等式

- $a \& b \leq \min(a, b)$, $a|b \geq \max(a, b)$ 。
- $a \oplus b \leq a + b$ (异或不会产生进位, 加法会)。
- $a + b = (a|b) + (a \& b) \geq a|b$ 。
- $(a \oplus b) + (a \& b) \cdot 2 = a + b$ 。
- 若 $a \& b = 0$ (无公共位), 则 $a + b = a|b = a \oplus b$ 。

竞赛意义:

- “选两个数使 AND 最大”——从高位贪心, 尽量让高位同时为 1。
- “选两个数使 XOR 最大”——用异或字典树从高位贪心取不同位。
- $a \& b = 0$ 时三种运算等价, 常用于状压 DP 中“两个集合不相交”的条件。

4.2.7 SOS DP (子集和 / 超集和)

给定数组 $f[mask]$, 求所有子集的和: $g[mask] = \sum_{sub \subseteq mask} f[sub]$ 。

暴力枚举子集是 $O(3^n)$, SOS DP 可以做到 $O(n \cdot 2^n)$:

```
for (int i = 0; i < n; i++)
    for (int mask = 0; mask < (1 << n); mask++)
        if (mask >> i & 1)
            f[mask] += f[mask ^ (1 << i)];
```

类似地, 超集和 ($g[mask] = \sum_{mask \subseteq sup} f[sup]$) 只需把条件改为 `if (!(mask >> i & 1))`。

竞赛应用：

- 求”有多少对 (i, j) 满足 $a_i \& a_j = a_i$ （即 a_i 是 a_j 的子集）”。
- 位运算卷积（AND/OR 卷积）的核心变换。
- 容斥原理在位运算上的高效实现。

4.3 常用技巧

```
// lowbit: 取出最低位的 1
int lowbit(int x) { return x & (-x); }

// 判断是否是 2 的幂
bool is_pow2(int x) { return x > 0 && (x & (x - 1)) == 0; }

// 枚举子集（枚举 mask 的所有非空子集）
for (int sub = mask; sub; sub = (sub - 1) & mask) {
    // sub 是 mask 的一个子集
}

// 枚举大小为 k 的子集（从 n 个元素中选 k 个）
int s = (1 << k) - 1;
while (s < (1 << n)) {
    // 处理 s
    int lb = s & (-s);
    int r = s + lb;
    s = ((r ^ s) >> 2) / lb | r;
}
```

- 位运算优先级极低！`if (x & 1 == 1)` 实际是 `x & (1 == 1)`，必须加括号 `if ((x & 1) == 1)`。
- 左移 `1 << i` 当 $i \geq 31$ 时溢出，必须写 `1LL << i`。

4.4 位运算与异或

- `x & -x`：最低位的 1；Fenwick 跳跃依赖此性质。
- `__builtin_popcountll(x)`：1 的个数。
- 枚举子集：`sub=(sub-1)&mask`。
- 异或满足交换律、结合律、自反性；“除一个数外都出现两次”直接全体异或。

- 线性基：维护若干最高位不同的向量，支持最大异或、判断可表示、计数不同异或值；见 08-进阶专题。

4.5 线性基（XOR Basis）

维护最高位不同的向量。插入时从高位向下消元；最大异或值从高位贪心尝试。可扩展为求第 k 小异或、判断异或方程组是否有解、求线性空间维数。

```
struct LinearBasis {
    long long b[61]{}; // b[i] 的最高位是 i；每个主元位只保留一个向量。
    bool insert(long long x) {
        for (int i = 60; i >= 0; --i) if (x >> i & 1) {
            // 从高位消元；如果该位已有主元，就异或消掉这一位。
            if (!b[i]) { b[i] = x; return true; }
            x ^= b[i];
        }
        // 完全消成 0：x 属于已有线性空间，插入不会增加秩。
        return false;
    }
    long long maximum(long long x = 0) const {
        // 从高位向低位贪心；异或后变大才保留，保证最高位最优。
        for (int i = 60; i >= 0; --i) x = max(x, x ^ b[i]);
        return x;
    }
};
```

4.6 线性基：参数、调用与改造

本节紧跟前面的基础模板。先看每个接口传什么，再看哪些不变量不能动，最后看如何替换维护信息或转移。

4.6.1 0.1 它解决什么问题

线性基处理的不是普通加法，而是异或意义下的线性空间。典型题面包括：

- 从若干数中选子集，使异或和最大/最小；
- 判断一个数能否由给定数异或得到；
- 在线加入数，查询当前所有数能得到的最大异或值；
- 两个集合的异或空间是否相交，或求异或值的第 k 小。

把每个整数看成二进制向量。异或就是在 $\mathbb{F}_2$ 上相加，所以每一位只有 0/1，消元时只需要“最高位主元”。若数据小于 2^{60} ，开 `LOG = 61`；若题目给到 2^{100} ，必须改用 `bitset` 或高精度

实现。

4.6.2 0.2 可直接使用的模板

```
template<int LOG = 61>
struct LinearBasis {
    using U = unsigned long long;
    U p[LOG]{}; // p[i] 的最高位是 i

    void clear() { memset(p, 0, sizeof(p)); }

    void insert(U x) {
        for (int i = LOG - 1; i >= 0; --i) {
            if ((x >> i) & 1ULL) continue;
            if (!p[i]) {
                p[i] = x;
                return;
            }
            x ^= p[i];
        }
    }

    bool can(U x) const {
        for (int i = LOG - 1; i >= 0; --i) {
            if (!((x >> i) & 1ULL)) continue;
            if (!p[i]) return false;
            x ^= p[i];
        }
        return true;
    }

    U max_xor(U x = 0) const {
        for (int i = LOG - 1; i >= 0; --i)
            x = max(x, x ^ p[i]);
        return x;
    }
};
```

insert 是异或版高斯消元；**can** 是消元判定；**max_xor** 从高位到低位贪心，若异或后变大就保留。这个顺序不能反过来，否则低位的选择可能破坏高位最优性。

4.6.3 0.3 例题：P3812 线性基

题目：P3812 【模板】线性基。题意是给出 n 个整数，选择任意子集，使它们的异或和最大。

套用步骤：

1. 每个输入数都代表一个可以选择或不选择的向量；
2. 不需要记录“选了哪些数”，因为目标只依赖最终异或值；
3. 逐个 `insert(x)`；
4. 输入结束后输出 `basis.max_xor()`。

例如输入集合为 1, 5, 9, 4: 插入后可以得到 $1 \oplus 4 \oplus 8 = 13$ ，模板从最高位开始尝试，得到最大值 13。完整调用如下：

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    LinearBasis<61> basis;
    while (n--) {
        unsigned long long x;
        cin >> x;
        basis.insert(x);
    }
    cout << basis.max_xor() << '\n';
}
```

测试样例：线性基必须按 `void solve()` 包装

下面约定每组输入为 n 个非负整数，输出任意子集异或和的最大值；第一行是测试组数 T 。前两组检查零向量、重复向量和多测重建，后两组检查独立主元以及高位贪心。

输入

```
4
1
0
3
5 5 5
4
1 2 4 8
5
8 7 3 10 5
```

输出

```
0
5
15
15
```

如果第二组大于 5，通常是消元时把重复向量当成了新主元；如果第三、四组小于 15，优先检查插入方向和 `max_xor` 是否从最高位向下贪心。

练习升级：P4301 新 Nim 游戏。它把“异或空间”放进博弈/删数语境中，看到“剩下的数不能异或出 0”时，应该想到线性基的秩和线性相关，而不是暴力枚举子集。

4.6.4 0.4 常见错误

- 把线性基当成普通的异或前缀；它维护的是一组向量，插入顺序不影响空间，但主元排列会影响某些“第 k 小”模板。
- `long long` 的符号位参与比较时容易出错。纯非负数据优先使用 `unsigned long long`。
- 求最大值时从高位到低位；求最小非零值时通常要先把基化成更适合贪心的形式。
- 线性基只能描述“每个数至多选一次”的异或组合；若某个数可以无限次使用，问题已经不是这个模板。

5 基础算法

5.1 前缀和

5.1.1 一维前缀和

```
// 预处理
int pre[N];
for (int i = 1; i <= n; i++)
    pre[i] = pre[i - 1] + a[i];
// 查询 [l, r] 的区间和
int sum = pre[r] - pre[l - 1];
```

5.1.2 二维前缀和

```
// 预处理
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        pre[i][j] = a[i][j] + pre[i-1][j] + pre[i][j-1] - pre[i-1][j-1];
// 查询 (r1,c1) 到 (r2,c2) 的矩形和
int sum = pre[r2][c2] - pre[r1-1][c2] - pre[r2][c1-1] + pre[r1-1][c1-1];
```

5.2 差分

5.2.1 一维差分

```
// 对区间 [l, r] 加上 val
d[l] += val;
d[r + 1] -= val;
// 还原：对 d 做前缀和即得原数组
for (int i = 1; i <= n; i++)
    d[i] += d[i - 1];
```

5.2.2 二维差分

```
// 对矩形 (r1,c1)~(r2,c2) 加 val
d[r1][c1] += val;
d[r2+1][c1] -= val;
d[r1][c2+1] -= val;
d[r2+1][c2+1] += val;
// 还原：对 d 做二维前缀和
```

5.3 二分查找

- 死循环魔咒：只要出现 $l = mid$ ，算 mid 就必须 $mid = (l + r + 1) / 2$ ；如果是 $r = mid$ ，则用 $mid = (l + r) / 2$ 。
- `check(mid)` 里如果用了全局数组（如 `vis`），每次调用前必须清空！二分会调用几十次 `check`。

5.3.1 整数二分

```
// 模板一：找满足条件的最小值（答案在右半边）
int l = lo, r = hi;
while (l < r) {
    int mid = (l + r) / 2;
    if (check(mid)) r = mid;
    else l = mid + 1;
}
// 答案是 l

// 模板二：找满足条件的最大值（答案在左半边）
int l = lo, r = hi;
while (l < r) {
    int mid = (l + r + 1) / 2; // 注意 +1 防死循环
    if (check(mid)) l = mid;
    else r = mid - 1;
}
// 答案是 l
```

5.3.2 浮点二分

```
// 固定循环次数，避免精度导致死循环
double l = lo, r = hi;
```

```
for (int i = 0; i < 100; i++) {
    double mid = (l + r) / 2;
    if (check(mid)) r = mid;
    else l = mid;
}
// 答案是 1
```

5.4 三分法

用于求单峰/单谷函数的极值点。

5.4.1 浮点三分法

```
// 求单峰函数的极大值点
double l = lo, r = hi;
for (int i = 0; i < 100; i++) {
    double m1 = l + (r - l) / 3;
    double m2 = r - (r - l) / 3;
    if (f(m1) < f(m2)) l = m1;
    else r = m2;
}
// 极值点在 l 附近, 极大值为 f(l)

// 求单谷函数的极小值点
double l = lo, r = hi;
for (int i = 0; i < 100; i++) {
    double m1 = l + (r - l) / 3;
    double m2 = r - (r - l) / 3;
    if (f(m1) < f(m2)) r = m2; // 极小值在左侧
    else l = m1;
}
// 极值点在 l 附近, 极小值为 f(l)
```

- 循环 100 次精度约为 10^{-15} , 一般足够。也可以用 `while (r - l > 1e-9)` 但要小心死循环。
- 三分法要求函数严格单峰/单谷。如果有平台（一段区间函数值相同），三分可能收敛到错误位置。
- 求极大值： $f(m1) < f(m2)$ 时砍左边（ $l = m1$ ）；求极小值： $f(m1) < f(m2)$ 时砍右边（ $r = m2$ ）。方向别搞反。

5.4.2 整数三分法

当定义域为整数时，不能用浮点三分。整数三分通过不断缩小区间直到 $r - l < 3$ ，最后暴力比较剩余点。

// 整数三分求单峰函数的极大值（定义域为整数）

```
int l = lo, r = hi;
while (r - l >= 3) {
    int m1 = l + (r - l) / 3;
    int m2 = r - (r - l) / 3;
    if (f(m1) < f(m2)) l = m1 + 1;
    else r = m2 - 1;
}
```

// 暴力检查剩余的 2~3 个点

```
int ans = l;
for (int i = l + 1; i <= r; i++)
    if (f(i) > f(ans)) ans = i;
// ans 即为极大值点，f(ans) 为极大值
```

// 整数三分求单谷函数的极小值（定义域为整数）

```
int l = lo, r = hi;
while (r - l >= 3) {
    int m1 = l + (r - l) / 3;
    int m2 = r - (r - l) / 3;
    if (f(m1) < f(m2)) r = m2 - 1;
    else l = m1 + 1;
}
```

```
int ans = l;
for (int i = l + 1; i <= r; i++)
    if (f(i) < f(ans)) ans = i;
// ans 即为极小值点，f(ans) 为极小值
```

- 整数三分不能像浮点三分那样用 $l = m1$ ，必须用 $l = m1 + 1$ 或 $r = m2 - 1$ ，否则区间可能不缩小导致死循环。
- 函数必须严格单峰/单谷（不能有平台），如果存在相邻两点函数值相等，三分可能失效。
- 常见应用：在整数定义域上求二次函数最值、求最小化某个凸代价函数等。

- 另一种写法: `while (l < r) { int mid = (l + r) / 2; if (f(mid) < f(mid + 1)) l = mid + 1; else r = mid; }`, 本质是对相邻差分做二分, 适用于严格单峰函数。
- 整数三分的时间复杂度为 $O(\log n)$, 其中 n 为区间长度。

5.5 双指针 / 滑动窗口

- 移动左指针时必须保证 $l \leq r$, 否则区间长度变负数或越界。
- 求最短区间: 在满足条件时更新答案; 求最长区间: 在修复完毕后更新答案。
- 遍历结束后, 最后一段可能没有触发结算! 可以在末尾加哨兵值强制触发。

```
// 最长满足条件的子数组
int l = 0, ans = 0;
for (int r = 0; r < n; r++) {
    // 加入 a[r] 到窗口
    while (/* 窗口不满足条件 */ && l <= r) {
        // 移除 a[l]
        l++;
    }
    ans = max(ans, r - l + 1);
}

// 最短满足条件的子数组
int l = 0, ans = INF;
for (int r = 0; r < n; r++) {
    // 加入 a[r] 到窗口
    while (/* 窗口满足条件 */) {
        ans = min(ans, r - l + 1);
        // 移除 a[l]
        l++;
    }
}
}
```

5.6 自定义排序

```
// 结构体排序
struct Node {
    int x, y;
    bool operator<(const Node& o) const {
```

```

        if (x != o.x) return x < o.x;
        return y < o.y;
    }
};

// lambda 排序
sort(a.begin(), a.end(), [](const auto& a, const auto& b) {
    return a.first < b.first; // 必须严格 <, 不能 <=
});

```

5.7 枚举与剪枝

5.7.1 子集枚举

`for (int s = mask; s; s = (s - 1) & mask)` 枚举 `mask` 的所有非空子集，总复杂度是 $O(3^n)$ （对所有 `mask` 求子集时）。

```

for (int mask = 0; mask < (1 << n); ++mask) {
    for (int sub = mask; sub = (sub - 1) & mask) {
        // 处理 sub; 这里包含空集
        if (sub == 0) break;
    }
}

```

5.7.2 回溯剪枝

固定顺序、提前判非法、记录当前最优上界、相同状态记忆化。对排列/组合先排序再跳过相同元素：`if (i > start && a[i] == a[i-1]) continue;`。

```

void dfs(int pos, long long sum) {
    if (sum == target) { ++answer; return; }
    if (pos == n || sum > target) return;
    for (int i = pos; i < n; ++i) {
        if (i > pos && a[i] == a[i - 1]) continue;
        dfs(i + 1, sum + a[i]);
    }
}

```

5.7.3 折半搜索 (Meet-in-the-middle)

$n \leq 40$ 、每个元素选/不选且目标为和/异或/最值时，把数组分两半枚举，排序一侧后二分或双指针，复杂度 $O(2^{(n/2)} \log 2^{(n/2)})$ 。


```
vector<long long> gen(const vector<long long>& a, int l, int r) {
    vector<long long> res{0};
    for (int i = l; i < r; ++i) {
        int old = (int)res.size();
        for (int j = 0; j < old; ++j) res.push_back(res[j] + a[i]);
    }
    return res;
}

auto L = gen(a, 0, n / 2), R = gen(a, n / 2, n);
sort(R.begin(), R.end());
long long best = 0;
for (long long x : L) {
    auto it = upper_bound(R.begin(), R.end(), limit - x);
    if (it != R.begin()) best = max(best, x + *prev(it));
}
```

5.8 前缀和、差分与扫描

5.8.1 一维前缀和

$pre[i+1] = pre[i] + a[i]$, 区间 $[l, r]$ 和为 $pre[r+1]-pre[l]$ 。二维前缀和使用容斥：
 $S(x_2, y_2) - S(x_1 - 1, y_2) - S(x_2, y_1 - 1) + S(x_1 - 1, y_1 - 1)$ 。

5.8.2 一维差分

对 $[l, r]$ 加 v : $d[l] += v$; $d[r+1] -= v$; 最后累加 d 恢复数组。二维矩形加法在四个角标记，二维前缀恢复。

```
vector<long long> pre(n + 1), diff(n + 2);
for (int i = 1; i <= n; ++i) pre[i] = pre[i - 1] + a[i];
auto range_sum = [&](int l, int r) { return pre[r] - pre[l - 1]; };

auto range_add = [&](int l, int r, long long v) {
    diff[l] += v;
    diff[r + 1] -= v;
};

for (auto [l, r, v] : updates) range_add(l, r, v);
long long cur = 0;
for (int i = 1; i <= n; ++i) {
    cur += diff[i];
    a[i] += cur;
}
```

```
}
```

5.8.3 扫描线思想

把“开始/结束”事件排序，按坐标推进并维护当前集合。矩形面积、区间覆盖、会议室数量都可转成事件；需要动态维护覆盖长度时配合线段树。

```
vector<pair<int, int>> event; // {坐标, +1 开始 / -1 结束}
for (auto [l, r] : intervals)
    event.push_back({l, 1}), event.push_back({r, -1});
sort(event.begin(), event.end());
int active = 0, peak = 0;
for (auto [x, delta] : event) {
    active += delta;
    peak = max(peak, active);
}
```

5.9 排序、二分、双指针

5.9.1 二分查找

`lower_bound` 找第一个 $\geq x$ ，`upper_bound` 找第一个 $> x$ 。手写时推荐半开区间 $[l, r)$ ：

```
int l = 0, r = n;
while (l < r) {
    int m = l + (r - l) / 2;
    if (a[m] < x) l = m + 1;
    else r = m;
}
// l 是第一个 a[l] >= x 的位置，可能等于 n
```

5.9.2 二分答案

当答案具有单调可行性 `check(x)` 时，在答案范围二分。先明确求“最小可行”还是“最大可行”，并把 `check` 写成无副作用函数。典型：最小最大分段和、最小速度、最大最小距离、满足容量的最少天数。

```
bool check(long long limit) {
    int groups = 1;
    long long sum = 0;
    for (long long x : a) {
        if (x > limit) return false;
```

```

        if (sum + x > limit) sum = x, ++groups;
        else sum += x;
    }
    return groups <= k;
}

long long l = *max_element(a.begin(), a.end());
long long r = accumulate(a.begin(), a.end(), 0LL);
while (l < r) {
    long long mid = l + (r - l) / 2;
    if (check(mid)) r = mid;
    else l = mid + 1;
}

```

5.9.3 双指针/滑动窗口

窗口右端只增不减，维护“当前窗口合法”不变量；若加入 r 使其非法，就移动 l 直到恢复。适用于正数和、至多/至少 k 个不同值、最长合法子串。含负数的和通常不能直接用双指针。

```

vector<int> cnt(alphabet);
int distinct = 0, left = 0, answer = 0;
for (int right = 0; right < n; ++right) {
    if (cnt[s[right]]++ == 0) ++distinct;
    while (distinct > k) {
        if (--cnt[s[left++]] == 0) --distinct;
    }
    answer = max(answer, right - left + 1);
}

```

5.9.4 三分与凸性

连续/离散单峰函数可三分；整数域更稳妥的做法是三分到很小区间后暴力。很多题可通过凸性改写为二分导数或斜率，不要盲目使用浮点三分。

```

while (r - l > 3) {
    long long m1 = l + (r - l) / 3;
    long long m2 = r - (r - l) / 3;
    if (f(m1) < f(m2)) l = m1 + 1; // 求最大值
    else r = m2 - 1;
}
long long ans = LLONG_MIN;
for (long long x = l; x <= r; ++x) ans = max(ans, f(x));

```

5.10 贪心

5.10.1 典型证明模式

- 区间调度：按右端点升序，选最早结束的可行区间；交换任意最优解的第一个区间即可证明。
- 最小生成树：割性质/环性质。
- Huffman：每次合并最小的两个权值（最优前缀码、合并果子）。
- 任务排序：按截止时间、比值、边际收益排序，必须写出交换论证，不要凭直觉。

```
sort(intervals.begin(), intervals.end(),
     [](auto &x, auto &y) { return x.second < y.second; });
int last = numeric_limits<int>::min(), chosen = 0;
for (auto [l, r] : intervals) {
    if (l >= last) last = r, ++chosen;
}
```

5.10.2 反悔贪心

先按某个顺序加入方案，用堆保存可撤销元素；若当前超限，撤掉代价最大/收益最小者。常见于“最多完成多少任务”“带截止时间的作业选择”。

```
sort(jobs.begin(), jobs.end(),
     [](auto &x, auto &y) { return x.deadline < y.deadline; });
priority_queue<int> chosen;
for (auto job : jobs) {
    chosen.push(job.cost);
    if ((int)chosen.size() > job.deadline) chosen.pop();
}
```

5.10.3 差分约束式贪心

“每个位置至少/至多”“相邻差不超过”等限制有时可从左到右维护最小需求，再从右到左修正；若约束包含图上的环，转为最短路/最长路更可靠。

```
// x[v] <= x[u] + w; 无解等价于存在可达负环
struct Edge { int v, w; };
vector<vector<Edge>> g(n);
vector<long long> d(n, 0);
vector<int> inq(n), pushed(n);
queue<int> q;
for (int s = 0; s < n; ++s) q.push(s), inq[s] = 1;
bool ok = true;
```

```

while (!q.empty() && ok) {
    int u = q.front(); q.pop(); inq[u] = 0;
    for (auto [v, w] : g[u]) if (d[v] > d[u] + w) {
        d[v] = d[u] + w;
        if (++pushed[v] > n) { ok = false; break; }
        if (!inq[v]) q.push(v), inq[v] = 1;
    }
}

```

5.11 单调栈与单调队列

5.11.1 单调栈

维护栈内下标对应值单调。遇到更小/更大元素时弹栈，弹出元素的右侧第一个更小/更大位置被确定。应用：下一个更大元素、柱状图最大矩形、最大子矩形、贡献法统计子数组最小值。

```

vector<int> nextGreater(n, -1), st;
for (int i = 0; i < n; ++i) {
    while (!st.empty() && a[st.back()] < a[i])
        nextGreater[st.back()] = i, st.pop_back();
    st.push_back(i);
}

```

5.11.2 单调队列

队首始终是窗口最优值，队尾删除不可能成为最优的元素。应用：滑动窗口最值、DP 转移 $dp[i] = \min(dp[j]) + w(i)$ 、有长度限制的最短/最长子数组。

```

deque<int> q;
for (int i = 0; i < n; ++i) {
    while (!q.empty() && q.front() <= i - k) q.pop_front();
    while (!q.empty() && a[q.back()] <= a[i]) q.pop_back();
    q.push_back(i);
    if (i >= k - 1) answer.push_back(a[q.front()]);
}

```

5.12 离散化与坐标压缩

收集所有会出现的坐标，排序去重，用 `lower_bound` 映射。若区间端点表示连续长度，通常还要把 x 与 $x+1$ 或相邻端点间距保留，避免把长区间压成一个点。值域大、操作数小的线段

树/树状数组必须先离散化。

```
vector<long long> xs = coordinates;
sort(xs.begin(), xs.end());
xs.erase(unique(xs.begin(), xs.end()), xs.end());
auto id = [&](long long x) {
    return int(lower_bound(xs.begin(), xs.end(), x) - xs.begin()) + 1;
};
```

5.13 常见构造套路

- 先处理奇偶、余数、最大值/最小值等不变量。
- 需要构造排列时，优先考虑循环移位、两端放置、按奇偶分组。
- 证明“无解”时找模数、颜色、度数、连通性或总和不变量。
- 证明“总能构造”时给出明确操作顺序，并说明每一步都保持前置条件。

```
// 常见构造：把 1..n 按奇偶分组输出
vector<int> ans;
for (int x = 1; x <= n; x += 2) ans.push_back(x);
for (int x = 2; x <= n; x += 2) ans.push_back(x);
```

5.14 随机化与哈希技巧

- 随机 `shuffle` + 快速选择求期望线性第 k 小。
- 64 位随机哈希表示集合，支持多重集合差异的高概率比较；严谨题目用排序/Trie/计数。
- Treap 随机优先级避免退化；不要把 `rand()` 作为高质量随机源，使用 `mt19937_64`。

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
shuffle(a.begin(), a.end(), rng);
nth_element(a.begin(), a.begin() + k, a.end());
int answer = a[k];
uint64_t random_hash = rng();
```

5.15 位集优化

`std::bitset` 将 64/机器字并行，适合传递闭包（布尔矩阵）、集合交并、子集计数；`bitset` 大小需编译期常量，动态场景使用 `vector<unsigned long long>`。

```
const int N = 2000;
bitset<N> reach[N];
for (int k = 0; k < n; ++k)
```

```
for (int i = 0; i < n; ++i)
    if (reach[i][k]) reach[i] |= reach[k];
```

5.16 快速小技巧

- `std::gcd`、`std::lcm` (C++17)。
- `__builtin_ctzll` 只对非零数调用。
- 排序后用 `unique` 去重，差异数组用 `adjacent_difference`。
- 大量模乘、叉积、斜率比较时先判断上界，必要时统一 `__int128`。

```
long long g = std::gcd(a, b);
long long l = a / g * b;
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());
long long lowbit = x & -x; // x != 0
long long safe_product = (long long)((__int128)x * y % MOD);
```

6 数学

6.1 GCD / LCM

```
int gcd(int a, int b) {
    while (b) { a %= b; swap(a, b); }
    return a;
}

int lcm(int a, int b) {
    return a / gcd(a, b) * b; // 先除后乘防溢出
}
```

- 求 LCM 必须先除后乘： $a / \gcd(a, b) * b$ ，写 $a * b / \gcd$ 会溢出。
- C++17 可直接用 `__gcd(a, b)` 或 `std::gcd(a, b)`。

6.2 快速幂（取模）

```
int qpow(int a, int b, int mod) {
    int res = 1;
    a %= mod;
    while (b > 0) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}
```

- 求逆元：`qpow(a, MOD - 2, MOD)`（MOD 为质数时，费马小定理）。
- a 可能大于 mod ，所以开头先 `a %= mod`。
- 不取模版本去掉 `% mod` 即可，但注意溢出。

6.3 逆元（费马小定理）

当 MOD 为质数时， $a^{-1} \equiv a^{MOD-2} \pmod{MOD}$ 。

```
int inv(int a, int mod) {
    return qpow(a, mod - 2, mod);
}
```

- 取模意义下绝对不能直接做除法！ $(a / b) \% MOD$ 是错的，必须写 $a * \text{inv}(b, MOD) \% MOD$ 。
- 减法取模可能出负数：必须写 $(a - b \% MOD + MOD) \% MOD$ 。
- 连乘时每一步都要取模，防止中间溢出： $a * b \% MOD * c \% MOD$ 。

6.4 质数筛

6.4.1 埃氏筛

```
// O(n log log n)
bool is_prime[N];
void sieve(int n) {
    fill(is_prime, is_prime + n + 1, true);
    is_prime[0] = is_prime[1] = false;
    for (int i = 2; i * i <= n; i++) {
        if (is_prime[i]) {
            for (int j = i * i; j <= n; j += i)
                is_prime[j] = false;
        }
    }
}
```

6.4.2 线性筛（欧拉筛）

```
// O(n)，每个合数只被最小质因子筛掉一次
int primes[N], cnt = 0;
bool vis[N];
void euler_sieve(int n) {
    for (int i = 2; i <= n; i++) {
        if (!vis[i]) primes[cnt++] = i;
        for (int j = 0; j < cnt && (long long)i * primes[j] <= n; j++) {
            vis[i * primes[j]] = true;
            if (i % primes[j] == 0) break;
        }
    }
}
```

```

        vis[i * primes[j]] = true;
        if (i % primes[j] == 0) break; // 关键：保证最小质因子
    }
}
}

```

分解质因数时，循环结束后如果 $n > 1$ ，说明剩下的 n 本身就是一个大质因数，不要漏掉！

6.5 组合数（预处理阶乘逆元）

```

const int MOD = 998244353;
int fact[N], inv_fact[N];

void init_comb(int n) {
    fact[0] = 1;
    for (int i = 1; i <= n; i++)
        fact[i] = fact[i - 1] * i % MOD;
    inv_fact[n] = qpow(fact[n], MOD - 2, MOD);
    for (int i = n - 1; i >= 0; i--)
        inv_fact[i] = inv_fact[i + 1] * (i + 1) % MOD;
}

int C(int n, int m) {
    if (m < 0 || m > n) return 0; // 边界特判！
    return fact[n] % MOD * inv_fact[m] % MOD * inv_fact[n - m] % MOD;
}

```

- $C(n, m)$ 必须特判 $m < 0$ 或 $m > n$ 返回 0，否则越界。
- $fact[0] = 1$ 必须初始化，否则所有涉及 0 的组合数变 0。

6.6 高斯消元

```

// 求解 n 元线性方程组，a[i][n] 是常数项
// 返回 0 无解，1 唯一解，2 无穷解
int gauss(int n, double a[][MAXN]) {
    for (int col = 0; col < n; col++) {
        // 找主元（绝对值最大的行）
        int mx = col;
        for (int i = col + 1; i < n; i++)

```

```

        if (fabs(a[i][col]) > fabs(a[mx][col])) mx = i;
    swap(a[col], a[mx]);
    if (fabs(a[col][col]) < 1e-9) return 2; // 自由变量
    // 消元
    for (int i = 0; i < n; i++) {
        if (i == col) continue;
        double rate = a[i][col] / a[col][col];
        for (int j = col; j <= n; j++)
            a[i][j] -= a[col][j] * rate;
    }
}
// 提取解: x[i] = a[i][n] / a[i][i]
return 1;
}

```

6.7 模运算与快速幂

对质数/任意模数都成立: $(a+b)\%p$ 、 $(a-b+p)\%p$ 、 $a \times b \% p$ 。除法不能直接取模, 必须证明逆元存在。

```

long long qpow(long long a, long long e, long long mod) {
    a %= mod; long long r = 1 % mod;
    while (e) {
        if (e & 1) r = (__int128)r * a % mod;
        a = (__int128)a * a % mod; e >>= 1;
    }
    return r;
}

```

6.8 最大公约数、扩展欧几里得与同余

```

long long exgcd(long long a, long long b, long long &x, long long &y) {
    if (!b) { x = 1; y = 0; return a; }
    long long x1, y1, g = exgcd(b, a % b, x1, y1);
    x = y1; y = x1 - (a / b) * y1; return g;
}

```

解 $ax \equiv b \pmod{m}$: 令 $g = \gcd(a, m)$, 若 $b \% g \neq 0$ 无解; 否则约去 g , 用扩展欧几里得求一个解, 通解模 m/g 。

中国剩余定理 (互质模数): $x = \sum a_i \times M_i \times \text{inv}(M_i \bmod m_i) \bmod M$ 。非互质模数用“合并两个同余方程”: 检查差值能否被 $\gcd$ 整除, 再扩展欧几里得求增量。

6.9 质数与筛法

6.9.1 试除与质因数分解

试除到 $i \times i \leq n$ ，每次除尽；单个 $n \leq 1e12$ 适用。若只需判质数，复杂度 $O(\sqrt{n})$ 。

```
vector<pair<long long, int>> factor(long long n) {
    vector<pair<long long, int>> res;
    for (long long p = 2; p * p <= n; ++p) {
        if (n % p) continue;
        int e = 0;
        while (n % p == 0) n /= p, ++e;
        res.push_back({p, e});
    }
    if (n > 1) res.push_back({n, 1});
    return res;
}
```

6.9.2 埃氏筛与线性筛

埃氏筛复杂度 $O(n \log \log n)$ ；线性筛保证每个合数只被最小质因子筛一次，复杂度 $O(n)$ ，可同时求 ϕ 、莫比乌斯等积性函数。

```
vector<int> primes, lp(N), phi(N);
phi[1] = 1;
for (int i = 2; i < N; ++i) {
    if (!lp[i]) lp[i] = i, primes.push_back(i), phi[i] = i - 1;
    for (int p : primes) {
        if (p > lp[i] || 1LL * i * p >= N) break;
        lp[i * p] = p;
        if (p == lp[i]) phi[i * p] = phi[i] * p;
        else phi[i * p] = phi[i] * (p - 1);
    }
}
```

6.9.3 约数枚举与整除分块

枚举约数成对出现， $O(\sqrt{n})$ 。对 $\text{floor}(n/i)$ 的求和，商相同的 i 区间为 $[i, n/(n/i)]$ ，可在 $O(\sqrt{n})$ 个块内完成（数论分块）。

```
for (long long l = 1, r; l <= n; l = r + 1) {
    long long quotient = n / l;
    r = n / quotient;
    // [l, r] 内 n / i 都等于 quotient
}
```

```
    answer += (r - l + 1) * quotient;
}
```

6.10 欧拉函数、莫比乌斯与积性函数

$\phi(n)=n\prod(1-1/p)$ 。求单点先分解；求前缀用线性筛。

莫比乌斯函数： $\mu(1)=1$ ，含平方因子为 0，否则质因数个数奇偶决定 ± 1 。常见反演：若 $f(n)=\sum_{d|n} g(d)$ ，则 $g(n)=\sum_{d|n} \mu(d)f(n/d)$ 。

```
vector<int> primes, lp(N), phi(N), mu(N);
phi[1] = mu[1] = 1;
for (int i = 2; i < N; ++i) {
    if (!lp[i]) lp[i] = i, primes.push_back(i), phi[i] = i - 1, mu[i] = -1;
    for (int p : primes) {
        if (1LL * i * p >= N) break;
        lp[i * p] = p;
        if (p == lp[i]) {
            phi[i * p] = phi[i] * p;
            mu[i * p] = 0;
            break;
        }
        phi[i * p] = phi[i] * (p - 1);
        mu[i * p] = -mu[i];
    }
}
```

6.11 组合数与阶乘（Binomial Coefficients）

6.11.1 5.1 质数模且 n 不大

预处理 $fac[i]$ 、 $ifac[i]$ ， $C(n,k)=fac[n]ifac[k]ifac[n-k]$ ，预处理 $O(N)$ ，查询 $O(1)$ 。

```
const long long MOD = 998244353;
vector<long long> fac(N), ifac(N);
fac[0] = 1;
for (int i = 1; i < N; ++i) fac[i] = fac[i - 1] * i % MOD;
ifac[N - 1] = qpow(fac[N - 1], MOD - 2, MOD);
for (int i = N - 1; i; --i) ifac[i - 1] = ifac[i] * i % MOD;
auto C = [&](int n, int k) -> long long {
    if (k < 0 || k > n) return 0;
    return fac[n] * ifac[k] % MOD * ifac[n - k] % MOD;
};
```

6.11.2 卢卡斯定理 (Lucas Theorem)

质数 p 下: $C(n, k) \equiv \prod C(n_i, k_i) \pmod{p}$, 其中 n_i, k_i 是 p 进制位。适用于 n 很大、 p 较小。

```
long long Csmall(long long n, long long k, long long p) {
    if (k < 0 || k > n) return 0;
    long long ans = 1;
    for (long long i = 1; i <= k; ++i) {
        ans = ans * (n - i + 1) % p;
        ans = ans * qpow(i, p - 2, p) % p;
    }
    return ans;
}

long long lucas(long long n, long long k, long long p) {
    if (!k) return 1;
    return Csmall(n % p, k % p, p) * lucas(n / p, k / p, p) % p;
}
```

6.11.3 非质数模

分解模数的质数幂, 计算每个质数幂下的阶乘 (跳过 p 因子), 再用 CRT 合并。实现复杂且容易错, 只有题目明确需要时使用。

```
// 合并  $x = a1 \pmod{m1}$ ,  $x = a2 \pmod{m2}$ , 允许模数不互质
bool merge_congruence(long long a1, long long m1,
                      long long a2, long long m2,
                      long long &a, long long &m) {
    long long x, y, g = exgcd(m1, m2, x, y);
    long long d = a2 - a1;
    if (d % g) return false;
    long long mod = m2 / g;
    long long t = (__int128)(d / g) * x % mod;
    if (t < 0) t += mod;
    m = m1 / g * m2;
    a = (a1 + (__int128)m1 * t) % m;
    if (a < 0) a += m;
    return true;
}
```

6.12 矩阵快速幂与线性递推 (Matrix Exponentiation and Linear Recurrence)

线性递推 $f(n)=c_1 f(n-1)+\dots+c_k f(n-k)$ 可转为 $k\times k$ 矩阵快速幂, 复杂度 $O(k^3 \log n)$; Fibonacci 为 2×2 。矩阵乘法注意模乘溢出。

```
struct Mat {
    int n; vector<vector<long long>> a;
    Mat(int n, bool ident = false): n(n), a(n, vector<long long>(n)) {
        if (ident) for (int i = 0; i < n; ++i) a[i][i] = 1;
    }
};

Mat operator*(const Mat& A, const Mat& B) {
    Mat C(A.n);
    for (int i = 0; i < A.n; ++i)
        for (int k = 0; k < A.n; ++k) if (A.a[i][k])
            for (int j = 0; j < A.n; ++j)
                C.a[i][j] = (C.a[i][j] + (__int128)A.a[i][k] * B.a[k][j]) %
MOD;
    return C;
}

Mat mpow(Mat a, long long e) {
    Mat r(a.n, true);
    while (e) { if (e & 1) r = r * a; a = a * a; e >>= 1; }
    return r;
}
```

6.13 容斥与组合计数

全集计数减去至少一个坏条件。 $n\leq 20$ 时枚举子集:

$ans = \sum (-1)^{|S|} \text{count}(\text{同时满足 } S \text{ 中条件})$ 。

条件独立/按质因子分解时可用容斥; 条件数量大时优先找补集、莫比乌斯反演或 DP。

```
long long ans = 0;
for (int mask = 0; mask < (1 << m); ++mask) {
    int bits = __builtin_popcount((unsigned)mask);
    long long ways = count_satisfying(mask); // 同时满足 mask 中条件
    ans += (bits & 1) ? -ways : ways;
}
```

6.14 概率与期望

- 期望线性： $E[X+Y]=E[X]+E[Y]$ ，即使不独立也成立。
- 终止状态递推： $E[\text{state}]=1+\sum p_i E[\text{next}_i]$ ；若有自环移项并检查系数非零。
- 取模概率需要模逆；分母与模数不互质时不能直接除。

```
double expected_dag(int u) {
    if (vis[u]) return E[u];
    vis[u] = true;
    E[u] = 1.0; // 走一步
    for (auto [v, p] : transitions[u]) E[u] += p * expected_dag(v);
    return E[u];
}
```

6.15 常见数列与技巧

- 等差/等比求和、前缀异或（周期 4）、 $1 \text{ xor } \dots \text{ xor } n$ 。
- $\text{gcd}(a,b)=\text{gcd}(a,b-a)$ ； $\text{lcm}(a,b)=a/\text{gcd}(a,b)*b$ 。
- 斐波那契 gcd: $\text{gcd}(F_m, F_n)=F_{\text{gcd}(m,n)}$ （用于识别构造题）。
- 反复应用 $x \rightarrow x/\text{gcd}(x,a)$ 时，质因数分解通常比模拟更快。

```
long long xor_prefix(long long n) {
    switch (n & 3) {
        case 0: return n;
        case 1: return 1;
        case 2: return n + 1;
        default: return 0;
    }
}

pair<long long, long long> fib(long long n) { // (F_n, F_{n+1})
    if (!n) return {0, 1};
    auto [a, b] = fib(n >> 1);
    long long c = a * (2 * b - a);
    long long d = a * a + b * b;
    return (n & 1) ? pair{d, c + d} : pair{c, d};
}
```


6.16 生成函数基础

把数列 a_n 视为形式幂级数 $A(x) = \sum a_n x^n$ 。加法对应逐项相加，乘法对应卷积； $1/(1-x)$ 是全 1 数列， $1/(1-x)^k$ 的系数是 $C(n+k-1, k-1)$ 。小规模直接卷积，规模大时换 NTT。

```
using Poly = vector<long long>;
Poly multiply(const Poly& a, const Poly& b, long long MOD) {
    Poly c(a.size() + b.size() - 1);
    for (int i = 0; i < (int)a.size(); ++i)
        for (int j = 0; j < (int)b.size(); ++j)
            c[i + j] = (c[i + j] + a[i] * b[j]) % MOD;
    return c;
}

// (1 + x + ... + x^m)^k 的系数: 重复卷积或用 NTT 加速
Poly ways(Poly base, int k, long long MOD) {
    Poly ans{1};
    while(k) {
        if (k & 1) ans = multiply(ans, base, MOD);
        k >>= 1;
        if (k) base = multiply(base, base, MOD);
    }
    return ans;
}
```

6.17 Burnside 引理与 Pólya 计数定理 (Burnside's Lemma and Pólya Enumeration Theorem)

Burnside 引理：轨道数等于所有群作用下不动点数的平均值。对“颜色数为 c 、置换 p 作用于位置”的染色问题，不动点数为 $c^{\text{cycles}(p)}$ ；模数需要能除以群大小。

```
long long mod_pow(long long a, long long e, long long mod) {
    long long r = 1;
    for (; e; e >>= 1, a = a * a % mod) if (e & 1) r = r * a % mod;
    return r;
}

long long burnside(const vector<vector<int>>& group, int colors, long long MOD) {
    long long sum = 0;
    for (const auto& perm : group) {
        int n = perm.size(), cycles = 0;
        vector<char> vis(n);
        for (int i = 0; i < n; ++i) if (!vis[i]) {
```

```

        ++cycles;
        for (int u = i; !vis[u]; u = perm[u]) vis[u] = true;
    }
    sum = (sum + mod_pow(colors, cycles, MOD)) % MOD;
}
return sum * mod_pow(group.size(), MOD - 2, MOD) % MOD;
}

```

6.18 Prüfer 序列与矩阵树定理 (Prüfer Sequence and Matrix-Tree Theorem)

Prüfer 序列长度为 $n-2$ ，每棵标号树与一个序列一一对应；度数满足 $\deg[v]=\text{出现次数}+1$ 。矩阵树定理把拉普拉斯矩阵删去一行一列后的行列式作为生成树数量。

```

vector<int> prufer_encode(const vector<vector<int>>& g) {
    int n = g.size() - 1;
    vector<int> deg(n + 1), code;
    priority_queue<int, vector<int>, greater<int>> leaves;
    for (int u = 1; u <= n; ++u) {
        deg[u] = g[u].size();
        if (deg[u] == 1) leaves.push(u);
    }
    for (int step = 0; step < n - 2; ++step) {
        int leaf = leaves.top(); leaves.pop();
        int v = 0;
        for (int x : g[leaf]) if (deg[x]) { v = x; break; }
        code.push_back(v);
        if (--deg[v] == 1) leaves.push(v);
        deg[leaf] = 0;
    }
    return code;
}

vector<pair<int, int>> prufer_decode(const vector<int>& code) {
    int n = code.size() + 2;
    vector<int> deg(n + 1, 1);
    for (int x : code) ++deg[x];
    priority_queue<int, vector<int>, greater<int>> leaves;
    for (int i = 1; i <= n; ++i) if (deg[i] == 1) leaves.push(i);
    vector<pair<int, int>> edges;
    for (int x : code) {
        int u = leaves.top(); leaves.pop(); edges.push_back({u, x});
        if (--deg[x] == 1) leaves.push(x);
    }
}

```

```

    }
    edges.push_back({leaves.top(), leaves.top()});
    int u = leaves.top(); leaves.pop(); int v = leaves.top();
    edges.back() = {u, v};
    return edges;
}

```

```

long long tree_count(vector<vector<long long>> lap, long long MOD) {
    int n = lap.size(); long long det = 1;
    for (int col = 0; col < n; ++col) {
        int pivot = col;
        while (pivot < n && lap[pivot][col] % MOD == 0) ++pivot;
        if (pivot == n) return 0;
        if (pivot != col) swap(lap[pivot], lap[col]), det = (MOD - det) % MOD;
        det = det * (lap[col][col] % MOD + MOD) % MOD;
        long long inv = mod_pow((lap[col][col] % MOD + MOD) % MOD, MOD - 2,
MOD);
        for (int i = col + 1; i < n; ++i) {
            long long factor = lap[i][col] % MOD * inv % MOD;
            for (int j = col; j < n; ++j)
                lap[i][j] = (lap[i][j] - factor * lap[col][j]) % MOD;
        }
    }
    return (det + MOD) % MOD;
}
// 调用时传入删除一行一列后的拉普拉斯矩阵。

```

6.19 二次剩余与 Tonelli - Shanks 算法 (Quadratic Residues and Tonelli-Shanks Algorithm)

Legendre 符号判断 a 是否为模奇素数 p 的二次剩余；Tonelli-Shanks 在存在平方根时以 $O(\log^2 p)$ 求出一个根。

```

long long legendre(long long a, long long p) {
    a %= p; if (a < 0) a += p;
    if (!a) return 0;
    long long x = mod_pow(a, (p - 1) / 2, p);
    return x == 1 ? 1 : -1;
}

long long tonelli_shanks(long long n, long long p) {
    n %= p; if (!n) return 0;
    if (p == 2) return n;

```

```

if (legendre(n, p) != 1) return -1;
if (p % 4 == 3) return mod_pow(n, (p + 1) / 4, p);
long long q = p - 1, s = 0;
while (!(q & 1)) q >>= 1, ++s;
long long z = 2;
while (legendre(z, p) != -1) ++z;
long long c = mod_pow(z, q, p), x = mod_pow(n, (q + 1) / 2, p);
long long t = mod_pow(n, q, p), m = s;
while (t != 1) {
    long long i = 1, tt = t * t % p;
    while (tt != 1) tt = tt * tt % p, ++i;
    long long b = mod_pow(c, 1LL << (m - i - 1), p);
    x = x * b % p; c = b * b % p; t = t * c % p; m = i;
}
return x;
}

```

费马小定理: $a^{(p-1)} \equiv 1 \pmod{p}$ (p 质数且 $p \nmid a$), 逆元为 $a^{(p-2)}$ 。欧拉定理推广到 $\gcd(a, m) = 1$: $a^{\phi(m)} \equiv 1 \pmod{m}$ 。

6.20 置换与循环

置换分解成不相交环；重复应用 k 次时每个位置在所属环内移动 $k \bmod \text{len}$ 。求置换逆、循环节、最少交换次数都从环分解出发。

```

vector<int> vis(n), result(n);
for (int i = 0; i < n; ++i) if (!vis[i]) {
    vector<int> cyc;
    for (int u = i; !vis[u]; u = p[u]) vis[u] = 1, cyc.push_back(u);
    int m = cyc.size();
    for (int j = 0; j < m; ++j) result[cyc[j]] = cyc[(j + k) % m];
}

```

6.21 快速傅里叶变换与数论变换（了解）

多项式卷积、计数合并、字符串匹配。NTT 要求模数为 $c \cdot 2^{k+1}$ （常用 998244353），每层蝶形乘原根；逆变换乘 n^{-1} 。只在卷积规模大于 $O(n^2)$ 时值得使用。

```

void ntt(vector<int>& a, bool inv) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1; for (; j & bit; bit >>= 1) j ^= bit; j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
}

```

```

    }
    for (int len = 2; len <= n; len <= 1) {
        int wlen = qpow(G, (MOD - 1) / len);
        if (inv) wlen = qpow(wlen, MOD - 2);
        for (int i = 0; i < n; i += len) {
            long long w = 1;
            for (int j = 0; j < len / 2; ++j) {
                int u = a[i + j], v = w * a[i + j + len / 2] % MOD;
                a[i + j] = (u + v) % MOD; a[i + j + len / 2] = (u - v + MOD) %
MOD;
                w = w * wlen % MOD;
            }
        }
    }
    if (inv) { long long ni = qpow(n, MOD - 2); for (int &x : a) x = x * ni %
MOD; }
}

```

6.22 高斯消元

解线性方程组、求秩、行列式。整数模数下用模逆；浮点版本选主元避免除以接近 0 的数。方程有自由变量时记录主元列并构造一组解。

```

int gauss(vector<vector<long long>> a) {
    int n = a.size(), m = a[0].size(), rank = 0;
    for (int col = 0; col < m && rank < n; ++col) {
        int pivot = rank;
        while (pivot < n && a[pivot][col] == 0) ++pivot;
        if (pivot == n) continue;
        swap(a[pivot], a[rank]);
        long long inv = qpow(a[rank][col], MOD - 2);
        for (int j = col; j < m; ++j) a[rank][j] = a[rank][j] * inv % MOD;
        for (int i = 0; i < n; ++i) if (i != rank && a[i][col]) {
            long long t = a[i][col];
            for (int j = col; j < m; ++j) a[i][j] = (a[i][j] - t * a[rank][j])
% MOD;
        }
        ++rank;
    }
    return rank;
}

```

6.23 数学与组合计数：参数、调用与改造

本节紧跟前面的基础模板。先看每个接口传什么，再看哪些不变量不能动，最后看如何替换维护信息或转移。

6.23.1 3.1 高级容斥与倍数容斥

集合容斥是“至少一个条件满足”：

$$|\cup A_i| = \sum |A_i| - \sum |A_i \cap A_j| + \dots$$

当条件是“能被 d 整除”，交集对应 lcm ；当条件是“所有因子贡献汇总”，常用莫比乌斯函数把“倍数和”反演为“恰好值”。

小范围条件最多二十个时可以枚举子集：

```
long long bad = 0;
for (int mask = 1; mask < (1 << m); ++mask) {
    long long l = 1;
    int bits = 0;
    bool overflow = false;
    for (int i = 0; i < m; ++i) if (mask >> i & 1) {
        ++bits;
        l = lcm_limit(l, a[i], n, overflow);
    }
    if (overflow || l > n) continue;
    long long ways = n / l;
    if (bits & 1) bad += ways;
    else bad -= ways;
}
```

测试样例：容斥的重复覆盖和最小公倍数溢出分支

下面约定每组输入 $n\ m$ ，后面给出 m 个除数，输出 $1..n$ 中不能被任何除数整除的数。

输入

```
4
1 1
2
10 2
2 3
30 3
2 3 5
100 2
```

4 6

输出

1

3

8

67

第二、三组检查交集是否按 lcm 加回；第四组的交集是 $\text{lcm}(4, 6)=12$ ，答案为 $100-25-16+8=67$ ，可以抓住把乘积当最小公倍数或符号写反的问题。

例题映射：题目问 $1..n$ 中不能被任何给定数整除的数，先求“至少被一个整除”的数量，再用 $n-\text{bad}$ 。若 m 很大，子集容斥不可用，检查数值范围后换莫比乌斯或筛法。

6.23.2 3.2 卢卡斯定理 (Lucas Theorem) 与组合数模质数

当模数 p 是质数、 n, m 很大但 p 不大时：

$$C(n, m) \equiv C(\lfloor n/p \rfloor, \lfloor m/p \rfloor) \cdot C(n \bmod p, m \bmod p) \pmod{p}.$$

```
long long mod_pow(long long a, long long e, long long p) {
    long long r = 1 % p; // p=1 时返回 0，避免写死为 1。
    for (; e; e >>= 1, a = a * a % p) {
        // e 的最低位为 1，说明当前 a 是答案乘积的一部分。
        if (e & 1) r = r * a % p;
    }
    return r;
}

long long Csmall(long long n, long long k, long long p,
                 const vector<long long>& fac,
                 const vector<long long>& ifac) {
    // 这里 n、k 必须小于 p；Lucas 会把大数拆成 p 进制位后调用本函数。
    if (k < 0 || k > n) return 0;

    // C(n, k)=n!/(k!(n-k)!), ifac 是阶乘逆元。
    return fac[n] * ifac[k] % p * ifac[n-k] % p;
}

long long Lucas(long long n, long long k, long long p,
                 const vector<long long>& fac,
                 const vector<long long>& ifac) {
    // 递归处理 n、k 的最低一位 p 进制数字。
    if (k == 0) return 1;

    // 当前位组合数乘上更高位的组合数；模 p 的进位不合法时 Csmall 返回 0。
```

```

    return Csmall(n % p, k % p, p, fac, ifac)
        * Lucas(n / p, k / p, p, fac, ifac) % p;
}

```

测试样例：Lucas 的低位进位、 $k=0$ 和组合数为零

每组输入 $n\ m\ p$ ，其中 p 是质数，输出 $C(n,m) \bmod p$ 。

输入

4

0 0 2

5 2 3

7 1 5

10 3 5

输出

1

1

2

0

$C(10,3)$ 在模 5 下因为五进制低位发生不合法进位而为 0；如果只计算 $n\%p$ 和 $m\%p$ 而不递归高位，最后一组会错误。

例题：P3807 Lucas 定理。预处理 $fac[0..p-1]$ 和逆阶乘；每组输入 (n,m,p) 调 $Lucas(n,m,p,fac,ifac)$ 。注意这份模板默认 p 不大且是质数；模合数或 p 很大时要用 exLucas/质因数分解，不能盲套。

6.23.3 3.3 中国剩余定理与扩展中国剩余定理

要合并

$$x \equiv a \pmod{m}, \quad x \equiv b \pmod{n},$$

令 $x=a+m \times t$ ，得到 $m \times t \equiv b-a \pmod{n}$ 。只有 $\gcd(m,n)$ 整除 $b-a$ 时有解；用扩展欧几里得求 m/g 的逆元。

```

long long exgcd(long long a, long long b,
                long long& x, long long& y) {
    // 返回 gcd(a,b)，同时求出  $a \times x + b \times y = g$ 。
    if (!b) { x = 1; y = 0; return a; }
    long long x1, y1;
    long long g = exgcd(b, a % b, x1, y1);
    // 由  $b \times x1 + (a \% b) \times y1 = g$  反推  $a \times y1 + b \times (x1 - a/b \times y1) = g$ 。
}

```



```

    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}
bool merge(long long& a, long long& m,
           long long b, long long n) {
    // 旧方程:  $x \equiv a \pmod{m}$ ; 新方程:  $x \equiv b \pmod{n}$ 。
    // 合并后仍用 a,m 表示一个最小非负解和新的最小公倍数模数。
    long long x, y;
    long long g = exgcd(m, n, x, y);
    long long d = b - a;
    // 两同余式相容的充要条件:  $\gcd(m,n) \mid (b-a)$ 。
    if (d % g) return false;
    // 令  $x=a+m \cdot t$ , 得到  $(m/g) \cdot t \equiv (b-a)/g \pmod{n/g}$ 。
    // exgcd 返回的 x 正是  $m/g$  在模  $n/g$  意义下的逆元。
    __int128 t = (__int128)(d / g) * x;
    long long mod = n / g;
    t %= mod;
    if (t < 0) t += mod;
    // 中间乘法可能超过 long long, 所以 na/nm 使用 __int128。
    __int128 na = (__int128)a + (__int128)m * t;
    __int128 nm = (__int128)m / g * n;
    a = (long long)(na % nm);
    if (a < 0) a += (long long)nm;
    m = (long long)nm;
    return true;
}

```

测试样例：扩展中国剩余定理的互质、非互质和无解

下面每组先输入方程数量 k , 随后输入 $b \bmod$, 表示 $x \equiv b \pmod{\bmod}$; 输出最小非负解, 无解输出 No。

输入

```

4
1
3 5
2
2 3
3 5
2
1 4
5 6
2

```

```
0 2
1 4

输出
3
8
5
No
```

第三组模数不互质但相容，答案是 5；第四组 x 必须同时为偶数和模 4 余 1，无解。连续多组还要检查合并后的 a, m 是否在下一组重新初始化为 0, 1。

例题：P4777 扩展中国剩余定理。初始化 $a=0, m=1$ ，依次读入每组 $x \equiv b \pmod{\text{mod}}$ ，调用 `merge(a, m, b, mod)`，最后输出最小非负 a 。本题明确提醒中间乘法可能溢出，`__int128` 不能省；若题目保证有解也仍然建议保留无解判断。

6.23.4 3.4 高斯消元、秩、解空间与行列式

高斯消元的现场顺序：选主元 $\rightarrow$ 交换到当前行 $\rightarrow$ 消去其他行/下面的行 $\rightarrow$ 判断矛盾或自由元。模意义下每次除以主元要乘逆元；普通实数消元则要用 `long double` 并带 `eps`。

```
int gauss_mod(vector<vector<long long>> a, long long mod) {
    // a 是系数矩阵；如果是增广矩阵，最后一列还要单独解释为常数项。
    // 这里采用 Gauss-Jordan 消元：每个主元列都会把其他行一起消成 0。
    int n = a.size(), m = a[0].size(), row = 0;
    for (int col = 0; col < m && row < n; ++col) {
        int p = row; // 在当前列寻找主元行。
        for (int i = row; i < n; ++i)
            if (a[i][col] > a[p][col]) p = i;
        // 没有非零主元，说明这一列是自由列，换下一列。
        if (a[p][col] == 0) continue;
        swap(a[p], a[row]);
        // 模意义下的除法必须乘逆元；这里默认 mod 是质数。
        long long inv = mod_pow(a[row][col], mod - 2, mod);
        for (int j = col; j < m; ++j) a[row][j] = a[row][j] * inv % mod;
        for (int i = 0; i < n; ++i) if (i != row) {
            long long k = a[i][col];
            // 用当前主元行消掉其它行的这一列。
            for (int j = col; j < m; ++j)
                a[i][j] = (a[i][j] - k * a[row][j]) % mod;
        }
        ++row;
    }
    return row; // 秩：增广矩阵要额外判断矛盾行
}
```

行列式在消元时记录交换次数，答案乘所有主元；若模数不是质数，不能用费马逆元，可用整数版消元或先保证主元可逆。矩阵树定理则是构造拉普拉斯矩阵，删去任意一行一列，求余下行列式。

测试样例：模高斯消元的零秩、满秩和相关行

下面约定每组输入 n m 和一个 $n \times m$ 的系数矩阵，输出模意义下的秩；模数固定为 7。

```

输入
4
1 1
0
1 1
1
2 2
1 0
0 1
2 3
1 2 3
2 4 6

```

```

输出
0
1
2
1

```

第一组检查没有主元时不能强行增加秩；最后一组两行线性相关，能抓住没有取模、主元交换和消元方向错误。若把它接到增广矩阵上，还要另外加入“系数全零但常数非零”的无解行。

例题：P6178 Matrix-Tree 定理。无向图中 $L[i][i]$ 加边数， $L[i][j]$ 减边数；删掉最后一行和最后一列后求模行列式。不要把邻接矩阵直接拿来求行列式，也不要将有向图和无向图的拉普拉斯构造混用。

6.23.5 3.5 莫比乌斯反演

若题目给出

$$F(n) = \sum_{d|n} f(d),$$

则

$$f(n) = \sum_{d|n} \mu(d) F(n/d).$$

竞赛中常见等价形式是“统计 gcd 恰好为 **d**”：先统计所有 gcd 是 **d** 的倍数，再从大到小用 $f[d] -= f[k]$ 反演。

```
vector<int> mu, primes;
vector<char> is_comp;
void linear_sieve_mu(int n) {
    // mu[1]=1; mu[p]=-1; 有平方因子时 mu=0; 不同质因子乘积时符号交替。
    mu.assign(n + 1, 0);
    is_comp.assign(n + 1, false);
    mu[1] = 1;
    for (int i = 2; i <= n; ++i) {
        // i 没被标记说明是质数。
        if (!is_comp[i]) primes.push_back(i), mu[i] = -1;
        for (int p : primes) {
            if (1LL * i * p > n) break;
            is_comp[i * p] = true;
            if (i % p == 0) {
                // p 已经在 i 中出现，再乘一次就有平方因子。
                mu[i * p] = 0;
                break;
            }
            // p 与 i 互质: mu(i*p)=-mu(i)。
            mu[i * p] = -mu[i];
        }
    }
}
```

测试样例：莫比乌斯函数的平方因子和多测重建

下面约定每组输入 n ，输出 $\text{mu}[1..n]$ 。

输入

```
3
1
5
10
```

输出

```
1
1 -1 -1 0 -1
1 -1 -1 0 -1 1 -1 0 0 1
```

$\mu(4)=0$ 和 $\mu(8)=0$ 是平方因子分支；连续调用时 `primes` 必须清空，否则线性筛的全局状态会残留。若后续用它统计互质对，还要单独测试全相同数组和全为质数的数组。

例如统计数对 (i, j) 的 $\gcd$ 恰为 1：先用频率数组求 $\text{cnt}[d] =$ 两数都是 d 的倍数的对数，再从大到小令 $\text{exact}[d] = \text{cnt}[d] - \sum \text{exact}[2d], \text{exact}[3d], \dots$ ；也可按 μ 直接加权。看到“所有倍数/恰好 $\gcd$ /互质对”就优先想反演。

6.23.6 3.6 原根、Baby-step Giant-step 离散对数与扩展算法

离散对数题的目标是求最小 x 使 $a^x \equiv b \pmod{p}$ 。当模数是质数且 a 可逆，用 Baby-step Giant-step：设 $m = \lceil \sqrt{p} \rceil$ ，枚举 a^j 存表，再枚举 $b \cdot a^{-im}$ 查表，复杂度 $O(\sqrt{p})$ 。

```
long long bsgs(long long a, long long b, long long mod) {
    // 求最小非负 x, 使 a^x = b (mod mod)。
    // 基础版要求 gcd(a, mod)=1; 下面用费马小定理求逆元,
    // 因而最安全的使用场景是 mod 为质数且 a!=0。
    a %= mod; b %= mod;
    long long m = sqrtl(mod) + 1;
    unordered_map<long long, long long> baby;
    long long cur = 1;
    // Baby step: 保存 a^j -> j, j=0..m-1。
    for (long long j = 0; j < m; ++j) {
        // 保留最小 j, 保证找到的是更小的答案。
        baby.emplace(cur, j);
        cur = (__int128)cur * a % mod;
    }
    // Giant step 每次乘 a^{-m}, 所以第 i 次对应 b * a^{-im}。
    long long factor = mod_pow(mod_pow(a, m, mod), mod - 2, mod);
    cur = b;
    for (long long i = 0; i <= m; ++i) {
        auto it = baby.find(cur);
        if (it != baby.end()) {
            // cur=a^j 时, b * a^{-im}=a^j, 即 x=i*m+j。
            return i * m + it->second;
        }
        cur = (__int128)cur * factor % mod;
    }
    return -1;
}
```

测试样例：离散对数的零次幂、最小解和无解

下面每组输入 $a \ b \ p$ ，输出最小非负 x 使 $a^x \equiv b \pmod{p}$ ；这里的基础 BSGS 只使用质数模数且 a 可逆。

输入

```
4
2 1 5
2 3 5
3 1 7
2 3 7
```

输出

```
0
3
0
-1
```

第一、三组检查 $x=0$ 不能被漏掉；最后一组 3 不在 2 的幂循环中，必须输出 -1。如果把扩展 BSGS 也接入，再另加 $\gcd(a, p) \neq 1$ 的样例，不能把基础模板直接套过去。

原根判定先分解 $\phi(\text{mod})$ 的质因子，候选 g 满足 $g^{(\phi/q)} \neq 1$ 对所有质因子 q 成立。若 $\gcd(a, \text{mod}) \neq 1$ ，普通 BSGS 不能直接用，需要扩展 BSGS 先剥离 $\gcd$ 。练习可用原根/离散对数专题中的对应题目；题目若要求模 10^9 级，先确认是否为质数，不能看见“指数很大”就自动套 BSGS。

6.23.7 3.7 生成函数、数论变换、Burnside 引理与 Pólya 计数定理

普通生成函数把数列 a_n 写成 $A(x) = \sum a_n x^n$ 。卷积就是多项式乘法：

```
vector<long long> multiply(const vector<long long>& a,
                          const vector<long long>& b) {
    // a[i] / b[j] 分别是  $x^i / x^j$  的系数，卷积后下标一定是  $i+j$ 。
    vector<long long> c(a.size() + b.size() - 1);
    for (int i = 0; i < (int)a.size(); ++i)
        for (int j = 0; j < (int)b.size(); ++j)
            // 先乘后加；每次取模避免系数过大。
            c[i + j] = (c[i + j] + a[i] * b[j]) % MOD;
    return c;
}
```

测试样例：卷积下标、单位多项式和重复系数

下面约定输入两段系数，输出普通卷积结果。

输入

```
3
1
1
```

```

1
2
1 1
2
1 1
1 2 3
2
4 5

```

输出

```

1
1 2 1
4 13 22 15

```

第一组检查单位元;第二组检查中间系数累加;第三组检查卷积长度是 `a.size()+b.size()-1`, 不能把下标写成 `i+j+1`。

当次数达到 `1e5`, 把 `multiply` 换成 NTT; 模板题是 P3803 多项式乘法。生成函数题的调用重点是先确定“指数代表什么”: 物品数量、总和还是选取个数; 卷积下标错一位会导致整个答案错。

Burnside 计算群作用下的不同染色数:

$$A = \frac{1}{|G|} \sum_{g \in G} k^{\text{cycles}(g)}.$$

```

long long burnside(const vector<vector<int>>& perm,
                   long long colors, long long mod) {
    long long sum = 0;
    for (auto& p : perm) {
        // p[i] 表示置换把位置 i 送到 p[i]。
        vector<char> vis(p.size());
        int cycles = 0;
        for (int i = 0; i < (int)p.size(); ++i) if (!vis[i]) {
            ++cycles;
            // 沿置换反复跳转，走一圈就是一个循环。
            for (int u = i; !vis[u]; u = p[u]) vis[u] = 1;
        }
        // 一个置换固定的染色要求同一循环内颜色相同，故有 colors^cycles 种。
        sum = (sum + mod_pow(colors, cycles, mod)) % mod;
    }
    // Burnside: 不动点总数除以群大小；这里默认 mod 与群大小互质。
    return sum * mod_pow(perm.size(), mod - 2, mod) % mod;
}

```

测试样例：Burnside 的单位群、交换群和旋转群

下面用置换数组直接调用 `burnside`，颜色数取 2，模数取 1000000007。

置换组一

```
perm = {{0}}
```

应输出：2

置换组二

```
perm = {{0,1}, {1,0}}
```

应输出：3

置换组三

```
perm = {{0,1,2}, {1,2,0}, {2,0,1}}
```

应输出：4

三组分别对应一个位置、长度二的交换、长度三的旋转；如果循环数统计错，结果会分别偏离 2、3、4。多测调用时还要确认 `vis` 是每个置换重新创建，而不是沿用上一次。

Pólya 是把大量具体置换按循环类型分组并求和。若旋转/翻转群很小，直接枚举置换最稳；若有 n 边正多边形，旋转的循环数是 $\gcd(n, i)$ ，翻转要分奇偶单独数循环。先把“允许的等价变换”写清楚，不要把颜色置换也默认算进群。

6.23.8 3.8 二次剩余

题目问 $x^2 \equiv n \pmod{p}$ 且 p 为奇质数时，先用欧拉判别： $n^{(p-1)/2} \equiv 1$ 才可能有解； $n=0$ 单独处理。Tonelli-Shanks 在 $O(\log^2 p)$ 内求平方根。

```
long long tonelli_shanks(long long n, long long p) {
    // 返回一个 r, 使 r^2=n (mod p); 无解返回 -1。
    // 使用前确认 p 是奇素数; p=2 要单独写分支。
    n %= p;
    if (n == 0) return 0;

    // 欧拉判别: 非零二次剩余的 (p-1)/2 次幂必须为 1。
    if (mod_pow(n, (p - 1) / 2, p) != 1) return -1;

    // p%4==3 时有直接公式, 优先走短分支。
    if (p % 4 == 3) return mod_pow(n, (p + 1) / 4, p);

    // 把 p-1 写成 q*2^s, 其中 q 为奇数。
    long long q = p - 1; int s = 0;
    while ((q & 1) == 0) q >>= 1, ++s;

    // 找一个二次非剩余 z, 使 z^((p-1)/2)=-1。
```



```

long long z = 2;
while (mod_pow(z, (p - 1) / 2, p) != p - 1) ++z;
long long c = mod_pow(z, q, p);
long long x = mod_pow(n, (q + 1) / 2, p);
long long t = mod_pow(n, q, p);
int m = s;
while (t != 1) {
    int i = 1; long long tt = t * t % p;
    // 找最小 i, 使 t^(2^i)=1; 这一步保证下一轮 m 变小。
    while (tt != 1) tt = tt * tt % p, ++i;

    // 这里的指数是 2^(m-i-1), 不是 m-i-1。
    // p 为 long long 时 m 很小, 移位安全; 若改用更大整数,
    // 请改写为专门的幂指数计算, 不能直接套 1LL 左移。
    long long b = mod_pow(c, 1LL << (m - i - 1), p);
    x = x * b % p;
    t = t * b % p * b % p;
    c = b * b % p; m = i;
}
return min(x, p - x);
}

```

测试样例：二次剩余的零、平方剩余、非剩余

下面每组输入 n p , 输出模板规定的较小根；无根输出 -1 。

输入

4

0 7

1 7

2 7

3 7

输出

0

1

3

-1

2 在模 7 下的根是 3 和 4, 模板若输出较小根应为 3; 3 是非二次剩余, 必须在 Tonelli-Shanks 主循环前返回 -1 。若题目要求两个根, 则分别输出 r 和 $p-r$, 不要只输出一个根。

例题: P5491 [模板] 二次剩余。输出两个根时输出 r 和 $p-r$, 并按题目要求排序; 没有根输出 -1 。1LL << (m-i-1) 在指数大时可能溢出, 实际模板应改成 `mod_pow(c, mod_pow(2, m`

$-i-1, p-1), p)$ 或用安全的幂次表示。

6.23.9 3.9 Prüfer 序列与矩阵树

Prüfer 序列把一棵 n 个点的标号树编码成长度 $n-2$ 的序列。编码每次删除最小叶子并记录它的邻点；解码维护度数和最小叶子堆。最常用结论是：度数为 d_i 的树的数量为

$$\frac{(n-2)!}{\prod_i (d_i - 1)!}$$

矩阵树定理的用法在上面的高斯小节已经给出：从图构造拉普拉斯矩阵，删一行一列求行列式。例题：P6178 Matrix-Tree 定理；若题目给定度数序列求树数，直接用 Prüfer 度数公式，不要真的枚举树。

6.23.10 7.1 矩阵快速幂与线性递推

看到“初值固定、每项只依赖前几项、询问第 n 项且 n 很大”，把状态向量写成矩阵乘法。以 Fibonacci 为例：

$$\begin{pmatrix} F_{i+1} \\ F_i \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_i \\ F_{i-1} \end{pmatrix}.$$

```
struct Mat {
    long long a[2][2]{}; // a[i][j]: 第 i 个新状态对第 j 个旧状态的系数。
};

Mat operator*(const Mat& A, const Mat& B) {
    Mat C{}; // 矩阵乘法是累加，必须从全零矩阵开始。
    for (int i = 0; i < 2; ++i)
        for (int k = 0; k < 2; ++k)
            for (int j = 0; j < 2; ++j)
                // C=A*B; 对列向量来说先应用 B，再应用 A。
                C.a[i][j] = (C.a[i][j] + A.a[i][k] * B.a[k][j]) % MOD;
    return C;
}

Mat mpow(Mat a, long long e) {
    // r 是单位矩阵，代表“不做任何转移”。
    Mat r{{{1, 0}, {0, 1}}};
    for (; e; e >>= 1, a = a * a)
        // 当前二进制位为 1，就把这一段转移乘进答案。
        if (e & 1) r = r * a;
    return r;
}

long long fibonacci(long long n) {
    // F_0=0 单独处理；n>=1 时 base^(n-1) 的左上角就是 F_n。
```

```

    if (n == 0) return 0;
    Mat base{{{1, 1}, {1, 0}}};
    return (mpow(base, n - 1).a[0][0]) % MOD;
}

```

测试样例：矩阵快速幂的零次幂、初值和普通项

下面直接调用 Fibonacci 示例，输出 F_n ，其中 $F_0=0, F_1=1$ 。

输入

```

5
0
1
2
10
20

```

输出

```

0
1
1
55
6765

```

第一组检查 $n=0$ 的特判； $n=1$ 检查幂指数为零时单位矩阵不能写成零矩阵； $n=20$ 检查快速幂多次平方和乘法顺序。

例题：P1939 矩阵加速（数列）。先把题目的递推式写成“新状态由旧状态线性组合”，矩阵第一行就是递推系数；不要看到数列就默认 Fibonacci，初值和下标要按题目重新代入。矩阵乘法有方向， $\text{left} * \text{right}$ 表示先做右边转移、再做左边转移。

6.23.11 7.2 高斯消元求实数方程组

如果题目要求概率、坐标或实数解，增广矩阵每一行是一条方程。选绝对值最大的主元可以减少误差；没有主元的行是自由变量，最后要区分唯一解、无解和多解。

```

int gauss_real(vector<vector<long double>> a,
               vector<long double>& ans) {
    // a[i][0..m-1] 是系数，a[i][m] 是常数项；这里默认方程数=未知数数。
    // 返回 0=无解，1=唯一解，2=多解。
    int n = a.size(), m = n, row = 0;
    for (int col = 0; col < m && row < n; ++col) {
        int p = row;
        for (int i = row + 1; i < n; ++i)

```

```

        if (fabsl(a[i][col]) > fabsl(a[p][col])) p = i;
// 选绝对值最大的主元可以减小浮点误差。
if (fabsl(a[p][col]) < 1e-12L) continue;
swap(a[p], a[row]);
for (int i = 0; i < n; ++i) if (i != row) {
    long double k = a[i][col] / a[row][col];
    // Gauss-Jordan: 把主元列在所有其它行都消成 0。
    for (int j = col; j <= m; ++j)
        a[i][j] -= k * a[row][j];
}
++row;
}
for (int i = row; i < n; ++i) {
    bool all_zero = true;
    for (int j = 0; j < m; ++j)
        all_zero &= fabsl(a[i][j]) < 1e-12L;
    // 0=0 是多解的一部分; 0=c(c!=0) 是矛盾, 直接无解。
    if (all_zero && fabsl(a[i][m]) > 1e-12L) return 0; // 无解
}
// 主元个数小于未知数个数, 存在自由变量。
if (row < m) return 2; // 多解; 题目若保证唯一可忽略
ans.assign(m, 0);
for (int i = 0; i < n; ++i) {
    int pivot = 0;
    while (pivot < m && fabsl(a[i][pivot]) < 1e-12L) ++pivot;
    if (pivot < m) ans[pivot] = a[i][m] / a[i][pivot];
}
return 1; // 唯一解
}

```

测试样例：实数高斯消元的唯一解、无解和多解

每组输入两个未知数的两个方程，输出解的类型；唯一解时再输出 x, y 。

测试一

$x+y=2$

$x-y=0$

应为：唯一解， $x=1, y=1$

测试二

$x+y=1$

$2x+2y=3$

应为：无解

测试三

$$x+y=2$$

$$2x+2y=4$$

应为：多解

第二组检查“系数全零但常数非零”的矛盾行，第三组检查秩小于未知数个数。不能只测试唯一解，否则 $\text{row} < m$ 和无解判断很容易被抄掉。

例题：P3389 高斯消元法。输入的是 n 个方程和 n 个未知数，读入 $a[i][0..n]$ 后直接调用；输出保留题目要求的小数位。模意义下则把除法换成逆元，且只能在主元与模数互质时做。

7 数据结构

7.1 单调栈

用于求每个元素左/右边第一个比它大（或小）的元素。

```
// 求每个元素右边第一个比它大的元素的下标
vector<int> nge(n, -1); // Next Greater Element
stack<int> stk;
for (int i = 0; i < n; i++) {
    while (!stk.empty() && a[stk.top()] < a[i]) {
        nge[stk.top()] = i;
        stk.pop();
    }
    stk.push(i);
}
```

7.2 单调队列

用于滑动窗口求区间最值， $O(n)$ 。

```
// 滑动窗口大小为 k，求每个窗口的最大值
deque<int> dq; // 存下标，维护递减队列
for (int i = 0; i < n; i++) {
    while (!dq.empty() && dq.front() <= i - k)
        dq.pop_front();
    while (!dq.empty() && a[dq.back()] <= a[i])
        dq.pop_back();
    dq.push_back(i);
    if (i >= k - 1)
        cout << a[dq.front()] << " ";
}
```

7.3 并查集

- 合并必须是根合根：fa[find(u)] = find(v)，写 fa[u] = v 是假合并！
- 初始化必须 fa[i] = i，否则所有点默认在同一集合（fa 全为 0）。
- 统计连通块数量必须用 find(i)，不能直接看 fa[i]（可能没路径压缩）。

```
int fa[N], sz[N];

void init(int n) {
    iota(fa + 1, fa + n + 1, 1); // fa[i] = i
    fill(sz + 1, sz + n + 1, 1);
}

int find(int x) {
    return fa[x] == x ? x : fa[x] = find(fa[x]); // 路径压缩
}

void unite(int x, int y) {
    x = find(x); y = find(y);
    if (x == y) return;
    if (sz[x] < sz[y]) swap(x, y); // 按大小合并
    fa[y] = x;
    sz[x] += sz[y];
}

bool connected(int x, int y) {
    return find(x) == find(y);
}
```

7.4 字典树（Trie）

```
int trie[N][26], cnt[N], idx = 0;

void insert(const string& s) {
    int p = 0;
    for (char c : s) {
        int u = c - 'a';
        if (!trie[p][u]) trie[p][u] = ++idx;
        p = trie[p][u];
    }
}
```

```

    cnt[p]++;
}

int query(const string& s) {
    int p = 0;
    for (char c : s) {
        int u = c - 'a';
        if (!trie[p][u]) return 0;
        p = trie[p][u];
    }
    return cnt[p];
}

```

- 异或字典树：把数字按二进制从高位到低位插入，用于求最大异或值。每个节点只有 0/1 两个子节点。
- 前缀计数：在每个节点维护经过该节点的字符串数量，可以快速统计以某前缀开头的字符串个数。
- 删除操作：沿路径将计数减 1 即可（逻辑删除）。

7.4.1 异或字典树 (01-Trie)

将整数按二进制从高位到低位插入 Trie，用于求一组数中与给定数异或的最大值。时间复杂度 $O(30)$ 或 $O(60)$ （取决于值域）。

```

const int MAXBIT = 30; // int 范围用 30, long long 用 60
int ch[N * 32][2], tot = 0;

void insert(int x) {
    int p = 0;
    for (int i = MAXBIT; i >= 0; i--) {
        int bit = (x >> i) & 1;
        if (!ch[p][bit]) ch[p][bit] = ++tot;
        p = ch[p][bit];
    }
}

// 查询与 x 异或的最大值
int query_max_xor(int x) {
    int p = 0, res = 0;
    for (int i = MAXBIT; i >= 0; i--) {
        int bit = (x >> i) & 1;

```



```

        if (ch[p][bit ^ 1]) { // 贪心走不同的位
            res |= (1 << i);
            p = ch[p][bit ^ 1];
        } else {
            p = ch[p][bit];
        }
    }
    return res;
}

```

- 数组大小： N 个数，每个数最多 $\text{MAXBIT} + 1$ 个节点，总共需要 $N \times (\text{MAXBIT} + 1)$ 个节点。
- 如果数可能为负数，需要先加偏移量变成非负数再插入。
- 多测时记得清空 Trie: `memset(ch, 0, sizeof(int) * (tot + 1) * 2); tot = 0;`。

7.5 哈夫曼树 (Huffman Tree)

哈夫曼树是一种带权路径长度 (WPL) 最小的二叉树，用于最优前缀编码 (哈夫曼编码)。
贪心策略：每次取权值最小的两个节点合并。时间复杂度 $O(n \log n)$ 。

```

// 求最小带权路径长度 (WPL)
// 输入: n 个叶子节点的权值
int huffman_wpl(vector<int>& weights) {
    priority_queue<int, vector<int>, greater<int>> pq;
    for (int w : weights) pq.push(w);
    int wpl = 0;
    while (pq.size() > 1) {
        int a = pq.top(); pq.pop();
        int b = pq.top(); pq.pop();
        wpl += a + b;
        pq.push(a + b);
    }
    return wpl;
}

```

```

// 构建哈夫曼树并生成编码
struct HuffNode {
    int val, left, right;
};
vector<HuffNode> tree;

```

```
string codes[N]; // codes[i] 存第 i 个字符的编码

int build_huffman(vector<int>& weights) {
    // pq 存 (权值, 节点编号)
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
    for (int i = 0; i < (int)weights.size(); i++) {
        tree.push_back({weights[i], -1, -1});
        pq.push({weights[i], i});
    }
    while (pq.size() > 1) {
        auto [w1, id1] = pq.top(); pq.pop();
        auto [w2, id2] = pq.top(); pq.pop();
        int newId = tree.size();
        tree.push_back({w1 + w2, id1, id2});
        pq.push({w1 + w2, newId});
    }
    return pq.top().second; // 返回根节点编号
}

// DFS 生成编码
void gen_codes(int u, string& path, int n) {
    if (tree[u].left == -1 && tree[u].right == -1) {
        codes[u] = path; // u < n 时为叶子节点
        return;
    }
    if (tree[u].left != -1) { path += '0'; gen_codes(tree[u].left, path, n); path.
pop_back(); }
    if (tree[u].right != -1) { path += '1'; gen_codes(tree[u].right, path, n); path.
pop_back(); }
}
```

- 经典应用：合并果子问题——每次合并代价为两堆之和，求最小总代价，就是求 WPL。
- k 叉哈夫曼树：每次取 k 个最小的合并。注意如果 $(n - 1) \% (k - 1) \neq 0$ ，需要先补 $k - 1 - (n - 1) \% (k - 1)$ 个权值为 0 的虚拟节点。
- 哈夫曼编码是最优前缀编码，任何字符的编码都不是另一个字符编码的前缀。

7.6 树状数组 (BIT)

支持单点加、前缀和与区间和，单次操作 $O(\log n)$ 。下标从 1 开始；区间加 + 单点查询可直接用下面的差分写法。

```
struct Fenwick {
    int n;
    vector<long long> bit;

    Fenwick(int n = 0) { init(n); }
    void init(int n_) { n = n_; bit.assign(n + 1, 0); }

    // a[i] += delta (i 从 1 开始)
    void add(int i, long long delta) {
        for (; i <= n; i += i & -i) bit[i] += delta;
    }

    // a[1] + ... + a[i]
    long long sum(int i) const {
        long long res = 0;
        for (; i > 0; i -= i & -i) res += bit[i];
        return res;
    }

    long long range_sum(int l, int r) const {
        return sum(r) - sum(l - 1);
    }
};

// 用差分实现“区间加 + 单点查询”
Fenwick diff(n);
diff.add(l, val);
diff.add(r + 1, -val);
long long value_at_i = diff.sum(i);
```

树状数组下标从 1 开始！下标为 0 会导致 $i += i \& (-i)$ 死循环。若原数组是 0-indexed，通常整体向右平移一位。

7.7 线段树

下面是“区间加 + 区间和”模板，使用动态 `vector`，不依赖全局数组大小。需要维护最大值/最小值时，把 `pull`、`apply` 中的合并和区间贡献替换即可。

- 本模板的 Lazy 标记表示“整个区间还要加多少”，因此下传时要用 `+=`；区间赋值需要额外的赋值标记。
- `pushdown` 必须同时更新子节点的值和标记，然后清空父节点标记。
- 区间长度用节点自身的 $[l, r]$ ，不是查询的 $[L, R]$ ！

```
struct SegTree {
    using ll = long long;
    int n;
    vector<ll> tree, lazy;

    // 传入 1-indexed 数组 a (a[0] 忽略)
    void init(const vector<ll>& a) {
        n = (int)a.size() - 1;
        tree.assign(4 * n + 4, 0);
        lazy.assign(4 * n + 4, 0);
        if (n) build(1, 1, n, a);
    }

    void build(int u, int l, int r, const vector<ll>& a) {
        if (l == r) { tree[u] = a[l]; return; }
        int mid = (l + r) / 2;
        build(u * 2, l, mid, a);
        build(u * 2 + 1, mid + 1, r, a);
        pull(u);
    }

    void pull(int u) { tree[u] = tree[u * 2] + tree[u * 2 + 1]; }

    void apply(int u, int l, int r, ll val) {
        tree[u] += val * (r - l + 1);
        lazy[u] += val;
    }

    void push(int u, int l, int r) {
        if (!lazy[u] || l == r) return;
        int mid = (l + r) / 2;
```

```
    apply(u * 2, l, mid, lazy[u]);
    apply(u * 2 + 1, mid + 1, r, lazy[u]);
    lazy[u] = 0;
}

void add(int u, int l, int r, int L, int R, ll val) {
    if (L <= l && r <= R) {
        apply(u, l, r, val);
        return;
    }
    push(u, l, r);
    int mid = (l + r) / 2;
    if (L <= mid) add(u * 2, l, mid, L, R, val);
    if (R > mid) add(u * 2 + 1, mid + 1, r, L, R, val);
    pull(u);
}

ll query(int u, int l, int r, int L, int R) {
    if (L <= l && r <= R) return tree[u];
    push(u, l, r);
    int mid = (l + r) / 2;
    ll res = 0;
    if (L <= mid) res += query(u*2, l, mid, L, R);
    if (R > mid) res += query(u*2+1, mid+1, r, L, R);
    return res;
}

void add(int l, int r, ll val) { add(1, 1, n, l, r, val); }
ll query(int l, int r) { return query(1, 1, n, l, r); }
};
```

8 数据结构

8.1 STL 必会组件

需求	容器/算法	复杂度
动态数组	<code>vector</code>	末尾均摊 $O(1)$ ，随机访问 $O(1)$
双端队列	<code>deque</code>	两端 $O(1)$
栈/队列	<code>stack</code> / <code>queue</code>	$O(1)$
优先队列	<code>priority_queue</code>	插入/删除顶 $O(\log n)$
有序集合	<code>set</code> / <code>multiset</code>	增删查 $O(\log n)$
有序键值	<code>map</code>	$O(\log n)$
哈希键值	<code>unordered_map</code>	期望 $O(1)$ ，最坏可能退化
去重/计数	<code>sort</code> + <code>unique</code> / <code>map</code>	$O(n \log n)$

`set` 的 `lower_bound`、`priority_queue<pair<...>>` 的字典序比较必须熟练。

`multiset::erase(iterator)` 与 `erase(value)` 的差别也要注意。

```
vector<int> a = input;
sort(a.begin(), a.end());
a.erase(unique(a.begin(), a.end()), a.end());
priority_queue<pair<int, int>> pq;
for (int i = 0; i < n; ++i) pq.push({value[i], i});
while (!pq.empty()) {
    auto [value, id] = pq.top(); pq.pop();
    // 处理当前最大值；必要时用 version[id] 判断是否过期
}
```

8.2 并查集 (Disjoint Set Union, DSU)

维护若干不相交集合并，支持合并和查询连通性。路径压缩 + 按大小合并后，单次摊销 $O(\alpha(n))$ 。

可扩展：维护集合大小、权值和、到父亲的距离/奇偶、撤销栈。带权并查集常用于维护 `dist[x]-dist[y]=w` 或敌友关系。

```

struct DSU {
    vector<int> p, sz;
    DSU(int n): p(n + 1), sz(n + 1, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b) {
        a = find(a); b = find(b); if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a; sz[a] += sz[b]; return true;
    }
};

```

Kruskal、离线加边、连通块统计、最小生成树、带限制的图连通性都优先考虑 DSU。

8.3 树状数组 (Fenwick Tree)

支持单点加、前缀和、区间和， $O(\log n)$ ；空间 $O(n)$ 。通过差分可支持区间加/单点查；两棵树可支持区间加/区间和。下标必须从 1 开始。

```

struct BIT {
    int n; vector<long long> t;
    BIT(int n): n(n), t(n + 1) {}
    void add(int x, long long v) { for (; x <= n; x += x & -x) t[x] += v; }
    long long sum(int x) const { long long r = 0; for (; x; x -= x & -x) r += t[x]; return r; }
    long long range(int l, int r) const { return sum(r) - sum(l - 1); }
};

```

应用：逆序对、动态排名、离线矩形计数、二维偏序（配合 CDQ/排序）、第 k 个值（二进制提升）。

8.4 线段树

支持任意可合并区间信息。节点存区间答案，`push_up` 合并左右儿子；区间修改要有懒标记并在访问子节点前 `push_down`。

8.4.1 常见维护

- 区间和 + 区间加：标记加法，子节点和增加 $\text{tag} \times \text{len}$ 。
- 区间和 + 区间乘/加：标记为仿射变换 $x \rightarrow \text{mul} \times x + \text{add}$ ，下传时 $\text{add} = \text{add} \times \text{mul_parent} + \text{add_parent}$ 。
- 区间最值 + 区间加：维护最大/最小值及懒加。

- 区间赋值：赋值标记覆盖旧加法；标记优先级必须明确。
- 扫描线：维护每个离散小区间是否被覆盖，保存 `cover` 与 `len`。

复杂度：建树 $O(n)$ ，单次查询/修改 $O(\log n)$ 。只要操作满足结合律，可把节点信息设计成结构体。

```
struct SegTree {
    int n; vector<long long> sum, lazy;
    SegTree(int n): n(n), sum(4 * n + 4), lazy(4 * n + 4) {}
    void apply(int p, int l, int r, long long v) {
        sum[p] += v * (r - l + 1); lazy[p] += v;
    }
    void push(int p, int l, int r) {
        if (!lazy[p] || l == r) return;
        int m = (l + r) / 2;
        apply(p * 2, l, m, lazy[p]);
        apply(p * 2 + 1, m + 1, r, lazy[p]);
        lazy[p] = 0;
    }
    void add(int p, int l, int r, int ql, int qr, long long v) {
        if (ql <= l && r <= qr) return apply(p, l, r, v);
        push(p, l, r); int m = (l + r) / 2;
        if (ql <= m) add(p * 2, l, m, ql, qr, v);
        if (qr > m) add(p * 2 + 1, m + 1, r, ql, qr, v);
        sum[p] = sum[p * 2] + sum[p * 2 + 1];
    }
    long long query(int p, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return sum[p];
        push(p, l, r); int m = (l + r) / 2; long long ans = 0;
        if (ql <= m) ans += query(p * 2, l, m, ql, qr);
        if (qr > m) ans += query(p * 2 + 1, m + 1, r, ql, qr);
        return ans;
    }
};
```

8.4.2 线段树上的二分

维护区间和/最大值时，可从根向下找“第一个满足条件的位置”，每层只走一个孩子， $O(\log n)$ ，比外层二分 + 查询更快。

```
int first_at_least(int p, int l, int r, long long x) {
    if (tree[p] < x) return -1;
    if (l == r) return l;
    int m = (l + r) / 2;
    if (tree[p * 2] >= x) return first_at_least(p * 2, l, m, x);
```



```

    return first_at_least(p * 2 + 1, m + 1, r, x);
}

```

8.5 稀疏表 (Sparse Table, ST)

静态数组的幂等运算 (min/max/gcd/and/or) 预处理 $O(n \log n)$, 查询 $O(1)$:

$k = \text{floor}(\log_2(\text{len}))$, 答案为 `merge(st[k][l], st[k][r - 2^k + 1])`。不可用于 sum 这类非幂等运算 (重叠会重复计数)。

```

int K = __lg(n);
vector<vector<int>> st(K + 1, vector<int>(n));
st[0] = a;
for (int j = 1; j <= K; ++j)
    for (int i = 0; i + (1 << j) <= n; ++i)
        st[j][i] = min(st[j - 1][i], st[j - 1][i + (1 << (j - 1))]);
auto range_min = [&](int l, int r) {
    int k = __lg(r - l + 1);
    return min(st[k][l], st[k][r - (1 << k) + 1]);
};

```

8.6 堆与可并堆思想

`priority_queue` 用于每步取最小/最大。双堆维护中位数 (小根维护大半边, 大根维护小半边)。懒删除: 堆中保留旧值, 弹出时与当前版本/计数比对。

左偏树/可并堆支持两个堆合并, 合并复杂度 $O(\log n)$; 题目要求“合并集合并取最值”时考虑。斜堆实现更短但无严格最坏保证。

```

priority_queue<int> low; // 小的一半, 堆顶是最大值
priority_queue<int, vector<int>, greater<int>> high;
void add_number(int x) {
    if (low.empty() || x <= low.top()) low.push(x); else high.push(x);
    if (low.size() > high.size() + 1) high.push(low.top()), low.pop();
    if (high.size() > low.size()) low.push(high.top()), high.pop();
}
int median() { return low.top(); }

```

8.7 单调结构与差分约束

单调栈/队列见 01-基础与实现规范。双端队列还能实现 0-1 BFS: 边权 0 从队首加入, 边权 1 从队尾加入。

```

vector<int> dist(n, INF);
deque<int> q;
dist[s] = 0; q.push_front(s);
while (!q.empty()) {
    int u = q.front(); q.pop_front();
    for (auto [v, w] : g[u]) {
        if (dist[v] <= dist[u] + w) continue;
        dist[v] = dist[u] + w;
        if (w == 0) q.push_front(v);
        else q.push_back(v);
    }
}

```

8.8 字典树 (Trie, Prefix Tree)

字符集固定时数组儿子最快；动态字符集用 `map`。插入/查询长度 L 的字符串均为 $O(L)$ 。二进制 Trie 支持最大异或、区间内最大异或（结合持久化或离线前缀）。

```

struct BinaryTrie {
    int ch[200000][2], cnt[200000], nodes = 1;
    void insert(int x) {
        int u = 1; ++cnt[u];
        for (int b = 30; b >= 0; --b) {
            int bit = (x >> b) & 1;
            if (!ch[u][bit]) ch[u][bit] = ++nodes;
            u = ch[u][bit]; ++cnt[u];
        }
    }
    int max_xor(int x) {
        int u = 1, ans = 0;
        for (int b = 30; b >= 0; --b) {
            int bit = (x >> b) & 1;
            if (ch[u][bit ^ 1] && cnt[ch[u][bit ^ 1]])
                ans |= 1 << b, u = ch[u][bit ^ 1];
            else u = ch[u][bit];
        }
        return ans;
    }
};

```

8.9 可持久化数据结构

每次修改只复制从根到叶子的 $O(\log n)$ 个节点，旧版本仍可查询。主席树（可持久化权值线段树）用于静态区间第 k 小： $\text{root}[r] - \text{root}[l-1]$ 在树上同步下行， $O(\log V)$ 。注意节点池容量约为 版本数 $\times \log V$ 。

```
struct Node { int l = 0, r = 0, sum = 0; } tr[4000000];
int tot = 0;
int update(int old, int l, int r, int pos) {
    int now = ++tot; tr[now] = tr[old]; ++tr[now].sum;
    if (l != r) {
        int m = (l + r) / 2;
        if (pos <= m) tr[now].l = update(tr[old].l, l, m, pos);
        else tr[now].r = update(tr[old].r, m + 1, r, pos);
    }
    return now;
}
int kth(int u, int v, int l, int r, int k) {
    if (l == r) return l;
    int left_count = tr[tr[v].l].sum - tr[tr[u].l].sum;
    int m = (l + r) / 2;
    if (k <= left_count) return kth(tr[u].l, tr[v].l, l, m, k);
    return kth(tr[u].r, tr[v].r, m + 1, r, k - left_count);
}
```

8.10 分块与莫队算法（Mo's Algorithm）

8.10.1 块状数组

把数组分成 $\sqrt{n}$ 块，块内暴力、块间整块处理；区间加/区间和约 $O(\sqrt{n})$ 。适合操作复杂、难以线段树维护的信息。

```
int B = max(1, (int)sqrt(n));
vector<long long> a(n), tag((n + B - 1) / B);
void add(int l, int r, long long v) {
    int bl = l / B, br = r / B;
    if (bl == br) { for (int i = l; i <= r; ++i) a[i] += v; return; }
    for (int i = l; i < (bl + 1) * B; ++i) a[i] += v;
    for (int b = bl + 1; b < br; ++b) tag[b] += v;
    for (int i = br * B; i <= r; ++i) a[i] += v;
}
```

8.10.2 莫队

离线排序询问的左右端点，维护当前区间并增删元素；移动总复杂度约 $O((n+q)\sqrt{n})$ 。需要可逆的 `add/remove`，如频次、不同数个数、答案贡献。带修改莫队多一个时间维，块大小约 $n^{2/3}$ 。

```
int block;
struct Query { int l, r, id; };
sort(qs.begin(), qs.end(), [&](const Query& x, const Query& y) {
    int bx = x.l / block, by = y.l / block;
    if (bx != by) return bx < by;
    return (bx & 1) ? x.r > y.r : x.r < y.r;
});
int L = 0, R = -1, distinct = 0;
auto add = [&](int p) { if (++cnt[a[p]] == 1) ++distinct; };
auto remove = [&](int p) { if (--cnt[a[p]] == 0) --distinct; };
for (auto qu : qs) {
    while (L > qu.l) add(--L); while (R < qu.r) add(++R);
    while (L < qu.l) remove(L++); while (R > qu.r) remove(R--);
    answer[qu.id] = distinct;
}
```

8.11 平衡树思想（了解 Treap）

随机优先级的 BST，支持按键插入、删除、排名、第 k 小、分裂/合并，期望 $O(\log n)$ 。无序序列维护时使用 FHQ Treap，节点存子树大小、区间反转标记、区间和/最大值。需要严格稳定最坏复杂度时优先线段树或 `std::set`。

```
struct Node { int l, r, key, pri, sz; } tr[MAXN];
int root, tot;
int size(int u) { return u ? tr[u].sz : 0; }
void pull(int u) { tr[u].sz = size(tr[u].l) + size(tr[u].r) + 1; }
void split(int u, int key, int &x, int &y) {
    if (!u) return x = y = 0, void();
    if (tr[u].key <= key) split(tr[u].r, key, tr[u].r, y), x = u;
    else split(tr[u].l, key, x, tr[u].l), y = u;
    pull(u);
}
int merge(int x, int y) {
    if (!x || !y) return x ? x : y;
    if (tr[x].pri > tr[y].pri) return tr[x].r = merge(tr[x].r, y), pull(x), x;
    return tr[y].l = merge(x, tr[y].l), pull(y), y;
}
```

8.12 撤销与可回滚结构

不做路径压缩的 DSU 可用栈记录每次修改，支持回滚到快照 $O(\log n)$ 合并、 $O(1)$ 回滚；配合线段树分治时间解决“边在一段时间内存在”的动态连通性。

```
struct RollbackDSU {
    vector<int> p, sz; vector<pair<int, int>> hist;
    RollbackDSU(int n): p(n + 1), sz(n + 1, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) { while (x != p[x]) x = p[x]; return x; }
    int snapshot() { return hist.size(); }
    bool unite(int a, int b) {
        a = find(a); b = find(b); if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        hist.push_back({b, sz[a]}); p[b] = a; sz[a] += sz[b]; return true;
    }
    void rollback(int snap) {
        while ((int)hist.size() > snap) {
            auto [b, old] = hist.back(); hist.pop_back();
            sz[p[b]] = old; p[b] = b;
        }
    }
};
```

8.13 李超线段树 (Li Chao Segment Tree)

维护直线集合在离散/整数横坐标上的最值。每个节点保存当前区间中在中点最优的直线，递归把另一条直线送入可能胜出的子区间；插入和查询均为 $O(\log V)$ 。

```
struct LiChao {
    using ll = long long;
    struct Line {
        // 这份代码维护“最小值”；空直线用极大截距表示。
        // 若改成最大值，把空值和所有比较方向一起反过来。
        ll k = 0, b = (ll)4e18;
        ll get(ll x) const { return k * x + b; }
    };
    int L, R; // 允许查询的整数横坐标闭区间 [L,R]。
    vector<Line> tr; // tr[p]: 节点区间内当前保留下来的候选直线。
    LiChao(int L, int R): L(L), R(R), tr(4 * (R - L + 1)) {}

    void add_line(Line nw, int p, int l, int r) {
        // 先比较新线和当前线在端点/中点的优劣。
        int m = (l + r) / 2;
```

```

bool lef = nw.get(l) < tr[p].get(l);
bool mid = nw.get(m) < tr[p].get(m);

// 中点更优的线留在当前节点，旧线递归到可能赢的半边。
if (mid) swap(nw, tr[p]);
if (l == r) return;

// 两线在左端和中点的相对优劣发生变化，交点在左半。
if (lef != mid) add_line(nw, p * 2, l, m);
else add_line(nw, p * 2 + 1, m + 1, r);
}

// 对外接口：只需要传入斜率 k 和截距 b，不要手动传根节点参数。
void add_line(ll k, ll b) { add_line({k, b}, 1, L, R); }

ll query(ll x, int p, int l, int r) const {
    // x 所在路径上的每个节点都可能保存一条更优直线，必须全部比较。
    ll ans = tr[p].get(x);
    if (l == r) return ans;
    int m = (l + r) / 2;
    if (x <= m) return min(ans, query(x, p * 2, l, m));
    return min(ans, query(x, p * 2 + 1, m + 1, r));
}

// x 必须落在构造时给出的 [L,R] 内；横坐标不连续时先离散化查询点。
ll query(ll x) const { return query(x, 1, L, R); }
};

```

求最大值时把比较符号整体反向；若横坐标不是连续整数，先离散化所有查询点。

8.14 区间最值势能线段树 (Segment Tree Beats)

“区间取 min + 区间和/最大值”不能用普通懒标记，因为只有当前最大值会被压低。维护最大值、次大值、最大值出现次数和区间和；当 x 严格大于次大值时可以整段打标记。

```

struct SegBeats {
    using ll = long long;
    static constexpr ll NEG = -(1LL << 60);
    int n;
    vector<ll> sum;          // 区间和。
    vector<ll> mx;           // 区间最大值。
    vector<ll> second_mx;    // 严格小于最大值的次大值。
    vector<int> cnt_mx;      // 最大值出现次数。
    SegBeats(const vector<ll>& a): n((int)a.size() - 1),
        sum(4 * n + 4), mx(4 * n + 4), second_mx(4 * n + 4),
        cnt_mx(4 * n + 4) { build(1, 1, n, a); }

```

```

void pull(int p) {
    // 父节点只保存“最大值这一类”和“次大值这一类”，
    // 不需要保存整个儿子数组，因此 chmin 才能整段快速处理。
    sum[p] = sum[p * 2] + sum[p * 2 + 1];
    if (mx[p * 2] == mx[p * 2 + 1]) {
        mx[p] = mx[p * 2]; cnt_mx[p] = cnt_mx[p * 2] + cnt_mx[p * 2 + 1];
        second_mx[p] = max(second_mx[p * 2], second_mx[p * 2 + 1]);
    } else {
        int x = p * 2, y = p * 2 + 1;
        if (mx[x] < mx[y]) swap(x, y);
        mx[p] = mx[x]; cnt_mx[p] = cnt_mx[x];
        second_mx[p] = max(second_mx[x], mx[y]);
    }
}

void build(int p, int l, int r, const vector<ll>& a) {
    if (l == r) {
        // 叶子只有一个最大值，次大值设为负无穷。
        sum[p] = mx[p] = a[l]; second_mx[p] = NEG; cnt_mx[p] = 1;
        return;
    }
    int m = (l + r) / 2;
    build(p * 2, l, m, a); build(p * 2 + 1, m + 1, r, a); pull(p);
}

void apply_chmin(int p, ll x) {
    // 调用者保证 second_mx[p] < x < mx[p],
    // 因而只有 cnt_mx 个最大值会下降，sum 可以 O(1) 更新。
    if (x >= mx[p]) return;
    sum[p] -= (mx[p] - x) * cnt_mx[p];
    mx[p] = x;
}

void push(int p) {
    // 父节点的 mx 变小后，儿子的最大值可能仍比父亲大，
    // 把父亲的上界传给儿子即可，不需要递归到底。
    if (mx[p * 2] > mx[p]) apply_chmin(p * 2, mx[p]);
    if (mx[p * 2 + 1] > mx[p]) apply_chmin(p * 2 + 1, mx[p]);
}

void chmin(int p, int l, int r, int ql, int qr, ll x) {
    // 无交集或当前最大值已经不超过 x 时，操作没有影响。
    if (qr < l || r < ql || x >= mx[p]) return;
    // x 大于次大值时只有最大值受影响，触发势能线段树的核心优化。
    if (ql <= l && r <= qr && x > second_mx[p]) return apply_chmin(p, x);
    // 否则必须下传并继续递归：删掉这个条件会退化甚至超时。
    push(p); int m = (l + r) / 2;

```

```

        chmin(p * 2, l, m, ql, qr, x);
        chmin(p * 2 + 1, m + 1, r, ql, qr, x); pull(p);
    }
    ll query_sum(int p, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return sum[p];
        push(p); int m = (l + r) / 2;
        return query_sum(p * 2, l, m, ql, qr)
            + query_sum(p * 2 + 1, m + 1, r, ql, qr);
    }
    void chmin(int l, int r, ll x) { chmin(1, 1, n, l, r, x); }
    ll query_sum(int l, int r) { return query_sum(1, 1, n, l, r); }
};

```

8.15 高级数据结构：参数、调用与改造

本节紧跟前面的基础模板。先看每个接口传什么，再看哪些不变量不能动，最后看如何替换维护信息或转移。

8.15.1 2.1 高阶前缀和与多阶差分

看到“每次给区间加一个一次/二次多项式，最后问点值或前缀”时，普通差分不够。若对差分数组再差分，区间加常数会变成两个端点；加一次函数需要更高阶差分或维护多个系数。

最实用的现场做法是先把更新写成基函数，再分别维护系数的差分。比如区间 $[l, r]$ 加 $a \cdot i + b$ ，维护两个差分数组 da, db ：

```

vector<long long> da(n + 2), db(n + 2);
void range_add_linear(int l, int r, long long a, long long b) {
    da[l] += a; da[r + 1] -= a;
    db[l] += b; db[r + 1] -= b;
}
long long point_value(int i) {
    static long long ca = 0, cb = 0;
    // 按 i 从小到大扫描时更新 ca += da[i], cb += db[i]
    return ca * i + cb;
}

```

若询问的是区间和，则要再维护系数的前缀贡献，或者直接使用带懒标记的线段树。不要在考场临时猜公式：先写出单点贡献 $a \cdot i + b$ ，再求 $l..r$ 的和 $a \cdot (l+r) \cdot (r-l+1)/2 + b \cdot (r-l+1)$ ，检查端点和取模。

8.15.2 2.2 多懒标记与仿射标记线段树

区间操作同时有“乘法”和“加法”时，标记不是两个互相独立的数，而是函数：

$$x \mapsto ax + b.$$

先执行旧标记 (a_1, b_1) ，再执行新标记 (a_2, b_2) ，合成结果为：

$$a = a_2 a_1, \quad b = a_2 b_1 + b_2.$$

```
struct Tag {
    // 一个懒标记表示：把区间内每个 x 变成 mul * x + add。
    // 初始值必须是恒等变换 x -> x，而不是 0。
    long long mul = 1;
    long long add = 0;
};

struct Node {
    long long sum = 0; // 当前区间的和，已经包含本节点自己的 tag。
    Tag tag;           // 尚未下传给儿子的变换。
};

void apply(int p, int l, int r, long long mul, long long add) {
    // 先更新“区间答案”：区间和乘 mul，再额外加 add * 区间长度。
    tr[p].sum = (tr[p].sum * mul + add * (r - l + 1)) % MOD;

    // 原来的标记是 f(x)=old_mul*x+old_add,
    // 新操作是 g(x)=mul*x+add，实际顺序是 g(f(x))。
    // 合成后：mul'=mul*old_mul, add'=mul*old_add+add。
    tr[p].tag.mul = tr[p].tag.mul * mul % MOD;
    tr[p].tag.add = (tr[p].tag.add * mul + add) % MOD;
}

void push(int p, int l, int r) {
    // 叶子没有儿子，标记留在自己这里即可。
    if (l == r) return;
    int mid = (l + r) >> 1;
    auto [m, a] = tr[p].tag;

    // 只有非恒等标记才需要下传。
    if (m != 1 || a != 0) {
        apply(p << 1, l, mid, m, a);
        apply(p << 1 | 1, mid + 1, r, m, a);

        // 父亲的变换已经交给两个儿子，父亲恢复为恒等变换。
        tr[p].tag = {1, 0};
    }
}
```

```
}

```

例题可用 P3373 模板线段树 2：题目有区间乘、区间加、区间和。题目中的“乘法操作”调用 `apply(..., k, 0)`，“加法操作”调用 `apply(..., 1, k)`。最容易错的是标记合成顺序，以及 `add` 作用在整个区间长度上。

测试样例：仿射标记的合成顺序

下面采用 P3373 风格：**1 l r k** 表示乘法，**2 l r k** 表示加法，**3 l r** 表示区间和；每组第一行是 `n m MOD`，随后是数组和 `m` 个操作。

输入

```
4
1 4 100
5
3 1 1
1 1 1 3
2 1 1 4
3 1 1
3 5 1000000007
1 2 3
2 1 2 5
3 1 3
1 2 3 2
3 2 2
2 4 1000
1 1
1 1 1 2
2 1 1 3
1 1 1 4
3 1 1
3 3 1000
100 200 300
1 1 3 2
2 2 3 900
3 1 3
```

输出

```
5
19
16
14
20
1000
```

0

第三组必须按 $((x \times 2) + 3) \times 4$ 计算，结果为 20；如果把标记合成顺序写反，或者把加法只加一次而不是乘区间长度，边界组和第三组都会暴露问题。

8.15.3 2.3 线段树上二分

当每个节点维护一个可合并的“容量/最大值/最小值”，并且可以判断“左子树是否存在答案”，就可以在线段树上二分，复杂度 $O(\log n)$ 。

```
int first_at_least(int p, int l, int r, long long x) {
    if (tr[p].mx < x) return -1;
    if (l == r) return l;
    push(p, l, r);
    int mid = (l + r) >> 1;
    int ans = first_at_least(p << 1, l, mid, x);
    if (ans != -1) return ans;
    return first_at_least(p << 1 | 1, mid + 1, r, x);
}
```

如果要找“第一个前缀和至少为 k ”，节点维护区间和且所有值非负；如果存在负数，不能用前缀和单调性，必须换树套树、平衡树或题目特定结构。例题可练 P3372 模板线段树 1 的改造版：每次单点加、询问第一个达到阈值的位置。

8.15.4 2.4 莫队、带修改莫队与回滚莫队

莫队适用于离线区间询问：移动 L/R 一格的代价很小，答案可以增量维护。普通莫队排序 $(l/block, r)$ ；带修改时再加时间 t ，状态是 (L, R, T) 。

```
// t 是时间维：这个询问发生以前，数组已经执行了多少次修改。
struct Query { int l, r, t, id; };

// oldv/newv 必须在读入时保存；处理询问时不能再从当前数组猜 oldv。
struct Change { int pos, oldv, newv; };

// 当前窗口是 [L,R]，当前已经应用到第 T 次修改。
// 初始设置为 L=1,R=0，表示窗口为空。
int L = 1, R = 0, T = 0;

// 这两个函数是“题目答案维护器”，不是莫队本身。
// 例如统计不同颜色：add 时 freq[a[p]] 从 0 变 1 就 ++answer,
// del 时从 1 变 0 就 --answer；统计频率平方和则要先减旧贡献再加新贡献。
void add_pos(int p) { /* 把 a[p] 加入当前答案 */ }
void del_pos(int p) { /* 从当前答案删除 a[p] */ }
```

```

void apply_change(const Change& c, bool forward) {
    // forward=true: 把 oldv 改成 newv; false: 撤销为 oldv。
    int v = forward ? c.newv : c.oldv;

    // 只有修改点位于当前区间时，它才会影响答案。
    // 顺序必须是“删除旧值 -> 改数组 -> 加入新值”。
    if (L <= c.pos && c.pos <= R) del_pos(c.pos);
    a[c.pos] = v;
    if (L <= c.pos && c.pos <= R) add_pos(c.pos);
}

// 典型的带修改莫队处理流程：
// 1. 先按 (l 所在块, r 所在块, t) 排序；
// 2. 把当前 L/R/T 一格一格移动到询问要求的位置；
// 3. 每次移动都调用同一套 add/del/apply_change，答案逻辑无需重复编写。
void process_query(Query q) {
    while (T < q.t) apply_change(changes[++T], true);
    while (T > q.t) apply_change(changes[T--], false);
    while (L > q.l) add_pos(--L);
    while (R < q.r) add_pos(++R);
    while (L < q.l) del_pos(L--);
    while (R > q.r) del_pos(R--);
    answer[q.id] = current_answer;
}

```

例题：P1903 数颜色。操作 **R P C** 是修改，**Q L R** 是询问。读入时给每个询问记录“它之前发生了多少个修改”作为 **t**；处理某个询问前先移动 **T**，修改覆盖当前区间时先删除旧值再加入新值。若只会加不支持删除，普通莫队不适用；若要求在线，也不能直接套莫队。

测试样例：带修改莫队的区间、时间双向移动

下面采用 P1903 的输入格式，每组第一行是 **n m**，数组后是 **Q l r** 或 **R p c**；查询输出区间内不同颜色数。

输入

```

2
1 4
5
Q 1 1
R 1 7
Q 1 1
Q 1 1
5 7

```

```

1 2 1 3 2
Q 1 5
R 2 3
Q 1 3
R 1 2
Q 1 2
Q 2 5
Q 1 5

```

输出

```

1
1
1
3
2
2
3
3

```

第一组检查修改点正好在当前区间内且同一查询重复；第二组要求时间前进到 2，并且查询区间来回变化。最容易抄错的是没有按“删除旧值、修改数组、加入新值”的顺序处理修改。回滚莫队适合“左端点块内、右端点可移动，修改只需撤销”的场景。核心是保存操作日志，不要真的把整个数组复制一份；每次离开一个块时回滚到块开始的快照。

8.15.5 2.5 三维偏序的 CDQ 分治与动态规划优化

CDQ 解决的不是“任意三维查询”，而是一个维度分治、另一个维度排序、剩余维度用树状数组/线段树合并。经典三维偏序：统计满足 $x_j \leq x_i, y_j \leq y_i, z_j \leq z_i$ 的点数。

```

// 这里给的是“统计三维偏序”的可改造骨架：
// 对每个点 i，统计有多少个点 j 满足  $x_j \leq x_i, y_j \leq y_i, z_j \leq z_i$ 。
// 调用前：按  $(x, y, z)$  排序、把完全相同的点合并，z 压缩成 1..Z。
struct Point {
    int x, y, z;
    int cnt = 1; // 完全相同点的个数；合并重复点后使用。
    long long ans = 0; // 不含自己这一组的答案。
};

struct Fenwick {
    vector<int> t;
    Fenwick(int n): t(n + 1) {}

    // 在压缩坐标 x 上加入 v 个点。
    void add(int x, int v) {

```

```

        for (; x < (int)t.size(); x += x & -x) t[x] += v;
    }

    // 返回  $z \leq x$  的点数。
    int sum(int x) const {
        int res = 0;
        for (; x > 0; x -= x & -x) res += t[x];
        return res;
    }
};

Fenwick bit(MAX_Z);

void cdq(vector<Point>& a, int l, int r) {
    // a 的“分治下标”仍然按 x 分组；递归返回时区间内部会按 y 有序。
    if (l == r) return;
    int m = (l + r) >> 1;

    // 先完成左半内部和右半内部的贡献。
    cdq(a, l, m);
    cdq(a, m + 1, r);

    // 此时 a[l..m]、a[m+1..r] 各自已经按 y 排序。
    // 左半只能贡献给右半，不能把右半反过来影响左半。
    int i = l;
    for (int j = m + 1; j <= r; ++j) {
        while (i <= m && a[i].y <= a[j].y) {
            // BIT 的下标是 z；加入左半中同时满足 x、y 限制的点。
            bit.add(a[i].z, a[i].cnt);
            ++i;
        }
        // BIT 前缀和再筛选  $z \leq a[j].z$ ，得到三维都不超过当前点的数量。
        a[j].ans += bit.sum(a[j].z);
    }

    // 必须撤销本次加入，否则别的分治区间会错误继承左半贡献。
    for (int k = l; k < i; ++k) bit.add(a[k].z, -a[k].cnt);

    // 合并为按 y 有序，供父层用双指针处理第二维。
    inplace_merge(a.begin() + l, a.begin() + m + 1,
                  a.begin() + r + 1,
                  [](const Point& A, const Point& B) {
                      return A.y < B.y;
                  });
}

```

```
}

```

使用时先离散化 z ，按 x 排序，把相同 (x, y, z) 合并并记录权值。例题：P3810 三维偏序。题目中的每个点就是模板的 `Point`，查询“左下后方点数量”就是 `bit.sum(z)`；重复点必须先合并，否则会漏算相等坐标。

测试样例：三维偏序的相等坐标与重复点

下面约定每组输入 n 个点，输出每个点的“包含自身、满足 $x_j \leq x_i, y_j \leq y_i, z_j \leq z_i$ 的点数”，顺序按输入编号输出。

输入

```
3
1
1 1 1
4
1 1 1
2 2 2
2 1 2
1 2 1
4
1 1 1
1 1 1
1 1 1
2 2 2
```

输出

```
1
1 4 2 2
3 3 3 4
```

第二组同时包含三个坐标方向上的相等关系；第三组专门检查完全相同点是否先合并并按 `cnt` 传播。若把比较条件写成严格小于，第二、三组都会错误。

CDQ 优化 DP 的识别句式是“转移来自满足下标/坐标偏序的前面状态”，把 `dp[j]` 看成事件，在 CDQ 合并阶段把左半的贡献加入数据结构，更新右半。若转移只依赖一维单调队列，不要为了 CDQ 增加复杂度。

8.15.6 2.6 可持久化字典树 (Persistent Trie)

当询问是“区间 $[l, r]$ 中与 x 异或最大/第 k 大”，前缀异或 Trie 的每个版本保存 $1..i$ 的所有数。区间查询用版本 `root[r]` 减去 `root[l-1]` 的子树大小。

```
const int LOG = 30; // 如果 a[i] 可能达到 2^31 以上，要提高到 60。
```

```
struct Node {
    int ch[2]{}; // 当前二进制位选择 0/1 后的儿子节点编号。
    int cnt = 0; // 该节点子树中数字的数量。
} tr[MAXNODE];

int tot = 0; // 已经使用的节点数，0 号节点表示空树。
int root[MAXN]; // root[i] 是前 i 个数构成的版本。

int insert(int old, int bit, int x) {
    // “持久化”的关键：复制旧根到新节点，只修改本次路径。
    int now = ++tot;
    tr[now] = tr[old];
    ++tr[now].cnt;

    // bit < 0 表示所有二进制位都处理完了。
    if (bit < 0) return now;

    int b = (x >> bit) & 1;
    // 旧版本的另一条分支仍然复用；当前分支递归创建新节点。
    tr[now].ch[b] = insert(tr[old].ch[b], bit - 1, x);
    return now;
}

int max_xor(int left_root, int right_root, int bit, int x) {
    // 两个版本相减：root[r] 的数量 - root[l-1] 的数量，正好保留 [l,r]。
    if (bit < 0) return 0;

    // 为了让异或结果当前位为 1，优先走和 x 当前位相反的分支。
    int want = ((x >> bit) & 1) ^ 1;
    int a = tr[tr[right_root].ch[want]].cnt
            - tr[tr[left_root].ch[want]].cnt;

    if (a > 0) {
        // 目标分支在区间中存在，贪心保留这一位的 1。
        return (1 << bit) | max_xor(tr[left_root].ch[want],
                                     tr[right_root].ch[want], bit - 1, x);
    }

    // 目标分支为空，只能走相同位；这一位贡献 0。
    int same = want ^ 1;
    return max_xor(tr[left_root].ch[same],
                   tr[right_root].ch[same], bit - 1, x);
}
```


调用时 `root[i] = insert(root[i-1], LOG, a[i])`，查询 `[l,r]` 用 `max_xor(root[l-1], root[r], LOG, x)`。注意节点数约为 $n * (LOG+1)$ ，不能只按 $4 * n$ 开。练习可从数据结构题单中的可持久化 Trie 开始，遇到“区间 + 异或最大”时不要用普通 Trie，因为普通 Trie 不能删除左端前缀。

测试样例：版本相减和区间端点

下面每组输入 n q 、数组，随后每个查询为 l r x ，输出区间 `[l,r]` 中某个数与 x 的最大异或值。

输入

```
3
1 2
0
1 1 0
1 1 7
4 4
1 2 4 8
1 4 0
1 4 7
2 3 3
3 3 0
3 2
5 5 1
1 2 1
2 3 7
```

输出

```
0
7
8
15
7
4
4
6
```

第一组检查单版本和零值；第二组检查 `[1,n]`、内部区间和单点；第三组检查重复值以及 `root[r]-root[l-1]` 的左端点是否写错。

8.15.7 2.10 李超线段树：整条直线与线段插入

维护若干直线 $y=kx+b$ ，询问某个整数 x 的最大/最小值，且插入和询问在线时，用李超树。与凸包不同，直线可以任意交叉；每个节点只保存当前区间中在某些位置更优的直线。

```

struct Line {
    long long k = 0;
    long long b = -(1LL << 60); // 最大值模板的负无穷空直线。

    // 题目数据若  $k \times x$  可能超过 long long, 要改成 __int128 比较。
    long long get(long long x) const { return k * x + b; }
};
Line tree[4 * MAXX];

void add_line(Line nw, int p, int l, int r) {
    // 节点 p 负责整数横坐标 [l,r], tree[p] 是当前候选直线。
    int m = (l + r) >> 1;

    // 比较新旧直线在左端点、中点的优劣。
    bool lef = nw.get(l) > tree[p].get(l);
    bool mid = nw.get(m) > tree[p].get(m);

    // 中点处更优的直线必须留在当前节点; 被换下来的直线只可能在某一侧更优。
    if (mid) swap(nw, tree[p]);
    if (l == r) return;

    // 如果两条线在左端和中点的优劣不同, 交点在左半, 否则在右半。
    if (lef != mid) add_line(nw, p << 1, l, m);
    else add_line(nw, p << 1 | 1, m + 1, r);
}

long long query(int x, int p, int l, int r) {
    // 答案必须比较沿根到 x 所在叶子的每一个节点保存的直线。
    long long res = tree[p].get(x);
    if (l == r) return res;
    int m = (l + r) >> 1;
    if (x <= m) return max(res, query(x, p << 1, l, m));
    return max(res, query(x, p << 1 | 1, m + 1, r));
}

```

插入“只在 $[ql, qr]$ 有效的线段”时, 把 `add_line` 外面再套区间递归, 只在覆盖节点调用 `add_line`。例题: P4097 李超线段树: 题目插入线段、询问横坐标 k 处最高的线段。坐标压缩不能随便做, 因为线性函数的比较依赖真实横坐标; 若题目横坐标本来是整数区间 $[1, 39989]$, 直接按整数域开树即可。

测试样例: 直线交叉、端点和空树

下面约定横坐标域为 $[1, X]$; 操作 1 k b 插入 $y=kx+b$, 操作 2 x 查询最大值。空树查询的输出按题目改成对应的负无穷标记。

输入

```
3
1 4
1 2 1
2 1
1 -1 5
2 1
5 5
1 1 0
1 -1 10
2 1
2 5
1 0 100
2 3
5 5
1 2 0
1 -2 10
2 1
2 5
2 3
```

输出

```
3
4
9
5
100
8
10
6
```

第二、三组必须在不同横坐标上选择不同的最优直线；如果只比较中点、不沿根到叶查询，或者把真实横坐标错误压成编号，通常会在这里出错。

8.15.8 2.11 区间最值势能线段树 (Segment Tree Beats)

当题目同时出现“区间取 $\min(x, \cdot)$ / 区间取 $\max(x, \cdot)$ ”与区间和，普通懒标记无法把所有元素统一修改，因为只有大于 x 的元素会变。Beats 维护：最大值 $mx1$ 、次大值 $mx2$ 、最大值个数 cnt_mx 、区间和；若 $mx2 < x < mx1$ ，说明只有最大值会下降，可以整段打标记。

```
struct BeatNode {
    long long sum; // 区间和。
    long long mx1; // 区间最大值。
    long long mx2; // 严格小于 mx1 的次大值；不存在时为 NEG。
```

```

    int cnt;          // 最大值出现次数。
};
const long long NEG = -(1LL << 60);

void push_chmin(int p, long long x) {
    // 只有 x < mx1 时才有变化；调用者还必须保证 x > mx2。
    // 这样区间内只有“最大值这一类”会被修改，才能整段打标记。
    if (x >= tr[p].mx1) return;
    tr[p].sum -= (tr[p].mx1 - x) * tr[p].cnt;
    tr[p].mx1 = x;
}

void push_up(int p) {
    // 合并两个儿子的 sum、最大值、次大值和最大值个数。
    tr[p].sum = tr[p<<1].sum + tr[p<<1|1].sum;
    tr[p].mx1 = max(tr[p<<1].mx1, tr[p<<1|1].mx1);
    tr[p].cnt = 0;
    tr[p].mx2 = NEG;
    for (int q : {p << 1, p << 1 | 1}) {
        if (tr[q].mx1 == tr[p].mx1) {
            tr[p].cnt += tr[q].cnt;
            tr[p].mx2 = max(tr[p].mx2, tr[q].mx2);
        } else {
            tr[p].mx2 = max(tr[p].mx2, tr[q].mx1);
        }
    }
}

void push_down(int p) {
    // 父节点的 mx1 可能已经被压低，儿子仍保留更大的旧最大值。
    // 把父亲的上界传给儿子，不能直接复制整个节点。
    if (tr[p<<1].mx1 > tr[p].mx1) push_chmin(p<<1, tr[p].mx1);
    if (tr[p<<1|1].mx1 > tr[p].mx1) push_chmin(p<<1|1, tr[p].mx1);
}

void range_chmin(int p, int l, int r, int ql, int qr, long long x) {
    // 没交集，或当前最大值已经不超过 x：无需处理。
    if (r < ql || qr < l || x >= tr[p].mx1) return;

    // x 严格大于次大值时，只有最大值会下降，可以 O(1) 修改当前节点。
    if (ql <= l && r <= qr && x > tr[p].mx2) {
        push_chmin(p, x);
        return;
    }

    // 否则区间内至少有两类值会被影响，必须下传后递归到儿子。
    push_down(p); // 把父节点较小的 mx1 下传给孩子
}

```

```

    int m = (l + r) >> 1;
    range_chmin(p<<1, l, m, ql, qr, x);
    range_chmin(p<<1|1, m+1, r, ql, qr, x);
    push_up(p);
}

```

完整题例：P6242 线段树 3。如果还要区间取 **max**，对称维护最小值 **mn1/mn2/cnt_mn**；如果要区间加，再额外维护 **add** 并在 **push_down** 中先传加法。这个算法的均摊复杂度依赖势能，不能把 **push_chmin** 的条件删掉后“递归到底”，否则会退化。

测试样例：区间取最小值的整段标记与递归分支

下面约定操作 **1 l r x** 是区间 **chmin**，操作 **2 l r** 查询区间和。

输入

```

3
1 4
5
2 1 1
1 1 1 5
1 1 1 3
2 1 1
4 4
1 3 3 7
2 1 4
1 1 4 5
2 1 4
1 2 3 2
2 2 3
5 5
10 1 8 8 3
1 1 5 8
2 1 5
1 1 5 5
2 1 5
1 2 4 4
2 2 4

```

输出

```

5
3
14
12
4

```

28
19
9

第二组的最大值有重复，第三组连续两次降低最大值后再对子区间降低，能同时检查 $mx1/mx2$ /cnt、push_down 和 “ $x > mx2$ 才能整段修改” 的条件。

8.15.9 7.3 可持久化并查集

题面出现 “第 k 个版本中连不连通” “回到历史版本再合并” 时，普通并查集不能回退。用可持久化线段树保存 fa 和秩/深度数组，每次只复制从根到叶子的 $O(\log n)$ 个节点。

```
struct PNode { int l, r, val; } tr[MAXNODE];
int tot;
int build(int l, int r) {
    // build 建立版本 0: 每个位置的父亲初始是自己。
    int p = ++tot;
    if (l == r) tr[p].val = l;
    else {
        int m = (l + r) >> 1;
        tr[p].l = build(l, m);
        tr[p].r = build(m + 1, r);
    }
    return p;
}
int update(int old, int l, int r, int pos, int val) {
    // 只复制从根到 pos 的路径；没有改动的子树直接复用 old 版本。
    int p = ++tot; tr[p] = tr[old];
    if (l == r) { tr[p].val = val; return p; }
    int m = (l + r) >> 1;
    if (pos <= m) tr[p].l = update(tr[old].l, l, m, pos, val);
    else tr[p].r = update(tr[old].r, m + 1, r, pos, val);
    return p;
}
int query(int p, int l, int r, int pos) {
    // 按 pos 在持久化线段树中向下找到 fa[pos]。
    if (l == r) return tr[p].val;
    int m = (l + r) >> 1;
    return pos <= m ? query(tr[p].l, l, m, pos)
        : query(tr[p].r, m + 1, r, pos);
}
int find_root(int root, int x) {
    // 不能做路径压缩：那会修改旧版本；用递归沿父指针查根。
    int f = query(root, 1, n, x);
```

```

    return f == x ? x : find_root(root, f);
}
int merge_version(int root, int x, int y) {
    // root 是“要从哪个历史版本继续操作”，不是固定写 root[n-1]。
    x = find_root(root, x); y = find_root(root, y);
    if (x == y) return root;
    // 应按秩/按大小把较浅树挂到较深树上；秩也要持久化。
    // 真实题目还要维护 rank/size 的第二棵持久化线段树。
    return update(root, 1, n, x, y);
}

```

调用测试：历史版本不能被后续合并污染

下面不强行规定 P3402 的操作编号，直接按接口描述测试：版本 0 是三个孤立点。

```

版本 1: 从版本 0 合并 (1,2)
版本 2: 从版本 0 合并 (2,3)
查询版本 1: connected(1,2) = true
查询版本 1: connected(2,3) = false
查询版本 2: connected(1,2) = false
查询版本 2: connected(2,3) = true

```

如果版本一和版本二都变成同一棵并查集，说明更新时修改了旧版本节点；如果从版本二继续合并时版本一也改变，说明没有真正复制根到叶子的路径。

例题：P3402 可持久化并查集。每个版本 `root[i]` 从 `root[i-1]` 继承；操作 0 复制某版本，操作 1 合并，操作 2 查询连通性。上面为了突出结构只展示 fa，正式提交要再维护按秩合并的深度，否则最坏会退化；不做路径压缩是为了保持版本不被修改。

9 字符串

9.1 KMP

KMP 算法用于在文本串 s 中查找模式串 p 的所有出现位置，时间复杂度 $O(n + m)$ 。核心是预处理 next 数组（最长公共前后缀长度）。

```
// next[i] 表示 p[0..i] 的最长公共前后缀长度
vector<int> get_next(const string& p) {
    int m = p.size();
    vector<int> nxt(m, 0);
    for (int i = 1, j = 0; i < m; i++) {
        while (j > 0 && p[i] != p[j]) j = nxt[j - 1];
        if (p[i] == p[j]) j++;
        nxt[i] = j;
    }
    return nxt;
}

// 返回所有匹配位置 (0-indexed)
vector<int> kmp(const string& s, const string& p) {
    vector<int> nxt = get_next(p);
    vector<int> res;
    int n = s.size(), m = p.size();
    for (int i = 0, j = 0; i < n; i++) {
        while (j > 0 && s[i] != p[j]) j = nxt[j - 1];
        if (s[i] == p[j]) j++;
        if (j == m) {
            res.push_back(i - m + 1);
            j = nxt[j - 1];
        }
    }
    return res;
}
```


10 字符串算法

10.1 选择算法

需求	算法	复杂度
单模式匹配	KMP / Z	$O(n+m)$
多模式匹配	AC 自动机	$O(\text{text} + \text{总模式长})$
子串相等、二分答案	双哈希	期望 $O(1)$ 查询
最长回文子串	Manacher	$O(n)$
所有后缀排序/不同子串	后缀数组 + LCP	$O(n \log n)$
子串出现次数/自动机	SAM	$O(n)$

10.2 Knuth–Morris–Pratt 字符串匹配算法（KMP）

$\text{nxt}[i]$ 表示模式前缀 $p[0..i]$ 的最长真前后缀长度。匹配失败时跳到 $\text{nxt}[j-1]$ ，不回退文本指针；可求出现位置、最小周期、前缀函数树。

最小周期：若 $n \% (n - \text{pi}[n-1]) == 0$ ，周期为 $n - \text{pi}[n-1]$ ，否则为 n 。

```
vector<int> prefix_function(const string& s) {
    vector<int> pi(s.size());
    for (int i = 1; i < (int)s.size(); ++i) {
        int j = pi[i - 1];
        while (j && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
    return pi;
}

vector<int> kmp(const string& text, const string& pat) {
    string t = pat + '#' + text; vector<int> pi = prefix_function(t), pos;
    for (int i = pat.size() + 1; i < (int)t.size(); ++i)
        if (pi[i] == (int)pat.size()) pos.push_back(i - 2 * pat.size());
    return pos;
}
```

```
}
```

10.3 Z 函数

$z[i]$ 是 $s[i..]$ 与 $s[0..]$ 的最长公共前缀。维护当前最右匹配段 $[l, r)$ ，落在段内时先继承 $z[i-l]$ 。把 `pattern + '#' + text` 拼起来即可匹配。

```
vector<int> z_function(const string& s) {
    int n = s.size(), l = 0, r = 0; vector<int> z(n);
    for (int i = 1; i < n; ++i) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
        if (i + z[i] > r) l = i, r = i + z[i];
    }
    return z;
}
```

10.4 字符串哈希

前缀哈希 $H[i+1]=H[i]*B+s[i]$ ，子串哈希： $H[r]-H[l]*powB[r-l]$ 。双模数或 64 位自然溢出降低碰撞；随机底数并不能提供形式化保证。比较子串、二分最长公共前缀、回文判定都可用。

```
const unsigned long long BASE = 911382323ull;
vector<unsigned long long> h(n + 1), pw(n + 1, 1);
for (int i = 0; i < n; ++i)
    h[i + 1] = h[i] * BASE + (unsigned char)s[i], pw[i + 1] = pw[i] * BASE;
auto get_hash = [&](int l, int r) { return h[r] - h[l] * pw[r - l]; }; // [l, r)
```

10.5 Manacher

把字符串改为 `^#a#b#$`，求每个中心向外的回文半径 $p[i]$ ；维护最右回文段 $(center, right)$ ，线性求最长回文。若不想处理奇偶，也可分别对原串做奇/偶中心版本。

```
string t = "^#";
for (char c : s) t += c, t += '#';
t += '$';
vector<int> p(t.size()); int center = 0, right = 0;
for (int i = 1; i + 1 < (int)t.size(); ++i) {
    int mir = 2 * center - i;
```

```

    if (i < right) p[i] = min(right - i, p[mir]);
    while (t[i + 1 + p[i]] == t[i - 1 - p[i]]) ++p[i];
    if (i + p[i] > right) center = i, right = i + p[i];
}

```

10.6 字典树与 Aho–Corasick 多模式匹配自动机（AC 自动机）

Trie 每个节点保存儿子、终止次数。AC 自动机用 BFS 建 fail：节点的 fail 指向最长真后缀状态，匹配时沿 fail 转移；把输出计数沿 fail 累加可统计所有模式出现次数。模式集合固定后一次建树、多次查询。

```

struct AC {
    int ch[MAXN][26]{}; // Trie 边; build 后也会补齐为自动机转移。
    int fail[MAXN]{};    // 最长真后缀节点。
    int out[MAXN]{};     // 模式串结束次数及 fail 链汇总次数。
    int tot = 0;
    void insert(const string& s) {
        int u = 0;
        for (char c : s) {
            int x = c - 'a';
            if (!ch[u][x]) ch[u][x] = ++tot;
            u = ch[u][x];
        }
        // 重复模式串要累加，而不是只记一个 bool。
        ++out[u];
    }
    void build() {
        queue<int> q;
        // 根节点的直接儿子的 fail 都是根。
        for (int c = 0; c < 26; ++c) if (ch[0][c]) q.push(ch[0][c]);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            // 如果要统计每个模式串，推荐建 fail 树后再统一 DFS 汇总；
            // 这里直接累加适合只问“当前位置结束的所有模式数”。
            out[u] += out[fail[u]];
            for (int c = 0; c < 26; ++c) {
                int &v = ch[u][c];
                if (v) {
                    // v 的 fail 是 fail[u] 沿同字符转移。
                    fail[v] = ch[fail[u]][c], q.push(v);
                } else {
                    // 补全转移，扫描时不需要反复 while 回退。
                    v = ch[fail[u]][c];
                }
            }
        }
    }
};

```

```

        }
    }
}
int query(const string& s) {
    int u = 0, ans = 0;
    for (char c : s) {
        // u 表示当前文本前缀的最长 Trie 后缀。
        u = ch[u][c - 'a'];
        ans += out[u];
    }
    return ans;
}
};

```

10.7 后缀数组与最长公共前缀 (Suffix Array and Longest Common Prefix, LCP)

倍增排序：按 $(\text{rank}[i], \text{rank}[i+k])$ 对后缀排序，重复 $O(\log n)$ 次，复杂度 $O(n \log^2 n)$ (用计数排序可到 $O(n \log n)$)。Kasai 求相邻后缀 LCP，RMQ 查询任意两个后缀 LCP。常见应用：不同子串数 $n(n+1)/2 - \sum \text{LCP}$ 、最长重复子串、字典序第 k 个子串。

```

vector<int> suffix_array(string s) {
    // 加入最小哨兵，保证所有真正后缀都排在哨兵之后。
    s.push_back(0); int n = s.size();
    vector<int> sa(n), r(n), nr(n);
    iota(sa.begin(), sa.end(), 0);
    for (int i = 0; i < n; ++i) r[i] = (unsigned char)s[i];
    for (int k = 1; k <= 1) {
        // 用长度 k 的前半排名和后半排名比较长度 2k 的前缀。
        sort(sa.begin(), sa.end(), [&](int i, int j) {
            if (r[i] != r[j]) return r[i] < r[j];
            int ri = i + k < n ? r[i + k] : -1;
            int rj = j + k < n ? r[j + k] : -1;
            return ri < rj;
        });
        // 重新压缩排名；二元组完全相同的后缀共享排名。
        nr[sa[0]] = 0;
        for (int i = 1; i < n; ++i)
            nr[sa[i]] = nr[sa[i - 1]] +
                (r[sa[i - 1]] != r[sa[i]] ||
                 (sa[i - 1] + k < n ? r[sa[i - 1] + k] : -1) !=
                 (sa[i] + k < n ? r[sa[i] + k] : -1));
    }
}

```

```

        r.swap(nr);
        if (r[sa.back()] == n - 1) break;
    }
    sa.erase(sa.begin()); return sa;
}

```

10.8 后缀自动机 (Suffix Automaton, SAM)

在线加入字符，状态保存 `len`、`link`、转移；每个状态代表一组 `endpos` 相同的子串。状态数最多 $2n-1$ 。按 `len` 降序累加出现次数，可求每个子串出现频次、不同子串数 $\sum(\text{len}[v]-\text{len}[\text{link}[v]])$ 。转移字符集大时用 `map`，小字符集用数组。

```

struct SAM {
    struct Node {
        int next[26]{}; // 从该 endpos 等价类沿字符转移。
        int link = -1; // suffix link, 指向更短的等价类。
        int len = 0; // 该状态能代表的最长子串长度。
        int occ = 0; // 直接作为前缀末尾出现的次数，之后沿 link 汇总。
    } st[2 * MAXN];
    int last = 0; // 当前整个字符串对应的状态。
    int sz = 1; // 状态 0 是初始状态。
    void extend(char c) {
        // cur 表示加入 c 后的新前缀及其所有后缀。
        int cur = sz++, p = last; st[cur].len = st[last].len + 1; st[cur].occ = 1;
        // 从 last 沿 suffix link 补齐缺失的 c 转移。
        while (p >= 0 && !st[p].next[c - 'a']) st[p].next[c - 'a'] = cur, p = st[p].link;
        if (p < 0) st[cur].link = 0;
        else {
            int q = st[p].next[c - 'a'];
            if (st[p].len + 1 == st[q].len) st[cur].link = q;
            else {
                // q 太长，复制出 clone 把长度截断到 st[p].len+1。
                int clone = sz++; st[clone] = st[q]; st[clone].len = st[p].len + 1; st[clone].occ = 0;
                // 把原来能经过 c 到 q 的一段 link 链改指 clone。
                while (p >= 0 && st[p].next[c - 'a'] == q) st[p].next[c - 'a'] = clone, p = st[p].link;
                st[q].link = st[cur].link = clone;
            }
        }
        last = cur;
    }
}

```

```

    }
};

```

10.9 回文自动机 (Palindromic Automaton, PAM, 了解)

每个节点代表一个不同回文串，fail 指向最长回文后缀；在线插入字符即可统计不同回文子串、每个前缀的回文后缀。代码比 Manacher 长，只有在“动态加入/回文后缀结构”时使用。

```

struct PAM {
    int next[MAXN][26]; // 回文串两端加字符后的转移。
    int fail[MAXN];      // 最长真回文后缀。
    int len[MAXN];       // 节点对应回文串长度。
    int s[MAXN];         // 已插入字符，寻找可扩展回文时要回看它。
    int occ[MAXN];       // 先记作最长回文后缀的次数，再沿 fail 汇总。
    int last, n, tot;    // last=当前最长回文后缀，n=长度，tot=节点数。
    void init() {
        // 0 号根长度 0，1 号根长度 -1；两个根让边界转移统一。
        memset(next, 0, sizeof(next)); memset(occ, 0, sizeof(occ));
        n = 0; tot = 1; last = 0; s[0] = -1;
        len[0] = 0; len[1] = -1; fail[0] = fail[1] = 1;
    }
    int get_fail(int u) {
        // 沿 fail 跳，直到 u 的两端可以接上新字符 s[n]。
        while (s[n - len[u] - 1] != s[n]) u = fail[u];
        return u;
    }
    void add(char c) {
        // 先把新字符写入串，再寻找能够扩展的最长回文后缀。
        s[++n] = c - 'a'; int u = get_fail(last);
        if (!next[u][s[n]]) {
            // 这是一个此前未出现过的新回文节点。
            int v = ++tot; len[v] = len[u] + 2;
            // 新节点的 fail: 从 fail[u] 开始寻找也能被该字符扩展的节点。
            fail[v] = next[get_fail(fail[u])][s[n]];
            next[u][s[n]] = v;
        }
        // last 是当前前缀的最长回文后缀；每个位置只直接计数一次。
        last = next[u][s[n]]; ++occ[last];
    }
    int build(const string& str) {
        init();
        for (char c : str) add(c);
        // fail 从长回文指向短回文，必须按长度降序汇总；
    }
};

```

```

// 许多简化板子用编号倒序，只在编号也随长度单调时才安全。
vector<int> ord(max(0, tot - 1));
iota(ord.begin(), ord.end(), 2); // 0/1 是两个虚拟根，不参与统计。
sort(ord.begin(), ord.end(), [&](int a, int b) { return len[a] > len[b]; });
for (int v : ord) occ[fail[v]] += occ[v];
return tot - 1; // 不同非空回文子串数量
}
};

```

10.10 字符串常见组合

- 子串出现次数：KMP 的前缀函数树或 SAM 的 endpos 计数。
- 多个模式在文本中出现：AC 自动机；需要撤销/动态模式时考虑 fail 树 + 树状数组。
- 比较循环字符串：最小表示法（Booth） $O(n)$ 。
- 最短覆盖/最小表示：先判周期，再用 KMP/哈希做边界验证。

```

int min_rotation(const string& s) {
    string t = s + s; int i = 0, j = 1, n = s.size();
    while (i < n && j < n) {
        int k = 0;
        while (k < n && t[i + k] == t[j + k]) ++k;
        if (k == n) break;
        if (t[i + k] > t[j + k]) i += k + 1, i += (i == j);
        else j += k + 1, j += (i == j);
    }
    return min(i, j);
}

```

10.11 字符串高级算法：参数、调用与改造

本节紧跟前面的基础模板。先看每个接口传什么，再看哪些不变量不能动，最后看如何替换维护信息或转移。

10.11.1 4.1 Booth 最小表示法

循环字符串的最小表示不是把字符串复制后排序所有起点。Booth 双指针在 $s+s$ 上比较两个候选起点，淘汰整段，复杂度 $O(n)$ 。

```

int booth(const string& s) {
    // t=s+s, 把所有循环移位统一成 t 中长度 n 的子串。

```

```

string t = s + s;
int n = s.size(), i = 0, j = 1, k = 0;
while (i < n && j < n && k < n) {
    // k 表示两个候选起点已经匹配的长度。
    if (t[i+k] == t[j+k]) { ++k; continue; }

    // 第一个不同字符决定哪个起点更大：较大的候选可以淘汰一整段。
    if (t[i+k] > t[j+k]) i = i + k + 1;
    else j = j + k + 1;

    // 两个指针不能相等，否则候选数会少一个。
    if (i == j) ++j;
    k = 0;
}

// 最小表示的起点一定在 [0,n-1] 中。
return min(i, j);
}

```

测试样例：最小表示的重复字符、周期和首尾交叉

下面每组输入一个字符串，输出它的字典序最小循环移位。

输入

4

a

aaaa

baca

caba

输出

a

aaaa

abac

abac

前两组检查 $i==j$ 和重复字符；后两组的最优起点不在原串开头，能抓住循环串复制后排序时的下标错误。

例题：P1368 工艺。把输入串交给 **booth**，从返回下标开始输出 n 个字符。题目求的是循环移位，不是反转后的最小串；若题面允许反转，要额外对反串再调用一次。

10.11.2 4.2 回文自动机 (Palindromic Automaton)

PAM 的每个节点代表一个不同的回文串；**fail** 指向去掉两端后最长的回文后缀。插入字符时沿 **fail** 找到可以向两端扩展的节点。节点数最多 $n+2$ ，所以它适合“所有回文子串”的统计。

```
struct PAM {
    static const int SIG = 26;
    int next[MAXN][SIG]; // next[u][c]: 回文串 u 两端加字符 c 后的节点。
    int fail[MAXN];      // fail[u]: u 的最长真回文后缀。
    int len[MAXN];       // len[u]: 节点 u 代表的回文串长度。
    int s[MAXN];         // 已插入的字符, get_fail 要访问历史字符。
    long long occ[MAXN]; // 先统计“作为最长回文后缀”的次数, 再沿 fail 汇总。
    int last, tot, n;     // last=当前前缀的最长回文后缀, tot=节点总数, n=串长。

    void init() {
        // 0 号根表示长度 -1 的虚拟回文, 1 号根表示长度 0 的空串。
        memset(next, 0, sizeof(next));
        memset(occ, 0, sizeof(occ));
        n = 0; tot = 1; last = 1;
        len[0] = -1; fail[0] = 0;
        len[1] = 0; fail[1] = 0;
    }

    int get_fail(int x) {
        // 从当前回文后缀沿 fail 向上跳, 直到它能和新字符首尾匹配。
        while (n - 1 - len[x] < 0 || s[n - 1 - len[x]] != s[n])
            x = fail[x];
        return x;
    }

    int add(char ch) {
        // 新字符暂存在 s[n], 此时 n 还是旧长度。
        s[n] = ch - 'a';
        int cur = get_fail(last), c = s[n];
        if (!next[cur][c]) {
            // 当前回文后缀加上两端字符, 产生一个此前没有出现过的新回文。
            int now = ++tot;
            len[now] = len[cur] + 2;

            // 长度 1 的回文 fail 到空串根; 其它节点继续从 fail[cur] 找。
            fail[now] = (len[now] == 1) ? 1 : next[get_fail(fail[cur])][c];
            next[cur][c] = now;
        }

        // 新前缀的最长回文后缀就是这条转移到达的节点。
        last = next[cur][c]; ++occ[last]; ++n;
    }
};
```

```

    return last;
}
void count_occurrence() {
    // fail 一定从长串指向短串，所以必须按 len 从大到小传递。
    // 节点编号不保证按长度排序，不能盲写 for(i=tot;i>=1;--i)。
    vector<int> ord(max(0, tot - 1));
    iota(ord.begin(), ord.end(), 2); // 0/1 是两个虚拟根，不参与统计。
    sort(ord.begin(), ord.end(), [](int a, int b) {
        return len[a] > len[b];
    });
    for (int u : ord) occ[fail[u]] += occ[u];
}
};

```

测试样例：回文自动机的根、重复回文和多测清空

下面每组输入一个小写字串，输出不同的非空回文子串数量，也就是 $\text{tot}-1$ 。

输入

4

a

aaaa

abacaba

ababa

输出

1

4

7

5

aaaa 的不同回文串是 a,aa,aaa,aaaa; abacaba 的七个不同回文串可用来检查 fail 链。连续多组调用前必须执行 `init()`，否则第二组的节点和出现次数会污染第三组。

例题：P5496 【模板】回文自动机（PAM）。题目要求每个位置结尾的回文串数量；插入第 i 个字符后，最长回文后缀是 `last`，所有回文后缀沿 `fail` 链连接，因此可以用 $\text{dp}[i] = \text{dp}[\text{fail}[i]] + 1$ 的题目特定统计，或沿 `fail` 链统计。若只问不同回文子串数量，答案是 $\text{tot}-1$ ；若问出现次数，要先 `count_occurrence()`。

10.11.3 4.3 后缀数组、最长公共前缀、后缀自动机与 Aho–Corasick 失配树

后缀数组适合“后缀排序、子串字典序、多个后缀的最长公共前缀”；LCP 用 Kasai 求相邻后缀公共前缀，再用 RMQ 回答任意两个后缀的 LCP。题目出现“第 k 个不同子串”时，SAM 往往更直接：每个状态记录 `len`、`link` 和转移，按 DAG DP 统计从状态出发的不同串数。

```

struct SAM {
    int next[MAXN][26]; // 状态转移；字符集大时改成 map/unordered_map。
    int link[MAXN];      // 后缀链接：指向最长的更短 endpos 等价类。
    int len[MAXN];       // 该状态能表示的最长子串长度。
    int last = 1;        // 整个当前前缀对应的状态。
    int tot = 1;         // 1 号状态是初始状态；0 号转移为空。

    void extend(int c) {
        // 新状态 cur 代表“以新字符结尾”的所有后缀。
        int cur = ++tot; len[cur] = len[last] + 1;
        int p = last;

        // 沿 suffix link 补上以前不存在的 c 转移。
        while (p && !next[p][c]) next[p][c] = cur, p = link[p];
        if (!p) link[cur] = 1;
        else {
            int q = next[p][c];
            if (len[p] + 1 == len[q]) link[cur] = q;
            else {
                // q 的长度太长，不能直接作为 cur 的 link。
                // clone 复制 q 的转移，但把最长长度截为 len[p]+1。
                int clone = ++tot;
                memcpy(next[clone], next[q], sizeof next[q]);
                len[clone] = len[p] + 1; link[clone] = link[q];

                // 把原来指向 q 的、位于同一等价区间的转移改指 clone。
                while (p && next[p][c] == q)
                    next[p][c] = clone, p = link[p];
                link[q] = link[cur] = clone;
            }
        }
        last = cur;
    }
};

```

调用测试：后缀自动机的空串、重复字符和克隆状态

当前代码片段只负责 `extend`，所以先测试最基本的不变量：初始状态编号为 `1`，字符串全相同时每个字符新增一个状态；如果要测试出现次数，还必须把 `occ` 统计和链接树排序一并接上。

调用输入

`s = ""`

```
s = "a"
s = "aaaa"
s = "ab"
```

应得到的状态总数 tot

```
1
2
5
3
```

第一组检查 **init**；第二、三组检查 **last** 和长度；第四组检查不同字符转移。真正测试克隆分支时，应使用一个会产生重复后缀等价类的字符串，并检查所有被旧状态指向的转移是否一起改到 clone。

例题：P3804 【模板】后缀自动机 (SAM) 问出现次数最多的子串：按 **len** 从大到小把 **occ[u]** 加到 **occ[link[u]]**，答案是 $\max(\text{len}[u] \times \text{occ}[u])$ 。若是第 **k** 个子串，先在状态 DAG 上做 $\text{dp}[u] = \sum(1 + \text{dp}[v])$ ，再按字符顺序跳转并扣减 $1 + \text{dp}[v]$ 。

AC 自动机把多个模式串建 Trie，再按 BFS 建 **fail**；把 fail 指针反向成树，就能把“文本中经过节点”转成 fail 子树求和。例题可用 P3796 AC 自动机加强版：插入模式串，扫描文本，沿 fail 链累计出现次数，最后找最大次数的模式串。查询中如果有“一个模式串是另一个模式串后缀”，不要只看 Trie 节点本身，必须沿 fail 树汇总。

哈希二分 LCP 的模板：预处理双模/64 位前缀哈希，比较 $\text{hash}(i, \text{len}) == \text{hash}(j, \text{len})$ ，二分最长长度。例题：P3370 【模板】字符串哈希。如果是高风险哈希题，使用双模或 **uint64_t** 随机基，不能把单模碰撞当成严格证明。

10.11.4 7.4 后缀数组与 LCP

后缀数组的倍增法按“前半排名、后半排名”排序。工程上可以使用 **sort** 版先保证正确；**n** 达到 **2e5** 以上再换基数排序优化。

```
vector<int> suffix_array(const string& s) {
    int n = s.size();
    vector<int> sa(n), rk(n), tmp(n);
    iota(sa.begin(), sa.end(), 0);
    // rk[i] 是长度 1 的后缀 s[i..] 的排名；不同字符最好先压成连续整数。
    for (int i = 0; i < n; ++i) rk[i] = s[i];
    for (int k = 1; k <= 1) {
        // 现在按长度 2k 的二元组（前 k 长排名，后 k 长排名）排序。
        sort(sa.begin(), sa.end(), [&](int i, int j) {
            if (rk[i] != rk[j]) return rk[i] < rk[j];
            int ri = i + k < n ? rk[i+k] : -1;
            int rj = j + k < n ? rk[j+k] : -1;
            return ri < rj;
        });
    }
}
```

```

    });
    // 根据排好序的后缀重新编号；相邻二元组相同才共享排名。
    tmp[sa[0]] = 0;
    for (int i = 1; i < n; ++i) {
        int a = sa[i-1], b = sa[i];
        bool diff = rk[a] != rk[b];
        int ra = a+k < n ? rk[a+k] : -1;
        int rb = b+k < n ? rk[b+k] : -1;
        tmp[b] = tmp[a] + (diff || ra != rb);
    }
    rk.swap(tmp);
    // 所有排名都不同后，sa 已经是最终后缀数组。
    if (rk[sa.back()] == n - 1) break;
}
return sa;
}

vector<int> kasai(const string& s, const vector<int>& sa) {
    int n = s.size(); vector<int> rank(n), lcp(n);
    for (int i = 0; i < n; ++i) rank[sa[i]] = i;
    for (int i = 0, h = 0; i < n; ++i) {
        int r = rank[i];
        if (r == 0) continue;
        int j = sa[r-1];
        while (i+h < n && j+h < n && s[i+h] == s[j+h]) ++h;
        lcp[r] = h;
        if (h) --h;
    }
    return lcp; // lcp[r] = sa[r] 与 sa[r-1] 的 LCP
}

```

测试样例：后缀排序和 LCP 的经典字符串

输入字符串

banana

应得到后缀数组

5 3 1 0 4 2

应得到 lcp 数组

0 1 3 0 0 2

banana 同时含有重复前缀和不同首字符，能检查倍增第二关键字的越界排名；lcp[2]=3 是 ana 与 anana 的公共前缀，能抓住 Kasai 的 h 是否正确复用。

例题：P3809 【模板】后缀排序。若要任意两个后缀的 LCP，把 `lcp` 做 RMQ；若要第 `k` 个不同子串，按后缀顺序累加 `n-sa[i]-lcp[i]`，第一次超过 `k` 的后缀就是答案所在位置。

10.11.5 7.5 AC 自动机与 fail 树

```
struct AC {
    int ch[MAXN][26]{}; // build 后不存在的转移也会补成 fail 转移。
    int fail[MAXN]{};    // 当前节点的最长真后缀节点。
    int out[MAXN]{};     // 节点本身及 fail 链上的模式串计数。
    int tot = 0;
    vector<int> tree[MAXN]; // fail 指针反图；用于子树汇总。
    int insert(const string& s) {
        int u = 0;
        for (char cc : s) {
            int c = cc - 'a';
            if (!ch[u][c]) ch[u][c] = ++tot;
            u = ch[u][c];
        }
        // 返回结束节点，题目要分别输出时保存这个编号。
        return u;
    }
    void build() {
        queue<int> q;
        // 根的 fail 是自己；根的直接儿子 fail 都是根。
        for (int c = 0; c < 26; ++c)
            if (ch[0][c]) q.push(ch[0][c]);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int c = 0; c < 26; ++c) {
                if (ch[u][c]) {
                    // 子节点的 fail: 沿 u 的 fail 走同一个字符。
                    fail[ch[u][c]] = ch[fail[u]][c];
                    q.push(ch[u][c]);
                } else {
                    // 补全自动机转移，扫描文本时就不需要 while 回退。
                    ch[u][c] = ch[fail[u]][c];
                }
            }
        }
        // fail[u] 是 u 的父亲，之后可以把文本经过次数沿 fail 树向上累加。
        for (int u = 1; u <= tot; ++u) tree[fail[u]].push_back(u);
    }
    void scan(const string& text) {
        int u = 0;
```

```
    for (char cc : text) {
        // 每读一个字符，u 就是当前前缀的最长 Trie 后缀状态。
        u = ch[u][cc - 'a'];
        ++out[u];
    }
}

void dfs_fail(int u) {
    for (int v : tree[u]) {
        dfs_fail(v);
        // v 的所有出现也意味着 fail[v] 对应的模式串出现。
        out[u] += out[v];
    }
}

};
```

测试样例：AC 自动机的后缀模式和重复出现

模式串依次为 a、ab、bab，文本为 abab。扫描后沿 fail 树汇总，三个模式的出现次数应为：

```
a    -> 2
ab   -> 2
bab  -> 1
```

ab 是 a 的延长，bab 又是文本中较长的后缀模式；如果只读取当前 Trie 节点的 out 而不沿 fail 树汇总，a 的次数通常会少算。

例题：P3808 【模板】AC 自动机（简单版）。每个模式串的结束节点记录 id；扫描文本后，调用 dfs_fail(0)，该节点的 out 就是模式串作为后缀出现的总次数。简单版只需问总出现次数，若要每个模式串分别输出，就保存插入时的节点编号。

11 图论

建图常见错误：

- 无向图边数必须开 $2M$ （正反各存一次），否则越界 RE。
- 多测时 `vector<int> G[N]` 必须根据上一组的 N 清空，不能只清当前组的范围。
- 节点编号是 0-based 还是 1-based？搞混会导致起点没连边。
- 没说“简单图”就可能有重边和自环，邻接矩阵存图时用 `g[u][v] = min(g[u][v], w)`。

11.1 最短路

11.1.1 Dijkstra（堆优化）

适用于非负权图， $O((n+m)\log n)$ 。有负权边时 Dijkstra 失效，必须用 Bellman-Ford。

```
vector<pair<int,int>> G[N]; // G[u] = {(v, w), ...}
int dist[N];
bool vis[N];

void dijkstra(int s, int n) {
    fill(dist + 1, dist + n + 1, inf);
    memset(vis, 0, sizeof(vis));
    dist[s] = 0;
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (vis[u]) continue; // 必须判重！否则 TLE
        vis[u] = true;
        for (auto [v, w] : G[u]) {
            if (dist[u] + w < dist[v]) { // dist[u] 不会是 inf（已出队）
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}
```



```

    }
    }
}

```

- 出队后必须 `if(vis[u]) continue`，否则同一节点被处理多次导致 TLE。
- 松弛时如果 `dist[u]` 可能是 `inf`，加上 `w` 会溢出变负数！要么保证出队时 `dist` 有限，要么加判断。

11.1.2 SPFA (Bellman-Ford 队列优化)

可以处理负权边，但最坏 $O(nm)$ 。正权图请用 Dijkstra，SPFA 容易被卡。可用于判负环。

```

int dist[N], cnt[N]; // cnt[i] 记录 i 入队次数
bool inq[N];

bool spfa(int s, int n) {
    fill(dist + 1, dist + n + 1, inf);
    memset(inq, 0, sizeof(inq));
    memset(cnt, 0, sizeof(cnt));
    dist[s] = 0; inq[s] = true; cnt[s] = 1;
    queue<int> q;
    q.push(s);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        inq[u] = false;
        for (auto [v, w] : G[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                if (!inq[v]) {
                    q.push(v);
                    inq[v] = true;
                    if (++cnt[v] > n) return true; // 存在负环
                }
            }
        }
    }
    return false; // 无负环
}

```

11.2 拓扑排序

- 跑完后必须检查出队节点数是否等于 n ，不等于说明有环！
- 要求字典序最小的拓扑序：把 `queue` 换成 `priority_queue<int, vector<int>, greater<int>>`。

```
int in_deg[N];
vector<int> topo;

bool toposort(int n) {
    queue<int> q;
    for (int i = 1; i <= n; i++)
        if (in_deg[i] == 0) q.push(i);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        topo.push_back(u);
        for (int v : G[u]) {
            if (--in_deg[v] == 0)
                q.push(v);
        }
    }
    return topo.size() == n; // false 说明有环
}
```

11.3 多源 BFS

多个起点同时扩展（如”所有怪物同时向外扩散”），将所有起点一次性入队即可。

```
int dist[N][N]; // 网格图距离
int dx[] = {0, 0, 1, -1};
int dy[] = {1, -1, 0, 0};

void multi_bfs(vector<pair<int,int>>& starts, int n, int m) {
    memset(dist, -1, sizeof(dist));
    queue<pair<int,int>> q;
    for (auto [x, y] : starts) {
        dist[x][y] = 0;
        q.push({x, y});
    }
    while (!q.empty()) {
```

```

    auto [x, y] = q.front(); q.pop();
    for (int d = 0; d < 4; d++) {
        int nx = x + dx[d], ny = y + dy[d];
        if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;
        if (dist[nx][ny] != -1) continue;
        dist[nx][ny] = dist[x][y] + 1;
        q.push({nx, ny});
    }
}
}

```

BFS 必须在入队时标记 vis（或设 dist），不能在出队时标记！否则同一节点被重复入队导致 TLE/MLE。

11.4 BFS 求最小环（无权图）

求无权无向图的最小环长度（围 girth）。对每个节点 BFS，当发现一条非树边 (u, v) （ v 已访问且不是 u 的 BFS 父亲）时，环长 = $\text{dist}[u] + \text{dist}[v] + 1$ 。对所有节点取最小值。时间复杂度 $O(n(n + m))$ 。

```

vector<int> G[N];

// 返回无权无向图的最小环长度，无环返回 inf
int min_cycle(int n) {
    int ans = inf;
    for (int s = 1; s <= n; s++) {
        vector<int> dist(n + 1, -1), par(n + 1, -1);
        dist[s] = 0;
        queue<int> q;
        q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : G[u]) {
                if (dist[v] == -1) {
                    dist[v] = dist[u] + 1;
                    par[v] = u;
                    q.push(v);
                } else if (par[u] != v) {
                    // 发现非树边，形成环
                    ans = min(ans, dist[u] + dist[v] + 1);
                }
            }
        }
    }
}

```

```

    }
    }
}
return ans;
}

```

- 该模板仅适用于**无权图**（边权全为 1）。带权图求最小环需用 Floyd 或枚举删边 + Dijkstra。
- 时间复杂度为 $O(n(n+m))$ ，适用于 $n \leq 1000$ 左右。 n 较大时考虑优化或换算法。
- 有向图的最小环需要不同算法（如枚举每个点跑 Dijkstra 找回到自身的最短路）。
- 发现非树边时不需要 `break`，因为同一轮 BFS 可能有多条非树边产生不同环长。

- **剪枝优化**：当 $\text{dist}[u] \geq \lceil \text{ans}/2 \rceil$ 时可以提前终止当前 BFS，因为后续不可能找到更短的环。
- **带权图最小环（Floyd 做法）**：在 Floyd 过程中，枚举中间点 k 之前，检查 $\text{dist}[i][j] + w(j, k) + w(k, i)$ ($i < k, j < k$)，复杂度 $O(n^3)$ 。
- **奇环 / 偶环判断**：二分图判定（BFS 染色）可以判断图中是否有奇环；如果图是二分图则所有环为偶数长度。

11.5 最小生成树（Kruskal）

贪心：按边权排序，依次加入不形成环的边（用并查集判断）。 $O(m \log m)$ 。

```

struct Edge {
    int u, v, w;
    bool operator<(const Edge& o) const { return w < o.w; }
};

int kruskal(int n, vector<Edge>& edges) {
    sort(edges.begin(), edges.end());
    init(n); // 并查集初始化
    int res = 0, cnt = 0;
    for (auto& [u, v, w] : edges) {
        if (find(u) != find(v)) {
            unite(u, v);
            res += w;
            if (++cnt == n - 1) break;
        }
    }
}

```

```

    }
}
// cnt < n-1 说明图不连通
return res;
}

```

11.6 欧拉路 (Hierholzer)

欧拉路径：经过每条边恰好一次的路径。存在条件：无向图中恰好 0 或 2 个奇度点；有向图中所有点入度 = 出度，或恰好一个点出度-入度 = 1（起点），一个点入度-出度 = 1（终点）。

```

// 有向图欧拉路径（邻接表，边需要标记是否用过）
vector<int> path;
vector<pair<int,int>> G[N]; // G[u] = {(v, edge_id), ...}
bool used[M];
int cur[N]; // 当前弧优化

void dfs(int u) {
    for (int& i = cur[u]; i < G[u].size(); ) {
        auto [v, id] = G[u][i];
        if (used[id]) { i++; continue; }
        used[id] = true;
        i++;
        dfs(v);
    }
    path.push_back(u);
}

// 调用后 reverse(path.begin(), path.end()) 得到欧拉路径

```

11.7 最大流 (Dinic)

时间复杂度 $O(V^2E)$ ，实际远快于此。用于求最大流和最小割（最大流 = 最小割）。

```

struct Edge { int to, cap, rev; };
vector<Edge> graph[N];
int level[N], iter[N];

void add_edge(int u, int v, int cap) {
    graph[u].push_back({v, cap, (int)graph[v].size()});
    graph[v].push_back({u, 0, (int)graph[u].size() - 1});
}

```

```
}

bool bfs(int s, int t, int n) {
    memset(level, -1, sizeof(int) * (n + 1));
    queue<int> q;
    level[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (auto& e : graph[u]) {
            if (e.cap > 0 && level[e.to] < 0) {
                level[e.to] = level[u] + 1;
                q.push(e.to);
            }
        }
    }
    return level[t] >= 0;
}

int dfs_flow(int u, int t, int f) {
    if (u == t) return f;
    for (int& i = iter[u]; i < graph[u].size(); i++) {
        Edge& e = graph[u][i];
        if (e.cap > 0 && level[e.to] == level[u] + 1) {
            int d = dfs_flow(e.to, t, min(f, e.cap));
            if (d > 0) {
                e.cap -= d;
                graph[e.to][e.rev].cap += d;
                return d;
            }
        }
    }
    return 0;
}

int max_flow(int s, int t, int n) {
    int flow = 0;
    while (bfs(s, t, n)) {
        memset(iter, 0, sizeof(int) * (n + 1));
        int d;
        while ((d = dfs_flow(s, t, inf)) > 0)
            flow += d;
    }
    return flow;
}
```

```
}
```

最小割 = 最大流。跑完最大流后，从源点 BFS 能到达的点属于 S 集，不能到达的属于 T 集，割边就是跨越 $S-T$ 的满流边。

12 图论

12.1 图的表示与基础遍历

稀疏图用邻接表，边结构保存 `to, weight, id`；需要删除/反向边时给每条有向边保存反向边下标。无向边应添加两条边。网格可把 (x,y) 映射为 $x \times m + y$ ，或直接四方向 BFS。

DFS 可求连通块、染色、子树大小、DFS 序、树高；BFS 求无权最短路和层数。递归 DFS 在 $n \approx 2e5$ 时可能爆栈，改手写栈或提高栈空间。

```
vector<vector<int>> g(n);
vector<int> dist(n, -1);
queue<int> q;
dist[s] = 0; q.push(s);
while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : g[u]) if (dist[v] == -1)
        dist[v] = dist[u] + 1, q.push(v);
}
```

12.2 有向无环图 (Directed Acyclic Graph, DAG) 与拓扑排序

Kahn：把入度为 0 的点入队，删除出边；处理点数小于 v 说明有环。DFS 三色标记也能判环。DAG 上按拓扑序做最长路/最短路、路径计数、DP；多源最短路可把所有源同时入队。

关键路径： $\text{earliest}[v] = \max(\text{earliest}[v], \text{earliest}[u] + w)$ 。若要恢复方案，保存前驱；同一最优值的多条路径计数要注意模数。

```
queue<int> q;
for (int u = 0; u < n; ++u) if (!indeg[u]) q.push(u);
vector<int> topo;
while (!q.empty()) {
    int u = q.front(); q.pop(); topo.push_back(u);
    for (auto [v, w] : g[u]) if (--indeg[v] == 0) q.push(v);
}
if ((int)topo.size() != n) throw runtime_error("cycle");
```


12.3 最短路

条件	算法	复杂度
无权	BFS	$O(V+E)$
边权 0/1	0-1 BFS	$O(V+E)$
非负权	Dijkstra + 堆	$O((V+E)\log V)$
允许负边、单源	Bellman-Ford	$O(VE)$
DAG	拓扑 DP	$O(V+E)$
全源、 $V \leq 500$	Floyd	$O(V^3)$

Dijkstra 的核心前提是所有边权非负；堆中允许重复状态，弹出时若距离不是当前 `dist[u]` 就跳过。多源时将所有起点距离设为 0。

负环检测：Bellman-Ford 第 V 轮仍有更新即存在从源可达负环；SPFA 可做队列优化但最坏仍为 $O(VE)$ ，不要无条件依赖。

差分约束：不等式 $x_v \leq x_u + w$ 建 $u \rightarrow v$ 权 w ，求最短路；若存在负环则无解。求最大下界时改写方向/用最长路。

```
using P = pair<long long, int>;
vector<long long> dist(n, INF);
priority_queue<P, vector<P>, greater<P>> pq;
dist[s] = 0; pq.push({0, s});
while (!pq.empty()) {
    auto [du, u] = pq.top(); pq.pop();
    if (du != dist[u]) continue;
    for (auto [v, w] : g[u]) if (dist[v] > du + w) {
        dist[v] = du + w;
        pq.push({dist[v], v});
    }
}
```

```
deque<int> q;
vector<int> d(n, INF); d[s] = 0; q.push_front(s);
while (!q.empty()) {
    int u = q.front(); q.pop_front();
    for (auto [v, w] : zero_one_graph[u]) if (d[v] > d[u] + w) {
        d[v] = d[u] + w;
        if (w == 0) q.push_front(v); else q.push_back(v);
    }
}
```

12.4 最小生成树

12.4.1 Kruskal

按边权排序，用 DSU 合并不同连通块；恰好选 $V-1$ 条边则图连通。复杂度 $O(E \log E)$ 。可用于“最小化最大边”“第二小生成树”的基础。

12.4.2 Prim

从任意点出发，每次选未加入点中连接代价最小者，堆优化 $O(E \log V)$ ；稠密图可用朴素 $O(V^2)$ 。

MST 性质：若一条非树边加入形成环，环上最大边被替换可得到候选；用于瓶颈路径、边权敏感性、树上倍增求环上最大值。

```
sort(edges.begin(), edges.end(),
      [](auto &a, auto &b) { return a.w < b.w; });
DSU dsu(n); long long cost = 0; int used = 0;
for (auto e : edges) if (dsu.unite(e.u, e.v)) {
    cost += e.w; ++used;
}
bool connected = (used == n - 1);
```

```
vector<long long> best(n, INF); vector<char> used_prim(n);
priority_queue<pair<long long, int>, vector<pair<long long, int>>, greater<>>
    pq2;
best[0] = 0; pq2.push({0, 0}); long long mst = 0;
while (!pq2.empty()) {
    auto [w, u] = pq2.top(); pq2.pop();
    if (used_prim[u]) continue;
    used_prim[u] = true; mst += w;
    for (auto [v, c] : weighted_graph[u]) if (!used_prim[v] && c < best[v])
        best[v] = c, pq2.push({c, v});
}
```

12.5 强连通分量 (Strongly Connected Components, SCC) 与缩点

Tarjan: DFS 维护 dfn/low 与栈， $low[u] == dfn[u]$ 时弹出一个 SCC， $O(V+E)$ 。Kosaraju: 正图后序 + 反图 DFS，代码更直观。SCC 缩成 DAG 后做 DP、计数、可达性。缩点后不要把同一分量内的边当作 DAG 边。

```
int timer = 0; vector<int> dfn(n), low(n), stk, in(n), comp(n); int cc = 0;
void tarjan(int u) {
```

```

    dfn[u] = low[u] = ++timer; stk.push_back(u); in[u] = 1;
    for (int v : g[u]) {
        if (!dfn[v]) tarjan(v), low[u] = min(low[u], low[v]);
        else if (in[v]) low[u] = min(low[u], dfn[v]);
    }
    if (low[u] != dfn[u]) return;
    ++cc;
    while (true) {
        int v = stk.back(); stk.pop_back(); in[v] = 0; comp[v] = cc;
        if (v == u) break;
    }
}

```

12.6 割点、桥、双连通

无向图 Tarjan: $\text{low}[v] \geq \text{dfn}[u]$ 时, u 是割点分隔子树; $\text{low}[v] > \text{dfn}[u]$ 时 (u, v) 是桥。根节点成为割点的条件是 DFS 子树数大于 1。边有重边时必须用边 id 判断父边, 不能只判断父节点。

点双/边双连通可在 Tarjan 过程中用边栈或 DSU 缩桥。桥树上路径问题常转为树上 LCA/差分。

```

int timer = 0; vector<int> dfn(n), low(n); vector<pair<int, int>> bridges;
void dfs(int u, int parent_edge) {
    dfn[u] = low[u] = ++timer;
    for (auto [v, id] : g[u]) {
        if (id == parent_edge) continue;
        if (!dfn[v]) {
            dfs(v, id); low[u] = min(low[u], low[v]);
            if (low[v] > dfn[u]) bridges.push_back({u, v});
        } else low[u] = min(low[u], dfn[v]);
    }
}

```

12.7 二分图

12.7.1 染色判定

BFS/DFS 交替染色; 发现相邻同色则不是二分图。每个连通块可独立选择两种颜色, 计数时乘法合并。

```

vector<int> color(n, -1);
bool ok = true;

```

```

for (int s = 0; s < n; ++s) if (color[s] < 0) {
    queue<int> q; q.push(s); color[s] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) {
            if (color[v] < 0) color[v] = color[u] ^ 1, q.push(v);
            else if (color[v] == color[u]) ok = false;
        }
    }
}

```

12.7.2 最大匹配

- 匈牙利 DFS: 适合 $V, E \leq$ 几千, 复杂度 $O(VE)$ 。
- Hopcroft-Karp: 分层增广, $O(E\sqrt{V})$, 适合更大二分图。
- 二分图最小点覆盖 = 最大匹配 (König); 最大独立集 = V - 最大匹配。

```

vector<int> matchR(n, -1), vis(n);
bool aug(int u) {
    for (int v : g[u]) if (vis[v] != stamp) {
        vis[v] = stamp;
        if (matchR[v] == -1 || aug(matchR[v]))
            return matchR[v] = u, true;
    }
    return false;
}
int matching = 0;
for (int u = 0; u < left_n; ++u) ++stamp, matching += aug(u);

```

12.7.3 带权匹配

KM 算法求完备二分图最大权匹配, 通常约 $O(n^3)$; 需要处理负权时整体平移或按模板维护顶标。

12.8 网络流

12.8.1 建模

容量表示资源上限, 源点连供应端, 需求端连汇点。拆点处理点容量; 上下界流增加下界需求并用超级源汇检查可行性。

12.8.2 Dinic

BFS 建层次图，DFS 在层次递增的图上发送阻塞流；一般题目规模足够使用，二分图匹配也可套用。每条边要维护反向边 `cap`。

```
struct Edge { int to, rev; long long cap; };
vector<vector<Edge>> fg;
vector<int> level, it;
void add_edge(int u, int v, long long c) {
    Edge a{v, (int)fg[v].size(), c}, b{u, (int)fg[u].size(), 0};
    fg[u].push_back(a); fg[v].push_back(b);
}
bool bfs(int s, int t) {
    fill(level.begin(), level.end(), -1); queue<int> q;
    level[s] = 0; q.push(s);
    while (!q.empty()) { int u = q.front(); q.pop();
        for (auto &e : fg[u]) if (e.cap && level[e.to] < 0)
            level[e.to] = level[u] + 1, q.push(e.to);
    }
    return level[t] >= 0;
}
long long dfs(int u, int t, long long f) {
    if (u == t) return f;
    for (int &i = it[u]; i < (int)fg[u].size(); ++i) {
        Edge &e = fg[u][i];
        if (e.cap && level[e.to] == level[u] + 1) {
            long long got = dfs(e.to, t, min(f, e.cap));
            if (got) { e.cap -= got; fg[e.to][e.rev].cap += got; return got; }
        }
    }
    return 0;
}
```

最大流最小割：最大流值等于最小割容量；割边可由残量网络中从源可达的点集确定。最小割常用于“选/不选且有冲突代价”的闭合图建模。

12.8.3 费用流

增广最短路（势能 + Dijkstra 或 SPFA）反复找最小费用路径；适合规模较小的带费用匹配/运输，注意负边与势能初始化。

```
struct MCEdge { int to, rev, cap, cost; };
vector<vector<MCEdge>> mg(n);
void add_edge(int u, int v, int cap, int cost) {
    MCEdge a{v, (int)mg[v].size(), cap, cost};
```

```

    MCEdge b{u, (int)mg[u].size(), 0, -cost};
    mg[u].push_back(a); mg[v].push_back(b);
}
pair<int, long long> min_cost_flow(int s, int t, int need) {
    int flow = 0; long long cost = 0;
    vector<long long> pot(n), dist(n);
    vector<int> pv(n), pe(n);
    while (flow < need) {
        fill(dist.begin(), dist.end(), (long long)4e18);
        priority_queue<pair<long long, int>, vector<pair<long long, int>>,
            greater<>> pq;
        dist[s] = 0; pq.push({0, s});
        while (!pq.empty()) {
            auto [du, u] = pq.top(); pq.pop();
            if (du != dist[u]) continue;
            for (int i = 0; i < (int)mg[u].size(); ++i) {
                auto const &e = mg[u][i];
                if (!e.cap) continue;
                long long nd = du + e.cost + pot[u] - pot[e.to];
                if (nd < dist[e.to])
                    dist[e.to] = nd, pv[e.to] = u, pe[e.to] = i,
                    pq.push({nd, e.to});
            }
        }
        if (dist[t] == (long long)4e18) break;
        for (int v = 0; v < n; ++v) if (dist[v] < (long long)4e18) pot[v] +=
dist[v];
        int add = need - flow;
        for (int v = t; v != s; v = pv[v]) add = min(add, mg[pv[v]][pe[v]].cap
);
        for (int v = t; v != s; v = pv[v]) {
            auto &e = mg[pv[v]][pe[v]];
            e.cap -= add; mg[v][e.rev].cap += add; cost += 1LL * add * e.cost
;
        }
        flow += add;
    }
    return {flow, cost};
}

```

12.9 树上算法

12.9.1 DFS 序与子树

$\text{tin}[u]$ 、 $\text{tout}[u]$ 使子树变成连续区间 $[\text{tin}[u], \text{tout}[u]]$ 。子树加/和直接用树状数组或线段树；路径则还需 LCA/重链。

```
int timer = 0; vector<int> tin(n), tout(n), depth(n), euler(n + 1);
void dfs(int u, int p) {
    tin[u] = ++timer; euler[timer] = u;
    for (int v : tree[u]) if (v != p)
        depth[v] = depth[u] + 1, dfs(v, u);
    tout[u] = timer;
}
```

12.9.2 最近公共祖先 (Lowest Common Ancestor, LCA) 倍增

预处理 $\text{up}[u][j]$ 与深度，查询时先抬高深度差，再从大到小跳；预处理 $O(V \log V)$ ，查询 $O(\log V)$ 。同时可维护路径最小/最大/和。

```
const int LOG = 20;
int up[MAXN][LOG], dep[MAXN];
void init_lca(int u, int p) {
    up[u][0] = p;
    for (int j = 1; j < LOG; ++j) up[u][j] = up[up[u][j - 1]][j - 1];
    for (int v : tree[u]) if (v != p) dep[v] = dep[u] + 1, init_lca(v, u);
}
int lca(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    int d = dep[u] - dep[v];
    for (int j = 0; j < LOG; ++j) if (d >> j & 1) u = up[u][j];
    if (u == v) return u;
    for (int j = LOG - 1; j >= 0; --j)
        if (up[u][j] != up[v][j]) u = up[u][j], v = up[v][j];
    return up[u][0];
}
```

12.9.3 树上差分

点路径 (u, v) 加一: $d[u]++$, $d[v]++$, $d[\text{lca}]--$, $d[\text{parent}(\text{lca})]--$; 边路径把 $d[\text{lca}] -= 2$ 。后序累加得到每点/边被经过次数。

```
void add_path(int u, int v) {
    int w = lca(u, v);
    ++diff[u]; ++diff[v]; --diff[w];
    if (up[w][0]) --diff[up[w][0]];
}
```

```

}
void collect(int u, int p) {
    for (int v : tree[u]) if (v != p)
        collect(v, u), diff[u] += diff[v];
}

```

12.9.4 直径与重心

树直径：从任一点 BFS/DFS 找最远 s ，再从 s 找最远 t ； $\text{dist}(s,t)$ 为直径。带权树同理（边权非负）。重心：删除后最大连通块最小的点，可由子树大小判断。

```

pair<int, long long> farthest(int s) {
    vector<long long> d(n, -1); queue<int> q;
    d[s] = 0; q.push(s);
    int best = s;
    while (!q.empty()) { int u = q.front(); q.pop();
        if (d[u] > d[best]) best = u;
        for (auto [v, w] : weighted_tree[u]) if (d[v] < 0)
            d[v] = d[u] + w, q.push(v);
    }
    return {best, d[best]};
}

```

12.9.5 树形 DP

先子树后父亲。常见状态：选/不选（独立集）、子树颜色、背包合并、最大匹配、距离和。换根 DP：第一次 DFS 求根答案，第二次用父节点答案推出子节点答案。

```

void tree_dp(int u, int p) {
    dp[u][0] = 0; dp[u][1] = weight[u];
    for (int v : tree[u]) if (v != p) {
        tree_dp(v, u);
        dp[u][0] += max(dp[v][0], dp[v][1]);
        dp[u][1] += dp[v][0];
    }
}

```

12.9.6 重链剖分（Heavy-Light Decomposition, HLD）

把每条重链映射到数组，路径拆成 $O(\log V)$ 个连续区间，配合线段树实现路径/子树修改查询，复杂度 $O(\log^2 V)$ 。先求 size , heavy ，再求 top , dfn 。

```

void dfs1(int u, int p) {

```



```

    parent[u] = p; size[u] = 1; heavy[u] = 0;
    for (int v : tree[u]) if (v != p) {
        dep[v] = dep[u] + 1; dfs1(v, u); size[u] += size[v];
        if (!heavy[u] || size[v] > size[heavy[u]]) heavy[u] = v;
    }
}
void dfs2(int u, int topf) {
    top[u] = topf; dfn[u] = ++timer;
    if (heavy[u]) dfs2(heavy[u], topf);
    for (int v : tree[u]) if (v != parent[u] && v != heavy[u]) dfs2(v, v);
}
long long path_sum(int u, int v) {
    long long ans = 0;
    while (top[u] != top[v]) {
        if (dep[top[u]] < dep[top[v]]) swap(u, v);
        ans += seg.query(dfn[top[u]], dfn[u]); u = parent[top[u]];
    }
    if (dep[u] > dep[v]) swap(u, v);
    return ans + seg.query(dfn[u], dfn[v]);
}

```

12.9.7 点分治（入门）

以重心分治树，统计跨重心的路径信息，递归处理子树；每层总计 $O(V)$ ，深度 $O(\log V)$ ，常见复杂度 $O(V \log V)$ 。适用于树上距离小于/等于 k 的点对计数、距离最小值。

```

int get_size(int u, int p) {
    sz[u] = 1;
    for (int v : tree[u]) if (v != p && !removed[v]) sz[u] += get_size(v, u);
    return sz[u];
}
int get_centroid(int u, int p, int total) {
    for (int v : tree[u]) if (v != p && !removed[v])
        if (sz[v] * 2 > total) return get_centroid(v, u, total);
    return u;
}
void divide(int entry) {
    int c = get_centroid(entry, 0, get_size(entry, 0));
    removed[c] = true;
    // 统计经过 c 的贡献
    for (int v : tree[c]) if (!removed[v]) divide(v);
}

```

12.9.8 树上启发式合并 (Disjoint Set Union on Tree, DSU on Tree)

先求子树大小和重儿子；轻儿子递归后清空，重儿子保留统计结果，再把所有轻子树合并进来。每个节点被清除/加入的次数受重轻性质控制，总复杂度通常为 $O(V \log V)$ 。

```
vector<int> sz(n + 1), heavy(n + 1); // 子树大小、重儿子。
vector<int> color(n + 1), freq(MAXC); // 颜色和当前统计集合中的频次。
vector<int> answer(n + 1);
int distinct = 0; // 示例维护量：当前集合中出现过的颜色数。

void dfs_size(int u, int p) {
    // 第一次 DFS 只决定轻/重儿子，不修改答案统计。
    sz[u] = 1;
    for (int v : tree[u]) if (v != p) {
        dfs_size(v, u); sz[u] += sz[v];
        if (!heavy[u] || sz[v] > sz[heavy[u]]) heavy[u] = v;
    }
}

void add_subtree(int u, int p, int delta) {
    // delta=+1 是加入，delta=-1 是清空。
    // 这里的“答案维护器”可以替换成最大频率、频率平方和等任何可增删统计。
    if (delta == 1 && freq[color[u]]++ == 0) ++distinct;
    if (delta == -1 && --freq[color[u]] == 0) --distinct;
    // 必须递归整棵子树；只处理 u 会漏掉大量颜色。
    for (int v : tree[u]) if (v != p) add_subtree(v, u, delta);
}

void dsu_on_tree(int u, int p, bool keep) {
    // 轻儿子先算并清空：它们只贡献一次，避免统计互相污染。
    for (int v : tree[u]) if (v != p && v != heavy[u]) dsu_on_tree(v, u, false);
    // 重儿子保留答案，后面把轻子树和 u 合并进这份大集合。
    if (heavy[u]) dsu_on_tree(heavy[u], u, true);
    // 把所有轻儿子整棵加入重儿子的统计集合。
    for (int v : tree[u]) if (v != p && v != heavy[u]) add_subtree(v, u, 1);
    // 最后加入 u 自己，此时集合恰好是 u 的整棵子树。
    if (freq[color[u]]++ == 0) ++distinct;
    answer[u] = distinct; // 换成题目所需的当前子树统计量
    if (!keep) {
        // u 是父节点的轻儿子时，当前统计不会被父节点复用，整棵删除。
        add_subtree(u, p, -1);
    }
}
```

12.9.9 长链剖分

长链剖分按“向下最长深度”而不是子树大小选重儿子，适合树上深度/祖先方向的 DP 优化和长链合并。它和 HLD 的选重标准不同，不能混用复杂度证明。

```
vector<int> parent2(n + 1), depth2(n + 1), heavy_len(n + 1);
vector<int> heavy2(n + 1), top2(n + 1);
void long_chain_dfs1(int u, int p) {
    // heavy_len[u] 是从 u 向下的最长链长度，不是子树大小。
    parent2[u] = p; heavy_len[u] = 1;
    for (int v : tree[u]) if (v != p) {
        depth2[v] = depth2[u] + 1;
        long_chain_dfs1(v, u);
        // 选择向下最深的儿子作为长链儿子。
        if (!heavy2[u] || heavy_len[v] > heavy_len[heavy2[u]]) heavy2[u] = v;
    }
    if (heavy2[u]) heavy_len[u] = heavy_len[heavy2[u]] + 1;
}
void long_chain_dfs2(int u, int topf) {
    // 长儿子沿用同一个链顶，其他儿子各自开新链。
    top2[u] = topf;
    if (heavy2[u]) long_chain_dfs2(heavy2[u], topf);
    for (int v : tree[u])
        if (v != parent2[u] && v != heavy2[u]) long_chain_dfs2(v, v);
}
```

12.9.10 Link-Cut Tree 动态树 (LCT)

LCT 用 splay 维护动态森林，支持 link/cut、换根和路径聚合。下面模板维护路径和；将 pull 中的 sum 换成结构体即可维护最大值、异或等可合并信息。

```
struct LCT {
    struct Node {
        int ch[2]{}; // splay 左右儿子。
        int fa{}; // 辅助树父亲；不等同于原树父亲。
        long long val{}; // 点权。
        long long sum{}; // 当前 splay 子树的维护信息。
        bool rev{}; // 是否有待下传的换根翻转标记。
    };
    vector<Node> t;
    LCT(int n): t(n + 1) {}
    bool is_root(int x) const {
        // x 没有 splay 父亲，或不是父亲的直接儿子时，是辅助树根。
        int f = t[x].fa;
        return !f || (t[f].ch[0] != x && t[f].ch[1] != x);
    }
```

```

}
void pull_sum(int x) {
    // 修改儿子后必须重新计算聚合信息。
    t[x].sum = t[x].val + t[t[x].ch[0]].sum + t[t[x].ch[1]].sum;
}
void apply_rev(int x) {
    if (!x) return;
    // 翻转只交换辅助树儿子；原树路径方向由 makeroot 实现。
    swap(t[x].ch[0], t[x].ch[1]); t[x].rev ^= 1;
}
void push(int x) {
    // 旋转前必须把从辅助树根到 x 的所有 rev 标记按顺序下传。
    if (t[x].rev) apply_rev(t[x].ch[0]), apply_rev(t[x].ch[1]), t[x].rev = false;
}
void rotate(int x) {
    // 一次旋转把 x 提升；b 是 x 原来靠旋转方向的另一棵子树。
    int y = t[x].fa, z = t[y].fa, dx = (t[y].ch[1] == x), b = t[x].ch[dx ^ 1];
    if (!is_root(y)) t[z].ch[t[z].ch[1] == y] = x;
    t[x].fa = z; t[x].ch[dx ^ 1] = y; t[y].fa = x; t[y].ch[dx] = b;
    if (b) t[b].fa = y;
    pull_sum(y); pull_sum(x);
}
void splay(int x) {
    // 先收集祖先，再从上到下 push，确保旋转时方向标记正确。
    vector<int> st{ x };
    for (int y = x; !is_root(y); y = t[y].fa) st.push_back(t[y].fa);
    while (!st.empty()) push(st.back()), st.pop_back();
    while (!is_root(x)) {
        int y = t[x].fa, z = t[y].fa;
        if (!is_root(y)) ((t[y].ch[0] == x) ^ (t[z].ch[0] == y)) ? rotate(x) : rotate(y);
        rotate(x);
    }
}
void access(int x) {
    // 依次把 x 到原树根的 preferred path 变成一条辅助树路径。
    // y 是上一轮处理的节点，挂到 x 的右儿子。
    for (int y = 0; x; x = t[y = x].fa) splay(x), t[x].ch[1] = y, pull_sum(x);
}
void makeroot(int x) {
    // access+splay 后 x 表示整条根到 x 的路径，翻转它即可让 x 成为原树根。

```

```

        access(x); splay(x); apply_rev(x);
    }
    int findroot(int x) {
        // 暴露 x 后不断向左走，左端点就是所在原树的根。
        access(x); splay(x);
        while (t[x].ch[0]) push(x), x = t[x].ch[0];
        splay(x); return x;
    }
    void split(int x, int y) {
        // 之后 y 的 splay 子树正好维护原树路径 x..y。
        makeroot(x); access(y); splay(y);
    }
    void link(int x, int y) {
        // 先换根保证 x 没有原树父亲，再确认两点不连通。
        makeroot(x); if (findroot(y) != x) t[x].fa = y;
    }
    void cut(int x, int y) {
        // 暴露边 x-y; 合法形态要求 y 的左儿子是 x 且 x 没有右儿子。
        split(x, y);
        if (t[y].ch[0] == x && !t[x].ch[1]) t[y].ch[0] = t[x].fa = 0, pull_sum
(y);
    }
    long long path_sum(int x, int y) {
        split(x, y);
        return t[y].sum;
    }
};

```

12.10 欧拉路、欧拉回路与哈密顿误区

无向图存在欧拉回路当且仅当非零度点连通且所有度为偶；欧拉路径有且仅有 0 或 2 个奇度点。有向图对应入度 = 出度，路径起点出度比入度多 1、终点相反。Hierholzer 用栈在线性时间构造。哈密顿路径一般是 NP-hard，不要把两者混淆。

```

vector<int> ptr(n), euler;
void hierholzer(int u) {
    while (ptr[u] < (int)g[u].size()) {
        auto [v, id] = g[u][ptr[u]++];
        if (used[id]) continue;
        used[id] = true; hierholzer(v);
    }
    euler.push_back(u);
}

```

12.11 图论组合套路

- “删掉一些边使图满足条件” 常考虑 MST、割、桥、最小割。
- “两点之间经过某类边最少” 常做分层图/状态图。
- “最多经过 k 次特殊边” 建 $k+1$ 层节点。
- “必须恰好经过/使用” 常用流、DP 或矩阵，不要只跑最短路。
- DAG + 计数要同时维护 `ways[v]`，相同最优值时累加、更新更优时覆盖。

13 树

13.1 树上 BFS / DFS

- 无向树 DFS 必须传父亲参数 `dfs(u, fa)`，并在循环中 `if(v == fa) continue`，否则死循环。
- 链状树递归 DFS 可能爆栈（ 10^5 层以上），考虑改用 BFS 或手动栈。
- 图不一定连通（可能是森林），外层要循环所有未访问的点。

```
vector<int> G[N];
int dep[N], sz[N], fa[N];

// 树上 BFS（求深度、父亲）
void bfs(int root) {
    queue<int> q;
    dep[root] = 0; fa[root] = 0;
    q.push(root);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : G[u]) {
            if (v == fa[u]) continue;
            fa[v] = u;
            dep[v] = dep[u] + 1;
            q.push(v);
        }
    }
}

// 树上 DFS（求子树大小、深度）
void dfs(int u, int f) {
    fa[u] = f;
    sz[u] = 1;
    dep[u] = dep[f] + 1;
    for (int v : G[u]) {
```

```

        if (v == f) continue;
        dfs(v, u);
        sz[u] += sz[v];
    }
}

```

13.2 LCA（倍增）

预处理 $O(n \log n)$ ，每次查询 $O(\log n)$ 。

- 倍增预处理：外层循环步长 i ，内层循环节点 u ，写反直接全错。
- $fa[u][i]$ 第二维通常开 20 ($2^{20} > 10^6$)。

```

const int LOG = 20;
int fa[N][LOG], dep[N];

void bfs_init(int root, int n) {
    memset(fa, 0, sizeof(fa));
    dep[root] = 1;
    queue<int> q;
    q.push(root);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : G[u]) {
            if (v == fa[u][0]) continue;
            fa[v][0] = u;
            dep[v] = dep[u] + 1;
            q.push(v);
        }
    }
    // 预处理倍增（外层步长，内层节点）
    for (int i = 1; i < LOG; i++)
        for (int u = 1; u <= n; u++)
            fa[u][i] = fa[fa[u][i-1]][i-1];
}

int lca(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    int diff = dep[u] - dep[v];
    for (int i = 0; i < LOG; i++)
        if ((diff >> i) & 1) u = fa[u][i];
}

```



```

    if (u == v) return u;
    for (int i = LOG - 1; i >= 0; i--)
        if (fa[u][i] != fa[v][i]) {
            u = fa[u][i];
            v = fa[v][i];
        }
    return fa[u][0];
}

```

树上两点距离: $\text{dist}(u, v) = \text{dep}[u] + \text{dep}[v] - 2 \cdot \text{dep}[\text{lca}(u, v)]$ 。

13.3 树上算法

13.3.1 DFS 序与子树

$\text{tin}[u]$ 、 $\text{tout}[u]$ 使子树变成连续区间 $[\text{tin}[u], \text{tout}[u]]$ 。子树加/和直接用树状数组或线段树；路径则还需 LCA/重链。

```

int timer = 0; vector<int> tin(n), tout(n), depth(n), euler(n + 1);
void dfs(int u, int p) {
    tin[u] = ++timer; euler[timer] = u;
    for (int v : tree[u]) if (v != p)
        depth[v] = depth[u] + 1, dfs(v, u);
    tout[u] = timer;
}

```

13.3.2 最近公共祖先 (Lowest Common Ancestor, LCA) 倍增

预处理 $\text{up}[u][j]$ 与深度，查询时先抬高深度差，再从大到小跳；预处理 $O(V \log V)$ ，查询 $O(\log V)$ 。同时可维护路径最小/最大/和。

```

const int LOG = 20;
int up[MAXN][LOG], dep[MAXN];
void init_lca(int u, int p) {
    up[u][0] = p;
    for (int j = 1; j < LOG; ++j) up[u][j] = up[up[u][j - 1]][j - 1];
    for (int v : tree[u]) if (v != p) dep[v] = dep[u] + 1, init_lca(v, u);
}
int lca(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    int d = dep[u] - dep[v];
    for (int j = 0; j < LOG; ++j) if (d >> j & 1) u = up[u][j];
}

```

```

    if (u == v) return u;
    for (int j = LOG - 1; j >= 0; --j)
        if (up[u][j] != up[v][j]) u = up[u][j], v = up[v][j];
    return up[u][0];
}

```

13.3.3 树上差分

点路径 (u, v) 加一: $d[u]++$, $d[v]++$, $d[lca]--$, $d[parent(lca)]--$; 边路径把 $d[lca] -= 2$ 。后序累加得到每点/边被经过次数。

```

void add_path(int u, int v) {
    int w = lca(u, v);
    ++diff[u]; ++diff[v]; --diff[w];
    if (up[w][0]) --diff[up[w][0]];
}
void collect(int u, int p) {
    for (int v : tree[u]) if (v != p)
        collect(v, u), diff[u] += diff[v];
}

```

13.3.4 直径与重心

树直径: 从任一点 BFS/DFS 找最远 s , 再从 s 找最远 t ; $\text{dist}(s, t)$ 为直径。带权树同理 (边权非负)。重心: 删除后最大连通块最小的点, 可由子树大小判断。

```

pair<int, long long> farthest(int s) {
    vector<long long> d(n, -1); queue<int> q;
    d[s] = 0; q.push(s);
    int best = s;
    while (!q.empty()) { int u = q.front(); q.pop();
        if (d[u] > d[best]) best = u;
        for (auto [v, w] : weighted_tree[u]) if (d[v] < 0)
            d[v] = d[u] + w, q.push(v);
    }
    return {best, d[best]};
}

```

13.3.5 树形动态规划 (Tree Dynamic Programming)

先子树后父亲。常见状态: 选/不选 (独立集)、子树颜色、背包合并、最大匹配、距离和。换根 DP: 第一次 DFS 求根答案, 第二次用父节点答案推出子节点答案。

```

void tree_dp(int u, int p) {
    dp[u][0] = 0; dp[u][1] = weight[u];
    for (int v : tree[u]) if (v != p) {
        tree_dp(v, u);
        dp[u][0] += max(dp[v][0], dp[v][1]);
        dp[u][1] += dp[v][0];
    }
}

```

13.3.6 重链剖分 (Heavy-Light Decomposition, HLD)

把每条重链映射到数组，路径拆成 $O(\log V)$ 个连续区间，配合线段树实现路径/子树修改查询，复杂度 $O(\log^2 V)$ 。先求 `size`, `heavy`, 再求 `top`, `dfn`。

```

void dfs1(int u, int p) {
    parent[u] = p; size[u] = 1; heavy[u] = 0;
    for (int v : tree[u]) if (v != p) {
        dep[v] = dep[u] + 1; dfs1(v, u); size[u] += size[v];
        if (!heavy[u] || size[v] > size[heavy[u]]) heavy[u] = v;
    }
}

void dfs2(int u, int topf) {
    top[u] = topf; dfn[u] = ++timer;
    if (heavy[u]) dfs2(heavy[u], topf);
    for (int v : tree[u]) if (v != parent[u] && v != heavy[u]) dfs2(v, v);
}

long long path_sum(int u, int v) {
    long long ans = 0;
    while (top[u] != top[v]) {
        if (dep[top[u]] < dep[top[v]]) swap(u, v);
        ans += seg.query(dfn[top[u]], dfn[u]); u = parent[top[u]];
    }
    if (dep[u] > dep[v]) swap(u, v);
    return ans + seg.query(dfn[u], dfn[v]);
}

```

13.3.7 点分治 (入门)

以重心分治树，统计跨重心的路径信息，递归处理子树；每层总计 $O(V)$ ，深度 $O(\log V)$ ，常见复杂度 $O(V \log V)$ 。适用于树上距离小于/等于 k 的点对计数、距离最小值。

```

int get_size(int u, int p) {
    sz[u] = 1;

```

```

    for (int v : tree[u]) if (v != p && !removed[v]) sz[u] += get_size(v, u);
    return sz[u];
}
int get_centroid(int u, int p, int total) {
    for (int v : tree[u]) if (v != p && !removed[v])
        if (sz[v] * 2 > total) return get_centroid(v, u, total);
    return u;
}
void divide(int entry) {
    int c = get_centroid(entry, 0, get_size(entry, 0));
    removed[c] = true;
    // 统计经过 c 的贡献
    for (int v : tree[c]) if (!removed[v]) divide(v);
}

```

13.3.8 树上启发式合并 (Disjoint Set Union on Tree, DSU on Tree)

先求子树大小和重儿子；轻儿子递归后清空，重儿子保留统计结果，再把所有轻子树合并进来。每个节点被清除/加入的次数受重轻性质控制，总复杂度通常为 $O(V \log V)$ 。

```

vector<int> sz(n + 1), heavy(n + 1), color(n + 1), freq(MAXC);
vector<int> answer(n + 1);
int distinct = 0;

void dfs_size(int u, int p) {
    sz[u] = 1;
    for (int v : tree[u]) if (v != p) {
        dfs_size(v, u); sz[u] += sz[v];
        if (!heavy[u] || sz[v] > sz[heavy[u]]) heavy[u] = v;
    }
}

void add_subtree(int u, int p, int delta) {
    if (delta == 1 && freq[color[u]]++ == 0) ++distinct;
    if (delta == -1 && --freq[color[u]] == 0) --distinct;
    for (int v : tree[u]) if (v != p) add_subtree(v, u, delta);
}

void dsu_on_tree(int u, int p, bool keep) {
    for (int v : tree[u]) if (v != p && v != heavy[u]) dsu_on_tree(v, u, false);
    if (heavy[u]) dsu_on_tree(heavy[u], u, true);
    for (int v : tree[u]) if (v != p && v != heavy[u]) add_subtree(v, u, 1);
    if (freq[color[u]]++ == 0) ++distinct;
    answer[u] = distinct; // 换成题目所需的当前子树统计量
    if (!keep) add_subtree(u, p, -1);
}

```

```
}
```

13.3.9 长链剖分

长链剖分按“向下最长深度”而不是子树大小选重儿子，适合树上深度/祖先方向的 DP 优化和长链合并。它和 HLD 的选重标准不同，不能混用复杂度证明。

```
vector<int> parent2(n + 1), depth2(n + 1), heavy_len(n + 1);
vector<int> heavy2(n + 1), top2(n + 1);
void long_chain_dfs1(int u, int p) {
    parent2[u] = p; heavy_len[u] = 1;
    for (int v : tree[u]) if (v != p) {
        depth2[v] = depth2[u] + 1;
        long_chain_dfs1(v, u);
        if (!heavy2[u] || heavy_len[v] > heavy_len[heavy2[u]]) heavy2[u] = v;
    }
    if (heavy2[u]) heavy_len[u] = heavy_len[heavy2[u]] + 1;
}
void long_chain_dfs2(int u, int topf) {
    top2[u] = topf;
    if (heavy2[u]) long_chain_dfs2(heavy2[u], topf);
    for (int v : tree[u])
        if (v != parent2[u] && v != heavy2[u]) long_chain_dfs2(v, v);
}
```

13.3.10 Link-Cut Tree 动态树 (LCT)

LCT 用 splay 维护动态森林，支持 link/cut、换根和路径聚合。下面模板维护路径和；将 pull 中的 sum 换成结构体即可维护最大值、异或等可合并信息。

```
struct LCT {
    struct Node { int ch[2]{}; fa{}; long long val{}; sum{}; bool rev{}; };
    vector<Node> t;
    LCT(int n): t(n + 1) {}
    bool is_root(int x) const {
        int f = t[x].fa;
        return !f || (t[f].ch[0] != x && t[f].ch[1] != x);
    }
    void pull_sum(int x) { t[x].sum = t[x].val + t[t[x].ch[0]].sum + t[t[x].ch[1]].sum; }
    void apply_rev(int x) { if (!x) return; swap(t[x].ch[0], t[x].ch[1]); t[x].rev ^= 1; }
    void push(int x) { if (t[x].rev) apply_rev(t[x].ch[0]), apply_rev(t[x].ch[1]), t[x].rev = false; }
```

```

void rotate(int x) {
    int y = t[x].fa, z = t[y].fa, dx = (t[y].ch[1] == x), b = t[x].ch[dx ^ 1];
    if (!is_root(y)) t[z].ch[t[z].ch[1] == y] = x;
    t[x].fa = z; t[x].ch[dx ^ 1] = y; t[y].fa = x; t[y].ch[dx] = b;
    if (b) t[b].fa = y; pull_sum(y); pull_sum(x);
}
void splay(int x) {
    vector<int> st{ x };
    for (int y = x; !is_root(y); y = t[y].fa) st.push_back(t[y].fa);
    while (!st.empty()) push(st.back()), st.pop_back();
    while (!is_root(x)) {
        int y = t[x].fa, z = t[y].fa;
        if (!is_root(y)) ((t[y].ch[0] == x) ^ (t[z].ch[0] == y)) ? rotate(x) : rotate(y);
        rotate(x);
    }
}
void access(int x) {
    for (int y = 0; x; x = t[y = x].fa) splay(x), t[x].ch[1] = y, pull_sum(x);
}
void makeroot(int x) { access(x); splay(x); apply_rev(x); }
int findroot(int x) {
    access(x); splay(x);
    while (t[x].ch[0]) push(x), x = t[x].ch[0];
    splay(x); return x;
}
void split(int x, int y) { makeroot(x); access(y); splay(y); }
void link(int x, int y) { makeroot(x); if (findroot(y) != x) t[x].fa = y; }
void cut(int x, int y) {
    split(x, y);
    if (t[y].ch[0] == x && !t[x].ch[1]) t[y].ch[0] = t[x].fa = 0, pull_sum(y);
}
long long path_sum(int x, int y) { split(x, y); return t[y].sum; }
};

```

13.4 点分治

每层找重心，把跨重心的路径贡献按子树统计并容斥，递归子树。统计距离、颜色、路径权值时要设计可合并的信息；重心分解树还能支持在线修改/查询。

```

int get_centroid(int u, int p, int total) {
    for (int v : tree[u]) if (v != p && !removed[v])
        if (sz[v] * 2 > total) return get_centroid(v, u, total);
    return u;
}
void decompose(int entry) {
    int c = get_centroid(entry, 0, calc_size(entry, 0));
    removed[c] = true;
    for (int v : tree[c]) if (!removed[v]) decompose(v);
}

```

13.5 虚树

给定树上少量关键点，只保留关键点及其两两相邻 LCA。按 DFS 序排序并用栈建边，节点数 $O(k)$ ；在虚树上做 DP，常用于多次询问的“关键点之间代价”。

```

sort(key.begin(), key.end(), [&](int a, int b) { return tin[a] < tin[b]; });
vector<int> nodes = key;
for (int i = 0, m = key.size(); i + 1 < m; ++i) nodes.push_back(lca(key[i],
    key[i + 1]));
sort(nodes.begin(), nodes.end(), [&](int a, int b) { return tin[a] < tin[b];
    });
nodes.erase(unique(nodes.begin(), nodes.end()), nodes.end());
vector<pair<int, int>> virtual_edges;
vector<int> st;
for (int u : nodes) {
    while (!st.empty() && !is_ancestor(st.back(), u)) st.pop_back();
    if (!st.empty()) virtual_edges.push_back({st.back(), u});
    st.push_back(u);
}

```

13.6 二进制提升的更多用法

不只求 LCA：沿跳跃表同时维护最大边、最小边、边权和、函数迭代。函数图（每点出度 1）先找环，再用倍增回答第 k 步位置，复杂度 $O((n+q)\log K)$ 。

```

int jump(int u, long long k) {
    for (int j = 0; j < LOG; ++j) if (k >> j & 1) u = up[u][j];
    return u;
}

```

13.7 树上高级算法：参数、调用与改造

本节紧跟前面的基础模板。先看每个接口传什么，再看哪些不变量不能动，最后看如何替换维护信息或转移。

13.7.1 2.7 树上启发式合并 (Disjoint Set Union on Tree)

识别信号：树不修改，每个节点有颜色/权值，每个询问问“子树中出现次数最多的颜色”“不同颜色数”“频率平方和”等。先求重儿子，保留重儿子的贡献，轻儿子算完后清空，复杂度通常 $O(n \log n)$ 。

```
vector<vector<int>> g;
vector<int> sz, son, color, ans;
vector<int> cnt;
long long cur_answer = 0; // 示例：维护所有颜色频次的平方和。

void dfs_size(int u, int fa) {
    // sz 和 son 只计算一次；son[u] 是子树最大的儿子。
    sz[u] = 1;
    for (int v : g[u]) if (v != fa) {
        dfs_size(v, u);
        sz[u] += sz[v];
        if (sz[v] > sz[son[u]]) son[u] = v;
    }
}

void add_subtree(int u, int fa, int delta) {
    // 先删除颜色旧贡献，再改变频次，最后加入新贡献。
    // 这里选 freq^2 作为演示；换题时只改这段“单点统计器”。
    cur_answer -= 1LL * cnt[color[u]] * cnt[color[u]];
    cnt[color[u]] += delta;
    cur_answer += 1LL * cnt[color[u]] * cnt[color[u]];

    // 加/删的是整棵子树，不是只有根 u。
    for (int v : g[u])
        if (v != fa) add_subtree(v, u, delta);
}

void dsu(int u, int fa, bool keep) {
    // 轻儿子先算完并清空，避免它们的统计污染当前节点。
    for (int v : g[u])
        if (v != fa && v != son[u]) dsu(v, u, false);

    // 重儿子的统计保留，之后所有轻子树都合并到这份大答案中。
    if (son[u]) dsu(son[u], u, true);
}
```



```

// 把轻儿子整棵子树逐个加入。
for (int v : g[u])
    if (v != fa && v != son[u]) add_subtree(v, u, +1);
add_subtree(u, fa, +1);

// 当前集合恰好是 u 的整棵子树，读取题目答案。
ans[u] = (int)cur_answer; // 换成题目要求的统计量

if (!keep) {
    // 父节点不需要这棵轻子树的贡献，因此整棵删除。
    add_subtree(u, fa, -1);
}
}

```

例题：CF600E Lomsat gelral。题目问每个子树中出现次数最多的颜色之和。上面代码中的 `cur_answer` 要改为“维护当前最大频次 `mx`，每次 `cnt[color]` 从旧频次变为新频次时更新 `sum_color[mx]`”；算法框架完全不变。不要把 `add_subtree` 写成只加节点 `u`，轻儿子整棵子树必须加入。

测试样例：树上启发式合并必须统计整棵子树

为了直接验证上面示例中的 `freq^2` 统计器，下面输出每个节点子树内各颜色频次平方和。输入每组为 `n`、颜色、`n-1` 条无向边。

输入

```

3
1
5
3
1 2 1
1 2
1 3
5
1 2 2 3 2
1 2
1 3
3 4
3 5

```

输出

```

1
5 1 1
11 1 5 1 1

```

第二组能抓住“只加入节点 u 、没有加入轻儿子整棵子树”的错误；第三组同时包含重儿子和轻儿子，检查清空轻子树后父节点统计是否恢复正确。

13.7.2 2.8 长链剖分

识别信号：树上大量询问“第 k 个祖先/深度为某值的节点”， k 很大，普通倍增内存或常数不理想。长链剖分按“向下最深”的儿子选重儿子，并把同一条长链连续存储，常配合 $O(1)$ 的第 k 级祖先。

```
void dfs_len(int u, int fa) {
    // up[u] 是父亲；len[u] 是从 u 向下走的最长链节点数。
    up[u] = fa; len[u] = 1; son[u] = 0;
    for (int v : g[u]) if (v != fa) {
        dfs_len(v, u);
        // 长链剖分按“深度最长”选重儿子，不是按子树大小。
        if (len[v] + 1 > len[u])
            len[u] = len[v] + 1, son[u] = v;
    }
}

void dfs_chain(int u, int top) {
    // 同一条长链上的节点共享链顶 top，并连续编号。
    chain_top[u] = top;
    chain_pos[u] = ++timer;

    // 先沿长儿子走，保证一条长链连续。
    if (son[u]) dfs_chain(son[u], top);

    // 其他儿子都是新长链的起点。
    for (int v : g[u])
        if (v != up[u] && v != son[u]) dfs_chain(v, v);
}
```

例题：P5903 树上 K 级祖先。读入查询 (x, k) 后，如果 $k > \text{depth}[x]$ 输出 0；否则沿长链的 chain_top 和预处理的链数组跳转。这个题还特别容易爆栈， $n=5e5$ 时要按题目环境决定是否使用非递归 DFS 或栈扩展。

测试样例：长链、星形树和超过根深度

每组输入 n q ，随后给出节点 $2..n$ 的父亲，再给出 q 个 (x, k) ；不存在的祖先输出 0。

```
输入
3
1 3
1 0
```

```

1 1
1 0
4 4
1 2 3
4 0
4 1
4 3
4 4
5 4
1 1 1 1
2 1
2 2
5 0
1 0

```

输出

```

1
0
1
4
3
1
0
1
0
5
1

```

第一组检查单点和 $k=0$ ；第二组是最长链，第三组是星形树。 k 超过深度时必须输出 0 ，不能访问未初始化的倍增表。

13.7.3 2.9 点分治与点分树

静态树上“距离为 k 的点数/距离小于等于 k 的点权和”是点分治；有在线修改则把每个节点到各层重心的距离放入重心树容器，形成点分树。

```

void collect(int u, int fa, int d, vector<int>& ds) {
    // ds 保存“当前重心 c 到该分支中所有节点的距离”。
    // 点分治的每层都只收集未被 removed 的节点。
    ds.push_back(d);
    for (int v : g[u])
        if (v != fa && !removed[v]) collect(v, u, d + 1, ds);
}

void decompose(int entry) {

```

```

// get_centroid 必须只在当前未删除连通块中找重心。
int c = get_centroid(entry); // 按子树大小找重心
removed[c] = true;

for (int v : g[c]) if (!removed[v]) {
    vector<int> ds;
    collect(v, c, 1, ds);

    // 把该分支的距离加入 c 的排序数组；查询时二分。
    // 同时还要把这些距离按“属于哪一个 c 的儿子分支”保存，
    // 查询时减掉同一分支的重复贡献（容斥）。
}

// 删除 c 后，剩下的是若干个互不相连的子问题。
for (int v : g[c]) if (!removed[v]) decompose(v);
}

```

例题可以从 P6329 点分树 | 震波开始：修改一个点的权值时，沿重心父链向上，把该点到每个重心的距离对应的贡献更新；查询重心时，对每层距离数组二分，并减去“同一子树方向”的重复贡献。点分树模板比普通点分治多一层“重心祖先链”，现场不要混用两个模板。

说明：当前代码只展示了点分治的分解和距离收集框架，没有给出完整的 `get_centroid`、查询容器、修改和答案接口。因此这里不伪造“输入/标准输出”样例；等这些接口补全后，测试必须至少包含单点、链、星形树、半径为 θ 和跨重心重复计数五类情况。

13.7.4 2.12 伸展树、文艺平衡树与 Link-Cut Tree 动态树

Splay 适合维护序列：节点有左右儿子、父亲、子树大小和翻转标记；把第 k 个节点旋到根，就能做区间翻转、插入、删除。文艺平衡树的区间反转调用流程是：把 $l-1$ 旋到根，把 $r+1$ 旋到根的右儿子，目标区间就是右儿子的左子树，打 `rev` 标记。

LCT 适合动态森林的路径操作，核心接口是 `makeroot(x)`；`access(y)`；`splay(y)`，此时 y 的 splay 子树就是原树路径 $x..y$ 。在已有 LCT 模板中，题目操作的对应关系通常是：

```

// 路径查询 x..y:
// makeroot(x) 把 x 变成原树根；split(x,y) 再 access(y)，
// 此时 y 的辅助树包含原树路径 x..y，维护值在 tr[y] 中汇总。
makeroot(x);
split(x, y);
answer = tr[y].sum;

// 连边：先确认两点不在同一棵树，否则会形成环。
makeroot(x);

```

```

if (findroot(y) != x) tr[x].fa = y;

// 断边：把边 x-y 暴露出来后，理想形态是 y 的左儿子恰为 x，
// 且 x 没有右儿子；这两个条件都满足才能安全删除。
makeroot(x);
access(y);
splay(y);
if (ch[y][0] == x && ch[x][1] == 0)
    ch[y][0] = fa[x] = 0;

```

例题：P3690 动态树要求路径异或、修改点权、连边和断边。先用已有板子完成 `is_root/rotate/splay/access/makeroot/findroot/split/link/cut`，再把 `sum` 的合并从加法换成异或。LCT 第一次现场学习风险很高，遇到它时优先复制完整模板，只修改“维护信息”和输入操作映射。

测试样例：动态树的路径、修改、连边和断边

下面采用 P3690 操作编号：0 x y 查询路径异或，1 x y 连边，2 x y 断边，3 x v 修改点权。

```

输入
3
1 3
5
0 1 1
3 1 7
0 1 1
3 7
1 2 4
1 1 2
1 2 3
0 1 3
3 2 5
0 1 3
2 2 3
0 1 2
4 7
1 1 1 1
1 1 2
1 3 4
0 1 2
1 2 3
0 1 4
2 2 3
0 1 2

```

输出

5

7

7

0

4

0

0

0

第二组检查路径合并和单点修改；第三组先建立两棵树，再连成一棵，最后断开，能抓住 **makeroot/access/splay** 顺序和断边条件错误。实际提交前还要根据题目保证避免查询不连通节点。

14 动态规划

DP 通用易错点：

- 求最大值初始化为 $-\text{inf}$ ，求最小值初始化为 inf 。初始化为 0 在有负数时会出错。
- 转移前判断前置状态是否合法：`if (dp[j] == inf) continue;`，防止从非法状态转移。
- 状态定义是“前 i 个”还是“以第 i 个结尾”？前者答案是 $dp[n]$ ，后者答案是 $\max(dp[1..n])$ 。
- 滚动数组每轮开头必须清空当前层，防止脏数据。

14.1 01 背包

n 个物品，每个只能选一次。容量 V ，第 i 个物品重 w_i ，价值 v_i 。

```
int dp[V + 1]; // dp[j] = 容量为 j 时的最大价值
memset(dp, 0, sizeof(dp));
for (int i = 0; i < n; i++)
    for (int j = V; j >= w[i]; j--) // 必须倒序!
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
// 答案: dp[V]
```

- 01 背包容量必须倒序遍历！正序会导致同一物品被重复选取（变成完全背包）。
- 内层循环下限是 $w[i]$ ，保证 $j - w[i] \geq 0$ 不越界。

14.2 完全背包

每个物品可以选无限次。

```
int dp[V + 1];
memset(dp, 0, sizeof(dp));
```

```
for (int i = 0; i < n; i++)
    for (int j = w[i]; j <= V; j++) // 正序! 允许重复选
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
```

14.3 多重背包（二进制拆分）

第 i 个物品最多选 c_i 次。将 c_i 拆成 $1, 2, 4, \dots$ 的组合，转化为 01 背包。

```
// 将每个物品按二进制拆分后加入 items
vector<pair<int,int>> items; // (weight, value)
for (int i = 0; i < n; i++) {
    int c = cnt[i], k = 1;
    while (k <= c) {
        items.push_back({w[i] * k, v[i] * k});
        c -= k;
        k *= 2;
    }
    if (c > 0) items.push_back({w[i] * c, v[i] * c});
}
// 然后跑 01 背包
for (auto& [wi, vi] : items)
    for (int j = V; j >= wi; j--)
        dp[j] = max(dp[j], dp[j - wi] + vi);
```

14.4 分组背包

k 组物品，每组最多选一个。

```
// groups[i] = 第 i 组的物品列表 {(w, v), ...}
for (int i = 0; i < k; i++) // 枚举每组
    for (int j = V; j >= 0; j--) // 倒序枚举容量
        for (auto& [wi, vi] : groups[i]) // 枚举组内物品
            if (j >= wi)
                dp[j] = max(dp[j], dp[j - wi] + vi);
```

分组背包的循环顺序：外层组、中层容量（倒序）、内层组内物品。容量和物品的顺序不能反！

14.5 最长上升子序列 (LIS)

$O(n \log n)$ 做法：维护一个数组 d ， $d[i]$ 表示长度为 $i + 1$ 的上升子序列的最小末尾元素。

```
// 严格递增 LIS
vector<int> d;
for (int i = 0; i < n; i++) {
    auto it = lower_bound(d.begin(), d.end(), a[i]);
    if (it == d.end()) d.push_back(a[i]);
    else *it = a[i];
}
int lis_len = d.size();
// 非严格递增：把 lower_bound 改成 upper_bound
```

- 严格递增用 `lower_bound`，非严格递增 ($\leq$) 用 `upper_bound`。
- 最长不下降子序列 (LIS 变体)：同样用 `upper_bound`。
- 注意： d 数组本身不是 LIS 的实际序列，只是辅助数组。

14.6 最长公共子序列 (LCS)

给定两个字符串 a, b ，求它们的最长公共子序列长度。 $dp[i][j]$ 表示 a 的前 i 个字符与 b 的前 j 个字符的 LCS 长度，时间复杂度 $O(nm)$ 。

```
int n = a.size(), m = b.size();
vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        if (a[i - 1] == b[j - 1])
            dp[i][j] = dp[i - 1][j - 1] + 1;
        else
            dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
    }
}
int lcs_len = dp[n][m];
```

14.6.1 还原一组 LCS

```

string lcs;
int i = n, j = m;
while (i > 0 && j > 0) {
    if (a[i - 1] == b[j - 1]) {
        lcs += a[i - 1];
        i--; j--;
    } else if (dp[i - 1][j] >= dp[i][j - 1]) {
        i--; // 相同长度时任选一边，得到的都是合法 LCS
    } else {
        j--;
    }
}
reverse(lcs.begin(), lcs.end());

```

- LCS 是子序列，字符不要求连续；要求连续匹配时是最长公共子串，转移不同。
- 只求长度时可滚动掉一维：vector<int> dp(m + 1)，空间降为 $O(m)$ 。
- 还原答案时遇到相等的最优转移可能有多条，题目通常接受任意一组 LCS；若要求字典序最小，需要额外设计。

14.7 区间 DP

区间 DP 的状态通常是 $dp[l][r]$ ，表示闭区间 $[l, r]$ 的最优值。按区间长度从短到长计算，再枚举最后一次分割点 k 。经典模型是“合并石子”：合并 $[l, r]$ 的代价为区间和。

```

// 1-indexed, 求把相邻石子合并成一堆的最小总代价
vector<long long> prefix(n + 1, 0);
for (int i = 1; i <= n; i++)
    prefix[i] = prefix[i - 1] + a[i];

const long long INF = (1LL << 62);
vector<vector<long long>> dp(n + 1,
    vector<long long>(n + 1, 0));
for (int len = 2; len <= n; len++) {
    for (int l = 1; l + len - 1 <= n; l++) {
        int r = l + len - 1;
        dp[l][r] = INF;
        long long w = prefix[r] - prefix[l - 1];
        for (int k = l; k < r; k++) {
            dp[l][r] = min(dp[l][r],
                dp[l][k] + dp[k + 1][r] + w);
        }
    }
}

```

```

    }
}
// 答案: dp[1][n]

```

- 区间长度从 1（或 2）递增，保证转移依赖的短区间已经计算。
- 求最小值时非空区间先设为 INF ；求最大值则设为 $-INF$ ，不能默认全为 0。
- 环形区间可把数组复制一遍，枚举长度不超过 n ，最后在所有长度为 n 的区间中取最优。
- 若转移只与端点有关（如回文串），可以省去分割点；不要套用 $O(n^3)$ 模板。

14.8 树上 DP（树形 DP）

状态 $dp[u][\dots]$ 表示以 u 为根的子树的某种最优值，自底向上：先递归处理所有子节点，再用子节点结果合并到 u 。无向树 DFS 必须传父亲。

- 无向树存边，DFS 必须传父亲 $dfs(u, fa)$ 并 `if(v == fa) continue;`，否则来回死循环。
- 链状数据递归深度可达 10^5 ，容易爆栈；必要时手写栈或调大栈空间。
- 状态在递归返回后才完整，合并子树的代码要写在 $dfs(v, u)$ ；之后。

14.8.1 树上最大权独立集（没有上司的舞会）

每个点有权 $w[u]$ ，选了一个点就不能选它的相邻点，求最大点权和。

```

vector<int> G[N];
int dp[N][2], w[N]; // dp[u][0]: 不选 u; dp[u][1]: 选 u
void dfs(int u, int fa) {
    dp[u][0] = 0;
    dp[u][1] = w[u];
    for (int v : G[u]) {
        if (v == fa) continue;
        dfs(v, u);
        dp[u][0] += max(dp[v][0], dp[v][1]); // 不选 u, 子可选可不选
        dp[u][1] += dp[v][0];                // 选 u, 子必不选
    }
}
// 答案: max(dp[root][0], dp[root][1])

```

14.8.2 树形背包（有依赖的背包 / 选课）

$dp[u][j]$ 表示在 u 的子树中、选 j 个点（或用容量 j ）的最大价值。合并子树时是分组背包，容量倒序枚举。整体复杂度 $O(n \cdot m)$ （精细实现）。

```
// 选课：选  $j$  门课，选子节点须先选其父（前置课）。建一个 0 号虚拟根连所有无前置的课
int dp[N][M], val[N]; // dp[u][j]: u 子树选  $j$  门的最大学分
void dfs(int u) {
    dp[u][1] = val[u]; // 至少选上 u 自己
    for (int v : G[u]) {
        dfs(v);
        for (int j = m; j >= 1; j--) // 倒序枚举分配给 u 子树的总数
            for (int k = 0; k <= j - 1; k++) // 给 v 子树 k 门，留  $\geq 1$  门给 u 链上
                dp[u][j] = max(dp[u][j], dp[u][j - k] + dp[v][k]);
    }
}
// 虚拟根 0: val[0]=0, 答案 dp[0][m+1] (含虚拟根那 1 门)
```

- 合并时枚举总容量 j 必须倒序，组内 k 正序，否则一个子树被重复计入。
- $dp[v][0]=0$ 表示 v 子树一个都不选，这要求“不选 v 则其后代也不能选”——依赖背包本就如此。

14.8.3 换根 DP（二次扫描）

求“以每个点为根”的某个量。先一次 DFS 求出以固定根的子树信息，再一次 DFS 把根从父亲“换”到孩子、 $O(1)$ 推出孩子做根的答案。下例求每个点到所有点的距离之和。

```
int n, sz[N];
long long down[N], f[N]; // down[u]: u 到子树内距离和; f[u]: u 到全树距离和
void dfs1(int u, int fa) {
    sz[u] = 1; down[u] = 0;
    for (int v : G[u]) {
        if (v == fa) continue;
        dfs1(v, u);
        sz[u] += sz[v];
        down[u] += down[v] + sz[v];
    }
}
void dfs2(int u, int fa) {
    for (int v : G[u]) {
        if (v == fa) continue;
        // 换根: v 上方的  $n - sz[v]$  个点各远 1 步，子树  $sz[v]$  个点各近 1 步
```

```

        f[v] = f[u] - sz[v] + (n - sz[v]);
        dfs2(v, u);
    }
}
// 调用: dfs1(root, 0); f[root] = down[root]; dfs2(root, 0);

```

14.8.4 树的直径（树形 DP 求法）

直径 = 树中最远两点的距离。对每个点统计”向下最长链 + 次长链”，其和的最大值即为直径（适用于含负权之外的一般情况；纯正权也可用两次 BFS/DFS）。

```

vector<pair<int,int>> G[N]; // (邻居, 边权)
long long down[N], dia = 0; // down[u]: u 向下最长链
void dfs(int u, int fa) {
    long long d1 = 0, d2 = 0; // 最长、次长向下链
    for (auto [v, w] : G[u]) {
        if (v == fa) continue;
        dfs(v, u);
        long long d = down[v] + w;
        if (d > d1) d2 = d1, d1 = d;
        else if (d > d2) d2 = d;
    }
    down[u] = d1;
    dia = max(dia, d1 + d2); // 经过 u 拐弯的最长路径
}
// 调用 dfs(root, 0) 后, dia 即为直径

```

- 无边权（边权视为 1）时直径也可两次 BFS：任取一点 BFS 找最远点 x ，再从 x BFS 的最远距离即直径。
- 树形 DP 的”次长链”思路同样可解”每个点的最长路径”等换根类问题。

14.9 DP 四问法

1. 状态表示什么最小信息？
2. 最后一步/最后一个决策是什么？
3. 转移是否覆盖且不重复所有情况？
4. 初值、遍历顺序、答案位置是什么？

先写朴素正确版本，再按状态数量、转移数量、内存逐层优化。dp 数组含义要写在代码注释里。

14.10 线性 DP

- LIS: $O(n \log n)$ 维护每个长度的最小结尾。
- 严格递增用 `lower_bound`, 非降用 `upper_bound`。
- 最长公共子序列: `dp[i][j]` 看前缀; 滚动到一维时保存左上角旧值。
- 最大子段和: `bestEnding = max(a[i], bestEnding+a[i])`, 也可用前缀最小值。
- 计数 DP: 同一状态的方案相加, 注意顺序是否导致重复计数。

```
vector<int> tail;
for (int x : a) {
    auto it = lower_bound(tail.begin(), tail.end(), x); // 非降改 upper_bound
    if (it == tail.end()) tail.push_back(x);
    else *it = x;
}
int lis = tail.size();

long long bestEnding = a[0], maxSubarray = a[0];
for (int i = 1; i < n; ++i) {
    bestEnding = max(a[i], bestEnding + a[i]);
    maxSubarray = max(maxSubarray, bestEnding);
}
```

```
vector<vector<int>> lcs(n + 1, vector<int>(m + 1));
for (int i = 1; i <= n; ++i)
    for (int j = 1; j <= m; ++j)
        lcs[i][j] = (a[i - 1] == b[j - 1])
            ? lcs[i - 1][j - 1] + 1
            : max(lcs[i - 1][j], lcs[i][j - 1]);
```

14.11 背包

类型	状态/遍历
0/1 背包	$dp[j] = \max(dp[j], dp[j-w] + v)$, j 从大到小
完全背包	同式, j 从小到大
多重背包	二进制拆分或单调队列优化
分组背包	每组只能选一个, 组内倒序并用上一层状态
依赖背包	树形 DP/附件分组
计数背包	把 <code>max</code> 换成加法, 模数下处理重复顺序

容量 $w \leq 1e5$ 时常用 $O(nw)$; 物品价值小则把“最小重量”作为状态; $n \leq 40$ 可折半搜索。

```
vector<long long> dp(W + 1, 0);
for (auto [w, v] : items)
    for (int j = W; j >= w; --j)
        dp[j] = max(dp[j], dp[j - w] + v); // 0/1 背包
for (auto [w, v] : items)
    for (int j = w; j <= W; ++j)
        dp[j] = max(dp[j], dp[j - w] + v); // 完全背包
```

14.12 区间 DP

按区间长度递增，枚举分割点： $dp[l][r] = \min/\max(dp[l][k] + dp[k+1][r] + cost)$ 。典型：石子合并、矩阵链乘、括号匹配、回文分割。若代价满足四边形不等式/决策单调性，可用四边形不等式优化，但先验证条件。

```
for (int len = 2; len <= n; ++len)
    for (int l = 1; l + len - 1 <= n; ++l) {
        int r = l + len - 1;
        dp[l][r] = INF;
        for (int k = l; k < r; ++k)
            dp[l][r] = min(dp[l][r], dp[l][k] + dp[k + 1][r] + cost(l, r));
    }
```

14.13 树形 DP 与换根 DP

树上没有环，后序即可保证子树状态已知。独立集： $f[u][0] = \sum \max(f[v][0], f[v][1])$ ， $f[u][1] = w[u] + \sum f[v][0]$ 。换根时先从任意根求子树贡献，再将父亲的答案传给儿子，常见于“每个点到其他点距离和”。

```
void reroot1(int u, int p) {
    sz[u] = 1; down[u] = 0;
    for (int v : tree[u]) if (v != p)
        reroot1(v, u), sz[u] += sz[v], down[u] += down[v] + sz[v];
}

void reroot2(int u, int p) {
    for (int v : tree[u]) if (v != p) {
        ans[v] = ans[u] - sz[v] + (n - sz[v]);
        reroot2(v, u);
    }
}
```

14.14 有向无环图动态规划与路径计数 (Directed Acyclic Graph Dynamic Programming)

拓扑序内转移；最长路用 $-\text{INF}$ 初始化，最短路用 INF 。若有多个源，把所有源初始化；要恢复路径保存 `pre`。在模数下统计方案时不要把不可达状态的 0 与真实方案混淆。

```
for (int u : topo) if (ways[u])
    for (auto [v, w] : dag[u]) {
        dp[v] = max(dp[v], dp[u] + w);
        ways[v] = (ways[v] + ways[u]) % MOD;
    }
```

14.15 状态压缩动态规划

`mask` 表示已选集合，转移 `mask | (1 << i)`。 $n \leq 20$ 的旅行商、集合覆盖、配对、棋盘放置都适用。优化：预处理 `popcount`、子集和、可行转移；枚举子集总复杂度 $O(3^n)$ 。

```
vector<vector<long long>> dp(1 << n, vector<long long>(n, INF));
dp[1][0] = 0;
for (int mask = 1; mask < (1 << n); ++mask)
    for (int u = 0; u < n; ++u) if (mask >> u & 1)
        for (int v = 0; v < n; ++v) if (!(mask >> v & 1))
            dp[mask | (1 << v)][v] = min(dp[mask | (1 << v)][v], dp[mask][u] + w[u][v]);
```

14.16 数位动态规划 (Digit Dynamic Programming)

按高位到低位处理，状态通常为 `(pos, tight, started, remainder/计数)`。`tight` 表示前缀是否贴合上界，`started` 区分前导零。求 $[L, R]$ 用 $F(R) - F(L-1)$ 。先写“不限制上界”的转移，再加入 `tight`。

```
long long memo[20][2][2][11];
string digits;
long long solve_digit(int pos, int tight, int started, int rem) {
    if (pos == (int)digits.size()) return started && rem == 0;
    long long &res = memo[pos][tight][started][rem];
    if (!tight && res != -1) return res;
    res = 0;
    int lim = tight ? digits[pos] - '0' : 9;
    for (int d = 0; d <= lim; ++d)
        res += solve_digit(pos + 1, tight && d == lim, started || d,
                           (rem * 10 + d) % mod);
    return res;
}
```



```
    return res;
}
```

14.17 概率动态规划与期望动态规划(Probability and Expected-Value Dynamic Programming)

状态转移可能含当前状态自身：整理成 $E = (1 + \text{other}) / (1 - p_{\text{self}})$ 。马尔可夫链有限状态可高斯消元；若转移无环则按拓扑/记忆化直接递推。概率取模时乘逆元，浮点题要保留足够精度。

```
// 无环状态图上的期望：先按题意保证 next[u] 都比 u 更靠后
vector<double> E(n, 0.0);
for (int u = n - 1; u >= 0; --u) {
    E[u] = 1.0;
    for (auto [v, p] : transitions[u]) E[u] += p * E[v];
}
```

14.18 记忆化搜索

当状态图是 DAG 或状态数量可控时，用 DFS + cache 比显式排序更自然。必须保证每个状态的所有后继已计算，含环问题不能直接记忆化递归。

```
int memo[MAXN];
int solve(int u) {
    int &res = memo[u];
    if (res != -1) return res;
    res = 0;
    for (int v : next[u]) res = max(res, solve(v) + gain[u][v]);
    return res;
}
```

14.19 动态规划优化 (Dynamic Programming Optimization)

14.19.1 滚动数组

只依赖上一层时把二维压一维；确认本轮不会读到已更新的状态，遍历方向是关键。

```
vector<long long> prev(m + 1), cur(m + 1);
for (int i = 1; i <= n; ++i) {
    cur[0] = 0;
    for (int j = 1; j <= m; ++j)
```

```

        cur[j] = transition(prev[j], cur[j - 1], a[i], b[j]);
        swap(prev, cur);
    }

```

14.19.2 单调队列优化

形如 $dp[i] = base(i) + \min/\max(dp[j] + g(j))$ 且 j 的合法范围滑动时，维护候选队列。若 g 可分离且窗口固定，复杂度从 $O(nk)$ 降到 $O(n)$ 。

```

deque<int> q;
for (int i = 0; i <= n; ++i) {
    while (!q.empty() && q.front() < i - k) q.pop_front();
    dp[i] = base[i] + (q.empty() ? INF : value(q.front()));
    while (!q.empty() && value(q.back()) >= value(i)) q.pop_back();
    q.push_back(i);
}

```

14.19.3 斜率优化 (Convex Hull Trick, CHT)

转移能化为 $dp[i] = x_i * k_j + b_j$ ，把每个 j 看作直线，按斜率/查询单调性维护凸包。斜率不单调时用 Li Chao 树；确认除法比较不溢出，必要时用 `__int128`。

```

struct Line { long long k, b; long long get(long long x) const { return k * x
    + b; } };
deque<Line> hull;
bool bad(Line a, Line b, Line c) {
    return (__int128)(b.b - a.b) * (b.k - c.k)
        >= (__int128)(c.b - b.b) * (a.k - b.k);
}
void add_line(Line x) {
    while (hull.size() >= 2 && bad(hull[hull.size() - 2], hull.back(), x))
        hull.pop_back();
    hull.push_back(x);
}

```

14.19.4 分治优化

若最优决策点单调， $dp[i][j]$ 在分治递归中只枚举决策区间的一部分，常见 $O(kn \log n)$ 。先证明四边形不等式或决策单调性。

```

void solve(int l, int r, int optl, int opttr) {
    if (l > r) return;
    int mid = (l + r) / 2, best = optl;

```

```
for (int k = optl; k <= min(mid, optr); ++k)
    if (cost(k, mid) < cost(best, mid)) best = k;
dp[mid] = cost(best, mid);
solve(l, mid - 1, optl, best);
solve(mid + 1, r, best, optr);
}
```

14.20 组合游戏动态规划 (Combinatorial Game Dynamic Programming)

无环游戏状态标记胜负：存在一个后继为必败则当前必胜；所有后继必胜则当前必败。将多个独立游戏的 Grundy 值 xor。棋盘/取石子题先找周期或 SG。

```
int grundy(int x) {
    if (memo[x] != -1) return memo[x];
    bool seen[MAXG] = {};
    for (int y : moves[x]) seen[grundy(y)] = true;
    int g = 0; while (seen[g]) ++g;
    return memo[x] = g;
}
```

14.21 常见 DP 误区

- 用“最后一个元素”定义状态，却漏掉了影响未来的最小信息。
- 把排列计数当组合计数，循环顺序写反。
- 区间 DP 长度顺序错误。
- 负权最大化仍用 0 初始化，导致不可达状态污染答案。
- 取模减法忘记加模；概率 DP 把浮点误差当精确相等。

15 博弈专题

本章只放竞赛中最常见的博弈模板。先判断游戏是否为无偏、有限、无环状态；能拆成独立子游戏时用 SG，双方都在完整状态上决策时用 Minimax。所有“必胜/必败”结论都要先确认终止规则和是否允许重复状态。

15.1 Nim、Bash 与反常 Nim 取石子游戏

标准 Nim 每次从一堆中取任意正数个，所有堆异或非零时先手必胜。Bash 游戏每次最多取 k 个，单堆 n 的必败位置是 $n \% (k + 1) == 0$ 。

```
bool nim_first_win(const vector<int>& piles) {
    int x = 0; // 整个无偏 Nim 游戏的 SG 异或和。
    for (int v : piles) x ^= v;
    return x != 0;
}

bool bash_first_win(int n, int k) {
    // 一堆石子每次取 1..k; 模 k+1 为 0 的位置是必败态。
    return n % (k + 1) != 0;
}
```

反常 Nim（最后取走者输）：若所有堆大小都为 1，先手在堆数为偶数时必胜；否则仍看异或和是否非零。

```
bool misere_nim_first_win(const vector<int>& piles) {
    // 只有“所有非空堆都是 1”时，最后一步输的规则会改变公式。
    bool all_one = true;
    int x = 0, ones = 0;
    for (int v : piles) {
        all_one &= (v == 1);
        x ^= v;
        ones += (v == 1);
    }
    if (all_one) return ones % 2 == 0;
    return x != 0;
}
```

15.2 通用 Sprague–Grundy 函数、最小未出现值与有向无环图博弈

无偏博弈的 Grundy 值满足 $sg[u] = \text{mex}(\{sg[v]\})$ ；独立子游戏的总值是异或。若状态图是 DAG，可以记忆化 DFS；若只是判断胜负，则“存在必败后继”为必胜，否则为必败。

```
int mex(vector<int> values) {
    // 排序去重后，从 0 开始找第一个断点；适合后继数量不太大的单次计算。
    sort(values.begin(), values.end());
    values.erase(unique(values.begin(), values.end()), values.end());
    int g = 0;
    for (int x : values) {
        if (x == g) ++g;
        else if (x > g) break;
    }
    return g;
}
```

```
vector<int> sg;
vector<char> sg_vis;
int grundy(int u) {
    // DAG 上记忆化求后继 SG；有环时必须换成专门的环博弈分析。
    if (sg_vis[u]) return sg[u];
    sg_vis[u] = true;
    vector<int> nxt;
    for (int v : dag[u]) nxt.push_back(grundy(v));
    return sg[u] = mex(nxt);
}
```

```
bool first_win(const vector<int>& starts) {
    // 只有子游戏互不影响时才能异或；共享资源的游戏不能套这里。
    int total = 0;
    for (int s : starts) total ^= grundy(s);
    return total != 0;
}
```

```
vector<int8_t> win; // 0: 未求, 1: 必胜, -1: 必败
int dag_game(int u) {
    // 先暂定为必败，发现一个必败后继就改成必胜。
    if (win[u]) return win[u];
    win[u] = -1;
    for (int v : dag[u])
        if (dag_game(v) == -1) return win[u] = 1;
}
```

```

    return win[u];
}

```

15.3 Nim 变体：取法集合与 Sprague–Grundy 周期

如果单堆允许取的数量集合固定，先计算一维 SG；`sg[n]` 出现周期后可以直接按周期回答超大 `n`。多个互相独立的堆仍然异或。

```

int solve_subtraction(int n, const vector<int>& moves) {
    vector<int> sg(n + 1);
    vector<int> seen(moves.size() + 2);
    for (int x = 1; x <= n; ++x) {
        vector<int> values;
        for (int take : moves) if (take <= x)
            values.push_back(sg[x - take]);
        sg[x] = mex(values);
    }
    return sg[n];
}

```

15.4 极大极小搜索（Minimax）与 Alpha-Beta 剪枝

双方零和、轮流行动且状态树有限时，Minimax 取当前方能保证的最优值。Alpha-Beta 剪枝维护祖先已知的下界 `alpha` 与上界 `beta`；排序优先搜索好着法会显著提高剪枝率。

```

int minimax(State& state, int depth, int alpha, int beta, bool maximizing) {
    // evaluate 始终站在固定一方视角；max 方取最大，min 方取最小。
    // alpha 是 max 已知下界，beta 是 min 已知上界。
    if (depth == 0 || terminal(state)) return evaluate(state);
    if (maximizing) {
        int best = INT_MIN;
        for (Move mv : ordered_moves(state)) {
            // apply/undo 必须成对，否则下一分支会继承上一分支的局面。
            apply(state, mv);
            best = max(best, minimax(state, depth - 1, alpha, beta, false));
            undo(state, mv);
            alpha = max(alpha, best);
            // beta cutoff: min 方已有更小上界，后面的分支不会被选择。
            if (alpha >= beta) break; // beta cutoff
        }
        return best;
    }
}

```

```

int best = INT_MAX;
for (Move mv : ordered_moves(state)) {
    apply(state, mv);
    best = min(best, minimax(state, depth - 1, alpha, beta, true));
    undo(state, mv);
    beta = min(beta, best);
    // alpha cutoff: max 方已有更大下界，后面的分支不会被选择。
    if (alpha >= beta) break; // alpha cutoff
}
return best;
}

```

实际写棋类题时，`State/Move/apply/undo/terminal/evaluate` 替换为题目状态；有深度限制时应加入深度奖励、置换表和走法排序，避免把“未搜到底”的局面误判成终局。

15.5 博弈题检查清单

- 终止状态是“不能走”输，还是“最后一步”输？这决定普通/反常规则。
- 是否无偏？若双方可用的操作不同，不能直接套 SG xor。
- 状态是否可拆成独立和？有共享资源时不能强行异或。
- 状态图是否有环？有环时需要周期、记忆化博弈或胜负状态方程。
- 题目只问胜负用 Win/Lose，问组合价值或多堆合并才引入 Grundy。

15.6 动态规划进阶：参数、调用与改造

本节紧跟前面的基础模板。先看每个接口传什么，再看哪些不变量不能动，最后看如何替换维护信息或转移。

15.6.1 6.1 复杂数位动态规划与自动机数位动态规划

数位 DP 处理 $0..N$ 中满足数字条件的数。状态通常是 `(pos, tight, started, extra)`；上下界用 `count(R)-count(L-1)`。若有“不能出现某模式/相邻数字限制”，把 `extra` 换成自动机节点。

```

long long memo[20][2][2][STATE];
long long dfs(int pos, bool tight, bool started, int st) {
    // pos: 正在处理第 pos 位; tight: 前缀是否仍等于上界;
    // started: 是否已经出现过非前导零; st: 额外限制的自动机状态。
    if (pos == len) return valid(st, started);
    // 只有 tight=false 的状态可以复用; tight=true 还依赖上界前缀。
    if (!tight && memo[pos][started][0][st] != -1)
        return memo[pos][started][0][st];
}

```

```

long long res = 0;
int up = tight ? digit[pos] : 9;
for (int d = 0; d <= up; ++d) {
    // 前导零通常不应触发“数字出现/相邻数字”等条件。
    bool ns = started || d != 0;
    // 尚未开始时是否把 0 送入自动机，必须按题意决定；这里留在起点 0。
    int nst = ns ? go[st][d] : 0;
    if (ns && nst == forbidden) continue;
    // 注意必须与真实上界位 digit[pos] 比较，不能写成 d==up。
    res += dfs(pos + 1, tight && d == digit[pos], ns, nst);
}
if (!tight) memo[pos][started][0][st] = res;
return res;
}

```

测试样例：数位 DP 的前导零和上下界差分

这里固定 `valid` 为“数字十进制表示中不出现数字 4”，并把 0 计入合法数。每组输入 `L R`，输出 `[L,R]` 中合法整数个数。

输入

4

0 0

0 4

0 10

0 99

输出

1

4

10

81

`[0,4]` 的合法数是 0,1,2,3；`[0,99]` 的答案是 $9 \times 9 = 81$ 。如果把前导零当成真实数字 4 的相邻字符，或者把 `tight && d == digit[pos]` 错写成 `d == up`，这些边界会出错。

这里 `up` 只表示当前位允许枚举到哪里；`tight` 更新必须比较真实上界位 `digit[pos]`。练习：P2602 数字计数；若题目问每位出现次数，把返回值从一个数改为“方案数 + 各位贡献”的结构体。

15.6.2 6.2 所有子集和动态规划、轮廓线动态规划与状态压缩

SOS DP 用来计算所有子集/超集的和：


```
for (int bit = 0; bit < K; ++bit)
    for (int mask = 0; mask < (1 << K); ++mask)
        if (mask >> bit & 1)
            // 把“去掉 bit 后的子集”贡献给当前 mask。
            // 处理完这一层后，f[mask] 已包含所有只在低 bit 位不同的子集。
            f[mask] += f[mask ^ (1 << bit)];
```

测试样例：SOS 动态规划的子集和方向

下面约定 $K=2$ ，初始 $f[\text{mask}]$ 是恰好等于 mask 的权值，输出所有子集和 $\text{sum}[\text{sub}]$ 。

输入

```
2
1
5 7
2
1 2 3 4
```

输出

```
5 12
1 3 4 10
```

第二组中 $\text{sum}[3]$ 必须是 $1+2+3+4=10$ ；如果把 $\text{mask} \wedge (1 \ll \text{bit})$ 写成加上不存在的超集，或循环顺序和转移方向不匹配，会在 $\text{sum}[1]$ 、 $\text{sum}[2]$ 同时出错。

若网格宽度 $m \leq 10/12$ ，每一列的填法只影响相邻列，维护 $\text{dp}[\text{col}][\text{mask}]$ ；这就是轮廓线 DP。先预处理从 mask 到 next_mask 的合法转移，再做列 DP。棋盘放置题可练 P2704 炮兵阵地。状态中相邻两行都可能影响当前行时，不能只保留一行。

15.6.3 6.3 单调队列、斜率优化与李超树优化动态规划

若转移有 $\text{dp}[i] = \min(\text{dp}[j] + w(j, i))$ ，且 j 的优劣具有区间单调性，先尝试单调队列；若整理后是直线最值，才上凸包/李超树。

```
deque<int> q;
for (int i = 0; i < n; ++i) {
    // 过期候选先删除；窗口条件由题目决定。
    while (!q.empty() && q.front() < i - window) q.pop_front();

    // 队首是当前窗口中最优候选，value(j, i) 是题目转移代价。
    dp[i] = value(q.front(), i);

    // 新候选 i 若在未来不可能优于队尾，就淘汰队尾。
    // next_x 和 value 的单调性必须由题目转移证明，不能盲套。
```

```

    while (!q.empty() && value(i, next_x(q.back()))
           <= value(q.back(), next_x(q.back())) q.pop_back();
    q.push_back(i);
}

// 把 dp[i] 整理成“查询某条候选直线在 x[i] 处的最值”时使用。
struct Line {
    long long k, b;
    long long value(long long x) const { return k * x + b; }
};

// 只有在斜率按单调顺序加入、查询 x 也按单调顺序时，
// 才能用普通 deque；任意插入/任意查询请改用前面的李超线段树。
deque<Line> hull;
bool useless(const Line& a, const Line& b, const Line& c) {
    // 交点顺序不再递增时，中间直线永远不会成为最优。
    // 用 __int128 防止交叉相乘时溢出。
    return (__int128)(b.b - a.b) * (b.k - c.k)
           >= (__int128)(c.b - b.b) * (a.k - b.k);
}

void add_line(Line line) {
    // 斜率顺序必须与 useless 的不等式方向一致；换成求最大值时通常要反号。
    while (hull.size() >= 2 &&
           useless(hull[hull.size() - 2], hull.back(), line))
        hull.pop_back();
    hull.push_back(line);
}

```

调用测试：斜率优化只能在顺序条件成立时使用

对下面三条直线做最小值查询：

加入顺序： $y=0$, $y=x$, $y=2x-1$

查询顺序： $x=0, 1, 3$

应得到： $0, 0, 1$

再把查询顺序改成 $x=3, 0, 1$ 。如果模板仍然使用只支持单调查询的双端队列，必须重新检查题目是否真的满足单调性；不能把这组非单调调用当成普通凸包优化的合法输入。

斜率优化的关键不是代码，而是把转移整理为 $y = kx + b$ ，并证明斜率和查询点单调；若斜率不单调或要在线插入，直接使用前面的李超线段树。例题：P5785 任务安排；先把平方项展开，再判断是否能维护直线。

15.6.4 6.4 分治优化、四边形不等式、树形背包与 Steiner 树状压动态规划

调用测试：分治优化的决策范围必须覆盖真实最优点

选一个可以暴力验证的代价： $\text{cost}(j,i)=(\text{prefix}[i]-\text{prefix}[j])^2$ ，对 $\text{prefix}=[0,1,3,6]$ 计算每一层的最优决策，并与 `divide_conquer_dp` 的结果逐项比较。重点不是某个固定答案，而是确认：

测试一： $\text{optL}=0, \text{optR}=i$ ，结果应与完整枚举一致。

测试二：把 optL/optR 缩到不包含真实最优 j 的范围，程序必须被判定为调用错误，不能把错误范围当成算法结论。

没有单调性证明时，不能因为测试一通过就直接把完整枚举范围缩成 $\text{optL}..\text{optR}$ 。

分治优化要求最优决策点满足单调性： $\text{opt}[l][r-1] \leq \text{opt}[l][r]$ 。模板递归 `solve(L,R,optL,optR)`，只在允许区间枚举决策点；没有证明时不能硬套。四边形不等式和决策单调性是证明工具，不是看到区间 DP 就默认成立。

// 典型形式： $\text{dp}[\text{layer}][i] = \min_j(\text{dp}[\text{layer}-1][j] + \text{cost}(j,i))$ 。

// optL/optR 是当前区间内“最优决策 j ”可能出现的范围。

```
void divide_conquer_dp(int L, int R, int optL, int optR) {
    if (L > R) return;
    int mid = (L + R) >> 1;
    long long best = INF;
    int best_j = -1;

    // 没有单调性证明时不能把枚举范围缩成 optL..optR;
    // 这份模板的正确性前提就是 opt[mid] 位于该范围内。
    for (int j = optL; j <= min(optR, mid); ++j) {
        long long cand = old_dp[j] + cost(j, mid);
        if (cand < best) best = cand, best_j = j;
    }
    new_dp[mid] = best;

    // 由 opt[l] <= opt[mid] <= opt[r]，左右区间可以分别缩小。
    divide_conquer_dp(L, mid - 1, optL, best_j);
    divide_conquer_dp(mid + 1, R, best_j, optR);
}
```

树形背包的状态是“子树里选了 j 个”，合并子树时做背包卷积；先处理小子树，循环边界必须使用当前已合并大小。Steiner 树状压 DP 适用于关键点数 $k \leq 10/15$ 的“连通所有关键点最小代价”：先对每个 `mask` 做子集合并，再以 Dijkstra 在图上松弛。若关键点数超过二十，通常要重新分析题意。

// Steiner 树状态： $\text{dp}[\text{mask}][v]$ = 连接 `mask` 中关键点，并以 v 作为当前末端的最小代价。

```
for (int mask = 1; mask < (1 << K); ++mask) {
```

```

// 把 mask 拆成两个非空子集，在同一个 v 汇合。
for (int sub = (mask - 1) & mask; sub; sub = (sub - 1) & mask) {
    int other = mask ^ sub;
    if (sub > other) continue; // 对称拆分只做一次，常数更小。
    for (int v = 0; v < n; ++v)
        dp[mask][v] = min(dp[mask][v],
                           dp[sub][v] + dp[other][v]);
}

// 合并后，末端 v 还可以沿图上的边移动；
// 用 Dijkstra 在“固定 mask 的状态层”上继续松弛。
dijkstra_on_state(mask);
}

```

动态 DP/矩阵线段树在本章 2.13 已给出调用思路；概率 DP 遇到自环时，把方程写成 $f = p \star f + \text{rest}$ ，得到 $f = \text{rest} / (1 - p)$ ；多个状态互相转移才需要高斯消元。不要在有自环时直接按拓扑序 DP。

15.6.5 2.13 动态动态规划与矩阵线段树

当树/序列上的一个点被修改后，答案仍然可以由局部状态转移合并，但普通 DP 不能重新算全局，就把每个位置的转移写成矩阵，线段树维护矩阵乘积。

```

struct Mat {
    long long a[2][2]; // a[i][j]: 第 i 个状态对第 j 个状态的线性系数。
};

Mat mul(Mat A, Mat B) {
    Mat C; // 必须清零；矩阵乘法是累加，不是覆盖。
    for (int i = 0; i < 2; ++i)
        for (int k = 0; k < 2; ++k)
            for (int j = 0; j < 2; ++j)
                // C = A * B: 先应用 B，再应用 A（按列向量约定）。
                C.a[i][j] = (C.a[i][j] + A.a[i][k] * B.a[k][j]) % MOD;
    return C;
}

Mat identity() {
    Mat I;
    I.a[0][0] = I.a[1][1] = 1;
    return I;
}

```

```

// 矩阵线段树的核心合并：左区间的转移先于右区间的转移。
// 如果状态向量按行向量存储，合并顺序会反过来，必须从建模时保持一致。

```

```
Mat merge_segment(Mat left, Mat right) {
    return mul(right, left);
}
```

现场映射方法：先在纸上写出“左边 DP 状态向右移动一格如何变成新状态”，把这一步写成矩阵；叶子存单点矩阵，线段树父节点存 `left * right`。矩阵乘法不可交换，区间方向不能写反。它是 C 级内容，不建议只看一遍标题就现场发明。

15.6.6 7.8 概率动态规划的自环方程

有自环的状态不能按拓扑序处理。例如从状态 i 以概率 p 留在 i ，以概率 q 转到已知状态，期望满足：

$$E_i = pE_i + qE_j + c \implies E_i = \frac{qE_j + c}{1 - p}.$$

多个状态互相转移时，把每个状态写成一行：

$$E_i - \sum_j p_{ij} E_j = c_i,$$

然后用高斯消元。 $1-p$ 很小时误差会放大，题目若要求模意义下概率，要先确认所有分母可逆；题目若允许无限期望，要识别并输出题目规定的无穷值。

测试样例：概率自环的除法和无限期望

把“每次转移产生的固定代价”也写进方程，测试不能只测没有自环的普通转移：

调用 1: $E = 0.5 E + 1$

期望: $E = 2$

调用 2: $E = 0.5 E + 0.5 * 4 + 1$

期望: $E = 6$

调用 3: $E = 1.0 E + 1$

期望: 不存在有限值；不能直接计算 $(1-p)$ 的逆。

调用 4:

$E1 = 0.5 E1 + 0.5 E2 + 1$

$E2 = 0.25 E1 + 0.75 E2 + 2$

期望: 先移项得到

$0.5 E1 - 0.5 E2 = 1$

$-0.25 E1 + 0.25 E2 = 2,$

两行线性相关但常数不相容，因此该系统无有限解。

前三组分别抓“漏除以 $1-p$ ”“把常数项放错侧”和“ $p=1$ 除零”。第四组抓高斯消元中的主元选择、无解判断与浮点误差。实际题目若保证期望存在，第四组应改成相容系统；测试代码本身仍应保留无解分支，避免模板在异常数据上产生假答案。

15.7 博弈论：参数、调用与改造

本节紧跟前面的基础模板。先看每个接口传什么，再看哪些不变量不能动，最后看如何替换维护信息或转移。

15.7.1 1.1 必胜态/必败态动态规划 (Win/Lose Dynamic Programming)

定义 $\text{win}[u]$ ：轮到当前玩家在状态 u 操作时，当前玩家是否能赢。

$$\text{win}(u) = \text{true} \iff \exists v \in \text{next}(u), \text{win}(v) = \text{false}.$$

没有合法操作的状态是必败态。对 DAG 从终点反推，复杂度是 $O(V + E)$ 。

```
// g[u]: 从状态 u 可以一步走到的所有状态。
// 这里假设状态图是 DAG; 如果有环，不能直接用这份递归模板。
vector<vector<int>> g;

// win[u] 的三种取值：
// 0 = 还没有计算；
// 1 = 必胜态：轮到当前玩家时存在一步走到必败态；
// -1 = 必败态：所有合法走法都会把对手送入必胜态。
vector<int8_t> win;

int solve_win(int u) {
    // 记忆化：同一个状态可能从很多地方到达，不能重复搜索。
    if (win[u] != 0) return win[u];

    // 先假设 u 是必败态。
    // 如果后面找到一个必败后继，再把它改成必胜态并立即返回。
    win[u] = -1;
    for (int v : g[u]) {
        // 如果能把对手送到必败态，当前状态就是必胜态。
        if (solve_win(v) == -1)
            return win[u] = 1;
    }

    // 没找到必败后继，说明所有走法都让对手赢；没有走法时也会走到这里。
    return win[u];
}
```

测试样例：必胜态/必败态的多测和状态清空

这里规定每组输入为 $n\ m\ s$ ，接着输入 m 条有向边 $u\ v$ ，从状态 s 开始；没有合法操作输出 **Lose**，能走到必败状态输出 **Win**。图保证无环。

输入

```
4
1 0 1
2 1 1 2
4 4 1 1 2 1 3 2 4 3 4
5 5 1 1 2 1 3 2 4 3 4 4 5
```

输出

```
Lose
Win
Lose
Win
```

第一组是终止态，第二组是一步必胜；第三组的两个后继都是必胜态，所以起点必败；第四组多加一层，检查结果是否按后继层数交替。四组连续运行还能检查 **g**、**win** 是否按当前 n 重新初始化。

例题：P2197 【模板】Nim 游戏。标准 Nim 的状态是所有堆大小组成的元组，直接写全状态会爆炸；理论告诉我们它等价于一个数 $a[1] \oplus \dots \oplus a[n]$ 。当异或和为 0 时输出 **No**，否则输出 **Yes**。这道题的价值是练习“先识别状态压缩后的不变量”，不是练习 DFS。

15.7.2 1.2 有向无环图上的 Sprague–Grundy 函数与最小未出现值

单个无偏游戏状态的 Grundy 值为：

$$sg(u) = mex\{sg(v) \mid u \rightarrow v\}.$$

mex 是没有出现在集合中的最小非负整数。终止状态没有后继，因此 $sg = 0$ 。

```
// 计算 mex (minimum excluded value)：集合中没有出现的最小非负整数。
// seen[x] == tag 表示“本次 mex 计算看到了 x”。
// 每次只递增 tag，就不需要把整个 seen 数组清零，适合大量重复调用。
int mex_by_stamp(const vector<int>& values, vector<int>& seen, int& tag) {
    ++tag; // 开启一次全新的标记批次。
    for (int x : values) {
        // SG 值一定非负；超出 seen 范围的值不会影响较小 mex。
        if (0 <= x && x < (int)seen.size()) seen[x] = tag;
    }

    int g = 0;
```

```

// 从 0 开始找第一个没有被本批次标记的数。
while (g < (int)seen.size() && seen[g] == tag) ++g;
return g;
}

// dag[u]: 有向无环图中从 u 出发的一步转移。
vector<vector<int>> dag;
vector<int> sg;          // sg[u]: 状态 u 的 Sprague-Grundy 值。
vector<char> vis;        // 是否已经完成记忆化搜索。
vector<int> mark;        // mex 的时间戳数组，大小要覆盖可能的 SG 值。
int mark_tag = 0;

int grundy(int u) {
    // DAG 中递归一定会走向更靠后的状态; vis 防止重复计算。
    if (vis[u]) return sg[u];
    vis[u] = 1;

    vector<int> values;
    for (int v : dag[u]) {
        // 先递归求所有后继状态的 SG 值。
        values.push_back(grundy(v));
    }

    // 当前状态的 SG 值只由“所有一步可达状态的 SG 值”决定。
    return sg[u] = mex_by_stamp(values, mark, mark_tag);
}

```

在实际题目中，`mark` 至少开到“可能出现的最大 SG 值 + 1”。如果不确定，简单 `sort + unique` 的 `mex` 更安全。状态数很大时，不能把整个状态对象直接当数组下标，要用哈希记忆化。

小例子：若 0 无后继， $1 \rightarrow 0$ ， $2 \rightarrow 0, 1$ ，则 $sg[0]=0$ ， $sg[1]=mex\{0\}=1$ ， $sg[2]=mex\{0,1\}=2$ 。两个独立游戏的值为 $sg[x] \oplus sg[y]$ ，不为 0 就是先手胜。

测试样例：`mex` 的空集合、单后继和时间戳复用

这里规定每组输入为 `N k moves...`，输出 `sg[N]`。四组连续执行，专门检查 `seen` 的时间戳是否污染下一组。

```

输入
4
0 1 1
1 1 1
4 2 1 2
10 3 1 3 4

```


输出

0
1
1
1

$N=0$ 必须直接得到 0；取法 $\{1,2\}$ 时序列从 0,1,2,0,1 开始， $sg[4]=1$ 。最后一组用于检查不能把上一组的 `seen` 标记当成当前组的结果。

15.7.3 1.3 多个独立子游戏为什么异或

若一次操作只能改变一个子游戏，而且各子游戏互不共享资源，那么整个局面是游戏和：

$$SG(G_1 + G_2 + \cdots + G_k) = SG(G_1) \oplus SG(G_2) \oplus \cdots \oplus SG(G_k).$$

“一堆石子”就是一个子游戏；“棋盘上多个互不相连、棋子不能跨组件移动的区域”也常常可以拆成多个子游戏。反例是：一次操作必须同时选择两堆，或某个操作会改变另一堆的合法操作，此时不能异或。

15.7.4 1.4 普通 Nim：不只判断，还要恢复必胜操作

令 $all = a[0] \oplus \cdots \oplus a[n-1]$ 。若 $all \neq 0$ ，找到最高位 b ，它在 all 中为 1。必然存在某个堆 $a[i]$ 的第 b 位为 1。令

$$target = a[i] \oplus all,$$

则 $target < a[i]$ ，把这堆从 $a[i]$ 减到 $target$ ，新异或和就为 0。

```
// 返回要操作的堆下标和操作后的大小；无必胜操作返回 {-1,-1}
pair<int, unsigned long long>
nim_move(const vector<unsigned long long>& a) {
    // all 是整个 Nim 局面的 SG 值，也就是所有堆大小的异或和。
    unsigned long long all = 0;
    for (auto x : a) all ^= x;

    // 异或和为 0 的局面是必败态，不存在一步必胜操作。
    if (all == 0) return {-1, 0};

    for (int i = 0; i < (int)a.size(); ++i) {
        // 只改变第 i 堆后，希望新异或和变成 0，目标大小必须是 a[i]^all。
        unsigned long long target = a[i] ^ all;
```

```

        // 合法操作要求只能减少石子；这个判断同时保证目标确实更小。
        if (target < a[i]) return {i, target};
    }

    return {-1, 0}; // 理论上不会到这里
}

```

演示：[3, 4, 5] 的异或为 2。对 3： $3 \oplus 2 = 1 < 3$ ，所以把第一堆从 3 变成 1；新局面 [1, 4, 5] 的异或为 0。如果题目要求输出“取走多少”，输出 $a[i] - \text{target}$ ，不要输出 target。

测试样例：普通 Nim 必胜操作恢复

下面约定输出为“从 1 开始编号的堆下标、修改后的堆大小”；没有必胜操作输出 0 0。

```

输入
4
1
0
2
1 1
3
3 4 5
4
1 2 4 8

输出
0 0
0 0
1 1
4 7

```

前两组是异或和为 0 的必败态；第三组检查示例中的 $3 \rightarrow 1$ ；第四组检查最高位选择，不能只输出目标值而不输出堆下标。

15.7.5 1.5 Bash、反常 Nim 与阶梯 Nim

Bash 游戏。单堆有 n 个石子，每次取 $1..k$ 个，必败态是 $n \% (k+1) == 0$ 。原因是长度为 $k+1$ 的每一轮，后手可以补到 $k+1$ 。

```

bool bash_win(long long n, long long k) {
    return n % (k + 1) != 0;
}

```

反常 Nim。最后一步的人输。若所有非空堆都为 1，轮流拿走一堆，堆数为偶数时先手胜；只要存在大于 1 的堆，结论仍然看普通 Nim 的异或和。

```

bool misere_nim_win(const vector<int>& a) {
    // 反常 Nim 只有在“所有非空堆大小都是 1”时与普通 Nim 不同。
    bool all_one = true;
    int x = 0;          // 普通 Nim 的异或和。
    int cnt = 0;        // 非空堆数量。
    for (int v : a) {
        if (v != 1) all_one = false;
        if (v) ++cnt;
        x ^= v;
    }

    // 每次只能拿走一整堆 1，最后拿的人输：
    // 非空堆为偶数时，先手可以把奇数局面留给后手。
    if (all_one) return cnt % 2 == 0;

    // 只要有一堆大于 1，就回到普通 Nim 的异或判定。
    return x != 0;
}

```

阶梯 Nim 的约定。位置 0 是地面，石子在 $i \geq 1$ 的台阶上；每次从台阶 i 向 $i-1$ 移任意正数，移到地面的石子不再参与游戏。此时只异或奇数编号台阶：

```

bool staircase_nim_win(const vector<long long>& a) {
    long long x = 0; // 阶梯 Nim 化成的等价 Nim 异或和。
    // 按“0 是地面、石子从 i 移到 i-1”的约定，只异或奇数层。
    for (int i = 1; i < (int)a.size(); i += 2) x ^= a[i];
    return x != 0;
}

```

演示 $a[0..5] = [0, 3, 1, 4, 2, 5]$ ：奇数台阶为 3, 4, 5，异或是 2。按普通 Nim 规则把台阶 1 的 3 变为 1，即移 2 个到地面，奇数台阶异或变为 0。题目若把“底层”编号成 1，先把下标约定翻译清楚再套公式；阶梯 Nim 最常见的错就是奇偶方向弄反。

测试样例： Bash、反常 Nim、阶梯 Nim 的分界

三段函数分别测试。输出 Win/Lose，不能把三种游戏共用同一个公式。

Bash 输入

```

4
0 3
1 1
4 3
5 3

```

Bash 输出

Lose

Win

Lose

Win

反常 Nim 输入

4

1 1

2 1 1

1 2

2 2 2

反常 Nim 输出

Lose

Win

Win

Lose

阶梯 Nim 输入

4

2 0 0

2 0 1

4 0 2 1 2

5 0 3 0 4 1

阶梯 Nim 输出

Lose

Win

Lose

Win

反常 Nim 的前两组专门检查“所有非空堆都是 1”的特殊分支；阶梯 Nim 的第三组两个奇数层异或为零，能抓住奇偶层写反的问题。

15.7.6 1.6 Sprague–Grundy 函数打表、周期与大范围取石子

固定取法集合 moves 时，先求：

```
vector<int> subtraction_sg(int N, const vector<int>& moves) {
    // sg[x]: 还剩 x 个石子时的 SG 值; sg[0] = 0 (不能操作)。
    vector<int> sg(N + 1);
    // 一次状态最多有 moves.size() 个后继, 所以 mex 不会大于 moves.size()。
    vector<int> seen(moves.size() + 5);
    for (int x = 1; x <= N; ++x) {
```

```

// 用 x 作为时间戳：本轮标记只写 seen，不需要清空数组。
int tag = x;
for (int take : moves) {
    // 只有拿得走时，x-take 才是合法后继。
    if (take <= x && sg[x - take] < (int)seen.size())
        seen[sg[x - take]] = tag;
}

int g = 0;
while (g < (int)seen.size() && seen[g] == tag) ++g;
sg[x] = g;
}
return sg;
}

```

发现周期不能只看“最后几个数像了”。可靠流程是：先打到足够大 N ，枚举候选周期 p ，检查一段连续窗口是否满足 $sg[i] == sg[i-p]$ ，再用更大的窗口或数学证明确认。模板如下：

```

int find_period(const vector<int>& sg, int min_start = 0,
               int min_len = 30) {
    int n = (int)sg.size();

    // p 是候选周期。至少需要两段长度为 p 的数据进行比较。
    for (int p = 1; p * 2 < n; ++p) {
        // st 是周期开始位置：不要默认周期从 0 开始，很多题有前导非周期段。
        for (int st = min_start; st + 2 * p + min_len <= n; ++st) {
            bool ok = true;

            // 检查从 st+p 开始的所有位置是否与 p 前的位置相同。
            for (int i = st + p; i < n; ++i)
                if (sg[i] != sg[i - p]) { ok = false; break; }
            if (ok) return p;
        }
    }
    return -1;
}

```

若题目给 $n \leq 10^{18}$ ，打表只是猜周期；必须确认周期确实适用于题目的完整取法，并把前导非周期段单独处理： $n < \text{start}$ 查表，否则查 $\text{start} + (n - \text{start}) \% \text{period}$ 。

测试样例：周期不能只看一小段

取法集合为 $\{1, 2\}$ 时， sg 从 0 开始按 0, 1, 2 周期重复。先打表到 $N=20$ ，再调用 `find_period`，应得到周期 3。

调用输入

```
N = 20, moves = {1, 2}
```

应得到

```
sg[0..8] = 0 1 2 0 1 2 0 1 2
find_period(sg) = 3
```

如果只比较最后几个值，容易把前导段误当周期；如果 `find_period` 返回 1 或 2，优先检查检查窗口和周期起点。

15.7.7 1.7 图游戏建模、状压博弈和记忆化

“棋子在图上移动，不能走到某些点”通常是图游戏。若图无环，节点本身就是状态；若有多个棋子，只有当棋子之间完全不影响时才拆成 SG 异或。

当 $n \leq 20$ ，状态压缩通常是 `mask`：位为 1 表示未用/仍在场，转移枚举一个合法位置并清位。

```
int n;
unordered_map<int, char> memo;

bool dfs_mask(int mask) {
    // mask 的每一位代表一个“仍然可用”的对象。
    // 如果状态还有 last、资源数等信息，必须一并编码进 key。
    auto it = memo.find(mask);
    if (it != memo.end()) return it->second;

    for (int i = 0; i < n; ++i) {
        // 第 i 位为 0，说明对象 i 已经被使用，不能再次选择。
        if (!(mask >> i & 1)) continue;

        // legal 由题目定义，例如不能与上一次选择相邻、不能违反图边限制等。
        if (!legal(mask, i)) continue;

        // 只要存在一个后继是必败态，当前状态就是必胜态。
        if (!dfs_mask(mask ^ (1 << i)))
            return memo[mask] = 1;
    }

    // 没有任何一步能把对手送入必败态。
    return memo[mask] = 0;
}
```

如果状态中还有 `last`、资源数 `r` 或轮到谁，就把它们一起放进 `key`，例如 `key = (mask <<`

5) | **last**; 不能只哈希 **mask** 而丢掉会影响后续转移的信息。状态压缩博弈的练习可从洛谷博弈题单里的 Nim 变体开始，再做带图/带限制的题。

调用测试：先固定 **legal**，再测试记忆化

原代码中的 **legal(mask,i)** 是题目相关接口，不能凭空编一个题目输入。这里先用最小调用测试：规定任何仍在 **mask** 中的元素都合法，每次只能拿走一个元素；因此剩余元素个数为奇数时先手胜。

调用输入 (n=3)

mask = 0, 1, 3, 7

应得到

Lose

Win

Lose

Win

四次调用之间必须清空 **memo** 或重新建立测试状态。正式题目若还有 **last**、资源数或当前玩家，必须把它们加入状态后再重新设计样例；只测试 **mask** 不能证明完整状态压缩正确。

15.7.8 1.8 极大极小搜索 (Minimax) 与 Alpha-Beta 剪枝

Minimax 适用于：双方目标相反、操作可能不同、不能拆成独立子游戏，且状态树能在给定深度内搜索。**evaluate** 必须站在固定一方视角，例如“对先手越大越好”；轮到另一方时取 **min**。

```
int alpha_beta(State& s, int depth, int alpha, int beta, bool maxing) {
    // State: 完整局面; depth: 还允许搜索多少层;
    // alpha: 极大方当前至少能保证的分数;
    // beta: 极小方当前至多允许极大方得到的分数。
    // evaluate 必须始终从同一方（例如先手）视角返回分数。
    if (depth == 0 || terminal(s)) return evaluate(s);

    // 好着法优先搜索会更早提高 alpha / 降低 beta, 剪枝效果更好。
    auto moves = ordered_moves(s); // 吃子、将军等好着法先搜
    if (maxing) {
        int value = INT_MIN;
        for (Move mv : moves) {
            // apply 和 undo 必须严格成对; 否则后续分支会读到被污染的棋盘。
            apply(s, mv);
            value = max(value, alpha_beta(s, depth - 1,
                                         alpha, beta, false));
            undo(s, mv);
        }
    }
}
```

```

        // 极大方已经知道至少能得到 value，因此 alpha 可以上调。
        alpha = max(alpha, value);

        // beta <= alpha 时，极小方不会允许进入这个分支，后面的着法无需搜索。
        if (alpha >= beta) break;
    }
    return value;
}
int value = INT_MAX;
for (Move mv : moves) {
    apply(s, mv);
    value = min(value, alpha_beta(s, depth - 1,
                                   alpha, beta, true));
    undo(s, mv);

    // 极小方希望分数更小，因此 beta 只能向下收紧。
    beta = min(beta, value);
    if (alpha >= beta) break;
}
return value;
}

```

例题式映射：井字棋中 **State** 是 3x3 棋盘加当前玩家，**Move** 是空格编号，**apply/undo** 落子/撤销，**terminal** 检查三连或棋盘已满，**evaluate** 对胜负返回 +100/-100/0。如果题目要求输出具体第一步，外层枚举每个 **Move**，对每个子局面调用 Alpha-Beta，保存得分最高的一步。深度搜索不到终局时，必须用启发式评估，并给将要赢的状态加深度奖励，否则可能“宁愿晚赢”。

调用测试说明：Alpha-Beta 代码中的 **State**、**Move**、**apply**、**undo**、**terminal** 和 **evaluate** 都没有固定定义，不能伪造统一的输入输出。实际接入井字棋时，至少手测三个局面：当前已经三连的终止局面、一步可以获胜的局面、必须先堵住对手的局面；并逐一检查每个分支的 **apply/undo** 后棋盘是否恢复。

15.7.9 1.9 状态去重、哈希与有环博弈

棋类搜索中同一局面可能由不同顺序到达。可以把规范化后的状态放入置换表：

```

struct StateHash {
    size_t operator()(const State& s) const {
        size_t h = 0;
        for (int x : s.board)
            h = h * 1000003ULL + (unsigned)x + 1;
        h = h * 2 + s.turn;
    }
};

```



```
        return h;
    }
};

unordered_map<State, int, StateHash> table;
```

若图上有环，普通 DFS 的“访问过就返回”是不对的，因为“正在递归路径上”和“已经确定为必胜/必败”不是一回事。无平局、且题目保证最终会结束时可用三色 DFS；允许双方无限循环时，要单独定义平局的收益，不能强行套 Win/Lose。

调用测试说明：**StateHash** 的样例必须和具体 **State** 一起写。至少构造两个“棋盘内容相同但轮到不同玩家”的状态，确认哈希表键不同；再用两个不同走法到达的同一规范局面，确认查表命中同一个状态。不能只打印哈希值判断正确，因为哈希相等不是状态相等。

16 计算几何

16.1 基础

```
const double EPS = 1e-9;
struct Point {
    double x, y;
    Point operator-(const Point& o) const { return {x-o.x, y-o.y}; }
    Point operator+(const Point& o) const { return {x+o.x, y+o.y}; }
    Point operator*(double t) const { return {x*t, y*t}; }
    double dot(const Point& o) const { return x*o.x + y*o.y; } // 点积
    double cross(const Point& o) const { return x*o.y - y*o.x; } // 叉积
    double len() const { return sqrt(x*x + y*y); }
};
```

- 浮点数比较相等必须用 $\text{fabs}(a - b) < \text{EPS}$ ，不能用 $==$ 。
- 整数题目尽量避免用 sqrt 和 pow ，用整数运算代替（如交叉相乘比大小）。
- $\text{pow}(5, 2)$ 可能返回 24.999...，赋给 int 变成 24！整数幂次用手写快速幂。

16.2 三分法求单峰极值（几何应用）

常见应用：求点到折线/曲线的最短距离、求凸函数极值等。

```
// 求函数 f 在 [l, r] 上的极小值点
double l = lo, r = hi;
for (int i = 0; i < 100; i++) {
    double m1 = l + (r - l) / 3;
    double m2 = r - (r - l) / 3;
    if (f(m1) < f(m2)) r = m2; // 极小值在左侧
    else l = m1;
}
// 极小值点在 l 附近，极小值为 f(l)
```

17 计算几何与计算技巧

17.1 浮点规范

整数坐标优先用 `long long` 叉积，乘法可能到 `1e18` 时用 `__int128`。实数几何统一 `long double`，比较用 `sgn(x) = (x>eps)-(x<-eps)`，不要直接 `==`。

```
using Real = long double;
const Real EPS = 1e-12L;
struct Point { Real x, y;
    Point operator+(Point b) const { return {x + b.x, y + b.y}; }
    Point operator-(Point b) const { return {x - b.x, y - b.y}; }
    Point operator*(Real k) const { return {x * k, y * k}; }
};
Real cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }
Real dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }
int sgn(Real x) { return (x > EPS) - (x < -EPS); }
```

17.2 向量与叉积

$\text{cross}(a, b) = a.x \times b.y - a.y \times b.x$ ：正为逆时针，负为顺时针，0 共线。点 p 在有向线段 ab 左侧当且仅当 $\text{cross}(b-a, p-a) > 0$ 。

点积判断投影/夹角： $\text{dot}(a, b) > 0$ 锐角， < 0 钝角；距离平方尽量不开放平方根。

```
Real dist2(Point a, Point b) { Point d = a - b; return dot(d, d); }
bool left_of(Point a, Point b, Point p) { return sgn(cross(b - a, p - a)) > 0; }
```

17.3 直线、线段与交点

线段相交：先做包围盒快速排除，再判断四个方向叉积；共线时检查投影是否重叠。求交点用参数方程： $a + t(b-a)$ ，分母是两方向向量叉积；平行/重合要单独处理。

点到直线距离: $\text{abs}(\text{cross}(\mathbf{b}-\mathbf{a}, \mathbf{p}-\mathbf{a})) / |\mathbf{b}-\mathbf{a}|$; 点到线段要先看投影是否落在段内, 否则取端点距离。

```
bool on_segment(Point a, Point b, Point p) {
    return sgn(cross(b - a, p - a)) == 0 && sgn(dot(p - a, p - b)) <= 0;
}
bool intersect(Point a, Point b, Point c, Point d) {
    Real c1 = cross(b - a, c - a), c2 = cross(b - a, d - a);
    Real c3 = cross(d - c, a - c), c4 = cross(d - c, b - c);
    if (sgn(c1) == 0 && on_segment(a, b, c)) return true;
    if (sgn(c2) == 0 && on_segment(a, b, d)) return true;
    return sgn(c1) * sgn(c2) < 0 && sgn(c3) * sgn(c4) < 0;
}
```

17.3.1 圆几何

直线与圆交点先投影到直线, 再根据圆心到直线的距离判断交点数; 两圆交点用圆心连线方向分解。判别相切时仍使用 EPS。

```
struct Circle { Point o; Real r; };
vector<Point> line_circle(Point a, Point b, Circle c) {
    Point d = b - a;
    Real dd = dot(d, d), t = dot(c.o - a, d) / dd;
    Point foot = a + d * t;
    Real h2 = c.r * c.r - dist2(foot, c.o);
    if (h2 < -EPS) return {};
    if (h2 <= EPS) return {foot};
    Point unit = d * (sqrtl(h2 / dd));
    return {foot - unit, foot + unit};
}

vector<Point> circle_circle(Circle a, Circle b) {
    Point d = b.o - a.o; Real dd = dot(d, d), dis = sqrtl(dd);
    if (dis < EPS || dis > a.r + b.r + EPS || dis < fabsl(a.r - b.r) - EPS)
        return {};
    Real x = (a.r * a.r - b.r * b.r + dd) / (2 * dis);
    Real h2 = max((Real)0, a.r * a.r - x * x);
    Point base = a.o + d * (x / dis);
    Point perp{-d.y / dis, d.x / dis};
    if (h2 <= EPS) return {base};
    Point off = perp * sqrtl(h2);
    return {base - off, base + off};
}
```

17.4 多边形

- 有向面积: $\sum \text{cross}(p[i], p[(i+1)\%n]) / 2$ 。
- 点在多边形: 射线法 (奇偶交点) 或绕数法; 边界先单独判定。
- 凸多边形内点可二分扇区, 查询 $O(\log n)$; 一般多边形射线法 $O(n)$ 。
- Pick 定理: 格点多边形 $A = I + B/2 - 1$, 其中 $B = \sum \gcd(|dx|, |dy|)$ 。

```
Real area2(const vector<Point>& p) {
    Real s = 0;
    for (int i = 0; i < (int)p.size(); ++i)
        s += cross(p[i], p[(i + 1) % p.size()]);
    return s;
}
```

17.5 凸包 (Andrew)

点按 (x, y) 排序去重; 维护下凸/上凸, 若 $\text{cross} \leq 0$ 弹出可得到不保留共线点的凸包, 若要保留边界改为 < 0 。复杂度 $O(n \log n)$ 。周长、面积、最远点对都以凸包为输入。

```
vector<Point> convex_hull(vector<Point> p) {
    sort(p.begin(), p.end(), [](Point a, Point b) {
        return a.x < b.x || (a.x == b.x && a.y < b.y);
    });
    p.erase(unique(p.begin(), p.end(), [](Point a, Point b) {
        return a.x == b.x && a.y == b.y;
    }), p.end());
    vector<Point> h;
    for (Point x : p) {
        while (h.size() >= 2 && cross(h.back() - h[h.size() - 2], x - h.back())
            <= 0) h.pop_back();
        h.push_back(x);
    }
    int lower = h.size();
    for (int i = (int)p.size() - 2; i >= 0; --i) {
        while ((int)h.size() > lower && cross(h.back() - h[h.size() - 2], p[i]
            - h.back()) <= 0) h.pop_back();
        h.push_back(p[i]);
    }
    if (h.size() > 1) h.pop_back();
    return h;
}
```

17.6 旋转卡壳

在凸包上用一个单调指针寻找对踵点，求直径（最远点对） $O(h)$ ；也可求最大三角形面积、两凸多边形距离。指针前进依据叉积面积是否增大，不要用浮点角度比较。

```
long double diameter2(const vector<Point>& p) {
    int n = p.size(), j = 1; long double ans = 0;
    for (int i = 0; i < n; ++i) {
        int ni = (i + 1) % n;
        while (fabsl(cross(p[ni] - p[i], p[(j + 1) % n] - p[i])) >
                fabsl(cross(p[ni] - p[i], p[j] - p[i]))) j = (j + 1) % n;
        ans = max(ans, max(dist2(p[i], p[j]), dist2(p[ni], p[j])));
    }
    return ans;
}
```

17.7 扫描线

矩形并面积：按 x 端点事件排序， y 区间用线段树维护覆盖长度，当前面积增量为 $\text{covered}_y \times \Delta x$ 。坐标压缩要保留相邻坐标差，节点代表小区间而非点。

```
struct Event { long long x, y1, y2; int delta; };
sort(events.begin(), events.end(), [](auto a, auto b) { return a.x < b.x; });
long long area = 0, last_x = events[0].x;
for (auto e : events) {
    area += covered_length() * (e.x - last_x);
    update(y1_id(e.y1), y2_id(e.y2) - 1, e.delta);
    last_x = e.x;
}
```

17.8 最近点对

分治按 x 排序，递归求左右最小距离，只检查中线附近按 y 排序的常数个点，复杂度 $O(n \log n)$ 。若只需竞赛常见规模，网格哈希也可用，但要证明桶大小与最小距离的关系。

```
long double closest(vector<Point>& p, int l, int r) {
    if (r - l <= 3) {
        long double ans = 1e100L;
        for (int i = l; i < r; ++i) for (int j = l; j < i; ++j) ans = min(ans, dist2(p[i], p[j]));
        sort(p.begin() + l, p.begin() + r, [](Point a, Point b) { return a.y < b.y; });
    }
```

```

    return ans;
}
int m = (l + r) / 2; Real mid = p[m].x;
Real ans = min(closest(p, l, m), closest(p, m, r));
vector<Point> strip;
for (int i = l; i < r; ++i) if ((p[i].x - mid) * (p[i].x - mid) < ans)
strip.push_back(p[i]);
for (int i = 0; i < (int)strip.size(); ++i)
    for (int j = i + 1; j < (int)strip.size() && strip[j].y - strip[i].y <
sqrtl(ans); ++j)
        ans = min(ans, dist2(strip[i], strip[j]));
return ans;
}

```

17.9 半平面交 (Half-Plane Intersection, HPI, 了解)

把每个半平面写成有向直线左侧，按极角排序并用双端队列维护交集；处理平行情形和角度相等是难点。若题目只问凸多边形裁剪，优先用逐边 Sutherland–Hodgman。

```

vector<Point> clip(const vector<Point>& poly, Point a, Point b) {
    // 保留有向边 a->b 的左侧；cp/cq 是端点相对切线的叉积符号。
    vector<Point> out;
    for (int i = 0; i < (int)poly.size(); ++i) {
        Point p = poly[i], q = poly[(i + 1) % poly.size()];
        Real cp = cross(b - a, p - a), cq = cross(b - a, q - a);
        // p 在合法侧就保留 p。
        if (cp >= -EPS) out.push_back(p);
        // 一进一出时，边 pq 与切线有交点，按线性插值求交。
        if ((cp >= 0) != (cq >= 0)) {
            Real t = cp / (cp - cq);
            out.push_back(p + (q - p) * t);
        }
    }
    return out;
}

```

下面是“有向直线左侧为合法半平面”的真正半平面交模板；返回空集表示交集无界或为空时需结合题目另行处理，常规竞赛题通常会额外给一个大矩形作边界。

```

struct HalfPlane { Point p, v; Real ang; };
Point line_inter(HalfPlane a, HalfPlane b) {
    // a.p+t*a.v 与 b.p+s*b.v 的交点参数。
    Real t = cross(b.p - a.p, b.v) / cross(a.v, b.v);
    return a.p + a.v * t;
}

```

```

}
bool outside(HalfPlane h, Point p) {
    // 有向直线左侧合法；叉积小于 0 表示点在右侧，应该删掉。
    return sgn(cross(h.v, p - h.p)) < 0;
}

vector<Point> half_plane_intersection(vector<HalfPlane> h) {
    // 极角排序让平面边界按方向排列，双端队列只需维护首尾交点。
    for (auto &x : h) x.ang = atan2l(x.v.y, x.v.x);
    sort(h.begin(), h.end(), [](const HalfPlane& a, const HalfPlane& b) {
        return a.ang < b.ang;
    });
    deque<HalfPlane> q;
    for (auto cur : h) {
        // 新半平面会淘汰队尾/队首不在新半平面内的交点。
        while (q.size() >= 2 && outside(cur, line_inter(q[q.size() - 2], q.back())) q.pop_back();
        while (q.size() >= 2 && outside(cur, line_inter(q[0], q[1]))) q.pop_front();
        if (!q.empty() && sgn(cross(q.back().v, cur.v)) == 0) {
            // 同向平行线只保留更严格的一条，反向平行通常意味着交集为空。
            if (outside(cur, q.back().p)) q.back() = cur;
        } else q.push_back(cur);
    }
    while (q.size() >= 3 && outside(q.front(), line_inter(q[q.size() - 2], q.back()))) q.pop_back();
    while (q.size() >= 3 && outside(q.back(), line_inter(q[0], q[1]))) q.pop_front();
    if (q.size() < 3) return {};
    vector<Point> poly;
    for (int i = 0; i < (int)q.size(); ++i)
        poly.push_back(line_inter(q[i], q[(i + 1) % q.size()]));
    return poly;
}

```

17.10 随机化与哈希技巧

- 随机 `shuffle` + 快速选择求期望线性第 k 小。
- 64 位随机哈希表示集合，支持多重集合差异的高概率比较；严谨题目用排序/Trie/计数。
- Treap 随机优先级避免退化；不要把 `rand()` 作为高质量随机源，使用 `mt19937_64`。

```

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
shuffle(a.begin(), a.end(), rng);

```



```
nth_element(a.begin(), a.begin() + k, a.end());
int answer = a[k];
uint64_t random_hash = rng();
```

17.11 位集优化

`std::bitset` 将 64/机器字并行，适合传递闭包（布尔矩阵）、集合交并、子集计数；`bitset` 大小需编译期常量，动态场景使用 `vector<unsigned long long>`。

```
const int N = 2000;
bitset<N> reach[N];
for (int k = 0; k < n; ++k)
    for (int i = 0; i < n; ++i)
        if (reach[i][k]) reach[i] |= reach[k];
```

17.12 快速小技巧

- `std::gcd`、`std::lcm`（C++17）。
- `__builtin_ctzll` 只对非零数调用。
- 排序后用 `unique` 去重，差异数组用 `adjacent_difference`。
- 大量模乘、叉积、斜率比较时先判断上界，必要时统一 `__int128`。

```
long long g = std::gcd(a, b);
long long l = a / g * b;
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());
long long lowbit = x & -x; // x != 0
long long safe_product = (long long)((__int128)x * y % MOD);
```

17.13 计算几何高级算法：参数、调用与改造

本节紧跟前面的基础模板。先看每个接口传什么，再看哪些不变量不能动，最后看如何替换维护信息或转移。

17.13.1 5.1 极角排序与凸包

极角排序常用于按方向扫描点或半平面。不要直接 `atan2` 排序，优先用半平面标记加叉积：

```
int half(Point a) { return (a.y > 0 || (a.y == 0 && a.x >= 0)) ? 0 : 1; }
bool polar_cmp(Point a, Point b) {
```

```

    if (half(a) != half(b)) return half(a) < half(b);
    return cross(a, b) > 0;
}

```

凸包单调链中，若题目要求保留边上的点用 `cross < 0`，只保留顶点通常用 `cross <= 0` 弹出。例题可配合 P1452 Beauty Contest G：先求凸包，再用旋转卡壳求直径平方。

17.13.2 5.2 旋转卡壳、凸多边形内点与最近点对

凸包直径的卡壳指针只向前走，面积/距离比较是单调的；不要对每个点重新扫描所有点。最近点对分治按 x 排序，递归解决左右，再只检查按 y 排序的窄条带，复杂度 $O(n \log n)$ 。

```

long double closest(int l, int r) {
    if (r - l <= 1) return INF;
    int m = (l + r) >> 1;
    long double midx = p[m].x;
    long double ans = min(closest(l, m), closest(m, r));
    vector<Point> strip;
    for (int i = l; i < r; ++i)
        if (fabs(p[i].x - midx) * fabs(p[i].x - midx) < ans)
            strip.push_back(p[i]);
    sort(strip.begin(), strip.end(), by_y);
    for (int i = 0; i < (int)strip.size(); ++i)
        for (int j = i + 1; j < (int)strip.size() &&
            (strip[j].y - strip[i].y) * (strip[j].y - strip[i].y) < ans; ++j)
            ans = min(ans, dist2(strip[i], strip[j]));
    return ans;
}

```

测试样例：最近点对的单点、重合点和跨分治边界

下面每组输入 n 个点，输出最小平方距离。题目若要求输出实际距离，再在最后开平方。

输入

```

4
2
0 0
3 4
2
1 1
1 1
4
0 0
0 3

```

```

4 0
4 3
5
0 0
10 10
5 5
5 6
20 0

```

输出

```

25
0
9
1

```

第二组检查重复点必须直接得到 0；第三组有多条同样的最短边；第四组的答案来自中间两个点，能检查合并后的 **strip** 是否按 **y** 排序以及带状区域边界是否写错。

例题：P1429 平面最近点对（加强版）。按 **x** 排序后调用分治；若保留平方距离，比较时避免不必要的 **sqrt**，最后输出时再开根号。

凸多边形点内判定可以先叉积判断扇区，再对扇区二分；切割则保留直线左侧点并对每条边和切线求交。题面已保证凸多边形时不要用通用半平面交替代简单二分。

17.13.3 5.3 半平面交（Half-Plane Intersection）

把每个不等式写成“有向直线左侧保留”，按极角排序，用双端队列维护当前交集。关键接口是 **outside(line, point)** 和相邻直线求交；平行线要保留更严格的一条。

```

struct HalfPlane { Point a, v; }; // a + t*v, 保留左侧
bool outside(const HalfPlane& h, Point p) {
    // cross(v, p-a) < 0 表示 p 在有向直线右侧，即不满足半平面。
    return cross(h.v, p - h.a) < -EPS;
}
Point intersection(HalfPlane a, HalfPlane b) {
    // 令 a.a + t*a.v = b.a + s*b.v, 叉乘消去 s 得 t。
    // 调用前必须保证两条直线不平行，否则分母为 0。
    long double t = cross(b.v, b.a - a.a) / cross(b.v, a.v);
    return a.a + a.v * t;
}

```

调用测试：半平面交的方向、平行边和空交集

当前片段只展示了 **outside** 和两条直线求交，完整的双端队列流程没有写出，因此不伪造完整输入输出。补齐 HPI 后必须至少验证：

测试一： $x \geq 0$ 、 $x \leq 2$ 、 $y \geq 0$ 、 $y \leq 2$ ，面积应为 4。

测试二：在测试一中再加入 $x \leq 1$ ，面积应为 2。

测试三：同时加入 $x \leq 0$ 和 $x \geq 1$ ，交集为空，面积应为 0。

其中测试一检查有向边左侧约定，测试二检查同向平行边保留更严格者，测试三检查双端队列清空和空交集判断。

例题：P4196 半平面交 / 凸多边形。每个输入凸多边形的每条边都转成“边的左侧”，所有多边形的边合在一起做 HPI，最后对双端队列中的相邻交点求面积。若交集为空，输出 0；平行同向边必须只留下更靠内的那条。

17.13.4 5.4 圆几何与扫描线

点到圆、圆与圆位置关系先比较距离平方和半径平方，避免无意义开根号；两圆相交时，沿圆心连线求基点，再沿垂直方向偏移。线圆交点用参数方程代入二次方程。

```
vector<Point> circle_circle(Point a, long double r,
                             Point b, long double R) {
    // d 是圆心向量，len 是圆心距；先判位置关系，再做开根号。
    Point d = b - a;
    long double len = hypotl(d.x, d.y);
    if (len > r + R + EPS || len < fabsl(r - R) - EPS) return {};

    // 两圆公共弦所在直线到圆心 a 的距离方向投影为 x。
    long double x = (r*r - R*R + len*len) / (2*len);

    // h2 是公共交点相对基点的垂直距离平方，浮点误差可能让它略小于 0。
    long double h2 = max((long double)0, r*r - x*x);
    Point e = d / len, base = a + e * x;
    Point per = {-e.y, e.x};

    // h2=0 是相切；否则关于圆心连线对称地产生两个交点。
    if (h2 < EPS) return {base};
    long double h = sqrtl(h2);
    return {base + per*h, base - per*h};
}
```

最小圆覆盖的现场策略是随机打乱点，增量维护当前圆：发现点在圆外，就以它为边界重新枚举之前点；双层/三层枚举保证期望 $O(n)$ 。例题：P1742 最小圆覆盖。

测试样例：圆相交的无交、相切和双交点

下面调用 `circle_circle`，只统计交点个数；重合圆必须在调用前单独分类，不能让 `len=0` 进入除法。

输入

```
4
0 0 1 5 0 1
0 0 1 2 0 1
0 0 1 1 0 1
0 0 1 0 3 1
```

输出

```
0
1
2
0
```

四组分别是外离、外切、两点相交和内离。第二组必须允许 `h2=0` 返回一个点；如果浮点误差使 `h2` 略小于零而没有截成 `0`，相切样例会出现异常。

矩形并面积用扫描线 + 线段树维护 x 轴覆盖长度；每个事件是 $(y, x1, x2, \text{delta})$ ，按 y 排序，先用上一层覆盖长度乘高度差，再更新 $[x1, x2]$ 。例题：P5490 扫描线。坐标离散化后叶子代表相邻坐标段，不是原始坐标点。

17.13.5 7.6 圆与直线、最小圆覆盖

直线写成 $p + t \cdot v$ ，代入圆方程后得到关于 t 的二次方程；判别式小于 `0` 无交点，等于 `0` 相切，大于 `0` 两交点。

```
vector<Point> line_circle(Point p, Point v, Point o, long double r) {
    // 直线参数式: P(t)=p+t*v; 如果题目给的是线段, 最后还要检查 t in [0,1]。
    Point q = p - o;
    long double A = dot(v, v), B = 2 * dot(q, v);
    long double C = dot(q, q) - r * r;
    long double D = B * B - 4 * A * C;
    // 判别式决定交点个数; 因为有误差, 接近 0 时按相切处理。
    if (D < -EPS) return {};
    D = max((long double)0, D);
    long double t1 = (-B - sqrtl(D)) / (2 * A);
    long double t2 = (-B + sqrtl(D)) / (2 * A);
    if (D < EPS) return {p + v * t1};
    return {p + v * t1, p + v * t2};
}
```

```

Circle min_circle(vector<Point> p) {
    // 随机打乱是期望线性复杂度的关键；不要用固定输入顺序硬套均摊结论。
    shuffle(p.begin(), p.end(), mt19937(chrono::steady_clock::now().
time_since_epoch().count()));
    Circle c{{0, 0}, -1};
    for (int i = 0; i < (int)p.size(); ++i) if (!inside(c, p[i])) {
        // p[i] 在当前圆外，新的最小圆必须把它放在边界上。
        c = {p[i], 0};
        for (int j = 0; j < i; ++j) if (!inside(c, p[j])) {
            // 两个边界点确定直径圆；若还不够，再由三个点确定外接圆。
            c = circle_diameter(p[i], p[j]);
            for (int k = 0; k < j; ++k) if (!inside(c, p[k]))
                c = circle_three_points(p[i], p[j], p[k]);
        }
    }
    return c;
}

```

例题仍可用P1742 最小圆覆盖。`inside` 要允许 `c.r<0` 作为空圆；三点共线时不能直接除以零，应退化为三点中最远的两点直径圆。几何模板里的 `Point/Circle/dot/cross` 名称必须统一，不能把 `Point::dot` 和全局 `dot` 重复定义成不同类型。

测试样例：直线圆交、最小圆覆盖的退化点集

先测 `line_circle` 的判别式分支。以下都把圆设为圆心 $(0,0)$ 、半径 1，直线方向为 $(1,0)$ ：

调用 1: $p=(0,0)$, $v=(1,0)$

结果: 2 个交点, 参数 $t=-1,1$

调用 2: $p=(0,1)$, $v=(1,0)$

结果: 1 个交点, 参数 $t=0$

调用 3: $p=(0,2)$, $v=(1,0)$

结果: 0 个交点

调用 4: $p=(2,0)$, $v=(1,0)$

结果: 无限直线仍有 2 个交点, 参数 $t=-3,-1$;

若题目要的是线段，必须额外检查 t 是否在 $[0,1]$ 。

再测 `min_circle`。输出使用误差 $1e-9$ 比较，不能因为随机打乱点的顺序不同而要求浮点输出逐位相同：

调用 1: $[(0,0)]$

期望: 圆心 $(0,0)$, 半径 0

调用 2: [(0,0),(2,0)]

期望: 圆心 (1,0), 半径 1

调用 3: [(0,0),(1,0),(3,0)]

期望: 圆心 (1.5,0), 半径 1.5

调用 4: [(0,0),(0,2),(2,0),(2,2)]

期望: 圆心 (1,1), 半径 $\sqrt{2}$

第一组专门抓 $D < 0$ / $D = 0$ / $D > 0$ 判断和“直线误当线段”；第二组专门抓空圆半径、两点直径、三点共线除零，以及三点外接圆的分支。

17.13.6 7.7 扫描线求矩形并面积

把每个矩形 $[x_1, x_2) \times [y_1, y_2)$ 变成两个事件：在 y_1 加入，在 y_2 删除。线段树维护当前 x 轴覆盖长度； $cover > 0$ 时整段都被覆盖，否则由左右儿子合并。

```
struct Event { long long y, x1, x2; int delta; };
struct CoverNode {
    int tag = 0;          // 当前 x 小区间被完整覆盖了多少层。
    long long len = 0;    // 该节点代表范围内至少被覆盖一次的长度。
};
vector<long long> xs;
CoverNode seg[MAXN * 4];
void pull(int p, int l, int r) {
    // xs[l..r] 是坐标点，节点覆盖的是 [xs[l], xs[r]]。
    if (seg[p].tag) seg[p].len = xs[r] - xs[l];
    else if (r - l == 1) seg[p].len = 0;
    else seg[p].len = seg[p<<1].len + seg[p<<1|1].len;
}
void modify(int p, int l, int r, int ql, int qr, int d) {
    // 更新的是离散下标半开区间 [ql, qr)，不是点 [ql, qr]。
    if (qr <= l || r <= ql) return;
    if (ql <= l && r <= qr) { seg[p].tag += d; pull(p, l, r); return; }
    int m = (l + r) >> 1;
    modify(p<<1, l, m, ql, qr, d);
    modify(p<<1|1, m, r, ql, qr, d);
    pull(p, l, r);
}
long long union_area(vector<Event> e) {
    // 事件按 y 扫描；在处理当前 y 前，seg[1].len 对应上一段高度。
    sort(e.begin(), e.end(), [](auto& a, auto& b) { return a.y < b.y; });
    long long ans = 0, last_y = e[0].y;
    for (int i = 0; i < (int)e.size(); i++) {
```

```

    ans += seg[1].len * (e[i].y - last_y);
    int j = i;
    while (j < (int)e.size() && e[j].y == e[i].y) {
        // 同一条扫描线上的事件必须先全部更新，再进入下一段高度。
        int l = lower_bound(xs.begin(), xs.end(), e[j].x1) - xs.begin();
        int r = lower_bound(xs.begin(), xs.end(), e[j].x2) - xs.begin();
        modify(1, 0, (int)xs.size()-1, l, r, e[j].delta);
        ++j;
    }
    last_y = e[i].y; i = j;
}
return ans;
}

```

例题：P5490 【模板】扫描线。`xs` 必须去重排序，线段树叶子代表 `xs[i]` 到 `xs[i+1]` 的小区间；所以树的右边界是 `xs.size()-1`，更新是半开区间 `[l,r)`。把离散点当成叶子会多算或少算一格。

测试样例：矩形并面积的相邻、重叠、嵌套

下面采用“输入矩形左下角和右上角”的调用格式。每个矩形对应半开区域 `[x1,x2) * [y1,y2)`；若你的题目允许任意顺序输入，先规范化端点。

调用 1：1 个矩形

`[(0,0,2,1)]`

期望面积：2

调用 2：两个只接边、不重叠的矩形

`[(0,0,1,1),(1,0,2,1)]`

期望面积：2

调用 3：两个有重叠的矩形

`[(0,0,2,2),(1,1,3,3)]`

期望面积：4+4-1=7

调用 4：一个矩形完全包含另一个矩形

`[(0,0,4,4),(1,1,3,3)]`

期望面积：16

第二组检查相邻坐标是否被错误重复计算；第三组检查覆盖层数从 2 降回 1 时是否错误清空长度；第四组检查 `cover>0` 时是否直接取整段长度。每个调用都必须清空 `xs`、事件数组、线段树 `tag/len`，否则把四组放进同一个 `t` 中会暴露多测污染。

17.14 超过十行模板的针对性自测

这一节专门给前面基础章节中的大板子做“抄完立即验”的小测试。每一组都按总模板包装：

```
int t = 1;
cin >> t;
while (t--) solve();
```

若某个板子本身只是函数或数据结构，没有统一题面输入，就写成“调用测试”；这不是少写样例，而是避免把不存在的接口硬编成错误题面。真正提交时，把调用测试里的对象和操作翻译到题目自己的 `solve()` 即可。所有多测都要在 `solve()` 开头重建图、清空节点池、重置时间戳和懒标记。

17.14.1 基础、枚举、差分与二分

折半搜索：空半边、没有可行解、恰好命中上界

代码维护的是“子集和不超过 `limit` 的最大值”，四个测试应分别得到：

```
T=4
1) a=[], limit=0                -> 0
2) a=[7], limit=6              -> 0
3) a=[3,5,6], limit=8          -> 8
4) a=[8,1,4,4], limit=9        -> 9
```

第三组抓两半枚举后的二分恰好命中，第四组抓重复值和“不能贪心取当前最大数”。第一组必须让 `gen` 返回只含 0 的数组，不能访问空半边。

一维差分：单点、整段、重叠更新和负数更新

每组先给初始数组，再执行所有区间加，输出最终数组：

```
T=4
1) [5]; add(1,1,+3)             -> [8]
2) [0,0,0,0]; add(1,4,+2)       -> [2,2,2,2]
3) [1,2,3,4]; add(2,3,+5), add(1,4,-1) -> [0,6,7,3]
4) [5,5,5]; add(1,2,-5), add(2,2,+10) -> [0,10,5]
```

第三、四组必须检查 `diff[r+1]` 的位置没有越界，并且多次更新的顺序是可叠加的；把 `r+1` 写成 `r` 会在第二组整段更新时立刻暴露。

二分答案：答案在端点、每组一段和多组切分

对“把正数数组分成至多 `k` 段，最小化最大段和”测试：

T=4

- | | |
|--------------------|-------|
| 1) [5], k=1 | -> 5 |
| 2) [1,2,3], k=3 | -> 3 |
| 3) [7,2,5,10], k=2 | -> 14 |
| 4) [4,4,4,4], k=2 | -> 8 |

第二组抓 $k=n$ ，第三组抓不能只按平均值切分，第四组抓多个相同最大值。 l 必须从数组最大值开始，不能从 0 开始；`check` 中遇到单个 $x>limit$ 要立即失败。

差分约束：无约束、可行环、负环和多条链

边 (u,v,w) 表示 $x[v] \leq x[u]+w$ ，只检查是否有解：

T=4

- | | |
|--------------------------------------|------------|
| 1) $n=1$, 0 条边 | -> 有解 |
| 2) $n=2$: 1->2(5), 2->1(5) | -> 有解 |
| 3) $n=2$: 1->2(-1), 2->1(-1) | -> 无解 (负环) |
| 4) $n=3$: 1->2(0), 2->3(0), 3->1(1) | -> 有解 |

第三组必须触发第 v 轮更新计数；第四组说明“有环”不等于“无解”，只有负环才无解。多测时 `pushed/in_queue/distance` 不清空，会让后一组错误继承负环状态。

17.14.2 数学、筛法、生成函数与树计数

试除分解、线性筛和欧拉函数：1、质数、平方数、重复质因子

质因数分解 T=4:

- | | |
|-------|---------------------|
| 1) 1 | -> 空分解 |
| 2) 2 | -> 2^1 |
| 3) 36 | -> $2^2 \times 3^2$ |
| 4) 97 | -> 97^1 |

欧拉函数 T=4:

- | | |
|---------------|-------|
| 1) $\phi(1)$ | -> 1 |
| 2) $\phi(2)$ | -> 1 |
| 3) $\phi(12)$ | -> 4 |
| 4) $\phi(36)$ | -> 12 |

筛法则单独检查 $N=1, 2, 10, 30$: $N=1$ 不应访问质数数组越界； $N=10$ 的质数是 2,3,5,7, 对应 $\phi(1..10)=1, 1, 2, 2, 4, 2, 6, 4, 6, 4$ 。 $p==lp[i]$ 的分支必须让 $\mu[i \times p]=0$ 并停止继续枚举。

整除分块：商相同区间和边界 n/i

计算 $\text{sum}(\text{floor}(n/i), i=1..n)$:

```
n=1 -> 1
n=2 -> 2+1=3
n=5 -> 5+2+1+1+1=10
n=10 -> 10+5+3+2+2+1+1+1+1+1=27
```

四组都要让 $r=n/(n/l)$ ，不能写成 $r=n/l$ ；最后一个商为 1 的长区间是最容易漏掉的部分。

非质数模的同余合并：互质、相容非互质、相同模数和无解

调用 `merge_congruence(a1,m1,a2,m2)`:

```
1) x=1 (mod 3), x=4 (mod 5)      -> x=4, mod 15
2) x=1 (mod 4), x=3 (mod 6)      -> x=9, mod 12
3) x=2 (mod 4), x=2 (mod 6)      -> x=2, mod 12
4) x=0 (mod 2), x=1 (mod 4)      -> 无解
```

第二、三组抓 $g=\text{gcd}(m1,m2)$ 后的最小公倍数；第四组抓 $d\%g!=0$ 。合并后的 a 必须归一到 $[0,m)$ ，中间乘法用 `__int128`，否则大模数样例会溢出。

异或前缀和与普通小卷积：周期四、单位元和重复系数

`xor_prefix(0)=0, xor_prefix(1)=1, xor_prefix(2)=3, xor_prefix(3)=0`

```
[1] * [1]          -> [1]
[1,1] * [1,1]      -> [1,2,1]
[1,2,3] * [4,5]    -> [4,13,22,15]
```

第一行抓 $n\&3==0$ ，不能把 0 和 4 的分支混淆；卷积第三组同时检查 $c[i+j]$ 下标和重复贡献累加。

Prüfer 序列与矩阵树定理：最小树、星形树、路径树和不连通图

Prüfer 解码:

```
1) code=[]          -> n=2, 边为 1-2
2) code=[1]         -> n=3, 边为 2-1、1-3
3) code=[1,1]       -> n=4, 得到以 1 为中心的星形树
```

Prüfer 编码:

```
4) 路径 1-2-3-4 -> [2,3] (使用最小编号叶子)
```

矩阵树的删除一行一列后的行列式:

- | | |
|--|------|
| 1) 单边图 1-2 的 1×1 矩阵 [1] | -> 1 |
| 2) 三角形的矩阵 $\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$ | -> 3 |
| 3) 四点路径的三对角余子式 | -> 1 |
| 4) 不连通图的零矩阵 | -> 0 |

第一组抓 $n-2=0$ 的空序列；第四组抓叶子优先队列和 $\text{pivot}==n$ 的行列式零分支。矩阵树代码若模数不是质数，不能直接使用费马逆元，这四组默认模数为质数。

17.14.3 数据结构：下标、懒标记、版本与回滚

并查集、树状数组和普通线段树：单点、整段、重复操作、多测清空

并查集 $T=4$:

- | | |
|--|-------------------|
| 1) $n=1$, <code>query(1,1)</code> | -> true |
| 2) $n=3$, <code>union(1,2)</code> , <code>query(1,3)</code> | -> false |
| 3) 接上 <code>union(2,3)</code> , <code>query(1,3)</code> | -> true |
| 4) 再次 <code>union(1,3)</code> | -> false (不能重复计数) |

树状数组 $T=4$:

- | | |
|--|------|
| 1) $n=1$, <code>add(1,5)</code> , <code>sum(1)</code> | -> 5 |
| 2) $n=4$, <code>add(1,4,2)</code> , <code>range(1,4)</code> | -> 8 |
| 3) [1,2,3,4] 通过单点加构造后 <code>range(2,3)</code> | -> 5 |
| 4) 先 <code>add(4,-3)</code> , 再 <code>query(1,4)</code> , 必须包含最后一个下标 | |

普通“区间加、区间和”线段树再按下列四组手算最终查询：

- | | |
|--|-------|
| 1) [5]: <code>query(1,1)</code> | -> 5 |
| 2) [0,0,0,0]: <code>add(1,4,2)</code> , <code>query(1,4)</code> | -> 8 |
| 3) [1,2,3,4]: <code>add(2,3,5)</code> , <code>query(1,4)</code> | -> 20 |
| 4) [1,1,1]: <code>add(1,3,2)</code> , <code>add(2,2,-1)</code> , <code>query(1,3)</code> | -> 8 |

第四组必须经过“整段懒标记后访问子区间”的 `push`；把 `push_down` 忘掉时通常只会在这组错。

线段树上二分与稀疏表：第一个位置、找不到、重复最值

对 `first_at_least`:

- | | |
|--------------------------|-------|
| 1) $a=[0]$, $x=0$ | -> 1 |
| 2) $a=[1,3,5]$, $x=4$ | -> 3 |
| 3) $a=[1,3,5]$, $x=6$ | -> -1 |
| 4) $a=[0,0,0,7]$, $x=7$ | -> 4 |

对静态区间最小值：

```

1) [5], query(1,1)          -> 5
2) [3,1,2,1], query(1,4)   -> 1
3) [3,1,2,1], query(2,3)   -> 1
4) [7,7,7], query(1,3)     -> 7

```

第二、三组同时检查 $k = \text{floor}(\log_2(\text{len}))$ 和右端点 $r - (1 \leq k) + 1$ ；稀疏表只能用于幂等运算，不能把这份代码改成区间和。

二进制字典树、主席树和块状数组：零值、重复值、跨边界

二进制字典树（先插入，再查询最大异或）：

```

1) 插入 [0], query(0)          -> 0
2) 插入 [1,2,4], query(3)      -> 7
3) 插入 [5,5], query(5)        -> 0
4) 插入 [8,15], query(7)       -> 15

```

主席树静态区间第 k 小：

```

1) [5], query(1,1,1)          -> 5
2) [1,2,3], query(1,3,1), query(1,3,3) -> 1,3
3) [4,4,1], query(1,3,2)      -> 4
4) [10,0,7,3], query(2,4,2)   -> 3

```

块状数组使用 0-based 下标测试 `add(l,r,v)`：

```

1) n=1, add(0,0,5)            -> [5]
2) n=4, add(0,3,1)            -> [1,1,1,1]
3) n=5, add(1,3,2)            -> [0,2,2,2,0]
4) n=6, add(0,5,3), add(2,4,-1) -> [3,3,2,2,2,3]

```

主席树第二、三组抓版本相减的左右端点；块状数组第三、四组抓“同一块”和“跨多个整块”两个分支。

Treap 与可回滚并查集：空树、重复键、快照恢复

Treap 不固定随机优先级，不能比较树形，只比较中序序列和子树大小：

```

1) 空树 split(key=0)          -> 左右均空
2) 插入 [5], split(4)          -> 左空，右为 [5]
3) 插入 [2,2,7]，中序必须保留两个 2 -> [2,2,7]
4) split(5) 后 merge(left,right) -> 恢复原中序和 size

```

可回滚并查集：

```

初始 1,2,3 独立; s0=snapshot()
union(1,2); s1=snapshot()
union(2,3); 此时 1 与 3 连通
rollback(s1); 此时 1 与 2 连通、1 与 3 不连通
rollback(s0); 三个点再次全部独立

```

这里专门抓 `rollback` 时恢复的是“合并前的大集合大小”，以及空合并不应压入历史栈；如果把路径压缩加回去，回滚会失效。

17.14.4 图论：路径、连通性、流和树上边界

拓扑排序：单点、链、分支和有向环

1) $V=1, E=0$	-> 拓扑序 [1]
2) $1 \rightarrow 2 \rightarrow 3$	-> [1, 2, 3]
3) $1 \rightarrow 3, 2 \rightarrow 3$	-> 1/2 任一先出, 3 必须最后
4) $1 \rightarrow 2, 2 \rightarrow 1$	-> 检测为环, 不能输出完整序列

第三组不能把拓扑序误认为唯一；第四组检查 `topo.size() != n`，并且每个 `solve()` 都要重新计算入度。

Dijkstra 与 0-1 BFS：重复堆项、不可达点和零边

Dijkstra 的四组边（源点均为 1）：

1) $V=1, E=0$	-> <code>dist[1]=0</code>
2) $1 \rightarrow 2(5), 1 \rightarrow 3(1), 3 \rightarrow 2(1)$	-> <code>dist[2]=2</code>
3) $1 \rightarrow 2(4)$, 点 3 无入边	-> <code>dist[3]=INF</code>
4) $1 \rightarrow 2(0), 2 \rightarrow 3(0), 1 \rightarrow 3(5)$	-> <code>dist[3]=0</code>

0-1 BFS 用同样的方向边测试：

1) 单点	-> 0
2) $1 \rightarrow 2(0), 2 \rightarrow 3(1)$	-> <code>dist[3]=1</code>
3) $1 \rightarrow 2(1), 1 \rightarrow 3(0), 3 \rightarrow 2(0)$	-> <code>dist[2]=0</code>
4) 点 4 不可达	-> <code>dist[4]=INF</code>

第三组抓 `w=0` 必须放队首；Dijkstra 第二组抓旧堆项，弹出时不能再次松弛旧距离。

最小生成树：单点、重边、环和不连通图

1) $V=1, E=0$	-> <code>cost=0, connected=true</code>
2) 三角形边权 1, 2, 3	-> <code>cost=3</code>
3) 两点有两条边权 5, 1	-> <code>cost=1</code>

4) $V=3$, 仅有边 1-2-> `connected=false`

第三组抓按边权排序和重复连边;第四组不能因为已经得到一个森林就误报为生成树。Kruskal 和 Prim 的结果应在四组上完全一致。

强连通分量、割点和桥：空边、全环、重边

强连通分量：

- | | |
|-----------------------|----------------------|
| 1) $V=1, E=0$ | -> 1 个分量 |
| 2) 1->2, 2->3 | -> 3 个分量 |
| 3) 1->2, 2->3, 3->1 | -> 1 个分量 |
| 4) 1<->2, 2->3, 3<->4 | -> {1,2}, {3,4} 两个分量 |

无向桥：

- | | |
|----------------------|--------------|
| 1) 单点 | -> 0 条桥 |
| 2) 链 1-2-3 | -> 2 条桥 |
| 3) 三角形 | -> 0 条桥 |
| 4) 两条平行边 1-2, 再加 2-3 | -> 只有 2-3 是桥 |

第四组必须使用边编号跳过父边；若只写 `v==parent`，平行边会被错误当成桥。SCC 第四组抓缩点后保留两个分量而不是把分量内边当成有向无环图边。

二分图染色与最大匹配：孤立点、增广链、奇环

染色：

- | | |
|----------------|---------|
| 1) $V=1, E=0$ | -> 是二分图 |
| 2) 一条边 1-2 | -> 是 |
| 3) 路径 1-2-3-4 | -> 是 |
| 4) 三角形 1-2-3-1 | -> 不是 |

最大匹配（左侧点数=右侧点数）：

- | | |
|---------------------|--------------|
| 1) 无边 | -> 0 |
| 2) 只有 1 条边 | -> 1 |
| 3) 边 1-{1,2}, 2-{2} | -> 2（必须回溯增广） |
| 4) 完全二分图 $K_{3,3}$ | -> 3 |

第三组是 `vis` 时间戳最关键的样例；每个左点开始增广前必须换新时间戳，不能每次都清空错误的那一侧或沿用旧访问标记。

最大流与最小费用最大流：零容量、瓶颈、拆路和需求不足

Dinic：

- | | |
|-------------------------------|------|
| 1) s, t 之间没有边 | -> 0 |
| 2) 唯一边 $s \rightarrow t$ 容量 5 | -> 5 |
| 3) 两条互不相交路径, 容量分别 3、2 | -> 5 |
| 4) 两条路径共享中间边容量 1 | -> 1 |

每次 BFS 后 it 必须清零; 第三、四组检查反向边和残量网络。费用流 `min_cost_flow(s, t, need)`:

- | | |
|------------------------------|-------------------|
| 1) $need=0$ | -> (0, 0) |
| 2) 唯一路径容量 3、单位费用 5, $need=2$ | -> (2, 10) |
| 3) 两条路径费用 2 和 8, $need=3$ | -> (3, 10) |
| 4) 总容量 1 但 $need=2$ | -> flow=1, 不能伪造满流 |

若题目允许负费用边, 再加一组 $s \rightarrow a(-5), a \rightarrow t(0), s \rightarrow t(0)$, 最优费用应为 -5; 这组用于检查初始势能是否正确, 不能把“所有费用非负”这个前提偷偷当成算法的一部分。

最近公共祖先、树上差分、直径和树形动态规划

用同一棵树 1-2, 1-3, 3-4, 3-5:

最近公共祖先: $lca(1, 1)=1, lca(2, 4)=1, lca(4, 5)=3, lca(1, 5)=1$

树上路径加: 路径 (4, 5) 加 1 后, 3、4、5 各增加 1, 1、2 不增加

直径: 端点可为 2、4/5, 长度为 3

再单独测: 单点树直径为 0, 链 1-2-3-4 直径为 3, 星形树 1-{2, 3, 4} 直径为 2。树形最大独立集用点权:

- | | |
|----------------------|-------|
| 1) 单点权 5 | -> 5 |
| 2) 边 1-2, 权 5, 7 | -> 7 |
| 3) 链权 1, 10, 1 | -> 10 |
| 4) 星形中心权 10, 三个叶子权 6 | -> 18 |

第一组抓根节点和 $lca(u, u)$; 路径差分要区分“点路径”和“边路径”, 不能把 lca 的减法公式混用。

深度优先序、重链剖分与长链剖分: 链、星、跨链路径

树 A: 1-2-3-4

- 子树(1) 应覆盖全部 DFS 序
- 子树(3) 只覆盖 {3, 4}
- HLD `path_sum(2, 4)` 必须包含 2、3、4
- 长链根 1 的长儿子必须是 2, 而不是按“子树大小”重新选择

树 B: 1-{2, 3, 4}

- 5) HLD `path_sum(2,3)` 必须拆成两条轻链并经过 1
- 6) 长链长度相同的儿子只要求结果正确，不能依赖某个固定 `tie-break`

这组不比较 `dfn` 的具体编号，因为同深度遍历顺序可以不同；只检查子树区间连续、路径和正确、长链选择标准确实是“最大向下深度”。

点分治代码若只保留重心分解而没有 `query/modify/answer`，只能做调用测试：单点、四点链、四点星、半径 0、两个子树各有一个距离相同的点。重点是验证重心被标记后不会再次进入同一分治层，并且跨重心的点对只计一次。

17.14.5 动态规划和字符串

线性动态规划、树形动态规划和数位动态规划

线性动态规划这一段同时测试“最长严格递增子序列”和“最大子段和”两段代码，输出写成 (LIS 长度, 最大子段和)：

1) <code>a=[5]</code>	<code>-> (1,5)</code>
2) <code>a=[-5]</code>	<code>-> (1,-5)</code>
3) <code>a=[3,1,2]</code>	<code>-> (2,6)</code>
4) <code>a=[5,1,5,1]</code>	<code>-> (2,12)</code>

第二组抓最大子段和不能把答案初值固定成 0；第三组抓 `lower_bound` 与“不能把 LIS 当成最长连续段”。树形 DP 用前面的四组独立集权值；换根 DP 再测单点、链、星和两个相同最大值，重点检查第二次 DFS 使用的是“父答案减去当前子树贡献”，不能把子树答案重复加回。

数位 DP 统计区间内不含数字 4 的数（包含 0）：

<code>T=4</code>	
1) <code>[0,0]</code>	<code>-> 1</code>
2) <code>[0,4]</code>	<code>-> 4</code>
3) <code>[0,10]</code>	<code>-> 10</code>
4) <code>[0,99]</code>	<code>-> 81</code>

第二组抓数字 4 的排除和前导零；第四组抓 `count(R)-count(L-1)`，以及 `tight` 只在仍贴着上界时为真。

Knuth-Morris-Pratt 字符串匹配：首位、重叠和无匹配

`T=4`（文本，模式，第一次出现位置/重叠出现次数）

1) <code>a</code>	<code>, a</code>	<code>-> 0 / 1</code>
2) <code>aaaaa</code>	<code>, aaa</code>	<code>-> 0 / 3</code>
3) <code>abc</code>	<code>, d</code>	<code>-> -1 / 0</code>
4) <code>abababa</code>	<code>, aba</code>	<code>-> 0 / 3</code>

第二、四组必须允许重叠匹配；把匹配成功后 j 直接清零会少算。模式为空串是否允许必须由题面规定，模板不要擅自把它当普通非空模式。

AC 自动机、后缀数组、后缀自动机和回文自动机的四组详细样例已经紧跟补充实现页；再加一组统一多测：**a、aaaa、abacaba、abab**。每组都必须重新 `init`，否则上一组的 Trie 节点和 `fail` 指针会污染下一组。后缀数组比较后缀时要检查空后缀，回文自动机要保留长度 `-1` 的虚根。

17.14.6 进阶专题：整体二分、时间分治、可满足性与匹配

下面这些模板的代码依赖题目状态，采用调用级测试，但每组仍然给出可核对的目标。

整体二分与线段树分治时间

整体二分（第 k 小）：

- | | |
|-------------------|------|
| 1) 数组 [5], $k=1$ | -> 5 |
| 2) [1,2,3], $k=1$ | -> 1 |
| 3) [1,2,3], $k=3$ | -> 3 |
| 4) [4,4,1], $k=2$ | -> 4 |

线段树分治时间（边存在区间）：

- | | |
|----------------------------|---------------------|
| 1) 单条边全程存在，询问其两端 | -> 连通 |
| 2) 边只在时间 [2,3] 存在，询问 $t=1$ | -> 不连通 |
| 3) 两条边的存在区间只在一个时刻重合 | -> 该时刻连通 |
| 4) 删除不存在的边 | -> 必须按题面判非法，不能修改回滚栈 |

整体二分中答案区间必须覆盖真实值域；时间分治中叶节点才回答询问，回到父节点时必须回滚到进入节点前的快照。

二可满足性算法与约束状态图

子句格式为 (x_i 或 x_j)：

- | | |
|--|-----------------------------------|
| 1) 0 个变量、0 个子句 | -> 可满足 |
| 2) (x_1 或 x_1) | -> $x_1=\text{true}$ |
| 3) (x_1 或 x_1) 且 ($\neg x_1$ 或 $\neg x_1$) | -> 不可满足 |
| 4) (x_1 或 x_2)、($\neg x_1$ 或 x_2)、(x_1 或 $\neg x_2$) | -> 可满足，唯一要求 $x_1=x_2=\text{true}$ |

第三组抓同一变量和反变量是否落在同一个强连通分量；第四组抓蕴含边方向。状态图搜索若没有定义“重复状态”和“终止状态”，不能只套 DFS，应分别测试空状态、一步可胜、重复到旧状态和无路可走四类调用。

快速傅里叶变换/数论变换与 Kuhn–Munkres 加权匹配

卷积用单位多项式、重复系数和非二次长度三组；结果应与前面生成函数的 $[1], [1, 1], [1, 2, 3]$ 三组完全一致，另加：

$[1, 0, 0, 1] \star [1, 1] \rightarrow [1, 1, 0, 1, 1]$

它专门抓变换长度是否至少为 $a.size()+b.size()-1$ ，以及逆变换除以长度是否在模意义下使用逆元。

Kuhn–Munkres 最大权完备匹配：

1) $[[5]]$	$\rightarrow 5$
2) $[[1, 2], [3, 4]]$	$\rightarrow 5$
3) $[[-1, -2], [-3, -4]]$	$\rightarrow -5$
4) 对角为 10、非对角为 1 的 3×3 矩阵	$\rightarrow 30$

第三组不能把答案初值设成 0；如果模板只支持非负权，必须先整体平移并在最后还原。

笛卡尔树、虚树、Steiner 树和高斯消元

最小笛卡尔树调用：

1) $[1]$	$\rightarrow$ 根为 1
2) $[1, 2, 3]$	$\rightarrow$ 根为 1，右链
3) $[3, 2, 1]$	$\rightarrow$ 根为 1，左链
4) $[2, 1, 3]$	$\rightarrow$ 根为 1，左右子树分别为 2、3

虚树四组关键点：单个关键点、关键点包含根、两个叶子、关键点的最近公共祖先不是原关键点。输出只检查保留了所有关键点及其必要的 LCA，不能把整棵原树都复制进虚树。

Steiner 树先用三点图手算： $1-2=1, 2-3=1, 1-3=3$ ，终端 $\{1, 3\}$ 的最小连通代价为 2；再测单终端代价 0、重复终端不能重复计费、不可达终端返回题面规定的无解。高斯消元则复用补充页的“唯一解、无解、多解”三组，另加交换行的主元为 0 的矩阵，确认会换到下面的非零行。

这些自测不是随机数据集：每组都至少命中一个边界分支和一个正常分支。需要做更大压力测试时，保留同一批核心模式，用生成器重复 t 次；不要把 $t=2345678$ 的数百万行文本直接塞进 PDF，现场测试应生成后通过标准输入喂给程序。

18 对拍

对拍 (Stress Testing) 是竞赛中查错的核心手段：用随机数据生成器产生大量测试数据，分别用暴力程序和待测程序运行，比较输出是否一致。一旦发现不一致，即可定位 bug。对拍三件套：

- **gen.cpp**: 随机数据生成器，输出一组合法的输入数据。
- **brute.cpp**: 暴力解法（正确但慢），作为对照标准。
- **sol.cpp**: 你的正式解法（快但可能有 bug）。

18.1 对拍脚本

18.1.1 Linux / macOS (Bash)

```
#!/bin/bash
# 编译三个程序
g++ -O2 -o gen gen.cpp
g++ -O2 -o brute brute.cpp
g++ -O2 -o sol sol.cpp

for ((i = 1; ; i++)); do
    # 生成随机数据（用循环变量 i 做种子）
    ./gen $i > in.txt
    # 分别运行暴力和正解
    ./brute < in.txt > out_brute.txt
    ./sol < in.txt > out_sol.txt
    # 比较输出
    if diff out_brute.txt out_sol.txt > /dev/null 2>&1; then
        echo "Test $i: AC"
    else
        echo "Test $i: WA !!!"
        echo "--- Input ---"
        cat in.txt
        echo "--- Brute ---"
        cat out_brute.txt
    fi
done
```

```
        echo "--- Sol ---"
        cat out_sol.txt
        break
    fi
done
```

18.1.2 Windows (BAT)

```
@echo off
g++ -O2 -o gen.exe gen.cpp
g++ -O2 -o brute.exe brute.cpp
g++ -O2 -o sol.exe sol.cpp

set i=0
:loop
set /a i+=1
gen.exe %i% > in.txt
brute.exe < in.txt > out_brute.txt
sol.exe < in.txt > out_sol.txt
fc out_brute.txt out_sol.txt > nul
if errorlevel 1 (
    echo Test %i%: WA !!!
    echo --- Input ---
    type in.txt
    echo --- Brute ---
    type out_brute.txt
    echo --- Sol ---
    type out_sol.txt
    pause
    goto :eof
)
echo Test %i%: AC
goto loop
```

- 编译时加 `-O2` 让暴力跑得更快，能测更多组。
- 如果正解有可能输出多种正确答案（如最优方案不唯一），不能直接 diff，需要写 checker 来判断正确性。
- 如果对拍跑很久都没问题，考虑增大数据范围或换生成策略（边界、特殊图等）。

18.2 随机数据生成器 (gen.cpp)

生成器的核心原则：

- 用命令行参数作为随机种子，保证每次运行的数据不同，且可复现。
- 生成的数据必须满足题目的输入格式和约束条件。
- 对拍时先用小数据 ($n \leq 10$)，确认暴力能跑出来。

```
#include <bits/stdc++.h>
using namespace std;

mt19937 rng; // 梅森旋转随机数引擎

// 生成 [lo, hi] 范围内的随机整数
int randint(int lo, int hi) {
    return uniform_int_distribution<int>(lo, hi)(rng);
}

// 生成 [lo, hi] 范围内的随机 long long
long long randll(long long lo, long long hi) {
    return uniform_int_distribution<long long>(lo, hi)(rng);
}

// 生成长度为 len 的随机小写字母字符串
string randstr(int len) {
    string s(len, ' ');
    for (char& c : s) c = 'a' + randint(0, 25);
    return s;
}

// 生成 n 个节点的随机树 (输出 n-1 条边)
// 方法：每个新节点随机连接到已有节点
vector<pair<int,int>> rand_tree(int n) {
    vector<pair<int,int>> edges;
    for (int i = 2; i <= n; i++) {
        int fa = randint(1, i - 1);
        edges.push_back({fa, i});
    }
    // 打乱边的顺序和方向
    shuffle(edges.begin(), edges.end(), rng);
    for (auto& [u, v] : edges)
        if (randint(0, 1)) swap(u, v);
    return edges;
}
```

```
// 生成 n 个节点 m 条边的随机简单图（无重边无自环）
vector<pair<int,int>> rand_graph(int n, int m) {
    set<pair<int,int>> used;
    vector<pair<int,int>> edges;
    while ((int)edges.size() < m) {
        int u = randint(1, n), v = randint(1, n);
        if (u == v) continue;
        if (u > v) swap(u, v);
        if (used.count({u, v})) continue;
        used.insert({u, v});
        edges.push_back({u, v});
    }
    shuffle(edges.begin(), edges.end(), rng);
    for (auto& [u, v] : edges)
        if (randint(0, 1)) swap(u, v);
    return edges;
}

// 生成 n 个不重复的随机整数（值域 [lo, hi]）
vector<int> rand_distinct(int n, int lo, int hi) {
    set<int> used;
    vector<int> res;
    while ((int)res.size() < n) {
        int x = randint(lo, hi);
        if (!used.count(x)) {
            used.insert(x);
            res.push_back(x);
        }
    }
    return res;
}

// 生成随机排列 [1, 2, ..., n]
vector<int> rand_perm(int n) {
    vector<int> p(n);
    iota(p.begin(), p.end(), 1);
    shuffle(p.begin(), p.end(), rng);
    return p;
}

int main(int argc, char* argv[]) {
    // 用命令行参数做种子
    rng.seed(atoi(argv[1]));
}
```

```
// ===== 根据题目修改以下内容 =====
int n = randint(1, 10); // 对拍时用小范围
cout << n << "\n";
for (int i = 0; i < n; i++) {
    cout << randint(1, 100);
    if (i + 1 < n) cout << " ";
}
cout << "\n";

return 0;
}
```

- mt19937 比 rand() 质量更好且可移植。用 mt19937_64 生成 64 位随机数。
- uniform_int_distribution<int>(lo, hi) 生成 $[lo, hi]$ 闭区间均匀分布。
- 生成随机树时，如果需要链状树（极端数据），让每个节点连接 $i - 1$ ；如果需要菊花图，让所有节点连接节点 1。
- 随机图的边数 m 不能超过 $\frac{n(n-1)}{2}$ ，否则死循环。

18.3 暴力解法（brute.cpp）

暴力程序的原则：正确性第一，效率无所谓。能用 $O(n^3)$ 就用 $O(n^3)$ ，能枚举就枚举，绝不优化。

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'

void solve() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];

    // ===== 暴力做法（保证正确，不管复杂度）=====
    // 例：暴力枚举所有子集 / 全排列 /  $O(n^2)$  枚举
    int ans = 0;
    for (int i = 0; i < n; i++)
        for (int j = i; j < n; j++) {
```



```
        // ...
    }

    cout << ans << endl;
}

signed main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int t = 1;
    // cin >> t; // 根据题目决定是否多测
    while (t--) solve();
    return 0;
}
```

18.4 交互题对拍

交互题无法用管道对拍，需要写一个 interactor 同时充当评测端和数据生成器。

```
// interactor.cpp —— 模拟交互评测
// 编译后用管道连接：
// Linux: mkfifo pipe; (./interactor < pipe | ./sol > pipe)
// 或者直接写成单文件 interactor，用 popen 启动 sol
#include <bits/stdc++.h>
using namespace std;

int main() {
    mt19937 rng(42);
    // 生成隐藏数据
    int secret = uniform_int_distribution<int>(1, 1000)(rng);

    // 模拟交互过程
    int queries = 0;
    while (true) {
        int guess;
        // 读取选手的输出（选手的 cout 是这里的 cin）
        if (!(cin >> guess)) break;
        queries++;
        if (guess == secret) {
            cout << "=" << endl; // 正确
            break;
        } else if (guess < secret) {
            cout << "<" << endl; // 太小
        }
    }
}
```

```
    } else {  
        cout << ">" << endl; // 太大  
    }  
}  
// 检查询问次数是否超限  
cerr << "Queries used: " << queries << endl;  
return 0;  
}
```

- 对拍时 gen 的数据范围要小！暴力程序通常是 $O(n^2)$ 或 $O(2^n)$ ， n 太大跑不完。一般 $n \leq 10$ 起步，跑上万组。
- 每次对拍脚本传入不同的种子 `./gen $i`，确保数据多样性。找到 WA 后用同一个种子可以复现。
- 注意行末空格和换行的差异可能导致 diff 报 WA，可以在对比前做 trim。
- 如果正解涉及浮点数输出，diff 无法判断精度误差，需要写 checker 比较相对/绝对误差。
- 生成的输入必须严格满足题目约束（如连通图、无重边、值域范围等），否则暴力和正解可能行为都不确定。

19 竞赛环境速查

19.1 Windows 命令行（CMD / PowerShell）

19.1.1 编译与运行

```
:: 基础编译
g++ -o sol.exe sol.cpp

:: 推荐编译选项（竞赛常用）
g++ -O2 -std=c++17 -o sol.exe sol.cpp

:: 开所有警告 + 调试信息（调试时用）
g++ -Wall -Wextra -Wshadow -g -fsanitize=address,undefined -o sol.exe sol.cpp

:: 编译并立即运行
g++ -O2 -o sol.exe sol.cpp && sol.exe

:: 从文件读入、输出到文件
sol.exe < in.txt
sol.exe < in.txt > out.txt

:: 同时输出到屏幕和文件（PowerShell）
.\sol.exe < in.txt | Tee-Object out.txt

:: 计时运行（PowerShell）
Measure-Command { .\sol.exe < in.txt > out.txt }
```

- `-fsanitize=address,undefined` 可以检测数组越界、未定义行为等，调试利器！但会显著变慢，提交前务必去掉。
- `-O2` 和 `-fsanitize` 不要同时用，优化可能把 sanitizer 的检测优化掉。
- Windows 下 PowerShell 和 CMD 的重定向语法一样，但管道行为有差异。推荐用 PowerShell。

19.1.2 文件操作

```
:: 查看文件内容
type in.txt                :: CMD
cat in.txt                 :: PowerShell (Get-Content 的别名)

:: 比较两个文件
fc out_brute.txt out_sol.txt :: CMD (按行比较)
diff out1.txt out2.txt      :: PowerShell (Compare-Object 的别名, 需 Git)

:: 复制 / 移动 / 删除
copy a.txt b.txt
move a.txt b.txt
del a.txt

:: 创建目录
mkdir contest

:: 列出当前目录文件
dir                        :: CMD
ls                         :: PowerShell
```

19.1.3 进程与调试

```
:: 强制终止程序 (死循环时用)
Ctrl + C                :: 终端中直接按

:: 查看正在运行的进程
tasklist | findstr sol.exe

:: 强制杀死进程
taskkill /F /IM sol.exe

:: 查看程序返回值 (上一条命令的退出码)
echo %errorlevel%        :: CMD
echo $LASTEXITCODE       :: PowerShell
```

19.1.4 常用技巧

```
:: 快速生成测试数据并对拍 (单行命令)
g++ -O2 -o gen.exe gen.cpp && g++ -O2 -o brute.exe brute.cpp && g++ -O2 -o sol.exe sol.
cpp
```

```

:: 批量编译当前目录所有 cpp 文件 (PowerShell)
Get-ChildItem *.cpp | ForEach-Object {
    g++ -O2 -o ($_.BaseName + ".exe") $_.FullName
}

:: 清理编译产物 (PowerShell)
Remove-Item *.exe, *.o, in.txt, out*.txt -ErrorAction SilentlyContinue

:: 查看 g++ 版本
g++ --version

:: 检查是否安装了某工具
where g++
where python

```

- 文件重定向 < in.txt > out.txt 在比赛中比在代码里写 freopen 更灵活，不用改代码。
- 如果需要在代码内重定向（某些 OJ 要求），用：


```
freopen("input.txt", "r", stdin);
freopen("output.txt", "w", stdout);
```
- PowerShell 中 Measure-Command 可以精确测量运行时间，判断是否会 TLE。

19.2 VS Code 快捷键速查

以下快捷键基于 Windows 默认键位。macOS 把 Ctrl 换成 Cmd。

19.2.1 文件与编辑

Ctrl + N	新建文件
Ctrl + O	打开文件
Ctrl + S	保存
Ctrl + Shift + S	另存为
Ctrl + W	关闭当前标签
Ctrl + Tab	切换已打开的文件
Ctrl + Z	撤销
Ctrl + Shift + Z	重做（或 Ctrl + Y）
Ctrl + X	剪切整行（光标在行上，不选中也可以）
Ctrl + C	复制整行（同上）
Ctrl + Shift + K	删除整行
Alt + Up/Down	上下移动当前行

Shift + Alt + Up/Down	向上/下复制当前行
Ctrl + D	选中当前单词，再按选下一个相同的
Ctrl + Shift + L	选中所有相同的单词（批量重命名变量超好用）

19.2.2 查找与替换

Ctrl + F	查找
Ctrl + H	查找并替换
Ctrl + Shift + F	全局搜索（跨文件）
Ctrl + G	跳转到指定行号
F12	跳转到定义
Alt + F12	速览定义（弹窗）
Ctrl + Shift + O	跳转到当前文件的符号（函数名等）

19.2.3 多光标与选择

Alt + Click	添加多光标
Ctrl + Alt + Up/Down	上下添加光标（竖向多行编辑）
Ctrl + Shift + L	选中所有匹配项并添加光标
Ctrl + L	选中当前行
Shift + Alt + 拖拽	列选择（矩形选择）
Ctrl + Shift + [/]	折叠/展开代码块

19.2.4 终端

Ctrl + `	打开/关闭集成终端
Ctrl + Shift + `	新建终端
Ctrl + Shift + 5	拆分终端（左右并排）
Ctrl + PageUp/Down	切换终端标签

- 在终端中按 `Up` 键可以翻出上一条命令，快速重复编译运行。
- 可以配置 VS Code 的 `tasks.json` 实现一键编译运行（见下方配置）。

19.2.5 调试

F5	启动调试
Shift + F5	停止调试
F9	切换断点
F10	单步跳过（Step Over）

F11	单步进入 (Step Into)
Shift + F11	单步跳出 (Step Out)
Ctrl + Shift + D	打开调试面板

19.2.6 其他实用操作

Ctrl + Shift + P	命令面板 (万能入口)
Ctrl + ,	打开设置
Ctrl + B	切换侧边栏显示/隐藏
Ctrl + J	切换底部面板显示/隐藏
Ctrl + K, Ctrl + S	查看所有快捷键
Ctrl + /	注释/取消注释当前行
Ctrl + Shift + A	块注释 (/* ... */)
Ctrl + K, Ctrl + F	格式化选中代码
Shift + Alt + F	格式化整个文件
Ctrl + Shift + V	预览 Markdown

19.3 VS Code 竞赛配置

19.3.1 一键编译运行 (tasks.json)

在 `.vscode/tasks.json` 中配置, 按 `Ctrl+Shift+B` 即可一键编译运行当前文件。

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "compile and run",
      "type": "shell",
      "command": "g++",
      "args": [
        "-O2", "-std=c++17",
        "-Wall", "-Wextra",
        "-o", "${fileDirname}/${fileBasenameNoExtension}.exe",
        "${file}"
      ],
      "group": {
        "kind": "build",
        "isDefault": true
      },
      "problemMatcher": "$gcc"
    }
  ]
}
```

```
]
}
```

19.3.2 调试配置 (launch.json)

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Debug C++",
      "type": "cppdbg",
      "request": "launch",
      "program": "${fileDirname}/${fileBasenameNoExtension}.exe",
      "args": [],
      "cwd": "${fileDirname}",
      "stopAtEntry": false,
      "externalConsole": true,
      "MIMode": "gdb",
      "miDebuggerPath": "gdb",
      "preLaunchTask": "compile and run"
    }
  ]
}
```

19.3.3 推荐插件

- **C/C++ (Microsoft)**: 语法高亮、智能补全、调试支持。
- **Code Runner**: 右键一键运行代码，支持输入重定向。
- **Competitive Programming Helper (cph)**: 自动抓取样例、一键测试，支持 Codeforces/AtCoder 等。
- **Error Lens**: 将编译错误/警告直接显示在代码行内，快速定位问题。
- **Bracket Pair Colorizer** (VS Code 已内置): 括号彩色匹配，嵌套多层时一目了然。

19.3.4 推荐用户设置 (settings.json 片段)

```
{
  // 自动保存
  "files.autoSave": "afterDelay",
  "files.autoSaveDelay": 1000,
  // 字体推荐等宽字体
```



```
"editor.fontFamily": "Consolas, 'Courier New', monospace",
"editor.fontSize": 15,
// 显示行号
"editor.lineNumbers": "on",
// Tab 转空格, 宽度 4
"editor.tabSize": 4,
"editor.insertSpaces": true,
// 括号匹配高亮
"editor.bracketPairColorization.enabled": true,
// 终端默认 PowerShell
"terminal.integrated.defaultProfile.windows": "PowerShell"
}
```

- 比赛时建议关闭自动补全的延迟弹出，避免打字时频繁弹出提示干扰思路：
"editor.quickSuggestions": false
- 使用 Ctrl+Shift+P 输入 “Snippets” 可以自定义代码片段，把常用模板设为一键插入。
- 如果终端太小，Ctrl+` 打开终端后，拖拽顶部边框可以调整高度。

20 11. 题型/关键词 → 算法索引

本页专门给“现场查名字”使用。下表中的算法先写完整中文/英文名称，括号内才是题解和洛谷题面中常见的简称；不要只记简称。标注“作用”的条目为 2100 分及以上的重点内容。

20.1 一、博弈论

约分	完整名称（常见写法）	2100 分及以上：作用
1800–2000	必胜态/必败态动态规划 (Win/Lose Dynamic Programming)	—
1800–2100	有向无环图上的 Sprague–Grundy 函数与最小 未出现值 (Directed Acyclic Graph、SG、mex)	把无偏无环博弈状态转成整数，便于判断胜负或合并子游戏。
1800–2100	多个独立子博弈的 Sprague–Grundy 值异或	判断游戏能否拆成互不影响的部分，并用异或合并各部分。
1800–2000	Bash 取石子游戏及限制取石子变形	—
1800–2000	普通 Nim 游戏的必胜操作构造	—
1900–2100	反常 Nim 游戏 (Misère Nim)	—
1900–2200	阶梯 Nim 游戏 (Staircase Nim)	只对特定编号台阶取异或，处理石子只能向下移动的游戏。
1900–2200	图游戏建模 (Graph Game Modeling)	把棋子移动、可达性和取点过程转成状态图，再做胜负或 Grundy 分析。

约分	完整名称（常见写法）	2100 分及以上：作用
2000–2300	Sprague–Grundy 函数打表与周期发现	将超大取石子数量压缩到前导段和周期内，避免按数量线性计算。
2100–2400	大范围取石子游戏的 Sprague–Grundy 周期压缩	对超大状态值利用已证明的周期直接回答，常用于 n 达到 10^{18} 的题。
1900–2200	普通 Nim 游戏的必胜策略恢复	根据异或和构造具体取哪一堆、取多少个。
2000–2300	状态压缩博弈动态规划 (State-Compressed Game Dynamic Programming)	用位掩码保存少量棋子/资源状态，枚举合法操作并记忆化。
1800–2100	记忆化博弈搜索 (Memoized Game Search)	对有大量重复状态的博弈搜索去重，避免重复递归。
1900–2200	极大极小搜索 (Minimax Search)	双方操作不同或不能拆成独立子游戏时，在完整博弈树上求最优决策。
2000–2300	Alpha-Beta 剪枝 (Alpha-Beta Pruning)	在极大极小搜索中利用上下界跳过不可能改变答案的分支。
2100–2400	博弈树状态去重与状态哈希 (Transposition Table and State Hashing)	合并不同走法到达的同一局面，降低棋类搜索的状态数。
2200–2500	多阶段/带资源限制的组合游戏	处理多个阶段、共享资源和操作规则变化，不能直接套独立游戏异或。
2300–2500	有环图上的博弈状态分析 (Cyclic Graph Game Analysis)	区分必胜、必败和平局，处理会回到旧状态的图博弈。

查阅位置：博弈专题基础模板后紧跟的博弈论详解与自测样例。

20.2 二、高级数据结构

约分	完整名称（常见写法）	2100 分及以上：作用
1800–2100	高阶前缀和与多阶差分 (Higher-Order Prefix Sums and Differences)	—
1900–2200	多重懒标记线段树 (Segment Tree with Multiple Lazy Tags)	同时维护多种区间修改，并正确合成和下传标记。
1900–2200	仿射标记线段树 (Affine-Transformation Segment Tree)	统一处理区间乘法、区间加法和区间和，标记形式为 $x \rightarrow ax+b$ 。
1800–2100	线段树上的二分 (Segment Tree Binary Search)	在区间最大值、容量或非负前缀和上找第一个满足条件的位置。
1800–2100	莫队算法 (Mo’s Algorithm)	—
2200–2400	带修改莫队算法 (Mo’s Algorithm with Modifications)	离线处理区间询问和时间维修改，状态移动为左端点、右端点、修改时间。
2200–2400	回滚莫队算法 (Rollback Mo’s Algorithm)	在不支持普通删除或需要撤销的区间统计中，用日志回退维护状态。
2000–2300	三维偏序的 CDQ 分治 (Three-Dimensional Dominance CDQ Divide and Conquer)	把一个维度分治、一个维度排序、最后一维交给树状数组统计。
2200–2400	CDQ 分治优化动态规划 (CDQ Divide and Conquer Optimization)	让左半状态只贡献给右半状态，优化具有时间/坐标偏序的动态规划转移。
1900–2200	可持久化字典树 (Persistent Trie / Persistent Prefix Tree)	对历史版本和区间版本做异或最大值、第 k 大等查询。
2200–2400	可持久化并查集 (Persistent Disjoint Set Union)	保存每个时刻的连通关系，支持复制历史版本、合并和查询连通性。
1900–2200	树上启发式合并 (Disjoint Set Union on Tree, DSU on Tree)	在线性树上统计每个子树的颜色频次、众数或频率函数。
2200–2500	长链剖分 (Long-Chain Decomposition)	用最长向下链优化树上深度 DP、 k 级祖先和同深度信息合并。

约分	完整名称（常见写法）	2100 分及以上：作用
2100–2400	点分树/重心树（Centroid Tree）	把树上距离查询转成重心祖先链上的若干距离容器，支持在线修改和查询。
2100–2400	李超线段树（Li Chao Segment Tree）	动态维护直线集合，在指定横坐标查询最大值或最小值。
2200–2400	线段插入版李超线段树（Li Chao Segment Tree for Line Segments）	让每条直线只在一个横坐标区间有效，处理线段插入与在线最值查询。
2300–2500	区间最值势能线段树（Segment Tree Beats）	维护区间取最小/最大与区间和，利用最大值、次大值和势能保证均摊复杂度。
2200–2500	伸展树（Splay Tree）	通过旋转把访问节点移到根，维护动态序列、排名和区间翻转。
2200–2400	文艺平衡树/序列伸展树（Reversible Splay Tree）	用伸展树维护序列区间反转、插入、删除和区间查询。
2400–2500	Link-Cut Tree 动态树（Link-Cut Tree, LCT）	维护动态森林的连边、断边、换根和路径信息。
2400–2500	动态动态规划与矩阵线段树（Dynamic Dynamic Programming and Matrix Segment Tree）	把局部状态转移写成矩阵，用线段树合并并支持单点修改后的全局答案。

20.3 三、数学与组合计数

约分	完整名称（常见写法）	2100 分及以上：作用
1800–2200	高级容斥与倍数容斥（Advanced Inclusion–Exclusion）	—
1900–2200	卢卡斯定理（Lucas Theorem）	—
1900–2200	扩展中国剩余定理（Extended Chinese Remainder Theorem，扩展 CRT）	合并模数不互质的同余方程，并处理解的存在性和最小非负解。

约分	完整名称（常见写法）	2100 分及以上：作用
2200–2500	模合数的组合数（Binomial Coefficients Modulo a Composite Number）	在不能直接使用阶乘逆元或 Lucas 定理时，按模数质因数分解计算组合数。
1800–2200	矩阵快速幂与线性递推（Matrix Exponentiation and Linear Recurrence）	—
2000–2300	高斯消元求秩与解空间（Gaussian Elimination for Rank and Solution Space）	判断方程组是否有解、自由元数量以及线性约束的解空间维数。
2100–2400	模意义高斯消元（Gaussian Elimination over a Finite Field）	在模质数或可逆环上求线性方程组，常用于概率、计数和线性约束。
2100–2400	高斯消元求行列式（Determinant by Gaussian Elimination）	计算矩阵行列式，作为矩阵树定理等组合计数的核心子程序。
2200–2500	莫比乌斯反演（Möbius Inversion）	把“所有倍数/所有约数”的累计统计反推出“恰好倍数/恰好最大公约数”的统计。
2200–2400	原根判定与原根求解（Primitive Root Testing and Construction）	构造模意义下生成整个乘法群的元素，为离散对数和周期问题提供坐标。
2200–2400	Baby-step Giant-step 离散对数算法（BSGS）	在模数较小时求 $a^x=b$ 的离散对数，复杂度约为平方根级别。
2300–2500	扩展 Baby-step Giant-step 离散对数算法（Extended BSGS）	处理底数与模数不互质的离散对数，先剥离公因子再进入 BSGS。
2300–2500	快速傅里叶变换与数论变换	在合适质数模数下快速计算多项式卷积、计数合并和字符串卷积。
2200–2400	普通生成函数基础（Ordinary Generating Functions）	把计数递推和组合对象转成多项式乘法或系数提取问题。
2300–2500	整数分拆生成函数（Generating Functions for Integer Partitions）	计算整数拆分、硬币组合和无序选取的方案数。

约分	完整名称（常见写法）	2100 分及以上：作用
2400–2500	集合分拆与排列计数生成函数（Generating Functions for Set Partitions and Permutations）	对集合结构、排列结构和组件合并进行高阶计数。
2000–2300	Burnside 引理（Burnside’s Lemma）	把等价染色的数量转成群元素固定点数量的平均值。
2200–2500	Pólya 计数定理（Pólya Enumeration Theorem）	按置换的循环结构批量计算旋转、翻转对称下的染色方案。
2100–2400	Legendre 符号（Legendre Symbol）	快速判断模奇质数下一个数是否为二次剩余。
2200–2500	Tonelli–Shanks 二次剩余算法（Tonelli–Shanks Algorithm）	在奇质数模下求平方根，处理 $x^2=a \bmod p$ 的有根和无根情况。
2300–2500	Cipolla 二次剩余算法（Cipolla’s Algorithm）	另一种模奇质数平方根算法，适合需要稳定实现的二次剩余题。
2000–2300	Prüfer 序列（Prüfer Sequence）	把标号树编码成长度为 $n-2$ 的序列，并利用度数公式计数。
2200–2400	Prüfer 序列计数应用	由点度数、叶子数量和限制条件反推标号树数量。
2200–2500	矩阵树定理（Matrix-Tree Theorem）	用拉普拉斯矩阵的余子式计算无向图生成树数量。
2300–2500	Lindström–Gessel–Viennot 引理（LGV 引理）	用行列式计算不相交路径组的数量，处理格点路径和网络路径计数。
1800–2200	鸽巢原理构造（Pigeonhole Principle Constructions）	—
1800–2200	线性递推的矩阵构造（Matrix Construction for Linear Recurrences）	—

20.4 四、字符串算法

约分	完整名称（常见写法）	2100 分及以上：作用
1800–2000	Booth 循环字符串最小表示法（Booth’s Algorithm）	—
2000–2300	回文自动机（Palindromic Automaton, PAM）	维护每个位置的最长回文后缀和所有不同回文串。
2100–2300	回文自动机出现次数统计	沿回文自动机失配指针汇总每个回文串的出现次数。
2300–2500	回文自动机与动态规划	把回文后缀链转成分割、计数或最优值转移。
2100–2300	后缀数组、最长公共前缀与区间最小值查询（Suffix Array、Longest Common Prefix、Range Minimum Query）	对后缀排序后用最长公共前缀和区间最小值查询回答任意后缀/子串关系。
2200–2400	字典序第 k 个不同子串	用后缀数组或后缀自动机统计每个前缀分支的子串数量并恢复答案。
2200–2400	多字符串后缀数组（Generalized Suffix Array）	把多个字符串放在一起比较后缀，求跨字符串最长公共子串等。
2100–2300	后缀自动机的结束位置集合树（Suffix Automaton Endpos Tree）	按后缀链接树汇总出现次数、包含关系和子串贡献。
2000–2300	后缀自动机出现次数与不同子串统计（Suffix Automaton）	线性构建所有子串状态，统计出现次数、不同子串数量和最大贡献。
2300–2500	多字符串后缀自动机（Multiple-String Suffix Automaton）	在多个文本上合并统计每个状态的覆盖串数或共同子串。
2200–2400	后缀自动机求字典序第 k 个子串	在后缀自动机状态有向无环图上做计数并按字符恢复路径。
2200–2400	Aho–Corasick 多模式匹配自动机的失配树（Aho–Corasick Automaton Fail Tree）	将文本经过节点的次数沿失配树汇总到模式串，处理后缀包含关系。
2200–2400	Aho–Corasick 自动机与树状数组	用失配树 DFS 序支持模式串动态开关、子树加和查询。

约分	完整名称（常见写法）	2100 分及以上：作用
2300–2500	动态插入/删除模式串的 Aho–Corasick 自动机	将动态模式集合拆成若干静态自动机，支持在线增删和匹配统计。
2300–2500	二进制分组动态 Aho–Corasick 自动机	用二进制分组控制多个自动机数量，使动态操作达到对数级组数。
1800–2100	哈希二分最长公共前缀 (Hash-Based Longest Common Prefix)	—

20.5 五、计算几何

约分	完整名称（常见写法）	2100 分及以上：作用
1800–2000	极角排序 (Polar-Angle Sorting)	—
1900–2200	射线排序与半平面方向判定 (Ray Sorting and Half-Plane Orientation Test)	用叉积和半平面标记稳定地排序方向、判断左右侧和旋转顺序。
2000–2200	旋转卡壳求凸包直径 (Rotating Calipers for Convex Polygon Diameter)	在凸包上在线性时间内求最远点对或最大距离。
2100–2300	旋转卡壳求最大三角形 (Rotating Calipers for Maximum-Area Triangle)	在凸多边形上优化三点枚举，求最大面积三角形。
1900–2200	凸多边形内点二分 (Binary Point Location in a Convex Polygon)	利用凸性把点内判定从线性扫描降到对数复杂度。
2100–2400	凸多边形半平面切割 (Convex Polygon Cutting)	用一条有向直线保留多边形一侧，连续切出新的凸多边形。
2300–2500	半平面交 (Half-Plane Intersection, HPI)	求多个线性不等式的公共区域及其面积、顶点和空集情况。

约分	完整名称（常见写法）	2100 分及以上：作用
2000–2300	最近点对分治（Closest Pair of Points Divide and Conquer）	在平面点集中求最小欧氏距离，复杂度为 $O(n \log n)$ 。
1900–2200	圆与直线交点（Line-Circle Intersection）	—
2000–2200	两圆交点（Circle-Circle Intersection）	—
1800–2100	点到圆/圆与圆位置关系	—
2200–2500	两圆公切线（Common Tangents of Two Circles）	求两圆的外公切线、内公切线及切点，处理相切和包含边界。
2100–2300	最小圆覆盖（Minimum Enclosing Circle）	随机增量维护包含所有点的最小圆，期望线性时间完成。
2300–2500	多边形与圆面积交（Polygon-Circle Intersection Area）	将多边形边分解为三角形/圆弓，精确累加相交面积。
2000–2300	扫描线求矩形并面积（Sweep Line for Rectangle Union Area）	通过事件排序和线段树维护当前横向覆盖长度。
2200–2400	扫描线求覆盖次数面积（Sweep Line for Area Covered at Least k Times）	在线段树中维护覆盖层数，统计覆盖至少 k 次的面积。
1800–2100	几何去重与方向向量规范化（Geometric Deduplication and Normalized Direction Vectors）	—

20.6 六、动态规划进阶

约分	完整名称（常见写法）	2100 分及以上：作用
2300–2500	复杂数位动态规划（Advanced Digit Dynamic Programming）	处理上界、前导零、数字和、禁止模式等多个数位状态。

约分	完整名称（常见写法）	2100 分及以上：作用
2200–2400	数位动态规划与自动机 (Digit Dynamic Programming with Automaton)	用自动机状态记录模式匹配、相邻限制和数位约束。
1900–2200	带自环方程的概率动态规划 (Probability Dynamic Programming with Self-Loops)	把自环期望方程移项，或建立线性方程组后求解。
2200–2500	概率动态规划与高斯消元 (Probability Dynamic Programming with Gaussian Elimination)	处理多个状态相互转移的期望和概率方程。
1800–2200	子集枚举与所有子集和动态规划 (Subset Enumeration and Sum Over Subsets Dynamic Programming, SOS DP)	—
2100–2400	轮廓线动态规划 (Profile Dynamic Programming)	用一列/几列的状态描述窄网格的铺砖、放置和相邻约束。
1800–2100	单调队列优化动态规划 (Monotone Queue Optimization)	—
2100–2400	斜率优化/凸包技巧 (Convex Hull Trick, CHT)	把二次或交叉项转成直线最值，利用斜率和查询单调性降复杂度。
2200–2400	李超线段树优化动态规划 (Li Chao Segment Tree Optimization)	在线插入斜率不单调的转移直线并查询最优值。
2200–2400	分治优化动态规划 (Divide and Conquer Optimization)	在决策点单调时缩小每层转移枚举范围。
2300–2500	四边形不等式优化 (Quadrangle Inequality Optimization)	证明区间动态规划的决策单调性，从而减少转移枚举。
2300–2500	决策单调性证明 (Decision Monotonicity)	证明最优决策点不向左移动，为分治优化或单调优化提供依据。

约分	完整名称（常见写法）	2100 分及以上：作用
2400–2500	动态动态规划与矩阵线段树	用矩阵乘积维护局部转移，支持单点修改后的全局答案。
2400–2500	生成函数优化计数动态规划	把动态规划转成多项式运算，利用卷积或生成函数提速。
2300–2500	Steiner 树状压动态规划 (Steiner Tree Subset Dynamic Programming)	在关键点数量很少时求连接所有关键点的最小代价。
2100–2400	树形背包优化 (Tree Knapsack Optimization)	合并子树选择数量，处理树上资源分配和选点限制。
2300–2500	长链剖分优化树形动态规划 (Long-Chain Decomposition for Tree DP)	利用长链合并同深度状态，优化树形 DP 的空间或时间。

20.7 七、按题面快速查找

- “区间加/区间乘/区间和”：多重懒标记线段树或仿射标记线段树。
- “离线区间统计”：莫队算法；有修改时用带修改莫队算法；只能撤销时用回滚莫队算法。
- “三维偏序/时间和坐标双重限制”：三维偏序 CDQ 分治。
- “动态维护直线最值”：李超线段树；直线只在一段横坐标有效时用线段插入版李超线段树。
- “区间取最小/最大并查询区间和”：区间最值势能线段树。
- “树上子树颜色频次”：树上启发式合并；“动态树路径”：Link-Cut Tree 动态树。
- “模数不互质的多个同余式”：扩展中国剩余定理。
- “ n 很大但递推阶数很小”：矩阵快速幂与线性递推。
- “所有倍数统计/恰好最大公约数”：莫比乌斯反演。
- “ $a^x \equiv b \pmod p$ ”：Baby-step Giant-step 离散对数算法；底数与模数不互质时用扩展版本。
- “多项式卷积”：数论变换；“对称染色”：Burnside 引理或 Pólya 计数定理。
- “ $x^2 \equiv a \pmod p$ ”：Legendre 符号先判定，Tonelli-Shanks 或 Cipolla 算法求根。
- “不同子串/出现次数”：后缀自动机；“回文后缀链”：回文自动机；“多个模式串”：Aho-Corasick 多模式匹配自动机。
- “凸包最远点”：旋转卡壳；“多个线性不等式公共区域”：半平面交；“矩形并面积”：扫描线与线段树。
- “胜负态”：必胜态/必败态动态规划；“无偏且可拆分”：Sprague-Grundy 函数；“双方完整决策”：极大极小搜索与 Alpha-Beta 剪枝。

20.8 八、常见算法组合

1. 离散化 + 树状数组：排名、逆序、区间计数。
2. DFS 序 + 线段树：树上子树修改/查询。
3. 最近公共祖先 + 树上差分：大量路径经过次数。
4. 强连通分量缩点 + 有向无环图动态规划：有向图上的可达/最大权。
5. 二分答案 + 广度优先搜索/贪心/网络流：判定型最优化。
6. Kruskal 重构树 + 倍增：按边权阈值查询连通块信息。
7. Aho-Corasick 自动机 + 动态规划：避免模式或统计匹配串。
8. 可持久化线段树 + 最近公共祖先：树上路径第 k 小。
9. 扫描线 + 线段树 + 坐标压缩：矩形并、覆盖次数。
10. 回滚并查集 + 线段树时间分治：离线动态连通性。

错题本

WA / RE / TLE 排查速查手册

打印用 • 比赛翻阅 • 提交前过一遍

2026 年 7 月 23 日

目录

第一章 板子	5
第二章 提交前速查	6
第三章 通用 WA 错误	7
3.1 溢出	7
3.2 多测	7
3.3 取模	7
3.4 边界	7
3.5 输出格式	8
3.6 杂项	8
3.7 15 分钟还没找到 WA	8
第四章 错题复盘与最小反例	9
4.1 一次 WA 的完整复盘流程	9
4.2 多测与 <code>void solve()</code> : 最容易被忽略的真实错误	9
4.2.1 中途 <code>return</code> 会把下一组输入读乱	10
4.2.2 全局容器和节点池必须按题目边界清理	10
4.2.3 多测回归样例的固定格式	11
4.3 边界样例设计 SOP	11
4.3.1 先列“变化维度”，再写数据	11
4.3.2 每个大型板子至少四组核心测试	12
4.4 最小反例库：按错误类型直接查	12
4.4.1 下标与半开区间	12
4.4.2 前缀和与差分	12

4.4.3	图论：不可达、重边、自环	13
4.4.4	最短路：旧堆项和不可达点	13
4.4.5	动态规划：全负、空转移和答案位置	13
4.4.6	线段树：整段标记后查询子区间	13
4.4.7	字符串：重叠和周期	14
4.4.8	计算几何：相切、共线和重复点	14
4.5	错误代码与修复代码对照	14
4.5.1	负数取模	14
4.5.2	区间最小值的错误懒标记	14
4.5.3	LCA 根节点初始化	15
4.6	可直接改名使用的对拍框架	15
4.7	错题卡：以后每道题只记录一页	16
4.8	提交前五分钟固定动作	16

第五章 通用 RE 错误 17

5.1	RE 排查流程	17
5.2	数组越界	17
5.3	栈溢出（递归过深）	18
5.4	除以零 / 取模零	18
5.5	非法内存访问	18
5.6	STL 误用	18
5.7	整数未定义行为	19
5.8	常见 RE 清单速查	19

第六章 图论与树 20

6.1	建图	20
6.2	特殊图	20
6.3	DFS / BFS	20
6.4	最短路	20
6.5	拓扑排序	21
6.6	LCA / 树形 DP	21
6.7	图论模型先判型	21

6.8	连通块、0-1 BFS 与最小生成树	21
6.8.1	连通块：外层循环不能漏	21
6.8.2	0-1 BFS：权值必须真的只有 0 和 1	22
6.8.3	Kruskal：最小生成树的三个返回值	23
6.8.4	强连通分量、割点和桥：最容易混淆的地方	24
6.9	图论回归样例：四组就能抓住大多数板子错误	24
第七章	动态规划	25
7.1	初始化	25
7.2	遍历顺序	25
7.3	转移遗漏	25
7.4	边界	25
7.5	先写状态三句话，再写循环	26
7.6	遍历顺序：是否会偷偷重复使用	26
7.7	不可达状态：不能把 INF 当成普通数字	27
7.8	滚动数组：清空的是“当前层”	27
7.9	答案位置、路径恢复与计数取模	27
7.10	DP 最小回归样例	28
第八章	数据结构	29
8.1	并查集	29
8.2	线段树	29
8.3	二分	29
8.4	双指针 / 滑窗	29
8.5	树状数组：下标、前缀和与差分	30
8.5.1	区间加、单点查：把差分藏进树状数组	30
8.6	单调栈与单调队列	30
8.6.1	下一个更大元素：栈内保存什么	30
8.6.2	滑动窗口最小值：队首才是答案	31
8.7	离散化与区间结构	31
8.8	数据结构回归样例	32

第九章 博弈论	33
9.1 必胜态与必败态	33
9.2 Bash 游戏：先找模周期	33
9.3 普通 Nim：异或和为零就是必败	34
9.4 Sprague-Grundy 函数与 mex	34
9.5 反常 Nim、阶梯 Nim 与常见变形	35
9.6 Minimax 极大极小搜索与 Alpha-Beta 剪枝	35
9.7 博弈题的现场检查顺序	36
9.8 博弈回归样例	36
第十章 数学与精度	38
10.1 取模（再强调）	38
10.2 组合计数	38
10.3 数论	38
10.4 浮点	38
第十一章 计算几何	39
11.1 精度	39
11.2 向量基础	39
11.3 线段与直线	39
11.4 凸包	40
11.5 多边形	40
11.6 圆	40
11.7 旋转卡壳	40
11.8 半平面交	40
11.9 常见坑总结	41
第十二章 读题错误	42
12.1 数值范围	42
12.2 核心对象	42
12.3 操作规则	42
12.4 坐标与格式	42

1 板子

比赛开局直接粘贴的模板。注意 `#define int long long` 之后 `main` 必须写 `signed main`。

```
#include <bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

const int inf = 2e18;
const int N = 1e6 + 10;

void solve() {
    int n;
    cin >> n;
}

signed main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int t = 1;
    cin >> t;
    while (t--) {
        solve();
    }
    return 0;
}
```

2 提交前速查

大多数 WA 不是算法错了，是细节把正确思路拖下水。提交前花 30 秒过一遍这页。

1. 有没有 `1 << i` 没写成 `1LL << i`?
2. `inf + x` 会不会爆？转移前加 `if(dist[u] != inf)`。
3. 两个大数相乘有没有步步取模？
4. 多测的数组/图清空了吗？`solve()` 里有没有提前 `return` 导致输入没读完？
5. 减法取模补 `+ MOD` 了吗？除法用逆元了吗？
6. $n = 0, 1, 2$ 、全相同、全负数、无解、不连通测过了吗？
7. 输出大小写对吗？下标是 0-based 还是 1-based？行末空格？

3 通用 WA 错误

3.1 溢出

- $1 \ll i$: 常数 1 是 32 位, $i \geq 31$ 就炸。写 $1LL \ll i$ 。
- $\text{inf} + x$: $2 \times 10^{18} + x$ 爆 long long 变负数。松弛/转移前判不可达。
- 大数乘法: $10^{18} \times 10^{18}$ 直接爆。用 `__int128` 或步步取模。
- $\text{pow}(5,2)$ 可能返回 24.999..., 强转 int 变 24。整数幂用快速幂。

3.2 多测

- 串流: `solve()` 里提前 return, 这组数据没读完, 后面全乱。
- 残留: 上一组 $n = 10$, 这组 $n = 5$, 只清 1 ~ 5 留下 6 ~ 10 的脏数据。
- 重名: 全局 `int n;` 和局部 `int n;` 同时存在, DFS 读到的是全局的 0。

3.3 取模

- 减法: $(a - b \% \text{MOD} + \text{MOD}) \% \text{MOD}$, 防负数。
- 除法: 不能直接除, 要乘逆元 $a * \text{qpow}(b, \text{MOD}-2) \% \text{MOD}$ 。
- 连乘: $A * B \% \text{MOD} * C \% \text{MOD}$, 步步取模防中间爆。

3.4 边界

- $n = 0, 1, 2$ 特判。
- 全相同、全为 10^9 、全为负数。

- 图：不连通、链、菊花、自环、重边。
- 无解时输出什么？-1? 0? Impossible?

3.5 输出格式

- YES/Yes/yes 大小写。
- 下标 +1 还是不 +1。
- 行末空格、最后换行。
- 多解时要不要字典序最小。

3.6 杂项

- 贪心假了：无法证明就怀疑，考虑 DP / 二分答案。
- 比较器：sort 的 cmp 用了 $\leq$ 会破坏严格弱序，必须用 $<$ 。
- 未初始化：本地对但提交 WA，检查局部变量有没有赋初值。
- 哈希被卡：unordered_map 在 CF 上容易被 hack，加 custom hash 或换 map。

3.7 15 分钟还没找到 WA

1. 停止看代码。手造小样例（含负数、0、1、乱序）。
2. 纸上跑一遍算法。
3. 写暴力（ $O(n^3)$ 也行），写随机数据生成器，对拍。
4. 找到最小错误数据，定位第一处不一致。

4 错题复盘与最小反例

这一章不是再罗列“某某算法会错”，而是把一次错误真正变成下一次不会再犯的检查项。每道题至少留下四样东西：

1. 最小错误输入：尽量短，但必须仍然能复现错误；
2. 错误原因：指出哪一个不变量、边界或题意被破坏；
3. 修复后的关键代码：只保留真正改变结果的几行；
4. 回归测试：以后每次改模板都重新运行，防止旧错误回来。

4.1 一次 WA 的完整复盘流程

1. 先确认输入输出没有错：样例是否完整复制，是否漏读了 T 、多组边、空行或字符串；输出大小写、空格和换行是否符合题面。
2. 再缩小数据：删除无关点、边、操作，优先保留第一个产生错误的操作。一个能在 4 个数上复现的错误，不要拿 4 万个数解释。
3. 写出不变量：例如堆顶是当前最短距离、线段树节点的和对应整个区间、动态规划状态已经处理完前缀、回滚前后的历史栈长度相同。
4. 对照错误发生前后：在第一次不符合不变量的位置打印变量，而不是只打印最终答案。
5. 写进回归样例：把这个最小输入和正确输出放入模板旁边，下一次修改代码时必须重新跑。

不要一看到 WA 就重写整份代码。先找到“第一个错误状态”，否则新代码即使过了当前样例，也不知道究竟修复了什么。

4.2 多测与 `void solve()`：最容易被忽略的真实错误

4.2.1 中途 return 会把下一组输入读乱

下面的代码在无解时直接返回，当前测试剩余的边仍在输入流中，下一组的 n, m 就会读成上一组的边：

```
void solve() {
    int n, m;
    cin >> n >> m;
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        if (u == v) return; // 错：剩余输入没有读完
    }
    cout << "YES\n";
}
```

如果输入是：

```
2
3 2
1 1
1 2
1 0
```

第一组提前返回后，第二组的 n, m 可能被读成 1, 2，错误会看起来像“第二组算法随机坏了”。正确写法是读完当前组，再用标志记录非法状态：

```
void solve() {
    int n, m;
    cin >> n >> m;
    bool ok = true;
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        if (u == v) ok = false;
    }
    cout << (ok ? "YES" : "NO") << '\n';
}
```

4.2.2 全局容器和节点池必须按题目边界清理

多测最小测试：第一组规模大，第二组规模小。它可以专门暴露“只清理前 n 个位置”或“节点编号没有归零”：

```
2
5
```



```
1 2 3 4 5
1
9
```

如果第二组的答案仍然包含第一组的数据，优先检查：

- `vector` 是否调用了 `clear()` 或重新赋值；
- 图的边表、`head`、`edge_count` 是否一起清空；
- 字典树、主席树、回文自动机的 `tot` 是否恢复到根节点之后；
- 线段树的 `sum`、`lazy`、`tag` 是否全部重新 `build`；
- 时间戳数组是否使用了新的 `stamp`，而不是只清空一半下标。

4.2.3 多测回归样例的固定格式

以后给任何板子补样例时，优先使用下面的四组结构：

1. $n = 1$ 或空结构：检查根、叶子、空集合和初始化；
2. 退化结构：链、星、全相同、全负、无边或单条边；
3. 正常结构：至少有两条竞争路径、两种状态或两个不同区间；
4. 两组连续测试：第一组故意较大，第二组很小，检查状态污染。

4.3 边界样例设计 SOP

4.3.1 先列“变化维度”，再写数据

不要随机生成一堆看起来很大的数。先列出模板真正依赖的变化维度：

- 数组：长度 0, 1, 2、重复值、负数、最大值、单调和逆序；
- 区间：单点、整段、相邻、不相交、完全包含、端点重合；
- 图：无边、链、星、环、重边、自环、不连通、负边；
- 树：单点、链、星、平衡树、根是答案、根不在查询路径；
- 字符串：空串、单字符、全相同、周期串、重叠匹配、无匹配；
- 几何：重合、相切、两点交、平行、共线、退化多边形；
- 多测：大组后小组、空组后正常组、节点池复用。

4.3.2 每个大型板子至少四组核心测试

建议每个超过十行的模板都保留以下四组，而不是只保留随机样例：

类型	目标	典型输入
边界一	空结构或最小规模	$n = 0, 1$ 、单点、空边集
边界二	退化或等号分支	全相同、相切、重复边、 $x = \text{second_max}$
正常一	核心转移真的发生	两条竞争路径、重叠区间、至少一次旋转/合并
正常二	多阶段组合	先修改再查询、先连边再断边、多个版本交叉查询

如果板子存在随机化或浮点误差，再增加一组“同答案不同顺序”的测试：输入随机打乱后，答案允许顺序不同，但数值和不变量必须相同。

4.4 最小反例库：按错误类型直接查

4.4.1 下标与半开区间

所有只处理 $[l, r)$ 的代码都要测试 $l=0, r=1$ 、 $l=0, r=n$ 和相邻区间：

```

数组：      [10, 20, 30]
区间一：    [0, 1)  只包含 10
区间二：    [1, 3)  包含 20, 30
区间三：    [0, 3)  包含全部元素
区间四：    [1, 2)  只包含 20
  
```

如果区间三答案少一个元素，通常是把右端点当成闭区间；如果区间一访问到 $a[1]$ ，通常是把右端点错误地包含进来了。

4.4.2 前缀和与差分

对数组 $[1, 2, 3]$ 依次执行 $\text{add}(1, 1, 5)$ 、 $\text{add}(2, 3, -2)$ ，最终应为 $[6, 0, 1]$ 。这个样例同时检查：

- 单点更新是否正确；
- 负数更新是否正确；
- $r+1$ 是否写在正确位置；
- 查询 $[1, 3]$ 是否为 7，而不是只返回最后一次更新。

4.4.3 图论：不可达、重边、自环

```
4
1 0
2 1 1 2
2 2 1 2 1 2
3 3 1 1 1 2 2 3
```

四组分别测试单点、单边、重边和自环。不要默认题目没有自环；如果算法是桥、割点、强连通分量或拓扑排序，自环的处理完全不同：自环会让有向图出现环，但不会把无向单点自动变成桥。

4.4.4 最短路：旧堆项和不可达点

```
3 3
1 2 10
1 3 1
3 2 1
```

从 1 到 2 的答案是 2。Dijkstra 会先把 2 以 10 放进堆，再通过 3 改成 2；弹出旧的 (10,2) 时必须跳过。再增加一个没有任何入边的点，检查不可达点是否保持 `INF`，而不是参与 `INF+w` 的溢出运算。

4.4.5 动态规划：全负、空转移和答案位置

最大子段和的回归样例：

```
4
1
-5
3
1 -2 1
4
-3 -2 -5 -1
5
2 -1 2 -1 2
```

答案分别是 -5,1,-1,4。如果把 `dp` 初值写成 0，第一组就会错误；如果只在转移时更新答案而不检查最后一个位置，第四组也容易错。

4.4.6 线段树：整段标记后查询子区间

初始数组 [1,1,1,1]，先对 [1,4] 加 5，再查询 [2,3]，答案必须为 12；随后对 [2,2] 加 -3，再查整段，答案为 18。这个顺序专门验证 `push_down`：只查整段时懒标记可能一直藏着，只有访

问子区间才会暴露错误。

4.4.7 字符串：重叠和周期

文本 abababa、模式 aba 的匹配起点为 0,2,4，总数为 3；文本 aaaaaa、模式 aaa 的总数也为 3。匹配成功后不能把模式指针直接归零，必须回到前缀函数给出的最长可复用前缀。

4.4.8 计算几何：相切、共线和重复点

- 两个半径为 1 的圆，圆心距 2：一个交点；圆心距 1：两个交点；圆心距 3：无交点。
- 三点 (0,0),(1,0),(3,0)：外接圆应退化成最远两点的直径圆，不能除以零。
- 最近点对中加入两个相同点：答案必须为 0，不能用一个正的初始答案掩盖重复点。

4.5 错误代码与修复代码对照

4.5.1 负数取模

```
// 错误：C++ 的负数取模结果仍可能为负
long long add = (a - b) % MOD;

// 修复：先把减数归一化，再整体加模
long long add = (a - b % MOD + MOD) % MOD;
```

回归输入：a=3,b=5,MOD=7，答案应为 5，而不是 C++ 直接取模得到的 -2。

4.5.2 区间最小值的错误懒标记

```
// 错误：把 chmin 当成加法，直接给整个区间加 tag
tree[p].mx = min(tree[p].mx, x);
tree[p].sum += x * len; // 完全错误

// 修复思路：只有当前最大值会下降，
// 必须维护最大值、次大值、最大值出现次数和区间和。
if (second_mx[p] < x && x < mx[p]) {
    sum[p] -= (mx[p] - x) * cnt_mx[p];
    mx[p] = x;
}
```

回归数组 [1,3,3,7]：执行整段 chmin(5) 后和为 12，再执行 [2,3] chmin(2) 后该区间和为 4。第二次操作不能只看根节点最大值，必须按次大值条件决定是否继续递归。

4.5.3 LCA 根节点初始化

```
// 错误：根的父亲没有明确设为 0，倍增表含有旧数据
dfs(root, parent[root]);

// 修复：每组数据先固定根和所有根祖先
parent[root] = 0;
depth[root] = 0;
for (int j = 0; j < LOG; ++j) up[root][j] = 0;
dfs(root, 0);
```

回归树：单点、链 1-2-3-4、星 1-{2,3,4}，分别查询 $\text{lca}(1,1)$ 、 $\text{lca}(2,4)$ 、 $\text{lca}(3,4)$ 。根表没有清空时，单点组往往会错误继承上一组根的祖先。

4.6 可直接改名使用的对拍框架

下面的框架不替代暴力解，它只负责反复生成小数据、运行标准程序和暴力程序，并在第一次不一致时停下。把三个可执行文件名替换成自己的文件即可：

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
    for (int tc = 1; tc <= 100000; ++tc) {
        // generator.exe 负责把一组随机数据写入 input.txt
        system("generator.exe > input.txt");
        system("std.exe < input.txt > std.out");
        system("brute.exe < input.txt > brute.out");
        ifstream a("std.out"), b("brute.out");
        string x, y;
        bool same = true;
        while (a >> x || b >> y) {
            if (x != y) { same = false; break; }
            x.clear(); y.clear();
        }
        if (!same) {
            cerr << "Mismatch on test " << tc << "\\n";
            cerr << "Input is saved in input.txt\\n";
            return 0;
        }
    }
    cerr << "All tests passed\\n";
}
```

对拍生成器不要一开始就追求大数据。先覆盖： $n = 1$ 、全相同、完全逆序、只有一条边、重复边、无解、答案在端点、随机小数据。暴力程序必须比标准程序更简单，即使慢到 $O(n^3)$ 也可以；对拍的目标是正确，不是速度。

4.7 错题卡：以后每道题只记录一页

字段	记录内容
题目与提交时间	题号、语言、提交结果、使用的模板
最小错误输入	删除哪些元素后仍然能复现
错误类别	题意、边界、下标、溢出、状态、清空、复杂度、输出
被破坏的不变量	例如“当前区间和应等于四个子区间之和”
错误代码	只贴最小的 3-10 行，不要整页复制
修复动作	改了哪一个条件、初始化或数据结构
回归样例	输入、正确输出、以后什么时候重跑

真正有价值的错题记录不是“这题用了线段树”，而是“我把整段懒标记下传前直接访问了子区间；最小反例是先改全区间再查中间一格”。后者可以迁移到下一道题，前者只能记住一道题。

4.8 提交前五分钟固定动作

1. 用题面约束重新确认 `int`、`long long`、`__int128` 和浮点类型；
2. 手算一组 $n = 1$ ，一组重复值/重边，一组无解或不可达；
3. 检查 `solve()` 是否完整读完当前测试，所有全局状态是否清空；
4. 检查所有区间端点是闭区间还是半开区间，下标是 0-based 还是 1-based；
5. 检查输出顺序、大小写、精度和无解格式；
6. 如果仍然不放心，立刻写暴力和对拍，不要继续凭感觉改条件。

5 通用 RE 错误

RE 不像 WA 有输出可以对比——程序直接崩了，没有任何线索。排查思路：先定位崩在哪一行，再判断哪类原因。

5.1 RE 排查流程

1. 本地复现：把样例贴进来跑，看能不能在本地炸。
2. 加输出定位：在关键位置加 `cerr << "checkpoint 1" << endl;`，逐步缩小崩溃区间。
3. 极端数据： $n = 0$ 、 $n = 1$ 、 $n = N_{\max}$ 、全相同、全极值，单独测。
4. 开编译选项：本地编译加 `-fsanitize=address,undefined -g`，能直接报出越界和 UB 的行号。
5. 对照常见原因清单：逐条过下面的列表。

5.2 数组越界

- 开小了： $N = 10^5 + 10$ 但输入 n 可以到 2×10^5 。仔细看数据范围再开数组。
- 下标为负：`a[i - k]` 当 $i < k$ 时变负数。用前先判 `if(i >= k)`。
- 下标为 0：1-based 存图但 `head[0]` 没初始化；或反过来 0-based 但循环从 1 开始漏了 `a[0]`。
- 邻接表边数：无向图每条边存两次，边数组至少开 $2M$ 。
- 线段树：节点数组至少 $4N$ （不是 $2N$ ）。
- 组合数/阶乘表：预处理到 N 但查询 $C(n, m)$ 时 n 可能到 $2N$ 。

5.3 栈溢出（递归过深）

- 链状图 DFS: $n = 10^5$ 的链递归深度 10^5 层，默认栈（1 MB / 8 MB）直接爆。
- 解决方案：
 - 改用 BFS / 手写栈模拟 DFS。
 - Linux 下加 `ulimit -s unlimited`（某些 OJ 支持）。
 - Windows 下编译加 `-Wl,--stack,268435456`（256 MB 栈）。
- 无限递归：忘记写 `return` / base case，或者 DFS 忘标 `vis[u] = true`，同一个节点反复进入。
- 树形 DP 忘跳父亲：`dfs(v, u)` 里没写 `if(v == fa) continue`，父子互相递归。

5.4 除以零 / 取模零

- a / b 或 $a \% b$ 当 $b = 0$ 时直接 RE。
- 常见场景：二分 $mid = (l + r) / 2$ 本身不会为零，但 n / mid 会在 $mid == 0$ 时炸。
- GCD 循环里 $a \% b$ ，如果 b 初始就是 0 则第一轮就崩。
- 解法：所有除法/取模前加 `if(b == 0)` 特判，或保证逻辑上 $b \neq 0$ 。

5.5 非法内存访问

- 空指针：链表/树结构访问 `nullptr->val`。
- 野指针：`delete` 后继续使用。竞赛中少用动态内存，优先用静态数组 + 内存池。
- VLA 过大：`int a[n]` 在栈上分配， $n = 10^6$ 就可能爆栈。改用全局数组或 `vector`。

5.6 STL 误用

- 空容器取元素：
 - `v.back()` / `v.front()` 在 `v.empty()` 时 UB/RE。
 - `s.top()` / `q.front()` 在空栈/空队列时崩。
 - 用前必须判 `!v.empty()`。
- 迭代器失效：

- vector 的 `push_back` 可能导致扩容，所有迭代器和指针失效。
- set/map 的 `erase` 后原迭代器失效，用 `it = s.erase(it)` 或先 `next(it)`。
- 越界访问：`v[i]` 不做边界检查（`v.at(i)` 会抛异常）。
- 比较器不满足严格弱序：`sort` 的 `cmp` 用 `<=` 会导致内部断言失败 → RE（不只是 WA!）。

5.7 整数未定义行为

- 有符号溢出：C++ 中有符号整数溢出是 UB，编译器可能生成崩溃代码。
- 移位越界：`1 << 31` 对 32 位 int 是 UB（符号位）；`1 << 64` 对 64 位也是 UB。
- 负数移位：`(-1) << k` 和 `x >> 负数` 都是 UB。
- 开 `-fsanitize=undefined` 可以在本地立刻暴露这些问题。

5.8 常见 RE 清单速查

提交 RE 后，按此表逐条排查：

1. 数组大小够吗？下标有没有负数或超上界？
2. 递归深度超过 10^4 了吗？有没有无限递归？
3. 有除法或取模吗？除数可能为 0 吗？
4. 有 `sort` 自定义比较器吗？是否满足严格弱序（`<` 而非 `<=`）？
5. 用了 `stack/queue/vector` 的 `top()/front()/back()` 吗？容器可能为空吗？
6. 用了 `1 << k` 吗？ $k \geq 31$ 时要写 `1LL << k`。
7. 有 VLA（`int a[n]`）吗？ n 大时改用全局数组。
8. 线段树开了 $4N$ 吗？并查集/倍增表第二维开够了吗？
9. 多测时上一组残留的大数组会不会越界访问？
10. 本地加 `-fsanitize=address,undefined -g` 编译，跑一遍样例。

RE 和 WA 的本质区别：WA 是逻辑错误，RE 是访问了不该访问的东西。绝大多数 RE 都是数组越界和栈溢出，优先查这两个。

6 图论与树

6.1 建图

- 无向图边数开 $2M$ （正反各一条）。
- 点属性开 N ，边属性开 M ，别搞混。
- 多测清空要按上一组的范围清，链式前向星 `cnt = 0`。
- 编号 0-based 还是 1-based，循环和起点要一致。

6.2 特殊图

- 不连通：没保证连通就外层 `for(i=1;i<=n;i++) if(!vis[i]) dfs(i)`。
- 重边：邻接矩阵 `g[u][v] = min(g[u][v], w)`。
- 自环：影响入度、度数、连通性。必要时 `if(u==v) continue`。
- 链：递归深搜会爆栈。菊花：暴力会退化成 $O(n^2)$ 。

6.3 DFS / BFS

- 无向树 DFS 传父亲：`dfs(u, fa)`，跳过 `if(v==fa) continue`。
- BFS 入队时立刻标 `vis[v]=true`，不要等出队。
- 找所有路径要回溯：离开前 `vis[u]=0`。

6.4 最短路

- Dijkstra 堆优化：出队判 `if(vis[u]) continue`。
- 有负权 $\Rightarrow$ Dijkstra 失效，用 Bellman-Ford。
- Floyd：初始化 `inf`，对角线 0，循环顺序 k, i, j 。
- 松弛防溢出：`if(dist[u]!=inf && dist[v]>dist[u]+w)`。

6.5 拓扑排序

- 跑完检查 `cnt==n`，不等说明有环。
- 要字典序最小 $\Rightarrow$ 用小根堆代替普通队列。

6.6 LCA / 树形 DP

- 倍增第二维开 $\lceil \log_2 N \rceil$ ，预处理外层枚举步长、内层枚举节点。
- 树形 DP：先 `dfs(v)` 再用 `dp[v]` 更新 `dp[u]`（后序）。
- 多测时在 `dfs(u)` 开头清当前点的 DP 状态。

图论一直 WA/RE 时：在测试数据里加孤立点、自环、重边、退化成链/菊花，很多隐藏 bug 会立刻暴露。

6.7 图论模型先判型

拿到图论题先不要急着敲模板，先把题面翻译成下面四个问题：

1. 边有没有方向？无向边必须加入两次；有向边不能反向乱加。
2. 权值范围是什么？只有 0/1 用 0-1 BFS，非负任意权值用 Dijkstra，有负权考虑 Bellman-Ford 或差分约束。
3. 题目问的是一条路径、所有点的可达性、连通块，还是删点删边之后的结构？对应的算法完全不同。
4. 图是否保证连通、无环、没有重边？题面没有保证的性质都不能默认。

最容易发生的误判是“看到图就写 DFS”。DFS 只能完成遍历，不能自动解决最短路、拓扑序、强连通分量或割点问题。先写出状态含义：例如 `dist[u]` 是源点到 u 的最短距离，`vis[u]` 只是“是否已经发现”，两者不是一回事。

6.8 连通块、0-1 BFS 与最小生成树

6.8.1 连通块：外层循环不能漏

只从 1 号点 DFS 只能得到 1 号点所在的连通块。要统计整个无向图的连通块，必须对所有点补一层外循环：

```

vector<vector<int>>> g;
vector<int> vis;

void dfs(int u) {
    vis[u] = 1;                // 先标记，避免无向边来回递归
    for (int v : g[u]) {
        if (!vis[v]) dfs(v);
    }
}

int count_components(int n) {
    int components = 0;
    for (int u = 1; u <= n; ++u) {
        if (!vis[u]) {
            ++components;
            dfs(u);            // 每次启动 DFS 就找到一个新连通块
        }
    }
    return components;
}

```

无向图的 DFS 如果只写 `if (v == fa) continue`，遇到重边时仍可能出错：两条平行边中只有一条是“父边”，另一条不能被跳过。需要区分边编号时传 `parent_edge`，不要只传父节点。

6.8.2 0-1 BFS：权值必须真的只有 0 和 1

边权为 0 或 1 时，双端队列可以代替优先队列。松弛后，走 0 边放到队首，走 1 边放到队尾。它的关键不在“用了 deque”，而在于队列顺序保证较小距离优先被处理：

```

struct Edge { int to, w; };      // w 必须是 0 或 1
const long long INF = (1LL << 60);

vector<long long> zero_one_bfs(int s, const vector<vector<Edge>>& g) {
    int n = (int)g.size() - 1;
    vector<long long> dist(n + 1, INF);
    deque<int> q;
    dist[s] = 0;
    q.push_front(s);

    while (!q.empty()) {
        int u = q.front();

```

```

    q.pop_front();
    for (auto e : g[u]) {
        if (dist[e.to] > dist[u] + e.w) {
            dist[e.to] = dist[u] + e.w;
            if (e.w == 0) q.push_front(e.to);
            else q.push_back(e.to);
        }
    }
}
return dist;
}

```

检查方式：把一条 0 边和一条 1 边交错放进图里，确认算法不会把“边数最少”误当成“权值和最小”；再故意放入权值 2，确认自己知道这时必须换 Dijkstra。

6.8.3 Kruskal: 最小生成树的三个返回值

Kruskal 不只要返回总权值，还经常要返回选了多少条边。若最后选边数不是 $n - 1$ ，图不连通，不能把当前总和当作一棵生成树：

```

struct DSU {
    vector<int> fa, sz;
    DSU(int n) : fa(n + 1), sz(n + 1, 1) {
        iota(fa.begin(), fa.end(), 0);
    }
    int find(int x) { return fa[x] == x ? x : fa[x] = find(fa[x]); }
    bool unite(int x, int y) {
        x = find(x); y = find(y);
        if (x == y) return false; // 会形成环，不能选
        if (sz[x] < sz[y]) swap(x, y);
        fa[y] = x; sz[x] += sz[y];
        return true;
    }
};

long long kruskal(int n, vector<tuple<int, int, int>> edges) {
    sort(edges.begin(), edges.end(),
        [](auto a, auto b) { return get<2>(a) < get<2>(b); });
    DSU dsu(n);
    long long answer = 0;
    int used = 0;
    for (auto [u, v, w] : edges) {
        if (dsu.unite(u, v)) {
            answer += w;
        }
    }
}

```

```
        ++used;
    }
}
return used == n - 1 ? answer : -1; // -1 代表不存在生成树
}
```

6.8.4 强连通分量、割点和桥：最容易混淆的地方

- 强连通分量只用于有向图；割点和桥通常讨论无向图。看到 `low` 不要直接套同一份 Tarjan 代码。
- 无向图 Tarjan 处理重边时要跳过“父边编号”，不能写成只跳过 `v == fa`。
- 桥判断是 `low[v] > dfn[u]`；割点的根节点需要单独判断 DFS 子树数是否至少为 2。
- 有向图拓扑排序的结果数量小于 n ，说明有环；这不是“少访问了几个点”，而是题目约束与 DAG 假设冲突。

6.9 图论回归样例：四组就能抓住大多数板子错误

1. 单点图： $n = 1, m = 0$ 。检查初始化、答案是否为 0、LCA 是否允许 `lca(1,1)`。
2. 不连通图： $n = 5$ ，边为 $(1,2), (2,3), (4,5)$ 。连通块应为 2，点 1 到点 5 应不可达。
3. 重边和自环：边为 $(1,2,7), (1,2,3), (2,2,0)$ 。最短路应选权值 3，拓扑排序不能把自环当成正常边。
4. 链和菊花：一组是 $1 - 2 - 3 - \dots - n$ ，另一组是 1 连接所有其他点。分别检查递归深度和邻接表遍历是否退化。

图论调试时建议同时打印“点数、边数、可达点数、选中边数、拓扑结果长度”。这些量一旦和理论值不符，比只打印最终答案更容易定位是建图、遍历还是转移错了。

7 动态规划

7.1 初始化

- 求最大值 $\Rightarrow$ 初始化 $-\infty$ (不是 0, 收益可能为负)。
- 求最小值 $\Rightarrow$ 初始化 $+\infty$ 。
- 不可达状态 ($dp[j]==inf$) 不要参与转移, 否则 $inf+cost$ 溢出。
- ”前 i 个的最优” vs ”以第 i 个结尾的最优”: 定义不同, 答案位置不同。

7.2 遍历顺序

- 区间 DP: 外层枚举长度 len , 内层枚举左端点 L , 推出 R 。
- 0-1 背包: 容量倒序。完全背包: 容量正序。
- 树形 DP: 先递归儿子, 再转移父亲。

7.3 转移遗漏

- LIS 每个元素可以单独成为长度 1, 别忘初始化 $dp[i]=1$ 。
- 计数 DP 有多个否定条件 $\Rightarrow$ 容斥, 注意别重复减。
- 滚动数组: 每轮开头清空当前层, 注意 $+=$ 和 $=$ 的区别。

7.4 边界

- 转移含 $dp[i-2] \Rightarrow$ 循环从 $i=2$ 或 $i=3$ 开始, 手写 base case。
- 背包: $j - weight[i]$ 必须 ≥ 0 。
- 答案位置: 状态是”以 i 结尾” $\Rightarrow$ 答案是 $\max(dp[1..n])$ 。

7.5 先写状态三句话，再写循环

DP 最常见的错误不是代码语法，而是状态含义在代码中途偷偷变了。开写前强制写下三句话：

1. **状态：** $dp[i][j]$ 精确表示什么，已经处理了哪些元素， i, j 是否包含当前位置。
2. **转移：** 最后一步是什么，从哪些更小的状态转移过来，是否会重复使用同一个物品或元素。
3. **答案：** 是 $dp[n][m]$ 、某一行的最大值，还是所有结尾状态的最大值。

例如“前 i 个数的最长上升子序列”和“以第 i 个数结尾的最长上升子序列”不是同一个状态。前者答案通常看 $dp[n]$ ，后者必须看 $\max(dp[1..n])$ ；只要把这两个答案位置混用，就会在答案不以最后一个元素结尾的样例上出错。

写完转移后，用 $n = 0, 1, 2$ 手算一遍。若连空前缀、单元素和不可能状态都说不清楚，继续写大模板只会把错误藏得更深。

7.6 遍历顺序：是否会偷偷重复使用

背包的循环方向不是格式问题，而是在决定一个物品能否在同一轮被重复使用：

```
// 0-1 背包：每个物品最多使用一次，容量必须倒序
for (int i = 1; i <= n; ++i) {
    for (int j = capacity; j >= weight[i]; --j) {
        dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
    }
}

// 完全背包：每个物品可以使用多次，容量必须正序
for (int i = 1; i <= n; ++i) {
    for (int j = weight[i]; j <= capacity; ++j) {
        dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);
    }
}
```

区间 DP 通常按区间长度从小到大；树形 DP 通常先算儿子再算父亲；依赖前一层的滚动数组必须先确认当前层是否还会读取旧值。一个非常有效的反例是：只有一个物品、重量刚好能放两次的容量。如果 0-1 背包输出了使用两次的价值，循环方向就错了。

7.7 不可达状态：不能把 `INF` 当成普通数字

最小值 DP 用正无穷初始化，最大值 DP 用负无穷初始化，但“无穷”只是标记，不是可以参与加法的真实数：

```
const long long INF = (1LL << 60);
vector<long long> dp(n + 1, INF);
dp[0] = 0;

for (int i = 1; i <= n; ++i) {
    if (dp[i - 1] == INF) continue; // 不可达状态不能继续转移
    dp[i] = min(dp[i], dp[i - 1] + cost[i]);
}
```

如果把不可达状态写成 0，求最小值时会错误地让它成为最优；如果直接计算 `INF + cost`，可能溢出成负数，之后的 `min` 会把这个负数当成答案。最大值 DP 同理，不能让 `-INF + value` 参与转移。

7.8 滚动数组：清空的是“当前层”

二维 DP 压成一维后，旧状态和新状态共用同一块内存。最常见的错误是上一轮的值残留，或者把本轮需要的 `+=` 写成覆盖：

```
vector<long long> old(m + 1), now(m + 1);
for (int layer = 1; layer <= n; ++layer) {
    fill(now.begin(), now.end(), NEG_INF); // 每轮先清当前层
    for (int j = 0; j <= m; ++j) {
        now[j] = old[j]; // 不选当前物品
        if (j >= w[layer] && old[j - w[layer]] != NEG_INF) {
            now[j] = max(now[j],
                          old[j - w[layer]] + value[layer]);
        }
    }
    swap(old, now);
}
```

计数 DP 的滚动数组还要注意加法顺序：每个来源状态可能贡献多次，不能因为看到一个来源就把目标状态清零。多测时，数组大小、组合数、前缀和和记忆化缓存都要在每组开始重新初始化，不能只改 `n`。

7.9 答案位置、路径恢复与计数取模

- 求“恰好装满”时，不能把所有容量都当成合法答案；只允许读取 `dp[capacity]`。

- 求“最多装多少”时，容量不必装满，答案通常是 $\max(dp[0..capacity])$ 。
- 需要输出方案时，必须在转移发生时记录前驱，或者在最终状态上反向判断；只输出最优值不等于完成题目。
- 计数 DP 每次加法和乘法都要及时取模；减法要先加模数，不能依赖 C++ 对负数取模的结果。
- 多个方案是否有顺序，要决定循环是“先物品后容量”还是“先容量后物品”；两种顺序可能分别统计组合和排列。

7.10 DP 最小回归样例

1. **全负收益**：数组 $[-5, -2, -7]$ ，最大值应为 -2 ，可以抓出把最大值 DP 错误初始化为 0 。
2. **答案不在最后**：数组 $[3, 1, 2]$ 的最长上升子序列长度为 2 ，若只输出最后一个状态，容易错误输出 1 。
3. **0-1 与完全背包**：一个物品重量 2 、价值 3 ，容量 4 ；0-1 背包答案为 3 ，完全背包答案为 6 。
4. **不可达容量**：重量只有 3 ，查询容量 1 和 2 ，必须输出题目规定的不可达标记，不能输出 0 或一个溢出的负数。
5. **多测残留**：第一组使用较大的容量并得到答案，第二组只有一个物品且容量为 0 ，检查答案数组和滚动数组是否真正清空。
6. **计数顺序**：硬币面值 $1, 2$ 、目标和 3 ，若统计组合答案为 2 ，若统计排列答案为 3 ；先确认题目问哪一种。

DP 怎么都调不对时：找最小样例，把 DP 表打印出来，手算一张表，对比第一处不同的格子——那里就是 bug。

8 数据结构

8.1 并查集

- 合并必须根对根: $fa[find(u)] = find(v)$ 。
- 初始化: `iota(fa+1, fa+n+1, 1)` 或 `for(i=1;i<=n;i++) fa[i]=i`。
- 统计连通块: 数 $find(i)$ 的种类, 或数 $fa[i]==i$ 的个数。

8.2 线段树

- Lazy tag: 区间加是累加 $tag[ls] += tag[u]$, 区间赋值才是覆盖。
- Pushdown: 传 tag $\rightarrow$ 更新子节点 tree 值 $\rightarrow$ 清父 tag。三步缺一不可。
- 区间长度: 左儿子用 $mid-l+1$ (节点范围), 不是查询边界 L。
- 越界返回: 最大值返回 $-\infty$, 最小值返回 $+\infty$, 求和返回 0。

8.3 二分

- 出现 $l = mid \Rightarrow mid$ 上取整 $mid = (l+r+1)/2$, 否则死循环。
- 防溢出: $mid = l + (r-l)/2$ 。
- 浮点二分: 固定循环 100 次, 别用 `while(r-l>eps)`。
- `check(mid)` 里的辅助数组每次都要清空。

8.4 双指针 / 滑窗

- 左指针不能越过右指针: `while(l<=r && ...)`。
- 最长区间: 修复合法后更新答案。最短区间: 即将破坏条件前更新。
- 分段统计漏最后一段 $\Rightarrow$ 加哨兵 `s += '#'` 或 `a[n+1] = inf`。

8.5 树状数组：下标、前缀和与差分

树状数组只维护“前缀可合并”的信息。最常见的单点加、区间和模板如下，所有下标都假定为 1-based:

```
struct Fenwick {
    int n;
    vector<long long> tree;
    Fenwick(int n) : n(n), tree(n + 1, 0) {}

    void add(int x, long long value) {
        for (; x <= n; x += x & -x) tree[x] += value;
    }

    long long prefix_sum(int x) const {
        long long result = 0;
        for (; x > 0; x -= x & -x) result += tree[x];
        return result;
    }

    long long range_sum(int l, int r) const {
        if (l > r) return 0;
        return prefix_sum(r) - prefix_sum(l - 1);
    }
};
```

`add(0, value)` 会在 `x += x & -x` 后永远停在 0，造成死循环；`prefix_sum(n+1)` 可能越界。离散化之后也必须把排名改成从 1 开始。

8.5.1 区间加、单点查：把差分藏进树状数组

对原数组做差分 $d_i = a_i - a_{i-1}$ 。给区间 $[l, r]$ 加 v ，只需 `add(l, v)`、`add(r+1, -v)`；查询位置 x 时求差分前缀和。测试时必须包含 $r = n$ ，否则 $r+1$ 越界的错误永远不会出现。

8.6 单调栈与单调队列

8.6.1 下一个更大元素：栈内保存什么

维护一个从栈底到栈顶单调递减的下标栈。处理 a_i 时，所有小于当前值的元素都找到答案；相等元素是否弹出要由题目要求决定：求“严格更大”通常弹出 $\leq$ ，求“更大或相等”通常只弹出 $<$ 。

```
vector<int> next_greater(const vector<int>& a) {
    int n = (int)a.size();
    vector<int> answer(n, -1), st; // st 保存还没有答案的下标
    for (int i = 0; i < n; ++i) {
        while (!st.empty() && a[st.back()] < a[i]) {
            answer[st.back()] = i;
            st.pop_back();
        }
        st.push_back(i);
    }
    return answer;
}
```

8.6.2 滑动窗口最小值：队首才是答案

单调队列保存下标，队列中的值从小到大。每次先删掉窗口左侧的下标，再弹出队尾所有不可能成为最小值的元素，最后读队首：

```
vector<long long> window_min(const vector<long long>& a, int k) {
    deque<int> q;
    vector<long long> answer;
    for (int i = 0; i < (int)a.size(); ++i) {
        while (!q.empty() && q.front() <= i - k) q.pop_front();
        while (!q.empty() && a[q.back()] >= a[i]) q.pop_back();
        q.push_back(i);
        if (i >= k - 1) answer.push_back(a[q.front()]);
    }
    return answer;
}
```

窗口长度 $k = 1$ 、 $k = n$ 、数组全相等、数组单调递增和单调递减，是单调队列最有价值的五组测试。尤其要确认队列存的是下标而不是值，因为只有下标才能判断元素是否过期。

8.7 离散化与区间结构

当数值很大但只关心相对大小时，先复制、排序、去重，再用 `lower_bound` 找排名。离散化不是“把值除以一个数”，重复值必须映射到同一个编号；若题目有区间端点和坐标长度，还要判断是否需要加入相邻点之间的空隙。

```
vector<long long> values = all_values;
sort(values.begin(), values.end());
values.erase(unique(values.begin(), values.end()), values.end());
```

```
int rank_of(long long x) {  
    return int(lower_bound(values.begin(), values.end(), x) - values.begin()) + 1;  
}
```

只把出现过的端点压成连续编号，适合统计点；如果要计算覆盖长度， $x = 1$ 和 $x = 2$ 之间的长度不能凭空消失，常需要把端点、端点加 1 或相邻坐标都放入离散化数组。先写清楚“一个编号代表点还是代表小区间”。

8.8 数据结构回归样例

1. **树状数组**： $n = 1$ ，先加在 1，再查询 $[1, 1]$ ；随后查询空区间 $[2, 1]$ ，答案应为 0。
2. **单调结构**：数组 $[3, 3, 3]$ ，分别测试严格大于与大于等于；窗口 $k = 1$ 和 $k = n$ 都必须输出完整正确结果。
3. **线段树懒标记**：初始 $[1, 1, 1, 1]$ ，整段加 5 后查询中间区间，再单点修改并查询整段，专门逼出漏写 `push_down` 的问题。
4. **离散化**：输入值 $[10^9, 10^9, -10^9, 0]$ ，确认重复值排名相同，负数不会因为数组下标类型转换而错位。

9 博弈论

博弈题先确认三件事：双方是否轮流操作、是否完全信息、是否存在后手无法操作的终止状态。默认下面讨论“双方最优、不能操作者输”的正常玩法；题目若写了反常玩法、和棋或同时操作，不能直接套用。

9.1 必胜态与必败态

把一个局面记为状态 s 。若当前玩家能走到某个必败态，当前态就是必胜态；若所有后继状态都是必胜态，当前态就是必败态。石子游戏的记忆化模板如下：

```
vector<int> moves;           // 允许拿走的石子数
vector<int> memo;           // -1 未计算, 0 必胜, 1 必败

bool win(int stones) {
    if (stones == 0) return false; // 没有合法操作, 当前玩家输
    int& answer = memo[stones];
    if (answer != -1) return answer;
    answer = 0;                  // 先假设所有走法都救不了
    for (int take : moves) {
        if (take <= stones && !win(stones - take)) {
            answer = 1;          // 走到对手必败态
            break;
        }
    }
    return answer;
}
```

这里的返回值是“当前局面是否必胜”，不是“谁最终获胜”。最容易错的是把 `memo` 按“上一个人是否赢”来理解，或者忘记先给终止状态赋值。若状态不是石子数而是多个变量，先把所有变量编码成一个整数，或用哈希表记录状态。

9.2 Bash 游戏：先找模周期

每次最多拿 k 个石子时， n 个石子的状态满足：

- $n \bmod (k+1) = 0$ 是必败态;
- 其他状态是必胜态, 拿走 $n \bmod (k+1)$ 个即可把对手送回必败态。

边界一定要包含 $n = 0$ 、 $n = k$ 、 $n = k+1$ 、 $n = 2(k+1)$ 。当允许拿走的集合不是连续的 1 到 k 时, Bash 公式通常失效, 应先用必胜必败 DP 打表, 再观察是否出现周期。

9.3 普通 Nim: 异或和为零就是必败

有若干堆石子, 每次从一堆拿任意正数。令

$$X = a_1 \text{ xor } a_2 \text{ xor } \cdots \text{ xor } a_n.$$

$X = 0$ 时是必败态; $X \neq 0$ 时必胜。构造操作不能只输出“存在”, 而要找出一堆:

```
long long xo = 0;
for (long long x : a) xo ^= x;

for (int i = 0; i < n; ++i) {
    long long target = a[i] ^ xo;
    if (target < a[i]) { // 只能减少, 不能把这一堆变大
        cout << i << ' ' << a[i] - target << '\n';
        break;
    }
}
```

恢复答案的检查式是 $a[i] \wedge xo < a[i]$ 。如果异或之后更大, 就不是合法的“拿走石子”。测试 $[0, 0]$ 、 $[1]$ 、 $[1, 1]$ 、 $[1, 2, 3]$ 和含有大于 2^{31} 的数, 分别检查全零、单堆、必败、异或抵消和整数类型。

9.4 Sprague-Grundy 函数与 mex

对无环有向状态图, 定义 $SG(s)$ 为所有后继状态的 Grundy 数集合的 mex, 即“不在集合中的最小非负整数”。终止状态的后继集合为空, 所以 $SG = 0$:

```
vector<vector<int>>> graph;
vector<int> sg;

int get_sg(int u) {
    int& result = sg[u];
    if (result != -1) return result;
    vector<int> seen(graph[u].size() + 2, 0);
    for (int v : graph[u]) {
        int value = get_sg(v);
```



```

    if (value < (int)seen.size()) seen[value] = 1;
}
result = 0;
while (result < (int)seen.size() && seen[result]) ++result;
return result;
}

```

多个互不影响的游戏的总 Grundy 数是各自 SG 值的异或。判断总局面时只需检查异或和是否为 0；若要恢复操作，就枚举某一子游戏的合法后继，寻找使总异或变为 0 的那个后继。

SG 记忆化搜索要求状态图无环，或者至少要先处理环。对有环图直接递归会无限递归；对“走过的点不能再走”的游戏，状态通常必须包含访问集合，不能只用当前顶点作为记忆化键。

9.5 反常 Nim、阶梯 Nim 与常见变形

- 反常 Nim 的终止规则相反：拿走最后一颗的人输。若所有堆都为 1，奇数堆是必败；否则仍检查异或和是否为 0。
- 阶梯 Nim 中，石子只能向更高或更低的相邻阶移动时，先确认题目规则；常见版本的关键量是奇数层堆的异或，而不是所有堆直接异或。
- “每次只能拿固定几种数量”通常是 SG 或必胜必败 DP；“可以从任意一堆拿任意数量”才是普通 Nim。
- 看到“最后一步得分”“操作次数限制”“有资源消耗”，先检查是否已经不是纯组合游戏，可能要把轮次或资源加入状态。

9.6 Minimax 极大极小搜索与 Alpha-Beta 剪枝

棋盘状态较小、双方都能看到完整局面时，可以枚举博弈树。下面模板返回“当前玩家从该状态能得到的最大得分”，其中轮到最大化玩家时取最大值，轮到最小化玩家时取最小值：

```

int minimax(State& state, int depth, int alpha, int beta, bool maximizing) {
    if (depth == 0 || terminal(state)) return evaluate(state);

    if (maximizing) {
        int best = INT_MIN;
        for (Move move : legal_moves(state)) {
            apply(state, move);
            best = max(best, minimax(state, depth - 1,
                                    alpha, beta, false));
        }
    }
}

```

```

        undo(state, move);
        alpha = max(alpha, best);
        if (alpha >= beta) break; // beta 剪枝
    }
    return best;
} else {
    int best = INT_MAX;
    for (Move move : legal_moves(state)) {
        apply(state, move);
        best = min(best, minimax(state, depth - 1,
                                alpha, beta, true));

        undo(state, move);
        beta = min(beta, best);
        if (alpha >= beta) break; // alpha 剪枝
    }
    return best;
}
}

```

调用前必须保证 `apply` 与 `undo` 完全对称；否则深层搜索结束后，父层状态已经被污染。若同一局面会从不同路径到达，可以把局面编码成字符串或整数做哈希去重，但哈希键必须包含轮到谁、剩余深度和所有影响后续决策的信息。

9.7 博弈题的现场检查顺序

1. 先写出“不能行动时谁输”，确认是正常玩法还是反常玩法。
2. 画出 0, 1, 2, 3 个石子的状态，手算前四个必胜必败态。
3. 状态无环且子游戏独立，优先考虑 SG；棋盘小且需要选择具体走法，考虑 Minimax。
4. 需要输出操作时，不能只判断胜负，必须验证操作后状态确实满足目标不变量。
5. 多测时清空 SG、记忆化数组、棋盘哈希和搜索计数器；尤其不要把上一组的周期结论直接带到下一组。

9.8 博弈回归样例

1. **必胜必败 DP**：允许拿 1 或 3 个，测试 $n = 0, 1, 2, 3, 4$ ，答案依次为败、胜、胜、胜、败。
2. **Bash**： $k = 2$ 时测试 $n = 0, 1, 2, 3, 4, 6$ ，所有 $n \bmod 3 = 0$ 的状态应为败。
3. **普通 Nim**： $[0, 0]$ 、 $[1]$ 、 $[1, 1]$ 、 $[1, 2, 3]$ ，分别检查零堆、单堆、异或为零和多堆抵消。

4. **反常 Nim:** 测试全是 1 的偶数堆和奇数堆, 再测试含有大于 1 的堆, 确认没有把普通 Nim 公式无条件套用。
5. **搜索状态:** 棋盘只有一个格子、无合法操作、只有一个合法操作、同一状态两条路径到达, 检查终止判断、回滚和状态去重。

博弈题不要一上来背结论。先用小数据把胜负序列写出来: 如果序列出现周期, 再证明周期; 如果是多个独立部分, 再验证操作是否真的只改变其中一部分。结论和样例对不上时, 优先怀疑游戏规则理解错, 而不是先改代码。

10 数学与精度

10.1 取模（再强调）

- 减法： $(a - b \% \text{MOD} + \text{MOD}) \% \text{MOD}$ 。
- 除法：乘逆元，不能直接除。
- 连乘：步步取模。

10.2 组合计数

- ”至少包含 A 或 B” $\neq$ 包含 A 的 + 包含 B 的（重复算了交集）。用容斥或正难则反。
- 组合数函数开头：`if(m<0 | m>n) return 0|`，保证 `fact[0]=1`。
- 球相同/不同、盒相同/不同、允不允许空——四个条件决定公式。

10.3 数论

- LCM 防溢出： $a / \text{gcd}(a,b) * b$ （先除后乘）。
- 分解质因数：循环结束后 `if(n>1)` 说明剩下的是一个大质因数。
- 负数整除：C++ 向零取整，数学向下取整，负数时结果不同。

10.4 浮点

- 整数幂不用 `pow`，用快速幂。
- 判完全平方：`long long r = round(sqrt(n)); if(r*r==n)`。
- double 判等：`abs(a-b) < 1e-9`。
- 分式比大小：交叉相乘 $a \cdot d$ vs $b \cdot c$ （注意分母符号和溢出）。

11 计算几何

11.1 精度

- 浮点比较必须用 eps: $\text{abs}(a-b) < 1e-9$, 直接 `==` 几乎必错。
- eps 取多少? 一般 10^{-9} 够用, 但坐标范围大 (10^9) 时误差会放大, 考虑用 10^{-7} 或整数化。
- 能用整数就不用浮点: 叉积、点积、距离平方都可以保持整数运算。
- `atan2` 精度有限, 角度排序优先用叉积比较。

11.2 向量基础

- 叉积 $\vec{AB} \times \vec{AC} = (B - A) \times (C - A)$, 正值 C 在 AB 左侧, 负值右侧, 零共线。
- 点积 $\vec{AB} \cdot \vec{AC}$: 正为锐角, 负为钝角, 零为直角。
- 向量旋转 90° : $(x, y) \rightarrow (-y, x)$ (逆时针)。
- 单位化: 除以模长, 注意模长为 0 的退化情况。

11.3 线段与直线

- 线段相交判定: 跨立实验 (两次叉积符号相反) + 共线时的投影重叠检查。
- 共线特判容易漏: 两线段共线且有重叠部分也算相交。
- 点在线段上: 叉积为 0 且点积 ≤ 0 (即投影在线段范围内)。
- 直线交点: 用叉积公式 $t = \frac{(C-A) \times d_2}{d_1 \times d_2}$, 分母为 0 说明平行。

11.4 凸包

- Andrew 算法排序后分上下凸壳，注意最后一个点不要重复加入。
- 叉积判断用 ≤ 0 还是 < 0 ：允许共线点在凸包上用 $<$ ，不允许用 $\leq$ 。
- 所有点共线时凸包退化成线段，面积为 0，周长要特判。
- $n \leq 2$ 时直接特判，不要跑凸包算法。

11.5 多边形

- 面积（Shoelace 公式）： $S = \frac{1}{2} \left| \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) \right|$ ，下标取模。
- 点在多边形内：射线法（奇偶计数）或转角法。射线法注意射线恰好过顶点的边界情况。
- 多边形方向：叉积求面积为正 $\Rightarrow$ 逆时针，为负 $\Rightarrow$ 顺时针。题目要求特定方向时先判断再 reverse。
- Pick 定理（格点）： $S = I + \frac{B}{2} - 1$ ，边上格点数 $= \gcd(|\Delta x|, |\Delta y|)$ 。

11.6 圆

- 两圆关系：比较圆心距 d 与 $r_1 \pm r_2$ ，注意内含/外切/相交/内切/外离五种。
- 直线与圆交点：圆心到直线距离 vs 半径，距离用叉积除以方向向量模长。
- 三点确定圆（外接圆）：解方程组或用垂直平分线交点，注意三点共线无解。
- 最小覆盖圆：随机增量法，期望 $O(n)$ ，注意先 `random_shuffle`。

11.7 旋转卡壳

- 求凸包直径（最远点对）：对踵点单调移动，不要每条边都从头扫。
- 叉积比较面积来移动对踵点：`cross(hull[i+1]-hull[i], hull[j+1]-hull[j]) > 0` 时 j 前进。
- 凸包点数 ≤ 2 时直接算距离，不要进旋转卡壳。

11.8 半平面交

- 有向直线：左侧为合法区域，方向一致才能排序。

- 按极角排序，平行的保留最左边那条。
- 双端队列维护：队首队尾都可能弹出，最后还要用队首检查队尾。
- 结果为空 $\Rightarrow$ 无解；结果无界 $\Rightarrow$ 加一个大矩形边界。

11.9 常见坑总结

- 坐标范围 10^9 时叉积会到 10^{18} ，必须用 `long long`。
- 三点共线、线段端点重合、多边形退化——这些边界情况是计算几何 WA 的重灾区。
- 排序比较函数里用叉积时，共线情况要用距离做二级排序，否则凸包/极角排序会乱。
- 输出精度：题目要求保留几位小数就用 `fixed << setprecision(k)`，别用默认格式。

12 读题错误

12.1 数值范围

- 变量可以是负数或 0 吗？ $-10^9 \leq a_i \leq 10^9$ 时别默认正数。
- 负数会破坏单调性、双指针的前提、乘法的符号。

12.2 核心对象

- **Permutation vs Array**: 排列意味着互不相同、值域 $1 \sim N$ ，可以用位置映射和置换环。
- **简单图**: 没写 “no self-loops and multiple edges” 就默认有。
- **有向 vs 无向**: 看清 directed/undirected, ”关注” 是单向, ”朋友” 是双向。

12.3 操作规则

- **同时 vs 依次**: simultaneously $\Rightarrow$ 开新数组存下一状态, 统一修改。
- **代价**: 是每删一个花 X , 还是执行一次操作总共花 X ?
- **Any vs All**: all 要求全部满足, any 只要存在一个。

12.4 坐标与格式

- 输入是 (row, col) 还是 (x, y) ? C++ 数组是 `grid[row][col]`。
- 输入顺序: 不一定先 N 后 M , 看清楚。
- 无解输出: 回题面 Output 段确认, 别凭经验写 -1 。